

Machine Learning Handbook

Part 2 — Classification (Excerpt: Chapters 7–13)

Contents

পার্ট ২ — ক্লাসিফিকেশন (Part 2 — Classification)	3
Logistic Regression	5
সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)	28
K-Nearest Neighbors (KNN)	34
সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)	59
GridSearchCV in KNN	61
সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)	81
Naive Bayes	85
সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)	101
Decision Tree	104
সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)	154
Random Forest	158
সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)	219
Support Vector Machine	222
সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)	264

পার্ট ২ — ক্লাসিফিকেশন (Part 2 — Classification)

Classification কী? (What is Classification?)

Classification হলো Supervised Machine Learning-এর একটি শাখা, যেখানে মডেলের কাজ হলো একটি ইনপুট স্যাম্পলকে পূর্বনির্ধারিত কয়েকটি ডিসক্রিট ক্যাটাগরি বা ক্লাসের (discrete class) মধ্যে একটিতে ফেলা। যেমন — একটি ইমেইল Spam নাকি Not Spam, একজন রোগীর রিপোর্ট Positive নাকি Negative, অথবা একটি ছবিতে থাকা প্রাণীটি Cat, Dog নাকি অন্য কিছু — এই সবই Classification সমস্যা। মডেল ইনপুট ফিচার (X) দেখে আউটপুট লেবেল (y) প্রেডিক্ট করে, যেখানে y একটি সসীম (finite) সেট থেকে আসা মান, যেমন {0, 1} বা {Cat, Dog, Rabbit}।

Classification বনাম Regression (How it Differs from Regression)

Regression-এ আউটপুট একটি Continuous সংখ্যা (যেমন — বাড়ির দাম, তাপমাত্রা), যা যেকোনো Real Number হতে পারে। কিন্তু Classification-এ আউটপুট একটি Discrete Label বা Category। উদাহরণস্বরূপ, "আগামীকাল তাপমাত্রা কত ডিগ্রি হবে" এটি Regression প্রশ্ন, কিন্তু "আগামীকাল বৃষ্টি হবে কি না" এটি একটি Classification প্রশ্ন (Yes/No)। মূল পার্থক্য হলো Output Space-এর ধরন — Regression-এ এটি Continuous, Classification-এ এটি Discrete/Categorical। এছাড়া Evaluation Metric-ও ভিন্ন হয়: Regression-এ MSE/RMSE ব্যবহার হয়, আর Classification-এ Accuracy, Precision, Recall, F1-Score, Confusion Matrix ইত্যাদি ব্যবহার হয়।

কোথায় ব্যবহার হয়? (Where it is Used)

Classification-এর ব্যবহার প্রায় প্রতিটি ইন্ডাস্ট্রিতে দেখা যায় — Email Spam Detection, Medical Diagnosis (রোগ আছে কি নেই), Credit Card Fraud Detection, Sentiment Analysis (রিভিউ Positive/Negative), Customer Churn Prediction (গ্রাহক থাকবে নাকি চলে যাবে), Image Recognition, এবং Loan Approval System-এ। যেকোনো জায়গায় যেখানে সিদ্ধান্ত একটি নির্দিষ্ট সংখ্যক বিকল্পের মধ্যে সীমাবদ্ধ, সেখানেই Classification প্রযোজ্য।

Classification কেন গুরুত্বপূর্ণ? (Why Classification is Important)

বাস্তব জীবনের বেশিরভাগ সিদ্ধান্ত-গ্রহণ প্রক্রিয়া মূলত Classification-ভিত্তিক — অনুমোদন করব নাকি করব না, রোগ শনাক্ত করব নাকি করব না, ব্যবহারকারীকে সতর্ক করব নাকি করব না। তাই Classification অ্যালগরিদমগুলো আধুনিক Automated Decision-Making সিস্টেমের মূল ভিত্তি। এই Part-এ আমরা সাতটি গুরুত্বপূর্ণ Classification অ্যালগরিদম শিখব — সহজতম Logistic Regression থেকে শুরু করে জটিল Support Vector Machine পর্যন্ত।

এই পার্টের অধ্যায়সমূহ (Chapters in this Part)

Chapter 7 — Logistic Regression: সবচেয়ে সহজ ও ফান্ডামেন্টাল Classification অ্যালগরিদম, যা Sigmoid Function ব্যবহার করে Probability প্রেডিক্ট করে।

Chapter 8 — K-Nearest Neighbors (KNN): একটি Instance-Based পদ্ধতি, যা নতুন স্যাম্পলের সবচেয়ে কাছের k-টি প্রতিবেশীর ভোটে সিদ্ধান্ত নেয়।

Chapter 9 — GridSearchCV: কীভাবে সঠিক Hyperparameter (যেমন KNN-এর k) বেছে নিতে হয় তার পদ্ধতিগত Validation-ভিত্তিক কৌশল।

Chapter 10 — Naive Bayes: Bayes' Theorem এবং Feature Independence অনুমানের উপর ভিত্তি করে তৈরি একটি দ্রুত ও কার্যকর Probabilistic Classifier।

Chapter 11 — Decision Tree: Entropy/Gini ব্যবহার করে ডেটাকে ধাপে ধাপে বিভক্ত করে একটি ট্রি-আকৃতির Decision Rule তৈরি করে।

Chapter 12 — Random Forest: অনেকগুলো Decision Tree-এর Ensemble, যা Bootstrap Sampling ও Majority Voting দিয়ে Robustness বাড়ায়।

Chapter 13 — Support Vector Machine (SVM): Maximum Margin Hyperplane খুঁজে বের করে ক্লাসগুলোকে সবচেয়ে নিরাপদভাবে আলাদা করে।

অধ্যায়গুলো কীভাবে একে অপরের উপর নির্ভরশীল (How These Chapters Build on One Another)

এই Part-টি সাজানো হয়েছে সরলতা থেকে জটিলতার দিকে। প্রথমে Logistic Regression দিয়ে Probability-ভিত্তিক Linear Decision Boundary বোঝা হবে। এরপর KNN দিয়ে একটি সম্পূর্ণ ভিন্ন, Non-Parametric ও Distance-ভিত্তিক দৃষ্টিভঙ্গি দেখানো হবে, এবং GridSearchCV অধ্যায়ে দেখানো হবে কীভাবে KNN-এর মতো মডেলের সেরা Hyperparameter বৈজ্ঞানিকভাবে নির্বাচন করতে হয়। এরপর Naive Bayes দিয়ে সম্ভাব্যতা-ভিত্তিক (Probabilistic) চিন্তাভাবনার একটি স্বতন্ত্র শাখা পরিচিত করানো হবে। তারপর Decision Tree থেকে Random Forest — অর্থাৎ একটি একক গাছ থেকে অনেকগুলো গাছের Ensemble পর্যন্ত ধারণাগত অগ্রগতি দেখানো হবে। সবশেষে SVM দিয়ে Margin Maximization-এর ধারণা শেখানো হবে, যা পরবর্তীতে Kernel Method-এর ভিত্তি তৈরি করে। এই Part শেষ করার পর পাঠক Unsupervised Learning (Clustering ও Dimensionality Reduction) অংশে প্রবেশ করবে, যেখানে ডেটাতে কোনো Label ছাড়াই প্যাটার্ন খুঁজে বের করার কৌশল শেখানো হবে।

Logistic Regression

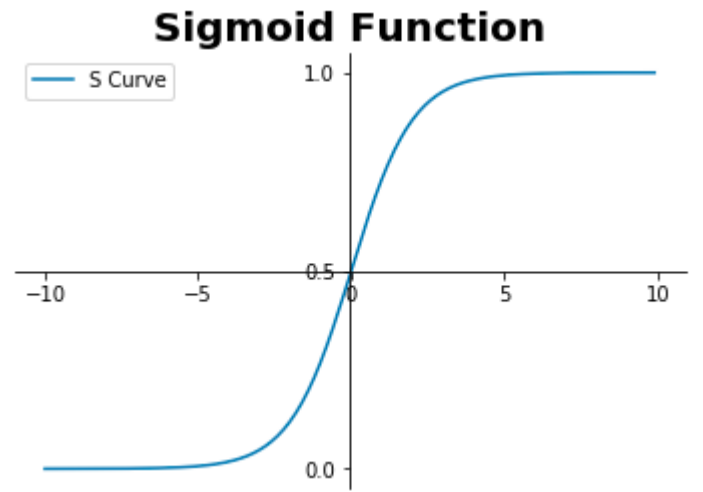
১. ভূমিকা (Introduction)

লজিস্টিক রিগ্রেশন মূলত একটি **ক্লাসিফিকেশন অ্যালগরিদম** (Classification Algorithm), যা বাইনারি ক্লাস (যেমন: 0 অথবা 1, হ্যাঁ অথবা না) প্রেডিক্ট করতে ব্যবহৃত হয়। লিনিয়ার রিগ্রেশন যেখানে $(-\infty$ থেকে $\infty)$ পর্যন্ত যেকোনো মান দিতে পারে, লজিস্টিক রিগ্রেশন সেখানে আউটপুটকে 0 থেকে 1 এর মধ্যকার একটি প্রোবাবিলিটি (Probability) বা সম্ভাবনায় রূপান্তর করে।

২. কেন এই অ্যালগরিদম তৈরি হয়েছিল (Why This Algorithm Was Developed)

Linear Regression প্রথম দিকে Classification সমস্যাতেও প্রয়োগের চেষ্টা করা হতো (আউটপুটে থ্রেশহোল্ড বসিয়ে), কিন্তু এতে বড় সমস্যা দেখা দেয়: Predicted Value যেকোনো Real Number হতে পারে $(-\infty$ থেকে $+\infty)$, অথচ Classification-এর জন্য দরকার একটি Bounded Probability (0 থেকে 1)। এছাড়া Linear Regression-এর Output-এ নতুন কোনো Outlier যোগ হলে

পুরো Decision Boundary সরে যেত, যা Classification-এর জন্য অস্থিতিশীল। এই সীমাবদ্ধতা কাটাতেই পরিসংখ্যানবিদরা মূলত Biology ও Population-Growth Model থেকে ধার করা Logistic Function ব্যবহার করে একটি নতুন মডেল তৈরি করেন, যার Output সবসময় 0 ও 1-এর মধ্যে Bound থাকে — এটাই Logistic Regression-এর জন্ম।



৩. বাস্তব জীবনের প্রেরণা (Real-life Motivation)

ধরা যাক একটি ব্যাংক ঠিক করতে চায় কোনো Loan Applicant ভবিষ্যতে Default করবে কি না — উত্তর হতে পারে শুধু দুটি: "হ্যাঁ" অথবা "না"। ব্যাংক যদি Linear Regression ব্যবহার করে, তাহলে Output হতে পারে -৫০ অথবা ৩০০ -এর মতো অর্থহীন সংখ্যা, যাকে কোনো Probability হিসেবে ব্যাখ্যা করা যায় না। কিন্তু Logistic Regression ব্যবহার করলে ব্যাংক সরাসরি বলতে পারবে, "এই Applicant-এর Default করার সম্ভাবনা ৭৮%" — যা বাস্তবসম্মত এবং সরাসরি সিদ্ধান্ত নিতে সাহায্য করে।

৪. স্বভাৱ (Intuition)

সহজভাবে চিন্তা করলে, Logistic Regression প্রথমে সাধারণ Linear Regression-এর মতোই একটি "স্কোর" (z) হিসাব করে — ফিচারগুলোর Weight-যুক্ত সমষ্টি। এরপর এই স্কোরকে Sigmoid নামক একটি S-আকৃতির ফাংশনের মধ্য দিয়ে পাঠানো হয়, যা যেকোনো সংখ্যাকে 0 ও 1-এর মধ্যে "চেপে" (squash) ফেলে। স্কোর যত বড় ধনাত্মক হবে, Probability তত 1-এর কাছাকাছি যাবে; স্কোর যত বড় ঋণাত্মক হবে, Probability তত 0-এর কাছাকাছি যাবে। শেষে একটি থ্রেশহোল্ড (সাধারণত 0.5) দিয়ে এই Probability-কে একটি নির্দিষ্ট ক্লাসে রূপান্তর করা হয়।

৫. কার্যপ্রণালী (Working Principle)

Logistic Regression মূলত চারটি ধাপে কাজ করে: (১) প্রতিটি ফিচারকে একটি Weight দিয়ে গুণ করে যোগ করে একটি Linear Score z তৈরি করা, (২) সেই Score-কে Sigmoid Function দিয়ে Probability-তে রূপান্তর করা, (৩) প্রকৃত লেবেলের সাথে এই Probability-র পার্থক্য (Error) পরিমাপ করে Loss গণনা করা, এবং (৪) Gradient Descent ব্যবহার করে Weight ও Bias-কে ধীরে ধীরে এমনভাবে Update করা যাতে Loss কমে যায়। এই চক্রটি বারবার পুনরাবৃত্তি হয় যতক্ষণ না মডেল Converge করে।

৬. গাণিতিক ভিত্তি (Mathematical Backend)

নিচে Logistic Regression-এর সম্পূর্ণ গাণিতিক কাঠামো ধাপে ধাপে দেখানো হলো — মূল সমীকরণ, Sigmoid Function-এর ডেরিভেশন, Cost Function, এবং Gradient।

৭. সূত্র (Formula)

১. মূল সমীকরণ ও সিগময়েড ফাংশন (Core Equation & Sigmoid)

প্রথমে linear combination হিসাব করা হয়:

$$z = XW + b$$

এই z -কে sigmoid function-এর মাধ্যমে probability-তে রূপান্তর করা হয়:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Final prediction:

$$\hat{y} = \sigma(XW + b) = \frac{1}{1 + e^{-(XW+b)}}$$

Sigmoid-এর গুরুত্বপূর্ণ বৈশিষ্ট্য:

$$\begin{aligned} z = 0 & \quad \text{তখন } \sigma(z) = 0.5 \\ z \rightarrow +\infty & \quad \text{তখন } \sigma(z) \rightarrow 1 \\ z \rightarrow -\infty & \quad \text{তখন } \sigma(z) \rightarrow 0 \end{aligned}$$

৮. সূত্রের ডেরিভেশন (Formula Derivation)

Odds $\rightarrow$ Sigmoid Function (Full Derivation)

১. Odds কী?

ধরি কোনো event-এর probability হলো p

তাহলে:

$$\text{odds} = \frac{p}{1-p}$$

মানে: success vs failure ratio

২. Log-Odds (Logit)

Odds-এর log নিলে:

$$\log(\text{odds}) = \log\left(\frac{p}{1-p}\right)$$

৩. Logistic Regression-এর assumption

$$z = \log\left(\frac{p}{1-p}\right)$$

অর্থাৎ:

- $z = \log\text{-odds}$

৪. Log undo করা (Exponential step)

আমরা জানি:

$$z = \log(\text{odds})$$

দুই পাশে exponential নিলে:

$$e^z = \text{odds}$$

৫. Odds বসানো

$$\frac{p}{1-p} = e^z$$

৬. এখন solve করি probability

$$\begin{aligned} p &= e^z(1-p) \\ p &= e^z - pe^z \\ p + pe^z &= e^z \\ p(1 + e^z) &= e^z \end{aligned}$$

৭. Final probability form

$$p = \frac{e^z}{1 + e^z}$$

৮. Sigmoid form এ রূপান্তর

উপরের expression কে rearrange করলে:

$$p = \frac{1}{1 + e^{-z}}$$

৯. Final Result (Sigmoid Function)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

৯. প্রতিটি চিহ্নের অর্থ (Meaning of Every Symbol)

Symbol	Meaning (অর্থ)
z	Linear Combination (Weighted Sum)
w	Weight / Coefficient
b	Bias / Intercept
x	Input Feature
y	True Label (0 অথবা 1)
$\hat{y}$	Predicted Probability
σ (sigma)	Sigmoid Function
e	Euler's Number (≈ 2.71828)

L	Loss (Binary Cross-Entropy)
α (alpha)	Learning Rate
n	মোট Sample সংখ্যা

২. কস্ট ফাংশন (Binary Cross-Entropy / Log Loss)

Logistic Regression-এ Mean Squared Error ব্যবহার করা হয় না, কারণ এটি non-convex সমস্যা তৈরি করে।

তাই ব্যবহার করা হয়:

$$J(W, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Loss Intuition

যখন $y = 1$:

$$\text{Loss} = -\log(\hat{y})$$

- $\hat{y} \approx 1 \rightarrow \text{loss} \approx 0$
- $\hat{y} \approx 0 \rightarrow \text{loss} \rightarrow \infty$

যখন $y = 0$:

$$\text{Loss} = -\log(1 - \hat{y})$$

- $\hat{y} \approx 0 \rightarrow \text{loss} \approx 0$
- $\hat{y} \approx 1 \rightarrow \text{loss} \rightarrow \infty$

৩. গ্রাডিয়েন্ট (Partial Derivatives)

Log loss + sigmoid combine করলে খুব সুন্দরভাবে gradient simplify হয়।

Linear part:

$$z = XW + b$$

$$\hat{y} = \sigma(z)$$

Weight gradient:

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

Bias gradient:

$$\frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

Important Note:

এখানে:

- $\hat{y}$ = sigmoid output
- error = $\hat{y} - y$

৪. প্যারামিটার আপডেট (Gradient Descent)

$$W = W - \alpha \cdot \frac{\partial J}{\partial W}$$
$$b = b - \alpha \cdot \frac{\partial J}{\partial b}$$

১০. সূত্রের ধাপে-ধাপে ব্যাখ্যা (Step-by-Step Formula Explanation)

ব্যাকএন্ড ওয়ার্কফ্লো (Full Process)

Step 1: Initialization

- $W = 0$ বা ছোট random value
- $b = 0$

Step 2: Linear computation

$$z = XW + b$$

Step 3: Activation (Sigmoid)

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

Step 4: Loss computation

$$J(W, b)$$

Step 5: Gradient computation

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

Step 6: Update parameters

W, b update

Step 7: Prediction rule

$$\begin{aligned}\hat{y} &\geq 0.5 \Rightarrow 1 \\ \hat{y} &< 0.5 \Rightarrow 0\end{aligned}$$

১১. সংখ্যাগত উদাহরণ (Numerical Example)

ধরা যাক একটি ইতিমধ্যে-Trained মডেলের প্যারামিটার হলো $w = 0.8$ এবং $b = -2.0$ । নতুন একজন শিক্ষার্থীর Study Hours $X = 3$ । তাহলে:

$$z = w \cdot X + b = (0.8)(3) + (-2.0) = 2.4 - 2.0 = 0.4$$

$$\hat{y} = \sigma(0.4) = 1 / (1 + e^{(-0.4)}) = 1 / (1 + 0.6703) = 1 / 1.6703 = 0.5987$$

যেহেতু $\hat{y} = 0.5987 \geq 0.5$, তাই প্রেডিক্টেড ক্লাস = 1 (Pass)। এই উদাহরণ দেখায় কীভাবে একবার Training শেষ হয়ে গেলে (w, b জানা থাকলে) নতুন যেকোনো স্যাম্পলের Prediction মাত্র দুটি ধাপে (z গণনা, তারপর Sigmoid) হয়ে যায়।

১২. ভিজ্যুয়ালাইজেশন (Visualization)

এই অধ্যায়ের শুরুতে দেখানো Sigmoid Curve চিত্রটি লক্ষ্য করুন — এটি একটি "S"-আকৃতির বক্ররেখা। X-অক্ষে z (Linear Score) এবং Y-অক্ষে $\hat{y}$ (Probability) দেখানো হয়েছে। লক্ষণীয় বিষয়: z যখন অনেক বড় ধনাত্মক (ডানদিকে), বক্ররেখা ১-এর দিকে Asymptotically এগোয়; z যখন অনেক বড় ঋণাত্মক (বামদিকে), বক্ররেখা ০-এর দিকে এগোয়; এবং $z = 0$ -তে $\hat{y}$ ঠিক ০.৫ — এটাই Decision Boundary-র কেন্দ্রবিন্দু। এই বক্ররেখার Steepness (খাড়াভাব) Weight w -এর মান দ্বারা নিয়ন্ত্রিত হয়: w যত বড় হবে, বক্ররেখা তত খাড়া (sharp) হবে।

১৪. স্কিট-লার্ন ইম্পোর্ট (Scikit-learn Import)

Python Code

```
1 import numpy as np
2
3 X = np.array([
4     [0.5, 1.5],
5     [1.0, 1.0],
6     [1.5, 0.5],
7     [3.0, 3.5],
8     [2.5, 4.5],
9     [4.0, 3.0]
10 ])
11
12 y = np.array([0, 0, 0, 1, 1, 1])
13
14 def sigmoid(z):
15     return 1 / (1 + np.exp(-z))
16
17 def predict_proba(X, W, b):
18     return sigmoid(np.dot(X, W) + b)
19
20 def predict(X, W, b, threshold=0.5):
21     return (predict_proba(X, W, b) >= threshold).astype(int)
22
23 def compute_cost(X, y, W, b):
24     y_pred = predict_proba(X, W, b)
25     return -np.mean(y*np.log(y_pred) + (1-y)*np.log(1-y_pred))
```

```

27- def compute_gradient(X, y, W, b):
28     error = predict_proba(X, W, b) - y
29     dW = (1/len(X)) * np.dot(X.T, error)
30     db = (1/len(X)) * np.sum(error)
31     return dW, db
32
33- def fit(X, y, lr, iters):
34     W = np.zeros(X.shape[1])
35     b = 0.0
36
37     for i in range(iters):
38         dW, db = compute_gradient(X, y, W, b)
39         W -= lr * dW
40         b -= lr * db
41
42     return W, b
43
44 W, b = fit(X, y, 0.1, 5000)
45
46 print(W, b)
47 print(predict(X, W, b))

```

Using Scikit Learn

```

1  from sklearn.linear_model import LogisticRegression, SGDClassifier
2  from sklearn.preprocessing import StandardScaler
3
4  lr = LogisticRegression(penalty=None)
5  lr.fit(X, y)
6
7  print(lr.coef_, lr.intercept_)
8  print(lr.predict(X))
9
10 scaler = StandardScaler()
11 X_scaled = scaler.fit_transform(X)
12
13 sgd = SGDClassifier(loss="log_loss", max_iter=5000, eta0=0.1, learning_rate
    ="constant")
14 sgd.fit(X_scaled, y)
15
16 print(sgd.coef_, sgd.intercept_)
17 print(sgd.predict(X_scaled))

```

LogisticRegression (sklearn.linear_model.LogisticRegression)

Logistic Regression classification সমস্যার জন্য ব্যবহৃত হয় এবং regularization-এর উপর অনেক বেশি নির্ভরশীল।

Logistic Regression হলো একটি Supervised Machine Learning Algorithm যা মূলত **Classification Problem** সমাধানের জন্য ব্যবহৃত হয়। নামের মধ্যে "Regression" থাকলেও এটি সাধারণত Class Predict করে। Binary Classification (Yes/No, Pass/Fail, Spam/Not Spam) থেকে শুরু করে Multiclass Classification পর্যন্ত এটি ব্যবহার করা যায়।

Linear Regression যেখানে সরাসরি একটি Continuous Value Predict করে, Logistic Regression সেখানে একটি Probability Predict করে এবং সেই Probability-এর ভিত্তিতে Final Class নির্ধারণ করে।

$$P(y = 1) = \frac{1}{1 + e^{-z}}$$

যেখানে,

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

এই Sigmoid Function-এর কারণে Output সবসময় 0 এবং 1-এর মধ্যে থাকে।

১৫. পাইথন উদাহরণ — From-Scratch Implementation (Python Example)

উপরে দেখানো Scikit-learn Implementation ছাড়াও, নিচে সম্পূর্ণ From-Scratch (NumPy দিয়ে হাতে-লেখা) Logistic Regression দেখানো হলো, যা এই অধ্যায়ের ম্যানুয়াল গণিতের সাথে হুবহু মেলে:

```
import numpy as np

class LogisticRegressionScratch:
    def __init__(self, lr=0.1, n_iters=1000):
        self.lr = lr
        self.n_iters = n_iters
        self.w = None
        self.b = 0.0

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.w = np.zeros(n_features)
        for _ in range(self.n_iters):
            z = np.dot(X, self.w) + self.b
            y_pred = self.sigmoid(z)
```

```

dw = (1/n_samples) * np.dot(X.T, (y_pred - y))
db = (1/n_samples) * np.sum(y_pred - y)
self.w -= self.lr * dw
self.b -= self.lr * db

def predict_proba(self, X):
    return self.sigmoid(np.dot(X, self.w) + self.b)

def predict(self, X, threshold=0.5):
    return (self.predict_proba(X) >= threshold).astype(int)

# Example usage
X = np.array([[1],[2],[3],[4],[5]])
y = np.array([0,0,0,1,1])
model = LogisticRegressionScratch(lr=0.1, n_iters=1000)
model.fit(X, y)
print(model.predict([[3.5]])) # -> predicted class

```

১৬. প্রেডিকশন প্রসেস (Prediction Process)

Training শেষ হওয়ার পর নতুন যেকোনো স্যাম্পলের ক্লাস প্রেডিক্ট করতে তিনটি সহজ ধাপ অনুসরণ করা হয়: (১) নতুন স্যাম্পলের ফিচার মান দিয়ে Learned w ও b ব্যবহার করে $z = w \cdot x + b$ গণনা করা, (২) সেই z -কে Sigmoid Function দিয়ে $\hat{y}$ (Probability)-তে রূপান্তর করা, এবং (৩) $\hat{y} \geq 0.5$ হলে ক্লাস 1, নাহলে ক্লাস 0 হিসেবে চূড়ান্ত সিদ্ধান্ত নেওয়া। কোনো Re-training দরকার হয় না — শুধু একবার শেখা w, b ব্যবহার করেই নতুন প্রতিটি স্যাম্পলের প্রেডিকশন মুহূর্তের মধ্যে করা যায়।

১৩. হাইপারপ্যারামিটার (Hyperparameters)

C

Default: 1.0

C হলো Regularization Strength-এর Inverse।

অনেক শিক্ষার্থী প্রথমে বিভ্রান্ত হয় কারণ SGDRegressor বা Ridge Regression-এ Regularization বাড়াতে Parameter-এর মান বাড়াতে হয়, কিন্তু Logistic Regression-এ ঠিক উল্টো।

$$C = \frac{1}{\lambda}$$

এখানে λ হলো Regularization Strength।

অর্থাৎ,

- ছোট $C \rightarrow$ বেশি Regularization
- বড় $C \rightarrow$ কম Regularization

যদি Model Training Data খুব ভালো শিখে ফেলে কিন্তু Test Data-তে খারাপ করে, তাহলে Overfitting হচ্ছে। এমন পরিস্থিতিতে C কমানো উচিত।

উল্টোভাবে, যদি Model খুবই Simple হয়ে যায় এবং Training Data-ও ঠিকমতো শিখতে না পারে, তাহলে C বাড়ানো যেতে পারে।

বাস্তবে Logistic Regression-এর সবচেয়ে গুরুত্বপূর্ণ Hyperparameter সাধারণত C ।

penalty

Default: 'l2'

penalty নির্ধারণ করে কোন ধরনের Regularization ব্যবহার করা হবে।

Machine Learning Model-এর একটি বড় সমস্যা হলো Overfitting। Regularization এই সমস্যা কমাতে সাহায্য করে।

$l2$ Regularization Weight-এর বড় মানকে Penalize করে এবং Coefficient-গুলোকে ছোট রাখে।

$$Loss + \lambda \sum w^2$$

এটি সবচেয়ে বেশি ব্যবহৃত Regularization।

$l1$ Regularization Weight-এর Absolute Value Penalize করে।

$$Loss + \lambda \sum |w|$$

এর ফলে অনেক Coefficient সরাসরি শূন্য হয়ে যায়। তাই Feature Selection-এর জন্য এটি খুব উপকারী।

elasticnet ব্যবহার করলে $L1$ এবং $L2$ দুটোর সুবিধা একসাথে পাওয়া যায়।

যদি Feature অনেক বেশি থাকে এবং কোনগুলো গুরুত্বপূর্ণ তা খুঁজতে চান, তাহলে $l1$ একটি ভালো Choice হতে পারে।

solver

Default: 'lbfgs'

Logistic Regression Training-এর সময় Optimal Weight বের করতে Optimization Algorithm ব্যবহার করে। solver Parameter সেই Algorithm নির্বাচন করে।

সব Solver একই কাজ করলেও তাদের Speed, Memory Usage এবং Supported Penalty আলাদা।

lbfgs

সবচেয়ে জনপ্রিয় এবং Default Solver।

- L2 Support করে
- Small ও Medium Dataset-এর জন্য ভালো
- Multiclass Support করে

সাধারণ ব্যবহারের জন্য এটি যথেষ্ট।

liblinear

- L1 এবং L2 দুটোই Support করে
- Small Dataset-এর জন্য উপযুক্ত
- Binary Classification-এ বেশি ব্যবহৃত

যদি `penalty='l1'` ব্যবহার করতে চান, তাহলে প্রায়ই `liblinear` অথবা `saga` ব্যবহার করতে হয়।

newton-cg

দ্বিতীয়-ধাপ (Second Order) Optimization ব্যবহার করে।

- Smooth Convergence
- Medium Dataset-এর জন্য ভালো

sag

- Large Dataset-এর জন্য দ্রুত
- শুধুমাত্র L2 Support করে

Dataset বড় হলে এটি Training অনেক দ্রুত করতে পারে।

saga

সবচেয়ে Flexible Solver।

- L1 Support করে
- L2 Support করে
- ElasticNet Support করে
- Large Dataset-এ কার্যকর

ElasticNet ব্যবহার করতে চাইলে সাধারণত `saga` Solver ব্যবহার করা হয়।

class_weight

Default: None

বাস্তব Dataset-এ অনেক সময় Class Distribution সমান থাকে না।

ধরুন,

Class	Count
No	950
Yes	50

এখানে Model যদি সবকিছুকে "No" Predict করে, তবুও Accuracy হবে 95%।

কিন্তু বাস্তবে Model কোনো কাজে আসবে না।

এই সমস্যা সমাধানের জন্য `class_weight='balanced'` ব্যবহার করা হয়।

`class_weight='balanced'`

এটি Minority Class-কে বেশি Weight দেয়।

ফলে Model শুধু Majority Class শেখার বদলে Minority Class-কেও গুরুত্ব দেয়।

Imbalanced Dataset-এ এটি অত্যন্ত গুরুত্বপূর্ণ Parameter।

max_iter

Default: 100

Training চলাকালে Solver কত Iteration পর্যন্ত চেষ্টা করবে, সেটি `max_iter` নিয়ন্ত্রণ করে।

যদি Training-এর সময় নিচের Warning আসে:

ConvergenceWarning

তাহলে বুঝতে হবে Model এখনো Converge করতে পারেনি।

এক্ষেত্রে সাধারণত:

`max_iter=1000`

অথবা

`max_iter=5000`

ব্যবহার করা হয়।

fit_intercept

Default: True

Logistic Regression-এর Equation হলো:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

এখানে,

b

হলো Bias বা Intercept।

fit_intercept=True হলে Model Bias Term শিখবে।

বাস্তবে অধিকাংশ ক্ষেত্রেই এটি True রাখা উচিত।

শুধুমাত্র Data যদি ইতোমধ্যে Perfectly Centered হয়, তখন False বিবেচনা করা যেতে পারে।

l1_ratio

l1_ratio শুধুমাত্র নিচের ক্ষেত্রে কাজ করে:

penalty='elasticnet'

এবং

solver='saga'

এটি নির্ধারণ করে L1 এবং L2 কত শতাংশ ব্যবহার হবে।

$$0 \leq l1_ratio \leq 1$$

যদি:

l1_ratio=0

হয়, তাহলে Pure L2।

আর যদি:

l1_ratio=1

হয়, তাহলে Pure L1।

মাঝামাঝি মান দিলে L1 এবং L2-এর মিশ্রণ পাওয়া যায়।

multi_class

Default: 'auto'

যখন Target Variable-এ দুইটির বেশি Class থাকে, তখন Logistic Regression Multiclass Problem কীভাবে Handle করবে তা multi_class নির্ধারণ করে।

One-vs-Rest (OVR)

multi_class='ovr'

এখানে প্রতিটি Class-এর জন্য আলাদা Binary Classifier তৈরি হয়।

ধরুন Class:

- Cat
- Dog
- Bird

তাহলে Model তিনটি আলাদা Binary Problem তৈরি করবে।

- Cat vs Others
- Dog vs Others
- Bird vs Others

Multinomial

`multi_class='multinomial'`

এখানে সরাসরি Softmax ব্যবহার করা হয়।

$$P(y = i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

সাধারণভাবে Multiclass Problem-এ এটি বেশি Accurate হয়।

বর্তমান Scikit-Learn Version-এ auto সাধারণত Multinomial Approach বেছে নেয় যখন Solver সেটি Support করে।

tol

Default: 1e-4

tol হলো Convergence Threshold।

Training চলাকালে যদি Loss Improvement এই মানের নিচে নেমে যায়, তাহলে Solver ধরে নেয় যে আর উল্লেখযোগ্য উন্নতি সম্ভব নয়।

ফলে Training বন্ধ হয়ে যায়।

বড় tol:

- দ্রুত Training
- কম Precision

ছোট tol:

- ধীর Training
- বেশি Precise Solution

random_state

Training-এর সময় Random Operation-এর জন্য Seed নির্ধারণ করে।

`random_state=42`

একই Code বারবার চালালে একই Result পাওয়ার জন্য এটি ব্যবহার করা হয়।

Research, Assignment এবং Experiment Tracking-এর ক্ষেত্রে এটি অত্যন্ত গুরুত্বপূর্ণ।

warm_start

Default: False

সাধারণত .fit() কল করলে Logistic Regression শুরু থেকে Training শুরু করে।

কিন্তু:

warm_start=True

দিলে আগের Training-এর Weight ব্যবহার করে নতুন Training শুরু হয়।

এটি Incremental Training অথবা বারবার Model Re-training-এর ক্ষেত্রে উপকারী হতে পারে।

n_jobs

n_jobs নির্ধারণ করে Training-এর সময় কতগুলো CPU Core ব্যবহার করা হবে।

n_jobs=-1

দিলে সিস্টেমের সব Available CPU Core ব্যবহার করা হয়।

এটি Model-এর Accuracy পরিবর্তন করে না।

শুধুমাত্র Training Speed বাড়ায়।

বিশেষ করে Large Dataset এবং Cross Validation-এর সময় এটি কার্যকর।

verbose

Default: 0

verbose নির্ধারণ করে Training-এর সময় কতটুকু Log বা Progress Information Console-এ দেখাবে।

verbose=0 মানে কোনো Output দেখাবে না। verbose=1 বা তার বেশি দিলে প্রতিটি Iteration-এর Progress দেখা যায়, যা Debugging বা দীর্ঘ Training-এর সময় কাজে লাগে।

dual

Default: 'auto'

dual Parameter নির্ধারণ করে Optimization Problem-এর Dual নাকি Primal Formulation ব্যবহার হবে।

শুধুমাত্র solver='liblinear' এবং penalty='l2' হলে এটি প্রযোজ্য। সাধারণত n_samples > n_features হলে dual=False (Primal) এবং n_samples < n_features হলে dual=True (Dual) বেশি কার্যকর। 'auto' দিলে Scikit-Learn নিজেই সঠিকটি বেছে নেয়।

LogisticRegression() Attributes

coef_

Default: Created after fit()

প্রতিটি Feature-এর Learned Weight (Coefficient) সংরক্ষণ করে। Positive Coefficient হলে Feature বাড়ার সাথে Positive Class-এর Probability বাড়ে এবং Negative Coefficient হলে Probability কমে। এটি Logistic Regression-এর সবচেয়ে গুরুত্বপূর্ণ Output।

intercept_

Default: Created after fit()

Model-এর Learned Bias (Intercept) সংরক্ষণ করে। সব Feature-এর মান 0 হলে Decision Function-এর প্রাথমিক মান নির্ধারণ করে।

classes_

Default: Created after fit()

Training Data-তে থাকা সকল Target Class-এর List সংরক্ষণ করে। Binary Classification-এ সাধারণত [0, 1] এবং Multi-class Classification-এ সব Unique Class থাকে।

n_iter_

Default: Created after fit()

Solver Converge হতে প্রতিটি Class-এর জন্য কতটি Iteration লেগেছে তা সংরক্ষণ করে। যদি max_iter-এ পৌঁছেও Converge না হয়, তাহলে Warning দেখা যেতে পারে।

n_features_in_

Default: Created after fit()

Training-এর সময় Model মোট কতটি Input Feature ব্যবহার করেছে তা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

n_classes_

Default: Created after fit() (*Some scikit-learn versions*)

Training Data-তে মোট কতটি Target Class রয়েছে তা সংরক্ষণ করে। এটি len(classes_)-এর সমান।

LogisticRegression-এর সবচেয়ে গুরুত্বপূর্ণ Parameter

IMPORTANT NOTE

Binary Cross-Entropy ভুল প্রেডিকশনকে exponentially বেশি penalize করে — একদম আত্মবিশ্বাসী কিন্তু ভুল প্রেডিকশনের loss প্রায় ∞ পর্যন্ত যেতে পারে। এটিই Log Loss-কে Classification-এর জন্য কার্যকর করে তোলে।

EXAM TIP

Odds $\rightarrow$ Log-Odds $\rightarrow$ Exponential $\rightarrow$ Algebraic rearrangement — এই ৪-ধাপের Derivation পরীক্ষায় প্রায়ই চাওয়া হয়। প্রতিটি ধাপ মুখস্থ না করে যুক্তি দিয়ে বোঝা ভালো, কারণ এতে ভুল করার সম্ভাবনা কমে যায়।

১৭-১৯. সুবিধা, অসুবিধা ও প্রয়োগক্ষেত্র (Advantages, Disadvantages & Applications)

Advantages, Disadvantages ও Real-World Applications

Advantages	Disadvantages
আউটপুট সরাসরি probability হিসেবে interpretable	শুধুমাত্র linearly (বা near-linearly) separable ডেটায় ভালো কাজ করে
কম্পিউটেশনালি সাশ্রয়ী ও দ্রুত train হয়	Outlier-এর প্রতি sensitive
L1/L2 Regularization-এর মাধ্যমে Overfitting নিয়ন্ত্রণযোগ্য	জটিল Non-linear সম্পর্ক ক্যাপচার করতে পারে না

REAL-WORLD EXAMPLE

Email Spam Detection, মেডিকেল ডায়াগনোসিসে রোগী আক্রান্ত/সুস্থ নির্ণয়, ব্যাংকিং-এ Credit Default Prediction, এবং Marketing-এ Customer Churn Prediction — এসব ক্ষেত্রে Logistic Regression একটি জনপ্রিয় Baseline Model।

COMMON MISTAKE

Multiclass সমস্যায় ভুলভাবে Binary Logistic Regression প্রয়োগ করা — `multi_class='multinomial'` বা One-vs-Rest ব্যবহার না করা।

Imbalanced Dataset-এ `class_weight` প্যারামিটার উপেক্ষা করা, যার ফলে Minority Class-এর Recall খুব কম থাকে।

INTERVIEW TIP

Q: কেন Logistic Regression-এ MSE ব্যবহার করা হয় না?

A: Sigmoid-এর সাথে MSE ব্যবহার করলে Cost Function Non-convex হয়ে যায়, ফলে Gradient Descent Local Minima-তে আটকে যেতে পারে। Binary Cross-Entropy ব্যবহার করলে ফাংশনটি Convex থাকে।

Q: Logistic Regression নামে 'Regression' থাকলেও এটি Classification Algorithm কেন?

A: কারণ এটি প্রথমে একটি Continuous Probability মান (Regression-এর মতো) প্রেডিক্ট করে, তারপর একটি Threshold (সাধারণত 0.5) ব্যবহার করে সেটিকে Discrete Class-এ রূপান্তর করে।

২০. পূর্ববর্তী অ্যালগরিদমের সাথে তুলনা (Comparison with Previous Algorithm)

Part 1-এ শেখা Linear Regression-এর সাথে Logistic Regression-এর তুলনা করলে মূল পার্থক্যগুলো স্পষ্ট হয়:

দিক (Aspect)	Linear Regression	Logistic Regression
Output ধরন	Continuous (যেকোনো Real Number)	Probability (0 থেকে 1-এর মধ্যে)
ব্যবহার	Regression সমস্যা	Classification সমস্যা
সমীকরণ	$\hat{y} = w \cdot x + b$	$\hat{y} = \sigma(w \cdot x + b)$
Loss Function	Mean Squared Error (MSE)	Binary Cross-Entropy (Log Loss)
Decision Boundary	প্রয়োজ্য নয়	একটি Threshold (সাধারণত 0.5)-এর মাধ্যমে নির্ধারিত
Outlier-এর প্রভাব	খুব বেশি সংবেদনশীল	Sigmoid-এর কারণে তুলনামূলক কম সংবেদনশীল

২১. সাধারণ ভুলত্রুটি (Common Mistakes)

- MSE (Mean Squared Error) ব্যবহার করা, যেখানে Binary Cross-Entropy ব্যবহার করা উচিত — MSE Non-Convex Loss Surface তৈরি করে, ফলে Gradient Descent ঠিকমতো Converge নাও করতে পারে।
- Feature Scaling (Standardization/Normalization) না করা — এতে Gradient Descent-এর Convergence অনেক ধীর হয়ে যায়, বিশেষ করে যখন ফিচারগুলোর Scale ভিন্ন ভিন্ন হয়।
- সবসময় 0.5 Threshold-কে সর্বোত্তম মনে করা — Imbalanced Dataset-এ (যেমন Fraud Detection) 0.5-এর বদলে অন্য Threshold বেছে নেওয়া ভালো ফলাফল দিতে পারে।
- Coefficient (w)-কে সরাসরি Linear Regression-এর মতো ব্যাখ্যা করা — Logistic Regression-এ Coefficient আসলে Log-Odds-এর পরিবর্তন বোঝায়, সরাসরি Probability-র পরিবর্তন নয়।
- max_iter বাড়ানো ছাড়াই ConvergenceWarning উপেক্ষা করা — এর ফলে মডেল Underfit থেকে যেতে পারে কারণ Optimization সম্পূর্ণভাবে শেষ হয়নি।

২২. সাক্ষাৎকারের প্রশ্ন (Interview Questions)

Q: Logistic Regression নাম হলেও এটি কেন Classification Algorithm, Regression নয়?

A: কারণ এর চূড়ান্ত লক্ষ্য একটি Discrete Class প্রেডিক্ট করা (Sigmoid দিয়ে Probability বের করে Threshold দিয়ে ক্লাস নির্ধারণ করা), যদিও ভিতরে একটি Linear Regression-এর মতো গণনা লুকিয়ে থাকে।

Q: কেন Logistic Regression-এ MSE-এর বদলে Binary Cross-Entropy ব্যবহার করা হয়?

A: কারণ Sigmoid ফাংশনের সাথে MSE ব্যবহার করলে Loss Surface Non-Convex হয়ে যায় (অনেক Local Minima তৈরি হয়), কিন্তু Binary Cross-Entropy Sigmoid-এর সাথে একটি Convex Loss Surface দেয়, যা Gradient Descent-কে নির্ভরযোগ্যভাবে Global Minimum-এ পৌঁছাতে সাহায্য করে।

Q: C Hyperparameter কী নিয়ন্ত্রণ করে?

A: C হলো Regularization Strength-এর Inverse। ছোট C মানে বেশি Regularization (সরল মডেল), বড় C মানে কম Regularization (মডেল Training Data-কে আরও নিবিড়ভাবে Fit করার চেষ্টা করে)।

Q: Logistic Regression কীভাবে Multiclass সমস্যা সামলায়?

A: সাধারণত দুইটি উপায়ে — One-vs-Rest (প্রতিটি ক্লাসের জন্য আলাদা Binary Classifier) অথবা Multinomial (Softmax ব্যবহার করে একবারে সব ক্লাসের Probability গণনা)।

Q: Decision Boundary Linear নাকি Non-Linear?

A: মূল (Vanilla) Logistic Regression-এ Decision Boundary সবসময় Linear (একটি সরলরেখা বা হাইপারপ্লেন), কারণ ভিতরের $z = w \cdot x + b$ একটি Linear সমীকরণ। Non-Linear Boundary পেতে হলে Polynomial Features যোগ করতে হয়।

২৩. সারাংশ (Summary)

Logistic Regression হলো Classification-এর সবচেয়ে সহজ, দ্রুত এবং সবচেয়ে বেশি ব্যাখ্যাযোগ্য (Interpretable) মডেল। এটি একটি Linear Score-কে Sigmoid Function দিয়ে Probability-তে রূপান্তর করে, Binary Cross-Entropy Loss কমানোর মাধ্যমে Gradient Descent দিয়ে Train হয়, এবং একটি Threshold ব্যবহার করে চূড়ান্ত ক্লাস প্রেডিক্ট করে। এটি Linear Decision Boundary তৈরি করে, তাই জটিল, Non-Linear প্যাটার্নের জন্য উপযুক্ত নয় — এই সীমাবদ্ধতা কাটাতেই পরবর্তী অধ্যায়গুলোতে KNN, Decision Tree, Random Forest ও SVM-এর মতো আরও শক্তিশালী পদ্ধতি শেখানো হবে।

সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)

Introduction

এই সেকশনে আমরা একটি অত্যন্ত ছোট ডেটাসেট নিয়ে সম্পূর্ণ Logistic Regression প্রক্রিয়া সম্পূর্ণ হাতে-কলমে (manually) সমাধান করব — Initialization থেকে শুরু করে Gradient Descent-এর প্রতিটি Iteration এবং শেষে নতুন একটি স্যাম্পলের উপর Prediction পর্যন্ত। প্রতিটি সংখ্যা ক্যালকুলেটর দিয়ে যাচাইযোগ্য।

Dataset

ডেটাসেট: একটি শিক্ষার্থী কত ঘণ্টা পড়াশোনা করেছে (X) তার ভিত্তিতে সে পরীক্ষায় Pass (1) করবে নাকি Fail (0) করবে তা প্রেডিক্ট করা হবে।

Sample	Study Hours (X)	Result (y)
1	1	0
2	2	0
3	3	0
4	4	1
5	5	1

Initialization

প্রাথমিক Weight ও Bias শূন্য (0) থেকে শুরু করা হলো, এবং Learning Rate (α) ধরা হলো 0.1:

$w = 0$, $b = 0$, $\alpha = 0.1$, $n = 5$

Core Formula

প্রথমে Linear অংশ z গণনা করা হয়, এরপর Sigmoid Function দিয়ে সেটিকে Probability-তে রূপান্তর করা হয়:

$$z = w \cdot X + b$$

$$\hat{y} = \sigma(z) = 1 / (1 + e^{(-z)})$$

z : Linear Combination-এর ফলাফল। $\hat{y}$: প্রেডিক্ট করা Probability (0 থেকে 1)। σ : Sigmoid Function। e : Euler's Number (≈ 2.71828)।

Loss Function

Binary Cross-Entropy (Log Loss) ব্যবহার করা হয়:

$$L = -(1/n) \cdot \sum [y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

y: প্রকৃত লেবেল (0 বা 1) | $\hat{y}$: প্রেডিক্ট করা Probability | n: মোট স্যাম্পল সংখ্যা।

Gradient / Decision Rule

Weight ও Bias-এর সাপেক্ষে Loss-এর Gradient:

$$dw = (1/n) \cdot \sum (\hat{y}_i - y_i) \cdot x_i$$

$$db = (1/n) \cdot \sum (\hat{y}_i - y_i)$$

Update Rule

$$w := w - \alpha \cdot dw$$

$$b := b - \alpha \cdot db$$

Iteration 1 (current: w = 0.0000, b = 0.0000)

$$x = 1, y = 0: z = 0.0000 \times 1 + (0.0000) = 0.0000 \rightarrow \hat{y} = 1/(1+e^{(-0.0000)}) = 0.5000 \rightarrow \text{loss} = -[0 \cdot \log(0.5000) + 1 \cdot \log(0.5000)] = 0.6931$$

$$x = 2, y = 0: z = 0.0000 \times 2 + (0.0000) = 0.0000 \rightarrow \hat{y} = 1/(1+e^{(-0.0000)}) = 0.5000 \rightarrow \text{loss} = -[0 \cdot \log(0.5000) + 1 \cdot \log(0.5000)] = 0.6931$$

$$x = 3, y = 0: z = 0.0000 \times 3 + (0.0000) = 0.0000 \rightarrow \hat{y} = 1/(1+e^{(-0.0000)}) = 0.5000 \rightarrow \text{loss} = -[0 \cdot \log(0.5000) + 1 \cdot \log(0.5000)] = 0.6931$$

$$x = 4, y = 1: z = 0.0000 \times 4 + (0.0000) = 0.0000 \rightarrow \hat{y} = 1/(1+e^{(-0.0000)}) = 0.5000 \rightarrow \text{loss} = -[1 \cdot \log(0.5000) + 0 \cdot \log(0.5000)] = 0.6931$$

$$x = 5, y = 1: z = 0.0000 \times 5 + (0.0000) = 0.0000 \rightarrow \hat{y} = 1/(1+e^{(-0.0000)}) = 0.5000 \rightarrow \text{loss} = -[1 \cdot \log(0.5000) + 0 \cdot \log(0.5000)] = 0.6931$$

$$\text{Avg Loss} = (0.6931 + 0.6931 + 0.6931 + 0.6931 + 0.6931) / 5 = 3.4657 / 5 = 0.6931$$

$$dw = [(0.5000-0) \times 1 + (0.5000-0) \times 2 + (0.5000-0) \times 3 + (0.5000-1) \times 4 + (0.5000-1) \times 5] / 5 = -0.3000$$

$$db = [(0.5000-0) + (0.5000-0) + (0.5000-0) + (0.5000-1) + (0.5000-1)] / 5 = 0.1000$$

$$w_{\text{new}} = 0.0000 - (0.1) \times (-0.3000) = 0.0300$$

$$b_new = 0.0000 - (0.1) \times (0.1000) = -0.0100$$

Iteration 2 (current: w = 0.0300, b = -0.0100)

$$x = 1, y = 0: z = 0.0300 \times 1 + (-0.0100) = 0.0200 \rightarrow \hat{y} = 1/(1+e^{(-0.0200)}) = 0.5050 \rightarrow \text{loss} = -[0 \cdot \log(0.5050) + 1 \cdot \log(0.4950)] = 0.7032$$

$$x = 2, y = 0: z = 0.0300 \times 2 + (-0.0100) = 0.0500 \rightarrow \hat{y} = 1/(1+e^{(-0.0500)}) = 0.5125 \rightarrow \text{loss} = -[0 \cdot \log(0.5125) + 1 \cdot \log(0.4875)] = 0.7185$$

$$x = 3, y = 0: z = 0.0300 \times 3 + (-0.0100) = 0.0800 \rightarrow \hat{y} = 1/(1+e^{(-0.0800)}) = 0.5200 \rightarrow \text{loss} = -[0 \cdot \log(0.5200) + 1 \cdot \log(0.4800)] = 0.7339$$

$$x = 4, y = 1: z = 0.0300 \times 4 + (-0.0100) = 0.1100 \rightarrow \hat{y} = 1/(1+e^{(-0.1100)}) = 0.5275 \rightarrow \text{loss} = -[1 \cdot \log(0.5275) + 0 \cdot \log(0.4725)] = 0.6397$$

$$x = 5, y = 1: z = 0.0300 \times 5 + (-0.0100) = 0.1400 \rightarrow \hat{y} = 1/(1+e^{(-0.1400)}) = 0.5349 \rightarrow \text{loss} = -[1 \cdot \log(0.5349) + 0 \cdot \log(0.4651)] = 0.6256$$

$$\mathcal{J}_{\text{Loss}} = (0.7032 + 0.7185 + 0.7339 + 0.6397 + 0.6256) / 5 = 3.4209 / 5 = 0.6842$$

$$dw = [(0.5050-0) \times 1 + (0.5125-0) \times 2 + (0.5200-0) \times 3 + (0.5275-1) \times 4 + (0.5349-1) \times 5] / 5 = -0.2251$$

$$db = [(0.5050-0) + (0.5125-0) + (0.5200-0) + (0.5275-1) + (0.5349-1)] / 5 = 0.1200$$

$$w_new = 0.0300 - (0.1) \times (-0.2251) = 0.0525$$

$$b_new = -0.0100 - (0.1) \times (0.1200) = -0.0220$$

Iteration 3 (current: w = 0.0525, b = -0.0220)

$$x = 1, y = 0: z = 0.0525 \times 1 + (-0.0220) = 0.0305 \rightarrow \hat{y} = 1/(1+e^{(-0.0305)}) = 0.5076 \rightarrow \text{loss} = -[0 \cdot \log(0.5076) + 1 \cdot \log(0.4924)] = 0.7085$$

$$x = 2, y = 0: z = 0.0525 \times 2 + (-0.0220) = 0.0830 \rightarrow \hat{y} = 1/(1+e^{(-0.0830)}) = 0.5207 \rightarrow \text{loss} = -[0 \cdot \log(0.5207) + 1 \cdot \log(0.4793)] = 0.7355$$

$$x = 3, y = 0: z = 0.0525 \times 3 + (-0.0220) = 0.1355 \rightarrow \hat{y} = 1/(1+e^{(-0.1355)}) = 0.5338 \rightarrow \text{loss} = -[0 \cdot \log(0.5338) + 1 \cdot \log(0.4662)] = 0.7632$$

$$x = 4, y = 1: z = 0.0525 \times 4 + (-0.0220) = 0.1880 \rightarrow \hat{y} = 1/(1+e^{(-0.1880)}) = 0.5469 \rightarrow \text{loss} = -[1 \cdot \log(0.5469) + 0 \cdot \log(0.4531)] = 0.6035$$

$$x = 5, y = 1: z = 0.0525 \times 5 + (-0.0220) = 0.2405 \rightarrow \hat{y} = 1/(1+e^{(-0.2405)}) = 0.5598 \rightarrow \text{loss} = -[1 \cdot \log(0.5598) + 0 \cdot \log(0.4402)] = 0.5801$$

$$\mathcal{J}_{\text{Loss}} = (0.7085 + 0.7355 + 0.7632 + 0.6035 + 0.5801) / 5 = 3.3909 / 5 = 0.6782$$

$$dw = [(0.5076-0) \times 1 + (0.5207-0) \times 2 + (0.5338-0) \times 3 + (0.5469-1) \times 4 + (0.5598-1) \times 5] / 5 = -0.1725$$

$$db = [(0.5076-0) + (0.5207-0) + (0.5338-0) + (0.5469-1) + (0.5598-1)] / 5 = 0.1338$$

$$w_new = 0.0525 - (0.1) \times (-0.1725) = 0.0698$$

$$b_new = -0.0220 - (0.1) \times (0.1338) = -0.0354$$

Iteration 4 (current: w = 0.0698, b = -0.0354)

$$x = 1, y = 0: z = 0.0698 \times 1 + (-0.0354) = 0.0344 \rightarrow \hat{y} = 1/(1+e^{(-0.0344)}) = 0.5086 \rightarrow \text{loss} = -[0 \cdot \log(0.5086) + 1 \cdot \log(0.4914)] = 0.7105$$

$$x = 2, y = 0: z = 0.0698 \times 2 + (-0.0354) = 0.1041 \rightarrow \hat{y} = 1/(1+e^{(-0.1041)}) = 0.5260 \rightarrow \text{loss} = -[0 \cdot \log(0.5260) + 1 \cdot \log(0.4740)] = 0.7466$$

$$x = 3, y = 0: z = 0.0698 \times 3 + (-0.0354) = 0.1739 \rightarrow \hat{y} = 1/(1+e^{(-0.1739)}) = 0.5434 \rightarrow \text{loss} = -[0 \cdot \log(0.5434) + 1 \cdot \log(0.4566)] = 0.7839$$

$$x = 4, y = 1: z = 0.0698 \times 4 + (-0.0354) = 0.2437 \rightarrow \hat{y} = 1/(1+e^{(-0.2437)}) = 0.5606 \rightarrow \text{loss} = -[1 \cdot \log(0.5606) + 0 \cdot \log(0.4394)] = 0.5787$$

$$x = 5, y = 1: z = 0.0698 \times 5 + (-0.0354) = 0.3134 \rightarrow \hat{y} = 1/(1+e^{(-0.3134)}) = 0.5777 \rightarrow \text{loss} = -[1 \cdot \log(0.5777) + 0 \cdot \log(0.4223)] = 0.5487$$

$$\overline{\text{Loss}} = (0.7105 + 0.7466 + 0.7839 + 0.5787 + 0.5487) / 5 = 3.3683 / 5 = 0.6737$$

$$dw = [(0.5086-0) \times 1 + (0.5260-0) \times 2 + (0.5434-0) \times 3 + (0.5606-1) \times 4 + (0.5777-1) \times 5] / 5 = -0.1356$$

$$db = [(0.5086-0) + (0.5260-0) + (0.5434-0) + (0.5606-1) + (0.5777-1)] / 5 = 0.1433$$

$$w_new = 0.0698 - (0.1) \times (-0.1356) = 0.0833$$

$$b_new = -0.0354 - (0.1) \times (0.1433) = -0.0497$$

Iteration 5 (current: w = 0.0833, b = -0.0497)

$$x = 1, y = 0: z = 0.0833 \times 1 + (-0.0497) = 0.0336 \rightarrow \hat{y} = 1/(1+e^{(-0.0336)}) = 0.5084 \rightarrow \text{loss} = -[0 \cdot \log(0.5084) + 1 \cdot \log(0.4916)] = 0.7101$$

$$x = 2, y = 0: z = 0.0833 \times 2 + (-0.0497) = 0.1169 \rightarrow \hat{y} = 1/(1+e^{(-0.1169)}) = 0.5292 \rightarrow \text{loss} = -[0 \cdot \log(0.5292) + 1 \cdot \log(0.4708)] = 0.7533$$

$$x = 3, y = 0: z = 0.0833 \times 3 + (-0.0497) = 0.2003 \rightarrow \hat{y} = 1/(1+e^{(-0.2003)}) = 0.5499 \rightarrow \text{loss} = -[0 \cdot \log(0.5499) + 1 \cdot \log(0.4501)] = 0.7983$$

$$x = 4, y = 1: z = 0.0833 \times 4 + (-0.0497) = 0.2836 \rightarrow \hat{y} = 1/(1+e^{(-0.2836)}) = 0.5704 \rightarrow \text{loss} = -[1 \cdot \log(0.5704) + 0 \cdot \log(0.4296)] = 0.5614$$

$$x = 5, y = 1: z = 0.0833 \times 5 + (-0.0497) = 0.3669 \rightarrow \hat{y} = 1/(1+e^{(-0.3669)}) = 0.5907 \rightarrow \text{loss} = -[1 \cdot \log(0.5907) + 0 \cdot \log(0.4093)] = 0.5264$$

$$\text{Average Loss} = (0.7101 + 0.7533 + 0.7983 + 0.5614 + 0.5264) / 5 = 3.3495 / 5 = 0.6699$$

$$dw = [(0.5084-0) \times 1 + (0.5292-0) \times 2 + (0.5499-0) \times 3 + (0.5704-1) \times 4 + (0.5907-1) \times 5] / 5 = -0.1096$$

$$db = [(0.5084-0) + (0.5292-0) + (0.5499-0) + (0.5704-1) + (0.5907-1)] / 5 = 0.1497$$

$$w_{\text{new}} = 0.0833 - (0.1) \times (-0.1096) = 0.0943$$

$$b_{\text{new}} = -0.0497 - (0.1) \times (0.1497) = -0.0647$$

Training Progress Table

Iteration	w	b	Loss
1	0.0000	0.0000	0.6931
2	0.0300	-0.0100	0.6842
3	0.0525	-0.0220	0.6782
4	0.0698	-0.0354	0.6737
5	0.0833	-0.0497	0.6699

Final Learned Model

৫টি Iteration-এর পর প্যারামিটারের বর্তমান মান: $w = 0.0943$, $b = -0.0647$ । বাস্তব Training-এ এই প্রক্রিয়া শত শত Iteration ধরে চলে যতক্ষণ না Loss ন্যূনতম মানে (Convergence) পৌঁছায়; এখানে পদ্ধতিটি হাতে-কলমে দেখানোর জন্য মাত্র ৫টি Iteration দেখানো হয়েছে।

$$\text{Decision Boundary: } 0.0943 \cdot X + (-0.0647) = 0$$

Prediction on a New Sample

ধরা যাক একজন নতুন শিক্ষার্থী $X = 3.5$ ঘণ্টা পড়াশোনা করেছে।

$$z = 0.0943 \times 3.5 + (-0.0647) = 0.2653$$

$$\hat{y} = 1/(1+e^{(-0.2653)}) = 0.5659$$

যেহেতু $\hat{y} = 0.5659 \geq 0.5$, তাই প্রেডিক্টেড ক্লাস = 1 (Pass)।

Interpretation

প্রতিটি Iteration-এ dw ঋণাত্মক (negative) থাকায় w ধীরে ধীরে বৃদ্ধি পাচ্ছে এবং db ধনাত্মক থাকায় b ধীরে ধীরে হ্রাস পাচ্ছে — এর অর্থ মডেল ধীরে ধীরে শিখছে যে বেশি Study Hours থাকলে Pass হওয়ার Probability বাড়া উচিত। Loss প্রতি Iteration-এ কমছে ($0.6931 \rightarrow 0.6699$), যা নিশ্চিত করে যে Gradient Descent সঠিক দিকে এগোচ্ছে। আরও বেশি Iteration চালালে w আরও বড় হবে এবং Decision Boundary আরও তীক্ষ্ণ (sharp) হয়ে উঠবে।

K-Nearest Neighbors (KNN)

১. ভূমিকা (Introduction)

1. Introduction to K-Nearest Neighbors

K-Nearest Neighbors (KNN) হলো Machine Learning-এর একটি জনপ্রিয় Supervised Learning Algorithm, যা Classification এবং Regression উভয় ধরনের সমস্যার সমাধান করতে ব্যবহৃত হয়। এটি একটি Non-Parametric এবং Lazy Learning Algorithm। অন্যান্য অনেক Machine Learning Algorithm-এর মতো KNN Training Phase-এ কোনো Mathematical Model তৈরি করে না। বরং সম্পূর্ণ Training Dataset মেমোরিতে সংরক্ষণ করে রাখে এবং Prediction করার সময় নতুন Data Point-এর সবচেয়ে কাছের প্রতিবেশীদের (Neighbors) খুঁজে বের করে সিদ্ধান্ত নেয়।

KNN-এর মূল ধারণা হলো:

Similar data points tend to exist close to each other in feature space.

অর্থাৎ, যেসব Data Point-এর বৈশিষ্ট্য একরকম, তারা সাধারণত Feature Space-এ কাছাকাছি অবস্থান করে।

২. কেন এই অ্যালগরিদম তৈরি হয়েছিল (Why This Algorithm Was Developed)

KNN মেশিন লার্নিং-এর সবচেয়ে পুরনো অ্যালগরিদমগুলোর একটি — এটি প্রথম প্রস্তাব করেন Evelyn Fix এবং Joseph Hodges ১৯৫১ সালে, এবং পরবর্তীতে Thomas Cover এটি আরও উন্নত করেন। সেই সময় বেশিরভাগ Statistical Model ডেটা সম্পর্কে কঠোর Assumption (যেমন Normal Distribution) করত, যা বাস্তব ডেটাতে প্রায়ই সত্য হতো না। KNN তৈরি হয়েছিল এমন একটি পদ্ধতি হিসেবে যা কোনো পূর্ব-অনুমান ছাড়াই শুধুমাত্র "কাছাকাছি অবস্থিত ডেটা পয়েন্টগুলো একই রকম আচরণ করে" — এই সাধারণ প্রাকৃতিক নীতির উপর ভিত্তি করে কাজ করে।

৩. বাস্তব জীবনের প্রেরণা (Real-life Motivation)

ধরা যাক আপনি একটি নতুন শহরে থাকতে এসেছেন এবং জানতে চান আপনার এলাকাটি নিরাপদ কি না। স্বাভাবিক প্রবৃত্তি হলো আশেপাশের প্রতিবেশীদের জিজ্ঞেস করা — যদি বেশিরভাগ প্রতিবেশী বলেন এলাকা নিরাপদ, আপনি সেটাই বিশ্বাস করবেন। KNN ঠিক এই স্বজ্ঞাটাই অনুসরণ করে — Netflix বা Spotify-এর মতো প্ল্যাটফর্ম যখন "Similar User"-দের পছন্দ দেখে আপনাকে Recommendation দেয়, তখন মূলত এটি KNN-এরই একটি রূপ ব্যবহার করে।

2. Why Do We Need KNN?

বাস্তব জীবনে অনেক সমস্যা আছে যেখানে আমাদের একটি নতুন Observation-এর Category বা Value নির্ধারণ করতে হয়।

উদাহরণস্বরূপ:

- Email Spam নাকি Not Spam?
- Customer Product কিনবে নাকি কিনবে না?
- একজন রোগী রোগে আক্রান্ত নাকি সুস্থ?
- একটি বাড়ির সম্ভাব্য মূল্য কত?

এই ধরনের সমস্যার সমাধানে KNN ব্যবহার করা যায়।

KNN-এর বিশেষত্ব হলো এটি Data Distribution সম্পর্কে আগে থেকে কোনো Assumption করে না। ফলে জটিল Non-Linear Data-এর ক্ষেত্রেও এটি অনেক সময় ভালো কাজ করে।

3. Types of Problems Solved by KNN

Classification

Classification সমস্যায় Target Variable একটি Category হয়।

উদাহরণ:

Input	Output
Patient Data	Disease / No Disease
Email	Spam / Not Spam
Student Info	Pass / Fail

Classification-এর ক্ষেত্রে KNN Majority Voting ব্যবহার করে।

Regression

Regression সমস্যায় Target Variable Continuous Numeric Value হয়।

উদাহরণ:

Input	Output
House Features	House Price
Weather Data	Temperature
Employee Data	Salary

Regression-এর ক্ষেত্রে KNN Neighbors-এর Mean বা Median ব্যবহার করে।

৪. স্বভাৱ (Intuition)

KNN-এৰ পেছনৈৰ স্বভাৱ অত্যন্ত সহজ: কোনো নতুন ডেটা পয়েণ্টেৰ ক্লাস জানতে হলে, তৰ সবচেয়ে কাছৰে প্ৰতিবেশীদেৰ দিকে তাকাও এবং তাৰে সংখ্যাগৰিষ্ঠ মতামত অনুসৰণ কৰো। এখানে কোনো জটিল Training প্ৰক্ৰিয়া নাই — মডেল কেবল সম্পূৰ্ণ Training Dataset মনে ৰাখে (Memorize কৰে), এবং Prediction-এৰ সময় সেই Memory থেকে দূৰত্ব হিচাব কৰে সিদ্ধান্ত নেয়।

৫. কাৰ্যপ্ৰণালী (Working Principle)

৪. Working Principle of KNN

KNN-এৰ সম্পূৰ্ণ কাজ Distance-এৰ উপৰ নিৰ্ভৰশীল।

ধৰা যাক এটি নতুন Data Point এসেছে।

KNN নিচৈৰ ধাপগুলো অনুসৰণ কৰে:

Step 1

এটি K Value নিৰ্বাচন কৰা হয়।

$$K = 3$$

Step 2

নতুন Data Point থেকে Training Dataset-এৰ প্ৰতিটি Observation-এৰ Distance বৈৰ কৰা হয়।

Step 3

Distance অনুযায়ী ছোট থেকে বড় ক্ৰমে Sort কৰা হয়।

Step 4

সবচেয়ে কাছৰে K সংখ্যক Neighbor নিৰ্বাচন কৰা হয়।

Step 5

Classification হলে Majority Voting এবং Regression হলে Average নেওয়া হয়।

৬. গাণিতিক ভিত্তি (Mathematical Backend)

৭. সূত্ৰ (Formula)

৫. Mathematical Foundation of KNN

KNN-এৰ সম্পূৰ্ণ Decision Making Distance Metrics-এৰ উপৰ ভিত্তি কৰে তৈরি।

দুটি Data Point-এর মধ্যকার Distance যত কম হবে, তারা তত বেশি Similar বলে বিবেচিত হবে।

5.1 Euclidean Distance

এটি KNN-এর সবচেয়ে ব্যবহৃত Distance Metric।

জ্যামিতিতে দুটি বিন্দুর সরলরেখার দূরত্ব নির্ণয়ের জন্য এটি ব্যবহার করা হয়।

যদি দুটি Point হয়:

$$P = (x_1, y_1)$$

এবং

$$Q = (x_2, y_2)$$

তাহলে তাদের Euclidean Distance:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

N-Dimensional Form

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

এই Formula-ই Scikit-Learn-এর KNN-এর Default Distance Metric।

5.2 Manhattan Distance

Euclidean Distance সরলরেখার দূরত্ব মাপে।

কিন্তু Manhattan Distance Grid Path ধরে Distance মাপে।

Formula:

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

High Dimensional Dataset-এর ক্ষেত্রে অনেক সময় Manhattan Distance Euclidean Distance-এর চেয়ে ভালো ফলাফল দেয়।

৮. সূত্রের ডেরিভেশন (Formula Derivation)

লক্ষ্য করুন, Euclidean ও Manhattan Distance আসলাদা আলাদা সূত্র মনে হলেও, উভয়ই একটি অভিন্ন সাধারণ সূত্র — Minkowski Distance-এর বিশেষ রূপ মাত্র। নিচে দেখানো হলো কীভাবে একটি একক প্যারামিটার p পরিবর্তন করে Minkowski Distance থেকে Manhattan ($p=1$) এবং Euclidean ($p=2$) Distance বের করা যায়:

5.3 Minkowski Distance

Euclidean এবং Manhattan Distance আসলে Minkowski Distance-এর Special Case।

$$d(x, y) = (\sum_{i=1}^n |x_i - y_i|^p)^{1/p}$$

যেখানে,

$$p = 1$$

হলে Manhattan Distance

এবং

$$p = 2$$

হলে Euclidean Distance পাওয়া যায়।

৯. প্রতিটি চিহ্নের অর্থ (Meaning of Every Symbol)

Symbol	Meaning (অর্থ)
A, B	দুইটি Data Point (একটি নতুন স্যাম্পল, একটি Training স্যাম্পল)
$A_1, A_2 \dots A_n$	Point A-এর প্রতিটি Feature-এর মান
$B_1, B_2 \dots B_n$	Point B-এর প্রতিটি Feature-এর মান
$d(A, B)$	A ও B-এর মধ্যে দূরত্ব
k	কতটি নিকটতম প্রতিবেশী বিবেচনা করা হবে
p	Minkowski Distance-এর Power Parameter

১০. সূত্রের ধাপে-ধাপে ব্যাখ্যা (Step-by-Step Formula Explanation)

১১. সংখ্যাগত উদাহরণ (Numerical Example)

6. Complete Numerical Example

ধরা যাক নিচের Dataset দেওয়া আছে।

Customer	Age	Income	Buy
C1	2	4	No

C2	4	6	Yes
C3	4	2	No
C4	6	8	Yes

এখন নতুন Customer:

(3, 5)

এবং

$K = 3$

প্রথম Customer-এর সাথে Distance:

$$d_1 = \sqrt{(3 - 2)^2 + (5 - 4)^2} = 1.41$$

দ্বিতীয় Customer-এর সাথে:

$$d_2 = 1.41$$

তৃতীয় Customer-এর সাথে:

$$d_3 = 3.16$$

চতুর্থ Customer-এর সাথে:

$$d_4 = 4.24$$

Distance অনুযায়ী Sort করলে:

Customer	Distance	Class
C1	1.41	No
C2	1.41	Yes
C3	3.16	No
C4	4.24	Yes

Nearest 3 Neighbor:

- No
- Yes
- No

Majority Vote:

$No = 2$

$Yes = 1$

Final Prediction:

No

১২. ভিজ্যুয়ালাইজেশন (Visualization)

KNN-কে কল্পনা করার সবচেয়ে সহজ উপায় হলো একটি 2D Scatter Plot, যেখানে প্রতিটি Training স্যাম্পল একটি রঙিন বিন্দু (ক্লাস অনুযায়ী রঙ ভিন্ন) হিসেবে দেখানো হয়। নতুন স্যাম্পলটিকে একটি ভিন্ন চিহ্ন (যেমন ★) দিয়ে বসানো হয়, এবং তার চারপাশে একটি কাল্পনিক বৃত্ত আঁকা যায় যার ভেতরে ঠিক k -টি নিকটতম প্রতিবেশী পড়ে। k ছোট হলে এই বৃত্ত ছোট ও Decision Boundary অসমান (jagged) হয়; k বড় হলে বৃত্ত বড় হয় এবং Boundary মসৃণ (smooth) হয়ে যায় — এভাবে Distance Metric Comparison চার্টেও (উপরে দেখানো হয়েছে) বিভিন্ন Metric-এর প্রভাব ভিজ্যুয়ালি বোঝা যায়।

7. Choosing the Optimal K Value

KNN-এর Performance-এর সবচেয়ে গুরুত্বপূর্ণ অংশ হলো K নির্বাচন করা।

K খুব ছোট হলে

$$K = 1$$

Model Noise Follow করতে শুরু করে।

ফলাফল:

- High Variance
- Overfitting

K খুব বড় হলে

$$K = 100$$

Model Local Pattern ভুলে যায়।

ফলাফল:

- High Bias

- Underfitting

সাধারণভাবে:

$$K = \sqrt{N}$$

দিয়ে শুরু করা হয়।

যেখানে N হলো Training Sample-এর সংখ্যা।

8. Feature Scaling in KNN

KNN Distance-Based Algorithm হওয়ার কারণে Feature Scaling অত্যন্ত গুরুত্বপূর্ণ।

ধরা যাক:

Feature	Range
Age	0-100
Salary	0-1,000,000

Distance Calculation-এর সময় Salary পুরো Distance-কে Dominant করে ফেলবে।

ফলে Age-এর Effect প্রায় হারিয়ে যাবে।

এই সমস্যার সমাধানের জন্য:

Standardization

$$z = \frac{x - \mu}{\sigma}$$

Min-Max Scaling

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

প্রয়োগ করা হয়।

9. Curse of Dimensionality

KNN-এর সবচেয়ে বড় সীমাবদ্ধতাগুলোর একটি হলো Curse of Dimensionality।

Feature সংখ্যা বাড়তে থাকলে Data Point-গুলোর মধ্যে Distance-এর পার্থক্য কমতে শুরু করে।

ফলে:

- Nearest Neighbor খুঁজে পাওয়া কঠিন হয়
- Prediction Accuracy কমে যায়
- Computation Cost বৃদ্ধি পায়

এই কারণে High-Dimensional Dataset-এ KNN অনেক সময় দুর্বল পারফর্ম করে।

10. Computational Complexity

Operation	Complexity
Training	$O(1)$
Memory	$O(N)$
Prediction	$O(ND)$

যেখানে,

- N = Number of Samples
- D = Number of Features

এই কারণেই KNN-এর Training Fast হলেও Prediction তুলনামূলক Slow হয়।

১৪. স্কিট-লার্ন ইম্পোর্ট (Scikit-learn Import)

১৫. পাইথন উদাহরণ — From-Scratch Implementation (Python Example)

নিচে সম্পূর্ণ From-Scratch (NumPy দিয়ে হাতে-লেখা) KNN দেখানো হলো, যা এই অধ্যায়ের ম্যানুয়াল দূরত্ব-গণনার সাথে হুবহু মেলে:

```
import numpy as np
from collections import Counter

class KNNClassifierScratch:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = np.array(X)
        self.y_train = np.array(y)

    def _euclidean(self, a, b):
        return np.sqrt(np.sum((a - b) ** 2))

    def predict_one(self, x):
```

```

        distances = [self._euclidean(x, xt) for xt in self.X_train]
        k_idx = np.argsort(distances)[:self.k]
        k_labels = [self.y_train[i] for i in k_idx]
        return Counter(k_labels).most_common(1)[0][0]

def predict(self, X):
    return [self.predict_one(np.array(x)) for x in X]

# Example usage
X_train = [[2,9],[3,7],[5,6],[7,5],[8,3]]
y_train = ['Fail','Fail','Pass','Pass','Pass']
model = KNNClassifierScratch(k=3)
model.fit(X_train, y_train)
print(model.predict([[4,6]]))    # -> ['Pass']

```

১৬. প্রেডিকশন প্রসেস (Prediction Process)

KNN-এ কোনো আলাদা "Training Phase" নেই — Training মানে শুধু পুরো Dataset মেমোরিতে সংরক্ষণ করা। প্রকৃত কাজ হয় Prediction-এর সময়: (১) নতুন স্যাম্পল থেকে প্রতিটি Training পয়েন্টের দূরত্ব গণনা, (২) দূরত্ব অনুযায়ী Sort করা, (৩) সবচেয়ে কাছের k-টি প্রতিবেশী বেছে নেওয়া, এবং (৪) Majority Voting-এর মাধ্যমে চূড়ান্ত ক্লাস নির্ধারণ করা। এই কারণেই KNN-কে "Lazy Learner" বলা হয় — সব গণনা Prediction Time-এ হয়, Training Time-এ নয়।

১৭. হাইপারপ্যারামিটার (Hyperparameters)

KNN ব্যবহার করার সময় নিচের Parameters সবচেয়ে গুরুত্বপূর্ণ।

```

KNeighborsClassifier(
    n_neighbors=5,
    weights='distance',
    metric='minkowski',
    p=2,
    algorithm='auto'
)

```

```

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

wine = load_wine()

X = wine.data
y = wine.target

print("Shape of X:", X.shape)
print("Shape of y:", y.shape)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

```

```

knn_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier(
        n_neighbors=5,
        weights="uniform",
        metric="minkowski",
        p=2
    ))
])

knn_pipeline.fit(X_train, y_train)
y_pred = knn_pipeline.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:")
print(accuracy)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

```

Distance Metric Comparison

Distance Metric	Formula (সংক্ষেপে)	ব্যবহারের প্রসঙ্গ
Euclidean (p=2)	$\sqrt{\sum (x_i - y_i)^2}$	সাধারণ, continuous low-dimensional data-তে default
Manhattan (p=1)	$\sum x_i - y_i $	High-dimensional বা grid-like data-তে কার্যকর
Minkowski (general)	$(\sum x_i - y_i ^p)^{1/p}$	p tune করে Euclidean/Manhattan-এর মধ্যে নমনীয়তা পাওয়া যায়

EXAM TIP

K নির্বাচনের rule of thumb $K \approx \sqrt{N}$ মুখস্থ রাখুন এবং Manual Distance Calculation (Euclidean) করে Majority Voting দেখানোর Numerical Example পরীক্ষায় প্রায়ই আসে।

COMMON MISTAKE

K = 1 ব্যবহার করে Model-কে অত্যন্ত Noise-sensitive করে ফেলা — এটি Training Accuracy-তে চমৎকার মনে হলেও Test Set-এ খারাপ Generalize করে।

Even K ব্যবহার করা, যা Binary Classification-এ Tie (সমান ভোট) তৈরি করতে পারে — সাধারণত Odd K ব্যবহার করাই নিরাপদ।

১৭-১৯. সুবিধা, অসুবিধা ও প্রয়োগক্ষেত্র (Advantages, Disadvantages & Applications)

Advantages ও Disadvantages

Advantages	Disadvantages
বোঝা এবং বাস্তবায়ন করা সহজ	Prediction Phase ধীরগতির (Computationally Expensive)
Data Distribution সম্পর্কে কোনো Assumption করে না	Curse of Dimensionality-তে আক্রান্ত হয়
Classification ও Regression উভয় ক্ষেত্রে ব্যবহারযোগ্য	Feature Scaling বাধ্যতামূলক
Multi-class সমস্যায় স্বাভাবিকভাবেই কাজ করে	Memory-ভিত্তিক (পুরো ডেটাসেট সংরক্ষণ করতে হয়)
REAL-WORLD EXAMPLE	

Recommendation System-এ Similar Users/Items খুঁজে বের করা, Image Recognition-এ Pattern Matching, এবং Medical Diagnosis-এ Similar Patient History অনুসন্ধান — এসব ক্ষেত্রে KNN ব্যাপকভাবে ব্যবহৃত হয়।

INTERVIEW TIP

Q: KNN-কে 'Lazy Learner' বলা হয় কেন?

A: কারণ এটি Training Phase-এ কোনো Model তৈরি করে না, শুধু ডেটা সংরক্ষণ করে রাখে; প্রকৃত Computation ঘটে শুধু Prediction Phase-এ।

Q: KNN-এ Feature Scaling কেন গুরুত্বপূর্ণ?

A: কারণ KNN সম্পূর্ণভাবে Distance-এর উপর নির্ভরশীল; বড় Scale-এর Feature ছোট Scale-এর Feature-কে Dominate করে ফেলে।

K-Nearest Neighbors (KNN) হলো একটি **Instance-Based** এবং **Lazy Learning Algorithm**। অন্যান্য Machine Learning Algorithm-এর মতো এটি Training-এর সময় কোনো Mathematical Model তৈরি করে না। বরং Training Data-কে Memory-তে সংরক্ষণ করে রাখে এবং নতুন Data আসলে তার কাছাকাছি থাকা Data Point-গুলোর Class দেখে Prediction করে।

ধরুন একটি নতুন Student-এর তথ্য দেওয়া হলো এবং জানতে হবে সে Pass করবে নাকি Fail। KNN প্রথমে Training Data-তে সবচেয়ে কাছের K জন Student খুঁজে বের করবে, তারপর তাদের Majority Vote-এর ভিত্তিতে Final Prediction দেবে।

এই কারণেই KNN-কে অনেক সময় "Similarity Based Learning" Algorithm বলা হয়।

n_neighbors

Default: 5

n_neighbors বা K হলো KNN-এর সবচেয়ে গুরুত্বপূর্ণ Hyperparameter। এটি নির্ধারণ করে Prediction করার সময় কতগুলো Nearest Neighbor বিবেচনা করা হবে।

ধরুন,

$$K = 5$$

তাহলে নতুন Data Point-এর সবচেয়ে কাছের 5টি Neighbor খুঁজে বের করা হবে এবং তাদের Vote গণনা করা হবে।

যদি K খুব ছোট হয়, যেমন:

`n_neighbors=1`

তাহলে Model Training Data-এর Noise পর্যন্ত শিখে ফেলতে পারে। ফলে Overfitting দেখা দেয়।

অন্যদিকে K খুব বড় হলে অনেক দূরের Data Point-ও Decision-এ অংশ নেয়। তখন Model অত্যন্ত Smooth হয়ে যায় এবং Underfitting হতে পারে।

সাধারণভাবে:

- Small Dataset → $K = 3$ বা 5
- Medium Dataset → $K = 5$ বা 7
- Large Dataset → $K = 11, 15, 21$ ইত্যাদি

ব্যবহার করা হয়।

KNN-এর Performance-এর উপর সবচেয়ে বেশি প্রভাব ফেলে `n_neighbors`।

weights

Default: 'uniform'

KNN-এ Neighbor-এর Vote কীভাবে গণনা করা হবে তা weights Parameter নিয়ন্ত্রণ করে।

Default Mode:

`weights='uniform'`

এখানে প্রতিটি Neighbor-এর Vote সমান গুরুত্ব পায়।

ধরুন $K=5$ ।

তাহলে ৫ জন Neighbor-এর প্রত্যেকের Vote-এর Weight হবে সমান।

অন্য Option:

weights='distance'

এখানে কাছের Neighbor-এর Vote বেশি গুরুত্বপূর্ণ হয়।

Weight গণনা হয়:

$$Weight = \frac{1}{Distance}$$

ফলে খুব কাছের Neighbor Prediction-এ বেশি প্রভাব ফেলে এবং দূরের Neighbor-এর প্রভাব কমে যায়।

যখন Dataset-এর Density সমান নয় অথবা Local Pattern বেশি গুরুত্বপূর্ণ, তখন distance সাধারণত ভালো Result দেয়।

metric

Default: 'minkowski'

KNN-এর মূল ভিত্তি হলো Distance Calculation।

কোন Formula ব্যবহার করে Distance বের করা হবে, সেটি metric Parameter নির্ধারণ করে।

ভুল Distance Metric নির্বাচন করলে "Nearest Neighbor" ভুলভাবে নির্ধারিত হতে পারে এবং Model-এর Accuracy কমে যেতে পারে।

Euclidean Distance

metric='euclidean'

Formula:

$$d = \sqrt{\sum (x_i - y_i)^2}$$

এটি Straight-Line Distance।

Continuous Numerical Data-এর জন্য এটি সবচেয়ে বেশি ব্যবহৃত Metric।

Manhattan Distance

metric='manhattan'

Formula:

$$d = \sum |x_i - y_i|$$

এটি Grid-Based Distance।

যদি Movement শুধুমাত্র Horizontal এবং Vertical Direction-এ সম্ভব হয়, তাহলে Manhattan Distance বেশি উপযুক্ত।

High-Dimensional Dataset-এ অনেক সময় এটি Euclidean-এর চেয়ে ভালো কাজ করে।

Minkowski Distance

metric='minkowski'

Formula:

$$d = (\sum |x_i - y_i|^p)^{1/p}$$

এটি Euclidean এবং Manhattan-এর General Form।

Scikit-Learn Default হিসেবে এটিই ব্যবহার করে।

Chebyshev Distance

metric='chebyshev'

Formula:

$$d = \max(|x_i - y_i|)$$

এখানে শুধুমাত্র Maximum Difference বিবেচনা করা হয়।

Cosine Distance

`metric='cosine'`

Text Mining, NLP এবং Document Similarity Problem-এ বেশি ব্যবহৃত হয়।

এটি Distance-এর পরিবর্তে Vector-এর Angle Measure করে।

p

Default: 2

p শুধুমাত্র তখন কাজ করে যখন:

`metric='minkowski'`

হয়।

এটি Minkowski Distance-এর Power Parameter।

যদি

`p=1`

হয়, তাহলে Minkowski Distance Manhattan Distance-এ রূপান্তরিত হয়।

$$d = \sum |x_i - y_i|$$

যদি

`p=2`

হয়, তাহলে এটি Euclidean Distance হয়ে যায়।

$$d = \sqrt{\sum (x_i - y_i)^2}$$

এই কারণে অনেক সময় আলাদা করে `metric='euclidean'` না লিখেও `metric='minkowski', p=2` ব্যবহার করা হয়।

algorithm

Default: 'auto'

KNN Prediction-এর সময় Nearest Neighbor Search করার জন্য কোন Internal Search Algorithm ব্যবহার হবে তা নির্ধারণ করে।

এটি Accuracy পরিবর্তন করে না।

শুধুমাত্র Computation Speed পরিবর্তন করে।

Auto

algorithm='auto'

Scikit-Learn Dataset দেখে নিজেই Best Algorithm নির্বাচন করে।

বেশিরভাগ ক্ষেত্রে এটিই যথেষ্ট।

Brute Force

algorithm='brute'

প্রতিটি Data Point-এর সাথে Distance Calculate করে।

Small Dataset-এর জন্য ভালো।

KD Tree

algorithm='kd_tree'

কম Dimension-এর Dataset-এ দ্রুত Search করতে পারে।

সাধারণত:

$$\text{Dimensions} < 20$$

হলে কার্যকর।

Ball Tree

algorithm='ball_tree'

High-Dimensional Dataset-এ KD Tree-এর চেয়ে ভালো কাজ করতে পারে।

leaf_size

Default: 30

KD Tree এবং Ball Tree-এর Leaf Node-এর Size নির্ধারণ করে।

এটি শুধুমাত্র Performance Optimization-এর জন্য ব্যবহৃত হয়।

ছোট Leaf Size:

- Tree গভীর হয়
- Memory বেশি লাগে
- Query দ্রুত হতে পারে

বড় Leaf Size:

- Tree কম গভীর হয়
- Memory কম লাগে
- Query ধীর হতে পারে

সাধারণ Dataset-এ Default Value পরিবর্তনের প্রয়োজন হয় না।

n_jobs

Default: None

Prediction বা Neighbor Search-এর সময় কতগুলো CPU Core ব্যবহার হবে তা নির্ধারণ করে।

n_jobs=-1

দিলে System-এর সব CPU Core ব্যবহার করা হয়।

এটি Accuracy পরিবর্তন করে না।

শুধুমাত্র Computation Time কমায়।

বিশেষ করে Large Dataset-এ এটি উপকারী।

KNN-এর আরও গুরুত্বপূর্ণ Parameter

অনেক Note-এ এগুলো বাদ যায়, কিন্তু বাস্তবে এগুলোও গুরুত্বপূর্ণ।

radius

Radius Neighbors Algorithm-এর ক্ষেত্রে ব্যবহৃত হয়।

radius=1.0

একটি নির্দিষ্ট Radius-এর ভেতরের Neighbor-গুলো বিবেচনা করা হয়।

তবে সাধারণ KNeighborsClassifier-এ এটি খুব বেশি ব্যবহৃত হয় না।

metric_params

Distance Metric-এর জন্য অতিরিক্ত Parameter পাঠাতে ব্যবহৃত হয়।

যখন Custom Distance Function ব্যবহার করা হয়, তখন এটি কাজে লাগে।

outlier_label

Radius-Based Neighbor Model-এ Outlier Handle করার জন্য ব্যবহৃত হয়।

effective_metric_

Training-এর পরে Model আসলে কোন Metric ব্যবহার করেছে তা জানায়।

effective_metric_params_

Metric-এর Final Parameter Value দেখায়।

KNeighborsClassifier() Attributes

classes_

Default: Created after fit()

Training Data-তে থাকা সকল Target Class-এর List সংরক্ষণ করে। Prediction করার সময় Model এই Class-গুলোর মধ্য থেকেই একটি Class নির্বাচন করে।

n_samples_fit_

Default: Created after fit()

Training-এর সময় মোট কতটি Sample ব্যবহার করা হয়েছে তা সংরক্ষণ করে। KNN পুরো Training Data-ই সংরক্ষণ করে, তাই এটি গুরুত্বপূর্ণ Attribute।

effective_metric_

Default: Created after fit()

Training শেষে আসলে কোন Distance Metric ব্যবহৃত হয়েছে তা সংরক্ষণ করে। যেমন euclidean, manhattan, minkowski ইত্যাদি।

effective_metric_params_

Default: Created after fit()

ব্যবহৃত Distance Metric-এর Parameter-গুলো সংরক্ষণ করে। উদাহরণস্বরূপ, metric='minkowski' এবং p=2 হলে এখানে সেই Parameter থাকবে।

outputs_2d_

Default: Created after fit()

Target Variable Single Output নাকি Multi-output তা নির্দেশ করে। False হলে Single Output এবং True হলে Multi-output Classification বোঝায়।

n_features_in_

Default: Created after fit()

Training-এর সময় Model মোট কতটি Input Feature ব্যবহার করেছে তা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

KNeighborsRegressor() Attributes

n_samples_fit_

Default: Created after fit()

Training-এর সময় মোট কতটি Sample ব্যবহার করা হয়েছে তা সংরক্ষণ করে। Prediction করার সময় এই Sample-গুলোর মধ্য থেকেই Nearest Neighbor খুঁজে বের করা হয়।

effective_metric_

Default: Created after fit()

Training শেষে ব্যবহৃত প্রকৃত Distance Metric সংরক্ষণ করে।

effective_metric_params_

Default: Created after fit()

Distance Metric-এর Parameter-গুলো সংরক্ষণ করে। যেমন $p=1$, $p=2$ ইত্যাদি।

outputs_2d_

Default: Created after fit()

Target Variable Single Output নাকি Multi-output Regression তা নির্দেশ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় ব্যবহৃত মোট Input Feature-এর সংখ্যা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

Quick Revision Notes — KNN

Category	Key Points
Key Formula	Euclidean: $\sqrt{\sum (x_i - y_i)^2}$; Minkowski: $(\sum x_i - y_i ^p)^{1/p}$
Type	Non-parametric, Lazy Learner, Instance-based
Complexity	Training $O(1)$, Prediction $O(N \cdot D)$
K too small	Overfitting (High Variance)
K too large	Underfitting (High Bias)
Must-do	Feature scaling before fitting
Exam Focus	Manual distance computation & majority voting
Interview Focus	Curse of dimensionality; lazy vs eager learning

২০. পূর্ববর্তী অ্যালগরিদমের সাথে তুলনা (Comparison with Previous Algorithm)

Logistic Regression-এর সাথে KNN-এর তুলনা করলে দুটি সম্পূর্ণ ভিন্ন দর্শন দেখা যায়:

দিক (Aspect)	Logistic Regression	KNN
ধরন	Parametric (নির্দিষ্ট সংখ্যক Parameter শেখে)	Non-Parametric (কোনো Fixed Parameter নেই)
Training	Gradient Descent দিয়ে w, b শেখে	কোনো প্রকৃত Training নেই, শুধু ডেটা সংরক্ষণ
Decision Boundary	সবসময় Linear	যেকোনো জটিল আকৃতির (Non-Linear) হতে পারে
Prediction Speed	দ্রুত (শুধু $w \cdot x + b$ গণনা)	ধীর (প্রতিটি Training পয়েন্টের সাথে দূরত্ব লাগে)
Feature Scaling	সাধারণত কম গুরুত্বপূর্ণ	অত্যন্ত গুরুত্বপূর্ণ (Distance-Based)

২১. সাধারণ ভুলত্রুটি (Common Mistakes)

- Feature Scaling না করা — Salary-এর মতো বড় Scale-এর ফিচার Distance-কে সম্পূর্ণ Dominate করে ফেলে।
- খুব ছোট k (যেমন $k=1$) ব্যবহার করা — এতে Noise-এর প্রতি অতি সংবেদনশীলতা (Overfitting) দেখা দেয়।
- খুব বড় k ব্যবহার করা — এতে Local Pattern হারিয়ে যায় এবং Model Underfit করে।
- Irrelevant বা Redundant Feature বাদ না দেওয়া — Curse of Dimensionality-এর কারণে Distance Calculation অর্থহীন হয়ে যেতে পারে।
- Large Dataset-এ KNN ব্যবহার করে ধীরগতির Prediction নিয়ে অভিযোগ করা — এক্ষেত্রে KD-Tree বা Ball-Tree Algorithm ব্যবহার করা উচিত।

২২. সাক্ষাৎকারের প্রশ্ন (Interview Questions)

Q: KNN-কে কেন "Lazy Learner" বলা হয়?

A: কারণ এটি Training Time-এ কোনো প্রকৃত শেখার (Model Fitting) কাজ করে না — শুধু ডেটা সংরক্ষণ করে। সমস্ত হিসাব-নিকাশ Prediction Time-এ ঘটে।

Q: k-এর মান খুব ছোট বা খুব বড় হলে কী সমস্যা হয়?

A: খুব ছোট k (Overfitting/High Variance) মডেলকে Noise-এর প্রতি সংবেদনশীল করে তোলে, আর খুব বড় k (Underfitting/High Bias) Local Pattern ভুলিয়ে দেয়।

Q: KNN-এ Feature Scaling কেন এত গুরুত্বপূর্ণ?

A: কারণ KNN সরাসরি Distance-এর উপর নির্ভর করে; বড় Scale-এর ফিচার (যেমন Salary) ছোট Scale-এর ফিচারকে (যেমন Age) সম্পূর্ণ চাপা দিয়ে ফেলতে পারে যদি Scaling না করা হয়।

Q: Euclidean ও Manhattan Distance-এর মধ্যে পার্থক্য কী?

A: Euclidean Distance সরলরেখায় দূরত্ব মাপে, আর Manhattan Distance Grid-ভিত্তিক (শুধু Horizontal ও Vertical) দূরত্ব মাপে। উভয়ই আসলে Minkowski Distance-এর বিশেষ রূপ।

২৩. সারাংশ (Summary)

KNN একটি সহজবোধ্য, Non-Parametric Algorithm যা কোনো প্রকৃত Training ছাড়াই শুধু Distance Calculation ও Majority Voting-এর মাধ্যমে কাজ করে। এটি Non-Linear প্যাটার্নেও ভালো কাজ করে, কিন্তু Large Dataset-এ ধীরগতির এবং Feature Scaling ছাড়া ভুল ফলাফল দিতে পারে। k-এর সঠিক মান নির্বাচন করাই KNN-এর সবচেয়ে গুরুত্বপূর্ণ চ্যালেঞ্জ — এই সমস্যার সমাধান হিসেবে পরবর্তী অধ্যায়ে GridSearchCV ব্যবহার করে পদ্ধতিগতভাবে সেরা k খুঁজে বের করা শেখানো হবে।

সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)

Introduction

KNN-এ কোনো Training Process নেই — এটি একটি Lazy Learner। নতুন স্যাম্পল এলে সেটি Training Data-র প্রতিটি পয়েন্টের সাথে দূরত্ব মেপে সবচেয়ে কাছের k-টি প্রতিবেশীর ভোটে সিদ্ধান্ত নেয়। নিচে দুটি ভিন্ন নতুন স্যাম্পলের জন্য সম্পূর্ণ ম্যানুয়াল হিসাব দেখানো হলো।

Dataset

ডেটাসেট: (Study Hours, Sleep Hours) → Result

Sample	Study Hours (X1)	Sleep Hours (X2)	Result
1	2	9	Fail
2	3	7	Fail
3	5	6	Pass
4	7	5	Pass
5	8	3	Pass

Euclidean Distance Formula

$$d(A, B) = \sqrt{[(A_1 - B_1)^2 + (A_2 - B_2)^2]}$$

A_1, A_2 : নতুন স্যাম্পলের ফিচার মান। B_1, B_2 : Training স্যাম্পলের ফিচার মান।

নতুন স্যাম্পল: A → (X1=4, X2=6)

Sample 1 (2,9,Fail) পর্যন্ত দূরত্ব: $d = \sqrt{[(2-4)^2 + (9-6)^2]} = \sqrt{[4 + 9]} = \sqrt{13} = 3.6056$

Sample 2 (3,7,Fail) পর্যন্ত দূরত্ব: $d = \sqrt{[(3-4)^2 + (7-6)^2]} = \sqrt{[1 + 1]} = \sqrt{2} = 1.4142$

Sample 3 (5,6,Pass) পর্যন্ত দূরত্ব: $d = \sqrt{[(5-4)^2 + (6-6)^2]} = \sqrt{[1 + 0]} = \sqrt{1} = 1.0000$

Sample 4 (7,5,Pass) পর্যন্ত দূরত্ব: $d = \sqrt{[(7-4)^2 + (5-6)^2]} = \sqrt{[9 + 1]} = \sqrt{10} = 3.1623$

Sample 5 (8,3,Pass) পর্যন্ত দূরত্ব: $d = \sqrt{[(8-4)^2 + (3-6)^2]} = \sqrt{[16 + 9]} = \sqrt{25} = 5.0000$

এখন দূরত্বগুলোকে ছোট থেকে বড় ক্রমে সাজানো হলো (Sort):

Rank (নিকটতম থেকে)	Class	Distance
1	Pass	1.0000
2	Fail	1.4142

3	Pass	3.1623
4	Fail	3.6056
5	Pass	5.0000

$k = 3$ নেওয়া হলো, তাই সবচেয়ে কাছের 3টি প্রতিবেশী বিবেচনা করা হবে: [('Pass', 1.0), ('Fail', 1.4142), ('Pass', 3.1623)]

Majority Voting: Pass = 2 ভোট, Fail = 1 ভোট

যেহেতু সংখ্যাগরিষ্ঠ প্রতিবেশী "Pass" ক্লাসের, তাই Final Prediction = Pass।

নতুন স্যাম্পল: B $\rightarrow$ (X1=6, X2=4)

Sample 1 (2,9,Fail) পর্যন্ত দূরত্ব: $d = \sqrt{[(2-6)^2 + (9-4)^2]} = \sqrt{[16 + 25]} = \sqrt{41} = 6.4031$

Sample 2 (3,7,Fail) পর্যন্ত দূরত্ব: $d = \sqrt{[(3-6)^2 + (7-4)^2]} = \sqrt{[9 + 9]} = \sqrt{18} = 4.2426$

Sample 3 (5,6,Pass) পর্যন্ত দূরত্ব: $d = \sqrt{[(5-6)^2 + (6-4)^2]} = \sqrt{[1 + 4]} = \sqrt{5} = 2.2361$

Sample 4 (7,5,Pass) পর্যন্ত দূরত্ব: $d = \sqrt{[(7-6)^2 + (5-4)^2]} = \sqrt{[1 + 1]} = \sqrt{2} = 1.4142$

Sample 5 (8,3,Pass) পর্যন্ত দূরত্ব: $d = \sqrt{[(8-6)^2 + (3-4)^2]} = \sqrt{[4 + 1]} = \sqrt{5} = 2.2361$

এখন দূরত্বগুলোকে ছোট থেকে বড় ক্রমে সাজানো হলো (Sort):

Rank (নিকটতম থেকে)	Class	Distance
1	Pass	1.4142
2	Pass	2.2361
3	Pass	2.2361
4	Fail	4.2426
5	Fail	6.4031

$k = 3$ নেওয়া হলো, তাই সবচেয়ে কাছের 3টি প্রতিবেশী বিবেচনা করা হবে: [('Pass', 1.4142), ('Pass', 2.2361), ('Pass', 2.2361)]

Majority Voting: Pass = 3 ভোট

যেহেতু সংখ্যাগরিষ্ঠ প্রতিবেশী "Pass" ক্লাসের, তাই Final Prediction = Pass।

Interpretation

লক্ষ্য করার বিষয় হলো, স্যাম্পল A (4,6) স্যাম্পল B (6,4)-এর তুলনায় Fail-ক্লাসের পয়েন্টগুলোর কাছাকাছি অবস্থিত, তাই এর প্রতিবেশীদের মধ্যে মতভেদ (2 Pass, 1 Fail) দেখা যায় যদিও Majority এখনো Pass-এর দিকে। অন্যদিকে B সম্পূর্ণভাবে Pass-ক্লাস অঞ্চলে অবস্থিত হওয়ায় তার 3টি প্রতিবেশীই Pass — ফলে সিদ্ধান্তে কোনো দ্বিধা নেই। এখান থেকে বোঝা যায়, k -এর মান এবং নতুন স্যাম্পলের অবস্থান উভয়ই চূড়ান্ত সিদ্ধান্তে প্রভাব ফেলে।

GridSearchCV in KNN

১. ভূমিকা (Introduction)

1. Why Do We Need GridSearchCV?

KNN-এর সবচেয়ে গুরুত্বপূর্ণ Hyperparameter হলো:

$$K = n_neighbors$$

কিন্তু প্রশ্ন হলো:

K কত নেব?

3?

5?

7?

11?

21?

যদি ভুল K নির্বাচন করি তাহলে Model-এর Performance অনেক কমে যেতে পারে।

ধরো:

K	Accuracy
3	88%
5	92%
7	95%
9	93%
11	90%

এখানে দেখা যাচ্ছে:

$$K = 7$$

সবচেয়ে ভালো।

কিন্তু Dataset দেখে আগে থেকে এটা জানা সম্ভব না।

তাই আমাদের বিভিন্ন K পরীক্ষা করতে হবে।

এই কাজটিই GridSearchCV স্বয়ংক্রিয়ভাবে করে।

২. কেন এই কৌশল তৈরি হয়েছিল (Why This Algorithm Was Developed)

শুরুর দিকে Machine Learning Practitioner-রা Hyperparameter (যেমন KNN-এর k) হাতে-কলমে Trial and Error করে নির্বাচন করতেন — একটি মান বসিয়ে দেখতেন, তারপর আরেকটি, এভাবে চলতে থাকত। এই প্রক্রিয়া শুধু সময়সাপেক্ষই নয়, বরং Subjective এবং Error-Prone ছিল, কারণ একটি মাত্র Train/Test Split-এর উপর ভিত্তি করে সিদ্ধান্ত নিলে Result-এ Random Variation চলে আসত। GridSearchCV তৈরি হয়েছিল এই সমস্যার একটি পদ্ধতিগত (Systematic) সমাধান হিসেবে — এটি Cross-Validation-এর সাথে Exhaustive Search সমন্বয় করে প্রতিটি সম্ভাব্য Combination নিরপেক্ষভাবে যাচাই করে।

৩. বাস্তব জীবনের প্রেরণা (Real-life Motivation)

ধরা যাক আপনি একটি রেসিপি নিখুঁত করতে চান, যেখানে চিনি ও লবণের পরিমাণ ভিন্ন ভিন্নভাবে মেশানো যায়। একবার একটি Combination চেখে সিদ্ধান্ত নেওয়ার বদলে, আপনি যদি প্রতিটি সম্ভাব্য Combination (কম চিনি-কম লবণ, কম চিনি-বেশি লবণ, ইত্যাদি) কয়েকবার রান্না করে গড় ফলাফল (Average Score) দেখে সবচেয়ে ভালোটি বেছে নেন — এটাই ঠিক GridSearchCV যা করে, শুধু রেসিপির বদলে Hyperparameter নিয়ে।

৪. স্বভাৱ (Intuition)

2. What is GridSearchCV?

GridSearchCV হলো Scikit-Learn-এর একটি Hyperparameter Tuning Technique।

এটি বিভিন্ন Parameter Combination পরীক্ষা করে এবং Best Combination খুঁজে বের করে।

ধরো তুমি KNN-এর জন্য নিচের Parameters পরীক্ষা করতে চাও:

n_neighbors = [3, 5, 7, 9]
weights = ['uniform', 'distance']

তাহলে মোট Combination হবে:

n_neighbors	weights
3	uniform

3	distance
5	uniform
5	distance
7	uniform
7	distance
9	uniform
9	distance

মোট:

$$4 \times 2 = 8$$

টি Model Train হবে।

৬. গাণিতিক ভিত্তি (Mathematical Backend)

৭. সূত্র (Formula)

3. Why Is It Called Grid Search?

ধরো Parameter Space হলো:

Parameter	Values
K	3,5,7
Weight	uniform,distance
p	1,2

তাহলে Grid তৈরি হবে:

K	Weight	p
3	uniform	1
3	uniform	2
3	distance	1
3	distance	2
5	uniform	1
5	uniform	2
5	distance	1

5	distance	2
7	uniform	1
7	uniform	2
7	distance	1
7	distance	2

মোট:

$$3 \times 2 \times 2 = 12$$

Combination।

GridSearchCV এই পুরো Grid Search করে।

তাই নাম:

Grid Search

4. What Does CV Mean?

CV = Cross Validation

Grid Search শুধু Train/Test Split ব্যবহার করে না।

বরং Cross Validation ব্যবহার করে।

৮. সূত্রের ডেরিভেশন (Formula Derivation)

5. Cross Validation Intuition

ধরো Dataset-এ:

$$1000$$

Samples আছে।

আমরা

$$cv = 5$$

দিলাম।

তাহলে Dataset 5 ভাগে ভাগ হবে।

Fold-1
Fold-2
Fold-3
Fold-4
Fold-5

Iteration 1:

Train:
2 3 4 5

Validation:
1

Iteration 2:

Train:
1 3 4 5

Validation:
2

Iteration 3:

Train:
1 2 4 5

Validation:
3

Iteration 4:

Train:
1 2 3 5

Validation:
4

Iteration 5:

Train:
1 2 3 4

Validation:
5

এরপর Average Accuracy বের করা হয়।

$$CV\ Score = \frac{Score_1 + Score_2 + Score_3 + Score_4 + Score_5}{5}$$

৫. কার্যপ্রণালী (Working Principle)

৯. প্রতিটি চিহ্নের অর্থ (Meaning of Every Symbol)

Symbol / Term	Meaning (অর্থ)
param_grid	যেসব Hyperparameter মান পরীক্ষা করা হবে তার তালিকা
cv	কতগুলো Fold-এ Cross-Validation করা হবে
Fold	Dataset-এর একটি ভাগ, যা পালানক্রমে Validation হিসেবে ব্যবহৃত হয়
best_params_	সর্বোচ্চ গড় Score পাওয়া Hyperparameter Combination
best_score_	সেরা Combination-এর গড় Cross-Validation Score
Total Combinations	প্রতিটি Hyperparameter-এর সম্ভাব্য মানের গুণফল

6. Backend Working of GridSearchCV

ধরি:

```
n_neighbors=[3,5,7]
weights=['uniform','distance']
```

Combination:

```
(3,uniform)
(3,distance)
```

```
(5,uniform)
(5,distance)
```

```
(7,uniform)
(7,distance)
```

মোট:

এবং

cv=5

তাহলে:

$$6 \times 5 = 30$$

বার Model Train হবে।

GridSearchCV:

1. First Combination নেয়
2. 5 Fold CV চালায়
3. Average Score বের করে
4. Next Combination নেয়
5. Repeat করে

শেষে Highest Score যেটা দেয় সেটাকে Best Model ঘোষণা করে।

7. Important KNN Hyperparameters

GridSearchCV সাধারণত KNN-এর নিচের Parameters Tune করে।

n_neighbors

সবচেয়ে গুরুত্বপূর্ণ Parameter।

n_neighbors = [3,5,7,9,11]

ছোট K

→ Overfitting

বড় K

→ Underfitting

weights

Uniform

সব Neighbor-এর Vote সমান।

weights='uniform'

Distance

কাছের Neighbor-এর Vote বেশি।

weights='distance'

Weight:

$$w = \frac{1}{d}$$

p

Minkowski Distance Power।

p = 1

Manhattan

Distance

p = 2

Euclidean

Distance

metric

metric='minkowski'

Default।

অন্যান্য:

euclidean
manhattan
cosine
chebyshev

১০. সূত্রের ধাপে-ধাপে ব্যাখ্যা (Step-by-Step Formula Explanation)

১১. সংখ্যাগত উদাহরণ (Numerical Example)

8. Example of Full Search Space

```
param_grid = {  
    "model__n_neighbors":[  
        3,5,7,9,11,15  
    ],  
    "model__weights":[  
        "uniform",  
        "distance"  
    ],  
    "model__p":[  
        1,  
        2  
    ]  
}
```

Total Models:

$$6 \times 2 \times 2 = 24$$

If

cv=5

Then

$$24 \times 5 = 120$$

Model Fits

9. Important Outputs

After Training:

Best Parameters

grid.best_params_

Output:

```
{  
'n_neighbors':7,  
'weights':'distance',  
'p':2  
}
```

Best Score

grid.best_score_

Output:

0.972

Meaning:

97.2%

Cross Validation Accuracy

Best Model

grid.best_estimator_

Returns:

Best Trained Pipeline

১২. ভিজুয়ালাইজেশন (Visualization)

GridSearchCV-এর ফলাফল সবচেয়ে ভালোভাবে একটি Heatmap দিয়ে কল্পনা করা যায়, যেখানে এক Axis-এ n_neighbors-এর বিভিন্ন মান, আরেক Axis-এ weights-এর বিভিন্ন Option, এবং প্রতিটি ঘরে (Cell) সেই Combination-এর গড় Cross-Validation Accuracy রঙ দিয়ে দেখানো থাকে (গাঢ় রঙ = বেশি Accuracy)। এভাবে এক নজরেই বোঝা যায় কোন অঞ্চলে (Hyperparameter Region) মডেল সবচেয়ে ভালো Perform করছে, এবং কোন অঞ্চলে Overfitting বা Underfitting ঘটছে।

১৯. প্রয়োগক্ষেত্র (Applications)

- যেকোনো Supervised Model-এর সেরা Hyperparameter (KNN-এর k, SVM-এর C ও gamma, Random Forest-এর n_estimators) নির্বাচন করতে।
- Model Comparison-এর সময় প্রতিটি Model-কে তার নিজস্ব সেরা Configuration-এ যাচাই করতে, যাতে তুলনা ন্যায্য হয়।
- Automated Machine Learning (AutoML) Pipeline-এর একটি মূল উপাদান হিসেবে।

১৭-১৮. সুবিধা ও অসুবিধা (Advantages & Disadvantages)

10. Advantages of GridSearchCV

Automatic Hyperparameter Tuning

নিজে নিজে Combination পরীক্ষা করে।

Uses Cross Validation

Result বেশি Reliable।

Finds Optimal Model

Best Parameters খুঁজে দেয়।

Easy to Use

Scikit-Learn-এ কয়েক লাইনের Code।

11. Disadvantages

Slow

Combination বেশি হলে সময় বেশি লাগে।

Computationally Expensive

Large Dataset-এ Costly।

Exhaustive Search

সব Combination পরীক্ষা করে।

কখনও কখনও অপ্রয়োজনীয়ও হতে পারে।

১৩. হাইপারপ্যারামিটার (Hyperparameters)

১৪. স্কিট-লার্ন ইম্পোর্ট (Scikit-learn Import)

```
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

১৫. পাইথন উদাহরণ — From-Scratch Implementation (Python Example)

নিচে সম্পূর্ণ From-Scratch একটি ছোট Grid Search + Manual Cross-Validation দেখানো হলো, যা sklearn-এর GridSearchCV ভিতরে যা করে তার সরলীকৃত রূপ:

```
import numpy as np
from collections import Counter

def knn_predict(X_train, y_train, x_new, k):
    dists = [np.linalg.norm(np.array(x_new)-np.array(xt)) for xt in X_train]
    idx = np.argsort(dists)[:k]
    labels = [y_train[i] for i in idx]
    return Counter(labels).most_common(1)[0][0]

def evaluate_k(X_train, y_train, X_val, y_val, k):
    correct = 0
    for x, y_true in zip(X_val, y_val):
        pred = knn_predict(X_train, y_train, x, k)
        correct += (pred == y_true)
    return correct / len(y_val)

X_train = [[1,9],[2,8],[3,6],[6,4],[7,3],[8,2]]
y_train = ['Fail','Fail','Pass','Pass','Pass','Pass']
X_val = [[3,7],[7,4]]
y_val = ['Fail','Pass']

best_k, best_acc = None, -1
for k in [1,3,5]:
    acc = evaluate_k(X_train, y_train, X_val, y_val, k)
    print(f'k={k}: accuracy={acc:.4f}')
    if acc > best_acc:
        best_k, best_acc = k, acc
print('Best k:', best_k)
```

১৬. প্রেডিকশন প্রসেস (Prediction Process)

GridSearchCV নিজে কোনো Prediction করে না — এটি শুধু সেরা Hyperparameter খুঁজে বের করে। সেরা Combination পাওয়ার পর (grid.best_params_), সেই Parameter দিয়ে একটি নতুন Model পুরো Training Data-তে পুনরায় Train (Refit) করা হয় (grid.best_estimator_), এবং এরপর সেই Final Model-ই ভবিষ্যতে নতুন স্যাম্পলের Prediction করার জন্য ব্যবহৃত হয়।

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

wine = load_wine()

X = wine.data
y = wine.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier())
])
```

```
param_grid = {  
    "model__n_neighbors": [3,5,7,9,11,15],  
    "model__weights": [  
        "uniform",  
        "distance"  
    ],  
    "model__p": [  
        1,  
        2  
    ]  
}
```

```
grid = GridSearchCV(  
    estimator=pipe,  
    param_grid=param_grid,  
    cv=5,  
    scoring="accuracy",  
    n_jobs=-1  
)
```

```
grid.fit(X_train, y_train)  
  
print("Best Parameters:")  
print(grid.best_params_)
```

```
print("Best Parameters:")  
print(grid.best_params_)
```

```
print("\nBest CV Score:")  
print(grid.best_score_)
```

```
print("\nBest Pipeline:")  
print(grid.best_estimator_)
```

```
test_accuracy = grid.score(X_test, y_test)
```

```
print("\nTest Accuracy:")  
print(test_accuracy)
```

refit

Default: True

refit নির্ধারণ করে GridSearchCV Best Parameters খুঁজে পাওয়ার পরে সম্পূর্ণ Training Data দিয়ে আবার Model Refit করবে কিনা। True রাখলে grid.best_estimator_ থেকে সরাসরি Prediction করা যায়। False করলে শুধু Best Parameters জানা যাবে কিন্তু সরাসরি Predict করা যাবে না।

verbose

Default: 0

verbose নির্ধারণ করে GridSearchCV-এর Search চলাকালে কতটুকু Progress দেখাবে। verbose=0 মানে কোনো Log নেই। verbose=2 বা 3 দিলে প্রতিটি Fold এবং Parameter Combination-এর Score Console-এ দেখা যায়, যা বড় Search Space-এ কতটুকু এগোলো তা বোঝার জন্য কার্যকর।

pre_dispatch

Default: '2*n_jobs'

pre_dispatch নির্ধারণ করে Parallel Execution-এর সময় একসাথে কতগুলো Job Memory-তে রাখা হবে। এটি Memory Usage নিয়ন্ত্রণ করে। অনেক বড় Dataset বা বেশি Combination-এ Memory সমস্যা হলে এই মান কমিয়ে দেওয়া যায়, যেমন pre_dispatch=8।

error_score

Default: np.nan

error_score নির্ধারণ করে কোনো Combination Train করার সময় Error বা Exception আসলে সেই Combination-এর Score হিসেবে কী মান বসানো হবে। Default nan মানে Error হলে সেই Combination-এর Score NaN হয়ে যাবে। 'raise' দিলে Error সরাসরি Raise হবে এবং পুরো Training বন্ধ হয়ে যাবে, যা Debugging-এর জন্য সুবিধাজনক।

return_train_score

Default: False

return_train_score=True করলে GridSearchCV প্রতিটি Fold-এ Validation Score-এর পাশাপাশি Training Score-ও cv_results_-এ সংরক্ষণ করে। Train এবং Validation Score একসাথে তুলনা করলে Overfitting বা Underfitting সহজে সনাক্ত করা যায়। তবে এটি True করলে Computation সময় কিছুটা বাড়ে।

GridSearchCV() Attributes

best_estimator_

Default: Created after fit()

সব Parameter Combination পরীক্ষা করার পর যে Model সবচেয়ে ভালো Performance করেছে, সেটিকে Best Parameter দিয়ে পুরো Training Data-তে পুনরায় (Refit) Train করা হয়। সেই Final Trained Model-টি এই Attribute-এ সংরক্ষণ থাকে।

best_params_

Default: Created after fit()

সব Parameter Combination-এর মধ্যে যেটি সর্বোচ্চ Cross-Validation Score পেয়েছে, সেই Parameter-এর Dictionary সংরক্ষণ করে।

best_score_

Default: Created after fit()

Best Parameter Combination-এর Mean Cross-Validation Score সংরক্ষণ করে। এটি বিভিন্ন Fold-এর Average Score।

best_index_

Default: Created after fit()

cv_results_-এর মধ্যে Best Parameter Combination কোন Index-এ রয়েছে তা সংরক্ষণ করে।

cv_results_

Default: Created after fit()

প্রতিটি Parameter Combination-এর বিস্তারিত ফলাফল Dictionary আকারে সংরক্ষণ করে। এর মধ্যে Mean Score, Standard Deviation, Rank, Fit Time, Score Time এবং প্রতিটি Fold-এর Score থাকে। এটি Grid Search-এর সবচেয়ে বিস্তারিত Output।

scorer_

Default: Created after fit()

Training-এর সময় ব্যবহৃত Scoring Function সংরক্ষণ করে। যেমন accuracy, f1, precision, recall, neg_mean_squared_error ইত্যাদি।

n_splits_

Default: Created after fit()

Cross-Validation-এর সময় মোট কতটি Fold ব্যবহার করা হয়েছে তা সংরক্ষণ করে।

refit_time_

Default: Created after fit() (*Only when refit=True*)

Best Parameter পাওয়ার পর পুরো Training Data-তে Final Model পুনরায় Train (Refit) করতে কত সময় লেগেছে তা সেকেন্ডে সংরক্ষণ করে।

multimetric_

Default: Created after fit()

একাধিক Scoring Metric ব্যবহার করা হয়েছে কিনা তা নির্দেশ করে। True হলে Multiple Metrics এবং False হলে একটি মাত্র Metric ব্যবহার হয়েছে।

classes_

Default: Created after fit() (*Only if the estimator is a classifier*)

যদি ভিতরের Estimator একটি Classifier হয়, তাহলে Training Data-তে থাকা সকল Target Class-এর List সংরক্ষণ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় Best Estimator মোট কতটি Input Feature ব্যবহার করেছে তা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

refit

Default: Available after initialization

এই Attribute নির্দেশ করে Best Parameter পাওয়ার পর Model-টিকে পুরো Training Data-তে পুনরায় Train করা হয়েছে কিনা। True হলে Refit করা হয়, False হলে করা হয় না।

multimetric_

Default: Created after fit()

Grid Search-এ একাধিক Scoring Function ব্যবহার করা হয়েছে কিনা তা নির্দেশ করে। এটি Boolean (True বা False) মান ধারণ করে।

PRACTICAL INSIGHT

Combination সংখ্যা বেশি হলে GridSearchCV-এর বদলে RandomizedSearchCV ব্যবহার করা ভালো, যা সব Combination না পরীক্ষা করে র‍্যান্ডমভাবে কিছু Combination Sample করে দ্রুত একটি ভালো Result দেয়।

INTERVIEW TIP

Q: GridSearchCV-তে cv=5 মানে কী?

A: ডেটাসেটকে ৫টি Fold-এ ভাগ করে প্রতিবার ৪টি Fold দিয়ে Train এবং ১টি Fold দিয়ে Validate করা হয়; এই প্রক্রিয়া ৫ বার পুনরাবৃত্তি হয়ে Average Score বের করা হয়।

Q: GridSearchCV এবং RandomizedSearchCV-এর মধ্যে পার্থক্য কী?

A: GridSearchCV সব Combination Exhaustively পরীক্ষা করে, যেখানে RandomizedSearchCV একটি নির্দিষ্ট সংখ্যক Combination র্যান্ডমভাবে Sample করে — তাই বড় Search Space-এ এটি অনেক দ্রুত।

Quick Revision Notes — GridSearchCV

Category	Key Points
Key Formula	Total Fits = (Combinations) × (cv folds)
Purpose	Exhaustive hyperparameter search with cross-validation
Key Outputs	best_params_, best_score_, best_estimator_
Trade-off	Reliability (CV) vs Computation cost (exhaustive)
Alternative	RandomizedSearchCV for large search spaces
Exam Focus	Compute total model fits given grid size and cv
Interview Focus	GridSearchCV vs RandomizedSearchCV vs Bayesian Optimization

২০. পূর্ববর্তী অ্যালগরিদমের সাথে তুলনা (Comparison with Previous Algorithm)

KNN অধ্যায়ে আমরা k হাতে-কলমে অনুমান করে বেছে নিয়েছিলাম; GridSearchCV সেই একই কাজ পদ্ধতিগতভাবে করে:

দিক (Aspect)	Manual k Selection (Chapter 8)	GridSearchCV (Chapter 9)
পদ্ধতি	হাতে-কলমে অনুমান করে একটি k চেষ্টা করা	একাধিক k স্বয়ংক্রিয়ভাবে পরীক্ষা করা
যাচাই	একটিমাত্র Validation Split	Cross-Validation (একাধিক Fold)
নির্ভরযোগ্যতা	Random Variation-এর ঝুঁকি বেশি	Average করার কারণে বেশি Stable
সময়	দ্রুত কিন্তু কম নিশ্চিত	ধীর কিন্তু বেশি নির্ভরযোগ্য

২১. সাধারণ ভুলত্রুটি (Common Mistakes)

- খুব বড় param_grid ব্যবহার করা, যার ফলে Training সময় বহুগুণ বেড়ে যায় (Combinatorial Explosion)।
- Test Set-কে GridSearchCV-এর ভেতরে ব্যবহার করা — Test Set শুধু চূড়ান্ত মূল্যায়নের জন্য আলাদা রাখা উচিত, GridSearchCV শুধু Train ডেটার উপর Cross-Validation করবে।
- cv-এর মান খুব ছোট (যেমন cv=2) রাখা, যার ফলে প্রতিটি Fold-এ পর্যাপ্ত ডেটা না থাকায় Score Unreliable হয়ে যায়।
- শুধু best_score_ দেখেই সিদ্ধান্ত নেওয়া, Overfitting-এর সম্ভাবনা (যেমন Train ও Validation Score-এর বড় ব্যবধান) যাচাই না করা।

২২. সাক্ষাৎকারের প্রশ্ন (Interview Questions)

Q: GridSearchCV-তে "CV" অংশটি কেন গুরুত্বপূর্ণ?

A: কারণ এটি নিশ্চিত করে প্রতিটি Hyperparameter Combination একাধিক ভিন্ন Train/Validation Split-এ যাচাই হয়, ফলে ফলাফল একটিমাত্র Random Split-এর উপর নির্ভরশীল থাকে না এবং বেশি নির্ভরযোগ্য হয়।

Q: GridSearchCV-এর প্রধান অসুবিধা কী?

A: এটি Exhaustive Search — সমস্ত সম্ভাব্য Combination পরীক্ষা করে, তাই Combination সংখ্যা বাড়লে (বা Dataset বড় হলে) Computationally অনেক ব্যয়বহুল হয়ে যায়।

Q: GridSearchCV এবং RandomizedSearchCV-এর মধ্যে পার্থক্য কী?

A: GridSearchCV সব Combination পরীক্ষা করে, কিন্তু RandomizedSearchCV Parameter Space থেকে একটি নির্দিষ্ট সংখ্যক Combination Randomly Sample করে পরীক্ষা করে — বড় Search Space-এ এটি অনেক দ্রুত।

Q: refit=True-এর কাজ কী?

A: সেরা Hyperparameter Combination খুঁজে পাওয়ার পর, refit=True থাকলে GridSearchCV পুরো Training Data দিয়ে সেই সেরা Configuration দিয়ে Model পুনরায় Train করে, যাতে সেটি সরাসরি Prediction-এ ব্যবহারযোগ্য হয়।

২৩. সারাংশ (Summary)

GridSearchCV হলো Hyperparameter Tuning-এর একটি পদ্ধতিগত ও নির্ভরযোগ্য পদ্ধতি, যা Cross-Validation-এর সাথে Exhaustive Search যুক্ত করে সেরা Model Configuration খুঁজে বের করে। এটি ধীরগতির হতে পারে যখন Search Space বড়, কিন্তু ফলাফলের নির্ভরযোগ্যতা তার প্রধান শক্তি। এই অধ্যায়ে আমরা শুধু KNN-এর k নিয়ে কাজ করেছি, কিন্তু একই কৌশল পরবর্তী অধ্যায়গুলোর Decision Tree, Random Forest ও SVM-এর হাইপারপ্যারামিটার Tune করতেও ব্যবহার করা যায়।

সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)

Introduction

এই সেকশনে দেখানো হবে কীভাবে KNN-এর সেরা k মান (Hyperparameter) নির্বাচন করার জন্য Train ও Validation সেট আলাদা করে, প্রতিটি সম্ভাব্য k -এর Validation Accuracy ম্যানুয়ালি গণনা করা হয়, এবং সবচেয়ে ভালো ফলাফল দেওয়া k বেছে নেওয়া হয়।

Dataset — Train ও Validation Split

Training Set (৬টি স্যাম্পল):

Sample	X1	X2	Result
1	1	9	Fail
2	2	8	Fail
3	3	6	Pass
4	6	4	Pass
5	7	3	Pass
6	8	2	Pass

Validation Set (২টি স্যাম্পল, Model যাচাই করার জন্য আলাদা রাখা হয়েছে):

Sample	X1	X2	True Result
V1	3	7	Fail
V2	7	4	Pass

চেষ্টা করা হবে যে মানগুলো: $k = 1$, $k = 3$, $k = 5$

প্রতিটি k -এর জন্য প্রতিটি Validation স্যাম্পলের ক্লাস প্রেডিক্ট করে True Label-এর সাথে তুলনা করা হবে।

$k = 1$

Validation স্যাম্পল V1 (3,7), True Label = Fail:

Train 1 (1,9,Fail) $\rightarrow d = \sqrt{[(1-3)^2 + (9-7)^2]} = \sqrt{4+4} = 2.8284$

Train 2 (2,8,Fail) $\rightarrow d = \sqrt{[(2-3)^2 + (8-7)^2]} = \sqrt{1+1} = 1.4142$

Train 3 (3,6,Pass) $\rightarrow d = \sqrt{[(3-3)^2 + (6-7)^2]} = \sqrt{0+1} = 1.0000$

Train 4 (6,4,Pass) $\rightarrow d = \sqrt{[(6-3)^2 + (4-7)^2]} = \sqrt{9+9} = 4.2426$

$$\text{Train 5 (7,3,Pass)} \rightarrow d = \sqrt{[(7-3)^2 + (3-7)^2]} = \sqrt{[16+16]} = 5.6569$$

$$\text{Train 6 (8,2,Pass)} \rightarrow d = \sqrt{[(8-3)^2 + (2-7)^2]} = \sqrt{[25+25]} = 7.0711$$

নিকটতম 1 প্রতিবেশী: [('Pass', 1.0)] $\rightarrow$ ভোট: {'Pass': 1} $\rightarrow$ Prediction = Pass (ভুল X)

Validation স্যাম্পল V2 (7,4), True Label = Pass:

$$\text{Train 1 (1,9,Fail)} \rightarrow d = \sqrt{[(1-7)^2 + (9-4)^2]} = \sqrt{[36+25]} = 7.8102$$

$$\text{Train 2 (2,8,Fail)} \rightarrow d = \sqrt{[(2-7)^2 + (8-4)^2]} = \sqrt{[25+16]} = 6.4031$$

$$\text{Train 3 (3,6,Pass)} \rightarrow d = \sqrt{[(3-7)^2 + (6-4)^2]} = \sqrt{[16+4]} = 4.4721$$

$$\text{Train 4 (6,4,Pass)} \rightarrow d = \sqrt{[(6-7)^2 + (4-4)^2]} = \sqrt{[1+0]} = 1.0000$$

$$\text{Train 5 (7,3,Pass)} \rightarrow d = \sqrt{[(7-7)^2 + (3-4)^2]} = \sqrt{[0+1]} = 1.0000$$

$$\text{Train 6 (8,2,Pass)} \rightarrow d = \sqrt{[(8-7)^2 + (2-4)^2]} = \sqrt{[1+4]} = 2.2361$$

নিকটতম 1 প্রতিবেশী: [('Pass', 1.0)] $\rightarrow$ ভোট: {'Pass': 1} $\rightarrow$ Prediction = Pass (সঠিক ✓)

$$\text{Validation Accuracy (k=1)} = 1/2 = 0.5000$$

$$\text{Validation Error (k=1)} = 1 - 0.5000 = 0.5000$$

k = 3

Validation স্যাম্পল V1 (3,7), True Label = Fail:

$$\text{Train 1 (1,9,Fail)} \rightarrow d = \sqrt{[(1-3)^2 + (9-7)^2]} = \sqrt{[4+4]} = 2.8284$$

$$\text{Train 2 (2,8,Fail)} \rightarrow d = \sqrt{[(2-3)^2 + (8-7)^2]} = \sqrt{[1+1]} = 1.4142$$

$$\text{Train 3 (3,6,Pass)} \rightarrow d = \sqrt{[(3-3)^2 + (6-7)^2]} = \sqrt{[0+1]} = 1.0000$$

$$\text{Train 4 (6,4,Pass)} \rightarrow d = \sqrt{[(6-3)^2 + (4-7)^2]} = \sqrt{[9+9]} = 4.2426$$

$$\text{Train 5 (7,3,Pass)} \rightarrow d = \sqrt{[(7-3)^2 + (3-7)^2]} = \sqrt{[16+16]} = 5.6569$$

$$\text{Train 6 (8,2,Pass)} \rightarrow d = \sqrt{[(8-3)^2 + (2-7)^2]} = \sqrt{[25+25]} = 7.0711$$

নিকটতম 3 প্রতিবেশী: [('Pass', 1.0), ('Fail', 1.4142), ('Fail', 2.8284)] $\rightarrow$ ভোট: {'Pass': 1, 'Fail': 2}
 $\rightarrow$ Prediction = Fail (সঠিক ✓)

Validation স্যাম্পল V2 (7,4), True Label = Pass:

$$\text{Train 1 (1,9,Fail)} \rightarrow d = \sqrt{[(1-7)^2 + (9-4)^2]} = \sqrt{[36+25]} = 7.8102$$

$$\text{Train 2 (2,8,Fail)} \rightarrow d = \sqrt{[(2-7)^2 + (8-4)^2]} = \sqrt{[25+16]} = 6.4031$$

$$\text{Train 3 (3,6,Pass)} \rightarrow d = \sqrt{[(3-7)^2 + (6-4)^2]} = \sqrt{[16+4]} = 4.4721$$

$$\text{Train 4 (6,4,Pass)} \rightarrow d = \sqrt{[(6-7)^2 + (4-4)^2]} = \sqrt{[1+0]} = 1.0000$$

$$\text{Train 5 (7,3,Pass)} \rightarrow d = \sqrt{[(7-7)^2 + (3-4)^2]} = \sqrt{[0+1]} = 1.0000$$

$$\text{Train 6 (8,2,Pass)} \rightarrow d = \sqrt{[(8-7)^2 + (2-4)^2]} = \sqrt{[1+4]} = 2.2361$$

নিকটতম 3 প্রতিবেশী: [('Pass', 1.0), ('Pass', 1.0), ('Pass', 2.2361)] $\rightarrow$ ভোট: {'Pass': 3} $\rightarrow$ Prediction = Pass (সঠিক ✓)

$$\text{Validation Accuracy (k=3)} = 2/2 = 1.0000$$

$$\text{Validation Error (k=3)} = 1 - 1.0000 = 0.0000$$

k = 5

Validation স্যাম্পল V1 (3,7), True Label = Fail:

$$\text{Train 1 (1,9,Fail)} \rightarrow d = \sqrt{[(1-3)^2 + (9-7)^2]} = \sqrt{[4+4]} = 2.8284$$

$$\text{Train 2 (2,8,Fail)} \rightarrow d = \sqrt{[(2-3)^2 + (8-7)^2]} = \sqrt{[1+1]} = 1.4142$$

$$\text{Train 3 (3,6,Pass)} \rightarrow d = \sqrt{[(3-3)^2 + (6-7)^2]} = \sqrt{[0+1]} = 1.0000$$

$$\text{Train 4 (6,4,Pass)} \rightarrow d = \sqrt{[(6-3)^2 + (4-7)^2]} = \sqrt{[9+9]} = 4.2426$$

$$\text{Train 5 (7,3,Pass)} \rightarrow d = \sqrt{[(7-3)^2 + (3-7)^2]} = \sqrt{[16+16]} = 5.6569$$

$$\text{Train 6 (8,2,Pass)} \rightarrow d = \sqrt{[(8-3)^2 + (2-7)^2]} = \sqrt{[25+25]} = 7.0711$$

নিকটতম 5 প্রতিবেশী: [('Pass', 1.0), ('Fail', 1.4142), ('Fail', 2.8284), ('Pass', 4.2426), ('Pass', 5.6569)] $\rightarrow$ ভোট: {'Pass': 3, 'Fail': 2} $\rightarrow$ Prediction = Pass (ভুল ✗)

Validation স্যাম্পল V2 (7,4), True Label = Pass:

$$\text{Train 1 (1,9,Fail)} \rightarrow d = \sqrt{[(1-7)^2 + (9-4)^2]} = \sqrt{[36+25]} = 7.8102$$

$$\text{Train 2 (2,8,Fail)} \rightarrow d = \sqrt{[(2-7)^2 + (8-4)^2]} = \sqrt{[25+16]} = 6.4031$$

$$\text{Train 3 (3,6,Pass)} \rightarrow d = \sqrt{[(3-7)^2 + (6-4)^2]} = \sqrt{[16+4]} = 4.4721$$

$$\text{Train 4 (6,4,Pass)} \rightarrow d = \sqrt{[(6-7)^2 + (4-4)^2]} = \sqrt{[1+0]} = 1.0000$$

$$\text{Train 5 (7,3,Pass)} \rightarrow d = \sqrt{[(7-7)^2 + (3-4)^2]} = \sqrt{[0+1]} = 1.0000$$

$$\text{Train 6 (8,2,Pass)} \rightarrow d = \sqrt{[(8-7)^2 + (2-4)^2]} = \sqrt{[1+4]} = 2.2361$$

নিকটতম 5 প্রতিবেশী: [('Pass', 1.0), ('Pass', 1.0), ('Pass', 2.2361), ('Pass', 4.4721), ('Fail', 6.4031)] $\rightarrow$ ভোট: {'Pass': 4, 'Fail': 1} $\rightarrow$ Prediction = Pass (সঠিক ✓)

$$\text{Validation Accuracy (k=5)} = 1/2 = 0.5000$$

Validation Error (k=5) = 1 – 0.5000 = 0.5000

Hyperparameter Evaluation Table (Comparison)

k	Correct Predictions	Validation Accuracy	Validation Error
1	1/2	0.5000	0.5000
3	2/2	1.0000	0.0000
5	1/2	0.5000	0.5000

Best Hyperparameter Selection

k=1 এ V1 ভুল প্রেডিক্ট হয়েছে (কারণ Train sample 3 (3,6,Pass) সবচেয়ে কাছের একমাত্র প্রতিবেশী হওয়ায় Noise-এর প্রভাব পুরোপুরি প্রেডিকশনে চলে আসে — একে Overfitting to Noise বলে)। k=5-তেও V1 ভুল হয় কারণ অনেক দূরের, কম প্রাসঙ্গিক প্রতিবেশীও ভোটে যুক্ত হয়ে যায় (Underfitting-এর দিকে ঝোঁকে)। কিন্তু k=3-তে উভয় Validation স্যাম্পলই সঠিকভাবে প্রেডিক্ট হয়েছে (Accuracy = 1.0000, Error = 0.0000)। তাই GridSearchCV এই তিনটি প্রার্থীর মধ্যে k=3 কে সেরা Hyperparameter হিসেবে নির্বাচন করবে, কারণ এটি Noise-Sensitivity (k খুব ছোট হলে) এবং Over-Smoothing (k খুব বড় হলে) — দুটির মধ্যে সবচেয়ে ভালো ভারসাম্য তৈরি করে।

Naive Bayes

১. ভূমিকা (Introduction)

Naive Bayes হলো একটি সহজ কিন্তু শক্তিশালী সুপারভাইজড মেশিন লার্নিং অ্যালগরিদম যা মূলত ক্লাসিফিকেশন বা শ্রেণিবিভাগের কাজে ব্যবহৃত হয়। এটি Bayes' Theorem বা বেইজ উপপাদ্যের গাণিতিক নীতির ওপর ভিত্তি করে তৈরি।

এই অ্যালগরিদমটিকে "Naive" বা "সরল" বলা হয় কারণ এটি একটি অত্যন্ত শক্তিশালী কিন্তু সরল ধারণা ধরে নেয়: প্রতিটি ইনপুট ফিচার একে অপরের থেকে সম্পূর্ণ স্বাধীন বা ইন্ডিপেন্ডেন্ট। অর্থাৎ, একটি ফিচারের উপস্থিতি অন্য কোনো ফিচারের উপস্থিতিতে প্রভাবিত করে না। বাস্তবে এমন স্বাধীনতা খুব কমই পাওয়া যায়, কিন্তু এই সহজ ধারণার কারণেই মডেলটি অত্যন্ত দ্রুত কাজ করতে পারে।

এর কাজের মূল ভিত্তি হলো প্রোবাবিলিটি বা সম্ভাবনা। যখনই আমাদের সামনে কোনো নতুন ডেটা পয়েন্ট আসে, এটি Bayes' Theorem ব্যবহার করে প্রতিটি সম্ভাব্য ক্লাসের জন্য সম্ভাবনা হিসাব করে। যে ক্লাসের সম্ভাবনা সবচেয়ে বেশি হয়, মডেলটি সেই ক্লাসটিকেই আউটপুট হিসেবে প্রেডিক্ট করে। এই অ্যালগরিদমটি বিশেষভাবে টেক্সট ক্লাসিফিকেশন, ইমেইল স্প্যাম ডিটেকশন এবং সেন্টিমেন্ট অ্যানালাইসিসের মতো বড় ও জটিল ডেটাসেটে অসাধারণ ফলাফল দেয়।

২. কেন এই অ্যালগরিদম তৈরি হয়েছিল (Why This Algorithm Was Developed)

Naive Bayes-এর ভিত্তি Thomas Bayes-এর ১৮ শতকের Probability Theorem-এর উপর দাঁড়িয়ে, কিন্তু একে ব্যবহারযোগ্য Machine Learning Algorithm হিসেবে গড়ে তোলা হয়েছিল মূলত Text Classification ও Spam Filtering-এর প্রয়োজনে ১৯৯০-এর দশকে। সেই সময় হাজার হাজার শব্দ (Feature) নিয়ে কাজ করা অন্যান্য Model-এর জন্য Computationally ব্যয়বহুল ছিল। Naive Bayes-এর "Naive" অনুমান — যে প্রতিটি ফিচার একে অপরের থেকে স্বাধীন — এই সমস্যার একটি চমৎকার সমাধান দেয়: এটি প্রতিটি ফিচারকে আলাদাভাবে বিশ্লেষণ করে, ফলে হাজার হাজার Feature থাকলেও Computation Cost তুলনামূলক অনেক কম থাকে।

৩. বাস্তব জীবনের প্রেরণা (Real-life Motivation)

প্রতিদিন Email Client (Gmail, Outlook) হাজার হাজার ইমেইল থেকে Spam আলাদা করে। মানুষ যেভাবে "Free", "Winner", "Click Here"-এর মতো শব্দ দেখলে সন্দেহ করে, Naive Bayes ঠিক একইভাবে প্রতিটি শব্দের Spam ও Non-Spam ইমেইলে উপস্থিতির সম্ভাবনা হিসাব করে এবং সবগুলো শব্দের প্রমাণ একসাথে মিলিয়ে একটি চূড়ান্ত সিদ্ধান্তে পৌঁছায় — এটাই বাস্তব জগতে Naive Bayes-এর সবচেয়ে পরিচিত ব্যবহার।

8. স্বভাৱ (Intuition)

সহজভাবে বললে, Naive Bayes প্ৰতিটি ফিচাৰকে একাটি "প্ৰমাণ" (Evidence) হিচাবে দেখে এবং প্ৰশ্ন কৰে, "এই প্ৰমাণগুলো দেখে কোন ক্লাসেৰ সম্ভাৱনা সবচেয়ে বেশি?" এটি শুৰু কৰে একাটি সাধাৰণ ধাৰণা (Prior Probability) দিয়ে, তাৰপৰ প্ৰতিটি নতুন প্ৰমাণ (Feature Value) দেখে সেই ধাৰণাকে ধীৰে ধীৰে আপডেট কৰে (Posterior Probability), ঠিক যেভাবে একজন গোয়েন্দা নতুন নতুন সূত্ৰ পেয়ে সন্দেহভাজনেৰ তালিকা সংকুচিত কৰেন।

Naive Bayes-এৰ প্ৰকাৰভেদ ও ব্যবহাৰেৰ উদাহৰণ

Naive Bayes অ্যালগৰিথমটি মূলত ডেটাৰ ডিস্ট্ৰিবিউশন বা বণ্টনেৰ ওপৰ ভিত্তি কৰে কয়েক ধৰনেৰ হয়। নিচে এদেৰ বিস্তাৰিত দেওয়া হলো:

Naive Bayes টাইপ	ডেটাৰ প্ৰকৃতি	ডেটাসেটেৰ উদাহৰণ
Gaussian NB	কন্টিনিউয়াস ডেটা যা নৰমাল ডিস্ট্ৰিবিউশন মেনে চলে	আইৰিস (Iris) ফ্লাওয়ার ডেটাসেট (যেখানে পাপড়িৰ দৈৰ্ঘ্য ও প্ৰস্থ কন্টিনিউয়াস মান)
Multinomial NB	ডিসক্ৰিট কাউন্ট (যেমন: শব্দেৰ সংখ্যা)	টেক্সট ক্লাসিফিকেশন বা স্প্যাম ডিটেকশন (ইমেইলে কোন শব্দ কতবাৰ আছে)
Complement NB	ইমব্যালেন্সড ডেটাসেট	টেক্সট ক্লাসিফিকেশন (যখন একাটি ক্লাসেৰ ডেটা অন্য ক্লাসেৰ চেয়ে অনেক বেশি থাকে)
Bernoulli NB	বাইনাৰি বা বুলিয়ান (০ অথবা ১) ডেটা	স্প্যাম ফিল্টাৰিং (একাটি শব্দ ইমেইলে আছে কি নেই—শুধু উপস্থিতি বা অনুপস্থিতি)
Categorical NB	ক্যাটাগৰিক্যাল ডেটা	শপিং ডেটাসেট (যেখানে বৈশিষ্ট্যগুলো নিৰ্দিষ্ট ক্যাটাগৰি যেমন: ৰং: লাল/নীল/সবুজ)

বিস্তাৰিত ব্যাখ্যা:

১. Gaussian Naive Bayes (GaussianNB)

এটি ধৰে নেয় যে ফিচাৰগুলো গাউসিয়ান (নৰমাল) ডিস্ট্ৰিবিউশন অনুসৰণ কৰে। যখন ডেটাৰ মানগুলো কন্টিনিউয়াস হয়, তখন এটি সবচেয়ে ভালো কাজ কৰে। উদাহৰণস্বৰূপ, আইৰিস ফ্লাওয়ার ডেটাসেটে ফুলেৰ বৈশিষ্ট্যগুলো সংখ্যাসূচক হয়।

২. Multinomial Naive Bayes (MultinomialNB)

এটি মাল্টিনোমিয়াল ডিস্ট্রিবিউশনের ডেটার জন্য ব্যবহৃত হয়, যা বিশেষ করে টেক্সট ক্লাসিফিকেশনে জনপ্রিয়। উদাহরণস্বরূপ, একটি ডকুমেন্টে কোন শব্দ কতবার এসেছে (word count), তার ভিত্তিতে ইমেইল স্প্যাম কি না তা নির্ণয় করা হয়।

৩. Complement Naive Bayes (ComplementNB)

এটি স্ট্যান্ডার্ড মাল্টিনোমিয়াল Naive Bayes-এর একটি উন্নত সংস্করণ, যা ইমব্যালেন্সড ডেটাসেটে খুব স্থিতিশীল ফলাফল দেয়। টেক্সট ক্লাসিফিকেশনের ক্ষেত্রে এটি প্রায়ই সাধারণ মাল্টিনোমিয়াল মডেলের চেয়ে অনেক ভালো পারফর্ম করে।

৪. Bernoulli Naive Bayes (BernoulliNB)

এই মডেলে প্রতিটি ফিচারকে বাইনারি (০ অথবা ১) হিসেবে ধরে নেওয়া হয়। টেক্সট ক্লাসিফিকেশনে এটি ওয়ার্ড কাউন্টের বদলে ওয়ার্ড অক্যুরেন্স (শব্দটি আছে কি নেই) ব্যবহার করে। এটি ছোট ইমেইল বা ডকুমেন্টের ক্ষেত্রে ভালো কাজ করতে পারে।

৫. Categorical Naive Bayes (CategoricalNB)

যখন ডেটা পুরোপুরি ক্যাটাগরিক্যাল হয় (যেমন: লিঙ্গ: পুরুষ/মহিলা, পেশা: ছাত্র/চাকরিজীবী), তখন এই মডেলটি ব্যবহৃত হয়। এটি প্রতিটি ক্যাটাগরির জন্য আলাদা ডিস্ট্রিবিউশন হিসাব করে।

৫. কার্যপ্রণালী (Working Principle)

Naive Bayes তিনটি ধাপে কাজ করে: (১) Training ডেটা থেকে প্রতিটি ক্লাসের Prior Probability গণনা করা, (২) প্রতিটি ফিচারের জন্য প্রতিটি ক্লাসের Likelihood গণনা করা (Categorical ডেটার জন্য Frequency Count, Continuous ডেটার জন্য Gaussian PDF), এবং (৩) নতুন স্যাম্পল এলে Bayes' Theorem ব্যবহার করে প্রতিটি ক্লাসের Posterior Probability (Score) গণনা করে সর্বোচ্চ Score-এর ক্লাসটি প্রেডিক্ট করা।

৬. গাণিতিক ভিত্তি (Mathematical Backend)

৭. সূত্র (Formula)

গাণিতিক সূত্র ও পরিভাষা (Mathematical Formula and Terms)

Naive Bayes-এর ভিত্তি হলো **Bayes' Theorem**, যা মূলত একটি শর্তাধীন সম্ভাবনা (Conditional Probability) নির্ণয়ের উপায়।

১. মূল সমীকরণ (The Core Formula)

যদি y একটি ক্লাস এবং X একটি ফিচার ভেক্টর (যা $x_1, x_2, \dots, x_n$ ফিচার নিয়ে গঠিত) হয়, তবে বেইজ থিওরেম অনুযায়ী:

$$P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)}$$

এখানে:

- $P(y|X)$ (**Posterior Probability**): ফিচার X দেওয়া থাকলে y ক্লাস হওয়ার সম্ভাবনা।
- $P(X|y)$ (**Likelihood**): y ক্লাসটির অধীনে ফিচার X -এর সম্ভাবনা।
- $P(y)$ (**Prior Probability**): কোনো ফিচার না দেখেই y ক্লাস হওয়ার প্রাথমিক সম্ভাবনা।
- $P(X)$ (**Evidence**): ফিচারটির নিজস্ব সম্ভাবনা।

৮. সূত্রের ডেরিভেশন (Formula Derivation)

২. Naive Assumption (শর্তাধীন স্বাধীনতা)

Naive Bayes ধরে নেয় যে, ক্লাসের মান জানা থাকলে প্রতিটি ফিচার একে অপরের থেকে স্বাধীন (Conditional Independence)। ফলে সমীকরণটি সরল হয়ে দাঁড়ায়:

$$P(y|x_1, \dots, x_n) = \frac{P(y) \prod_{i=1}^n P(x_i|y)}{P(x_1, \dots, x_n)}$$

যেহেতু $P(x_1, \dots, x_n)$ একটি ধ্রুবক (Constant), তাই ক্লাসিফিকেশনের নিয়মটি হয়:

$$y = \arg \max_y P(y) \prod_{i=1}^n P(x_i|y)$$

৯. প্রতিটি চিহ্নের অর্থ (Meaning of Every Symbol)

Symbol	Meaning (অর্থ)
P(C)	Prior Probability — ফিচার না দেখেই ক্লাস C-এর প্রাথমিক সম্ভাবনা
P(X C)	Likelihood — ক্লাস C-এর অধীনে ফিচার X দেখা যাওয়ার সম্ভাবনা
P(C X)	Posterior Probability — ফিচার X দেখার পর ক্লাস C হওয়ার সম্ভাবনা
P(X)	Evidence — ফিচার X-এর সামগ্রিক সম্ভাবনা (Normalizing Constant)
MAP	Maximum A Posteriori — সর্বোচ্চ Posterior Probability দেওয়া ক্লাস বেছে নেওয়া
α (Laplace Smoothing)	শূন্য Probability সমস্যা এড়াতে যোগ করা একটি ছোট ধ্রুবক

৩. গুরুত্বপূর্ণ পরিভাষা

- MAP (Maximum A Posteriori):** মডেলটি $P(y|X)$ কে সর্বোচ্চ করার জন্য যে অনুমান বা এস্টিমেশন ব্যবহার করে, তাকে MAP এস্টিমেশন বলা হয়।
- Laplace Smoothing:** যদি কোনো ফিচার ট্রেনিং সেটে না থাকে, তবে তার সম্ভাবনা ০ হয়ে যায়। এই সমস্যা দূর করার জন্য সম্ভাবনা গণনায় একটি ছোট মান যোগ করা হয়, একে Laplace Smoothing বলে।
- Likelihood:** প্রতিটি নির্দিষ্ট ক্লাসের জন্য ফিচারের ডিস্ট্রিবিউশন কেমন হবে, তা এটি নির্দেশ করে।

১০. সূত্রের ধাপে-ধাপে ব্যাখ্যা (Step-by-Step Formula Explanation)

১১. সংখ্যাগত উদাহরণ (Numerical Example)

১. ডেটাসেট (Dataset)

আমরা ৫ জন শিক্ষার্থীর তথ্য নিয়ে একটি ছোট ডেটাসেট তৈরি করি:

ID	Study Hours (x1)	Attendance (x2)	Pass (y)
1	2	60	No
2	3	70	No

3	5	80	Yes
4	6	90	Yes
5	4	75	Yes

- **টার্গেট:** Pass (Yes/No)
- **ফিচার:** Study Hours, Attendance

২. ট্রেনিং ধাপ (Training Phase)

ট্রেনিংয়ের কাজ হলো প্রতিটি ক্লাসের জন্য ফিচারগুলোর Prior Probability এবং Likelihood (Mean ও Variance) বের করা।

ক. Prior Probability $P(y)$

- মোট ডেটা $N = 5$
- $P(Yes) = 3/5 = 0.6$
- $P(No) = 2/5 = 0.4$

খ. Likelihood (Mean ও Variance নির্ণয়)

আমাদের প্রতিটি ফিচারের জন্য Mean (μ) এবং Variance (σ^2) বের করতে হবে।

Pass = Yes (ID 3, 4, 5):

- Study Hours: Mean $\mu = (5 + 6 + 4)/3 = 5$, Var $\sigma^2 \approx 1$
- Attendance: Mean $\mu = (80 + 90 + 75)/3 = 81.67$, Var $\sigma^2 \approx 58.33$

Pass = No (ID 1, 2):

- Study Hours: Mean $\mu = (2 + 3)/2 = 2.5$, Var $\sigma^2 = 0.5$
- Attendance: Mean $\mu = (60 + 70)/2 = 65$, Var $\sigma^2 = 50$

৩. প্রেডিকশন ধাপ (Prediction Phase)

ধরুন, একটি নতুন শিক্ষার্থীর ডেটা আসলো: Study Hours = 4, Attendance = 78।

ধাপ ১: গাউসিয়ান PDF ব্যবহার করে সম্ভাবনা বের করা আমরা প্রতিটি ফিচারের জন্য সম্ভাবনা বের করব:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \cdot \exp\left(-\frac{(x_i-\mu_y)^2}{2\sigma_y^2}\right)$$

- **Pass = Yes এর জন্য:**

- $P(Study = 4|Yes)$ ও $P(Atten = 78|Yes)$ গণনা করি।
- Likelihood $L_{Yes} = P(Study = 4|Yes) \times P(Atten = 78|Yes) \times P(Yes)$

- **Pass = No এর জন্য:**

- $P(Study = 4|No)$ ও $P(Atten = 78|No)$ গণনা করি।
- Likelihood $L_{No} = P(Study = 4|No) \times P(Atten = 78|No) \times P(No)$

ধাপ ২: চূড়ান্ত সিদ্ধান্ত (MAP Estimation) যেহেতু আমাদের ফিচারগুলো স্বাধীন ধরা হয়েছে, আমরা উপরের Likelihood গুলো তুলনা করব:

- যদি $L_{Yes} > L_{No}$ হয়, তবে প্রেডিকশন হবে "Yes"।
- যদি $L_{No} > L_{Yes}$ হয়, তবে প্রেডিকশন হবে "No"।

১. ট্রেনিং থেকে প্রাপ্ত প্যারামিটার (আমাদের আগের হিসাব অনুযায়ী):

- **Pass = Yes:** $\mu_{study} = 5, \sigma_{study}^2 = 1; \mu_{atten} = 81.67, \sigma_{atten}^2 = 58.33$
- **Pass = No:** $\mu_{study} = 2.5, \sigma_{study}^2 = 0.5; \mu_{atten} = 65, \sigma_{atten}^2 = 50$
- **Prior:** $P(Yes) = 0.6, P(No) = 0.4$

২. সম্ভাবনা গণনা (Gaussian PDF)

$$\text{সূত্র: } P(x|y) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

ক. Pass = Yes এর জন্য সম্ভাবনা:

$$১. \text{ Study Hours (4): } P(4|Yes) = \frac{1}{\sqrt{2\pi(1)}} \exp\left(-\frac{(4-5)^2}{2(1)}\right) \approx 0.3989 \cdot e^{-0.5} \approx \mathbf{0.2419}$$

$$২. \text{ Attendance (78): } P(78|Yes) = \frac{1}{\sqrt{2\pi(58.33)}} \exp\left(-\frac{(78-81.67)^2}{2(58.33)}\right) \approx 0.0523 \cdot e^{-0.116} \approx \mathbf{0.0466}$$

$$\text{Likelihood } (L_{Yes}): 0.2419 \times 0.0466 \times 0.6 \approx \mathbf{0.00676}$$

খ. Pass = No এর জন্য সম্ভাবনা:

$$১. \text{ Study Hours (4): } P(4|No) = \frac{1}{\sqrt{2\pi(0.5)}} \exp\left(-\frac{(4-2.5)^2}{2(0.5)}\right) \approx 0.5642 \cdot e^{-2.25} \approx \mathbf{0.0594}$$

$$২. \text{ Attendance (78): } P(78|No) = \frac{1}{\sqrt{2\pi(50)}} \exp\left(-\frac{(78-65)^2}{2(50)}\right) \approx 0.0564 \cdot e^{-1.69} \approx \mathbf{0.0105}$$

$$\text{Likelihood } (L_{No}): 0.0594 \times 0.0105 \times 0.4 \approx \mathbf{0.00025}$$

৩. চূড়ান্ত সিদ্ধান্ত (Comparison)

এখন আমরা প্রাপ্ত Likelihood দুটির তুলনা করব:

- $L_{Yes} = 0.00676$
- $L_{No} = 0.00025$

যেহেতু $L_{Yes} > L_{No}$ ($0.00676 > 0.00025$), তাই মডেলটি প্রেডিক্ট করবে যে শিক্ষার্থীটি **"Pass = Yes"**।

১২. ভিজ্যুয়ালাইজেশন (Visualization)

Naive Bayes-কে ভিজ্যুয়ালাইজ করার একটি ভালো উপায় হলো প্রতিটি ফিচারের জন্য প্রতিটি ক্লাসের Probability Distribution (Bell Curve) একই গ্রাফে আঁকা। যেখানে দুটি ক্লাসের Curve ওভারল্যাপ করে, সেখানেই Model সবচেয়ে বেশি বিভ্রান্ত হয় (Misclassification-এর সম্ভাবনা বেশি); যেখানে Curve দুটি স্পষ্টভাবে আলাদা, সেখানে Model অনেক আত্মবিশ্বাসের সাথে (High Probability) সঠিক সিদ্ধান্ত নিতে পারে।

১৯. প্রয়োগক্ষেত্র (Applications)

Over Iris Dataset:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

gnb = GaussianNB()

gnb.fit(X_train, y_train)

y_pred = gnb.predict(X_test)

print(f"Accuracy Score: {accuracy_score(y_test, y_pred):.4f}")
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

Over News Groups:

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline

data = fetch_20newsgroups(subset='train', categories=['alt.atheism', 'comp
    .graphics'])
X_train = data.data
y_train = data.target

model = make_pipeline(CountVectorizer(), MultinomialNB())

model.fit(X_train, y_train)

sample_text = ["This is about computer graphics and software rendering"]
prediction = model.predict(sample_text)

print(f"Predicted category index: {prediction[0]}")
print(f"Predicted category name: {data.target_names[prediction[0]]}")
```

১৭-১৮. সুবিধা ও অসুবিধা (Advantages & Disadvantages)

Advantages (সুবিধা)	Disadvantages (অসুবিধা)
অত্যন্ত দ্রুত Training ও Prediction, এমনকি অনেক Feature থাকলেও	Naive Assumption বাস্তবে প্রায়ই ভুল (ফিচারগুলো সবসময় স্বাধীন হয় না)
অল্প ডেটাতেও ভালো কাজ করে	Continuous Feature-এর জন্য প্রায়ই Gaussian Distribution ধরে নেয়, যা সবসময় সঠিক নয়
Multiclass Classification-এ স্বাভাবিকভাবেই কাজ করে	predict_proba() থেকে পাওয়া Probability প্রায়ই সঠিকভাবে Calibrated থাকে না
Text Classification ও Spam Detection-এ বিশেষভাবে কার্যকর	একটি Feature সম্পূর্ণ অনুপস্থিত থাকলে Zero-Frequency সমস্যা দেখা দিতে পারে (Smoothing ছাড়া)

১৩. হাইপারপ্যারামিটার (Hyperparameters)

Naive Bayes: Hyperparameter Hierarchy

Naive Bayes মডেলগুলো মূলত তাদের গাণিতিক ডিস্ট্রিবিউশন অনুমানের ওপর ভিত্তি করে কাজ করে, তাই এতে অন্যান্য অ্যালগরিদমের (যেমন: Random Forest) তুলনায় টিউন করার মতো প্যারামিটার কিছুটা কম।

গুরুত্ব	প্যারামিটার	কেন গুরুত্বপূর্ণ?	কোন মডেলে প্রযোজ্য?
সবচেয়ে বেশি	alpha	এটি Laplace/Lidstone Smoothing নিয়ন্ত্রণ করে। এটি নিশ্চিত করে যে কোনো ফিচার অনুপস্থিত থাকলেও মডেল যেন ০-এর কারণে পুরো গুণফল ০ না করে ফেলে।	Multinomial, Bernoulli, Complement, Categorical
সবচেয়ে বেশি	var_smoothing	এটি ছোট ভ্যারিয়েন্সের মানগুলোকে স্টেবল করে গাণিতিক ত্রুটি রোধ করে। মডেলের স্ট্যাবিলাইটির জন্য এটি অত্যন্ত কার্যকর।	Gaussian
মাঝারি	fit_prior	এটি মডেলকে শেখায় যে ডেটাসেটে ক্লাসের অনুপাত (Prior) বিবেচনা করতে হবে কি না। False দিলে সব ক্লাসকে সমান গুরুত্ব দেওয়া হয়।	সব মডেল
মাঝারি	binarize	এটি ফিচারগুলোকে নির্দিষ্ট সীমার ভিত্তিতে বাইনারি (০ বা ১) মান দেয়। টেক্সট ক্লাসিফিকেশনে এটি অত্যন্ত প্রভাবশালী।	Bernoulli

কম	min_categories	প্রতিটি ফিচারের নূন্যতম ক্যাটাগরি সংখ্যা নির্ধারণ করে, যা ডেটার সঠিক ডিস্ট্রিবিউশন এস্টিমেশনে সাহায্য করে।	Categorical
----	----------------	--	-------------

বিস্তারিত বিশ্লেষণ

১. alpha (Smoothing Factor)

এটি সবচেয়ে গুরুত্বপূর্ণ প্যারামিটার কারণ এটি মডেলের **জেনারেলাইজেশন ক্ষমতা** নিয়ন্ত্রণ করে।

- যদি $\alpha=1.0$ হয়, তবে এটি Laplace Smoothing হিসেবে কাজ করে এবং জিরো-প্রোবাবিলিটি সমস্যা দূর করে।
- যদি আপনার ডেটাসেটে এমন কোনো শব্দ বা ফিচার থাকে যা ট্রেনিং ডেটায় নেই কিন্তু টেস্ট ডেটায় আছে, তবে α সেই ফিচারকে একটি ক্ষুদ্র সম্ভাবনা প্রদান করে মডেলকে অকেজো হওয়া থেকে বাঁচায়।

২. var_smoothing (Variance Smoothing)

Gaussian Naive Bayes-এর জন্য এটিই সবচেয়ে বড় ব্রেক।

- অনেক সময় ফিচারগুলোর ভ্যারিয়েন্স খুব কম থাকে, যা ক্যালকুলেশনের সময় জিরো বা নেগেটিভ মান আসার ঝুঁকি তৈরি করে।
- var_smoothing এই ভ্যারিয়েন্সের সাথে সর্বোচ্চ ভ্যারিয়েন্সের একটি অংশ যোগ করে মডেলকে গাণিতিকভাবে স্থিতিশীল রাখে।

৩. binarize (Threshold for Binary Conversion)

BernoulliNB-এর ক্ষেত্রে এটি একটি গেম-চেঞ্জার।

- যদি আপনার ইনপুট ডেটা কন্টিনিউয়াস হয়, তবে binarize মানটি একটি নির্দিষ্ট থ্রেশহোল্ড হিসেবে কাজ করে যা ডেটাকে ০ বা ১ এ রূপান্তর করে দেয়। এটি মডেলের সিদ্ধান্ত গ্রহণ প্রক্রিয়ায় সরাসরি প্রভাব ফেলে।

৪. fit_prior (Class Prior)

এটি মডেলের সিদ্ধান্তের ওপর ভিত্তি হিসেবে কাজ করে। ডিফল্ট True রাখা ভালো কারণ এটি ডেটার প্রকৃত ক্লাসের অনুপাত শিখতে পারে। তবে যদি আপনার ডেটাসেট প্রবলভাবে ইমব্যালেন্সড হয় এবং আপনি চান মডেল পক্ষপাতমুক্ত সিদ্ধান্ত নিক, তবে fit_prior=False ব্যবহার করে সব ক্লাসকে সমান অগ্রাধিকার দেওয়া যেতে পারে।

১৪. স্কিট-লার্ন ইম্পোর্ট (Scikit-learn Import)

```
from sklearn.naive_bayes import GaussianNB
```

১৫. পাইথন উদাহরণ — From-Scratch Implementation (Python Example)

নিচে দেখানো হলো Scikit-learn-এর সাহায্যে এবং সম্পূর্ণ From-Scratch (NumPy দিয়ে) Naive Bayes বাস্তবায়ন:

```
from sklearn.naive_bayes import GaussianNB
import numpy as np
```

```
X_train = np.array([[8, 90], [7, 85], [3, 60], [2, 55], [9, 95]])
y_train = np.array(['Yes', 'Yes', 'No', 'No', 'Yes'])
```

```
model = GaussianNB()
model.fit(X_train, y_train)
print(model.predict([[4, 78]]))      # predicted class
print(model.predict_proba([[4, 78]])) # class probabilities
```

```
import numpy as np
from collections import defaultdict
```

```
class NaiveBayesScratch:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.priors = {}
        self.likelihoods = defaultdict(dict)
        n = len(y)
        for c in self.classes:
            X_c = X[y == c]
            self.priors[c] = len(X_c) / n
            for col in range(X.shape[1]):
                vals, counts = np.unique(X_c[:, col], return_counts=True)
                total = counts.sum()
                self.likelihoods[c][col] = dict(zip(vals, counts / total))

    def predict_one(self, x):
        scores = {}
        for c in self.classes:
            score = self.priors[c]
            for col, val in enumerate(x):
                score *= self.likelihoods[c][col].get(val, 1e-6)
            scores[c] = score
        return max(scores, key=scores.get)

    def predict(self, X):
```

```
return [self.predict_one(x) for x in X]
```

১৬. প্রেডিকশন প্রসেস (Prediction Process)

Training শেষে (Prior ও Likelihood মুখস্থ হয়ে গেলে), নতুন স্যাম্পলের জন্য প্রেডিকশন তিন ধাপে হয়: (১) প্রতিটি ক্লাসের জন্য $\text{Score} = \text{Prior} \times \text{সব ফিচারের Likelihood}$ -এর গুণফল গণনা করা, (২) সবগুলো Score-কে Normalize করে প্রকৃত Posterior Probability-তে রূপান্তর করা (ঐচ্ছিক, শুধুমাত্র Classification-এর জন্য Normalization আবশ্যিক নয়), এবং (৩) সর্বোচ্চ Score/Probability যে ক্লাসের, সেটিকে চূড়ান্ত Prediction হিসেবে নেওয়া।

২২. সাক্ষাৎকারের প্রশ্ন (Interview Questions)

Naive Bayes প্রশ্ন ও উত্তর

প্রশ্ন ১: Naive Bayes-কে কেন 'Naive' বা 'সরল' বলা হয়?

উত্তর: Naive Bayes-কে 'Naive' বলা হয় কারণ এটি একটি শক্তিশালী কিন্তু সরল ধারণা ধরে নেয় যে, একটি ডেটাসেটের প্রতিটি ফিচার একে অপরের থেকে সম্পূর্ণ স্বাধীন (Conditional Independence)। বাস্তবে এমন স্বাধীনতা খুব কমই পাওয়া যায়, কিন্তু এই ধারণার কারণেই মডেলটি অত্যন্ত দ্রুত কাজ করতে পারে।

প্রশ্ন ২: Naive Bayes কেন টেক্সট ক্লাসিফিকেশন বা স্প্যাম ডিটেকশনের জন্য সবচেয়ে জনপ্রিয়?

উত্তর: টেক্সট ডেটায় সাধারণত ফিচারের সংখ্যা (Vocabulary size) অনেক বেশি থাকে। Naive Bayes ফিচারের স্বাধীনতার ধারণা ব্যবহার করে জটিল এই সমস্যাতে প্রতিটি ফিচারের জন্য আলাদা ওয়ান-ডাইমেনশনাল ডিস্ট্রিবিউশনে ভাগ করে ফেলে, যা কম্পিউটেশনাল খরচ কমায় এবং 'Curse of Dimensionality' সমস্যা সমাধানে সাহায্য করে।

প্রশ্ন ৩: 'Zero Frequency' সমস্যা বলতে কী বোঝায় এবং এটি কীভাবে সমাধান করা যায়?

উত্তর: যদি কোনো ফিচার বা শব্দ ট্রেনিং সেটে না থাকে, তবে তার সম্ভাবনা (Likelihood) ০ হয়ে যায়, যার ফলে পুরো গুণফল ০ হয়ে যায়। একে 'Zero Frequency' সমস্যা বলে। এটি সমাধানের জন্য Laplace Smoothing বা Lidstone Smoothing ব্যবহার করা হয়, যেখানে সম্ভাবনা গণনায় একটি ছোট মান (যেমন ১) যোগ করা হয় যাতে ফলাফল কখনো ০ না হয়।

প্রশ্ন ৪: লজিস্টিক রিগ্রেশন এবং Naive Bayes-এর মধ্যে মূল পার্থক্য কী?

উত্তর: লজিস্টিক রিগ্রেশন একটি 'Discriminative' মডেল, যা সরাসরি ক্লাসের সীমানা (Decision Boundary) শেখার চেষ্টা করে। অন্যদিকে Naive Bayes একটি 'Generative' মডেল, যা প্রতিটি ক্লাসের জন্য ডেটার ডিস্ট্রিবিউশন কেমন হবে তা শেখে এবং বেইজ থিওরেম ব্যবহার করে সম্ভাবনা নির্ণয় করে।

প্রশ্ন ৫: Naive Bayes কি আউটলায়ারের (Outliers) প্রতি সংবেদনশীল?

উত্তর: Naive Bayes আউটলায়ারের প্রতি লিনিয়ার রিগ্রেশন বা KNN-এর মতো অতটা সংবেদনশীল নয়। কারণ এটি পুরো ডেটাসেটের গড় (Mean) ও ভেদাঙ্ক (Variance) বা ফ্রিকোয়েন্সির ওপর ভিত্তি করে সম্ভাবনা নির্ণয় করে, কোনো নির্দিষ্ট দূরত্বের (Distance) ওপর নয়।

প্রশ্ন ৬: কখন Gaussian Naive Bayes বনাম Multinomial Naive Bayes ব্যবহার করবেন?

উত্তর: যখন ডেটার ফিচারগুলো কন্টিনিউয়াস (Continuous) এবং নরমাল ডিস্ট্রিবিউশন মেনে চলে, তখন GaussianNB ব্যবহার করা হয় (যেমন: ফুলের দৈর্ঘ্য/প্রস্থ)। আর যখন ডেটাগুলো ডিসক্রিট কাউন্ট (যেমন: শব্দের সংখ্যা) বা মাল্টিনোমিয়াল ডিস্ট্রিবিউশন মেনে চলে, তখন MultinomialNB ব্যবহার করা হয় (যেমন: টেক্সট ক্লাসিফিকেশন)।

২৩. সারাংশ (Summary)

- **মৌলিক ধারণা:** Naive Bayes অ্যালগরিদমগুলো সুপারভাইজড লার্নিংয়ের একটি সেট যা বেইজ থিওরেম এবং প্রতিটি ফিচারের মধ্যে শর্তাধীন স্বাধীনতার (conditional independence) সরল অনুমানের ওপর ভিত্তি করে কাজ করে।
- **গাণিতিক ভিত্তি:** এটি ক্লাস ভ্যারিয়েবল এবং ডিপেন্ডেন্ট ফিচার ভেক্টরের সম্পর্কের ওপর ভিত্তি করে কাজ করে। ইনপুট সাপেক্ষে ক্লাসটি ধ্রুবক হওয়ায় এটি Maximum A Posteriori (MAP) এস্টিমেশন ব্যবহার করে প্রেডিকশন রুল নির্ধারণ করে।
- **ব্যবহারিক সুবিধা:** এগুলোর অনুমান সরল হওয়া সত্ত্বেও বাস্তব জীবনে, বিশেষ করে ডকুমেন্ট ক্লাসিফিকেশন এবং স্প্যাম ফিল্টারিংয়ে অত্যন্ত কার্যকর। এগুলো ট্রেনিং ডেটার অল্প পরিমাণের ওপর ভিত্তি করেই প্যারামিটার নির্ধারণ করতে পারে এবং ক্লাসিফিকেশনের গতি অত্যন্ত দ্রুত।
- **ডাইমেনশনালিটি:** প্রতিটি ডিস্ট্রিবিউশনকে আলাদাভাবে এক-মাত্রিক হিসেবে অনুমান করার ফলে এগুলো 'কার্স অফ ডাইমেনশনালিটি' (curse of dimensionality) সংক্রান্ত সমস্যা কমাতে সাহায্য করে।
- **সীমাবদ্ধতা:** এটি একটি ভালো ক্লাসিফায়ার হলেও 'বাজে এস্টিমেটর' হিসেবে পরিচিত, তাই predict_proba থেকে প্রাপ্ত প্রোবাবিলিটি আউটপুটগুলোকে খুব গুরুত্বের সাথে নেওয়া উচিত নয়।

বিভিন্ন ধরন:

- **GaussianNB:** ফিচারগুলো গাউসিয়ান ডিস্ট্রিবিউশন অনুসরণ করে বলে ধরে নেয়।
- **MultinomialNB:** মাল্টিনোমিয়ালি ডিস্ট্রিবিউটেড ডেটার জন্য ব্যবহৃত হয়, যা টেক্সট ক্লাসিফিকেশনের একটি ক্লাসিক ভেরিয়েন্ট।

- **ComplementNB**: স্ট্যান্ডার্ড মাল্টিনোমিয়াল Naive Bayes-এর একটি অ্যাডাপ্টেশন যা ইমব্যালেন্সড ডেটাসেটের জন্য উপযোগী।
- **BernoulliNB**: বাইনারি বা বুলিয়ান ডেটার ক্ষেত্রে ব্যবহৃত হয়।
- **CategoricalNB**: ক্যাটাগরিক্যাল ডিস্ট্রিবিউশন অনুসরণকারী ডেটার ক্ষেত্রে প্রযোজ্য।
- **ইনক্রিমেন্টাল লার্নিং**: বড় স্কেলের ডেটার জন্য partial_fit মেথড ব্যবহার করা যায় যা পুরো ডেটাসেট মেমোরিতে ধরে না এমন সমস্যার সমাধান করে।

২০. পূর্ববর্তী অ্যালগরিদমের সাথে তুলনা (Comparison with Previous Algorithm)

GridSearchCV/KNN-এর সাথে Naive Bayes-এর তুলনা করলে একটি মৌলিক দার্শনিক পার্থক্য ধরা পড়ে:

দিক (Aspect)	KNN	Naive Bayes
মডেলের ধরন	Instance-Based (Lazy)	Probabilistic (Eager — Training-এই Prior/Likelihood গণনা হয়ে যায়)
Feature Independence	কোনো Assumption নেই	Feature-গুলো স্বাধীন ধরে নেয় (Naive Assumption)
Prediction Speed	ধীর (প্রতিটি Training পয়েন্টের সাথে তুলনা)	অত্যন্ত দ্রুত (শুধু সংরক্ষিত Probability ব্যবহার)
High-Dimensional Data	Curse of Dimensionality-তে ভোগে	হাজার হাজার Feature-এও ভালো কাজ করে (Text Classification)

২১. সাধারণ ভুলত্রুটি (Common Mistakes)

- Continuous ডেটার জন্য ভুল Naive Bayes Variant বেছে নেওয়া (যেমন Categorical ডেটার জন্য GaussianNB ব্যবহার করা)।
- Laplace/alpha Smoothing ব্যবহার না করা, যার ফলে অদেখা Feature Value এলে Probability সম্পূর্ণ শূন্য হয়ে যায় এবং পুরো Prediction নষ্ট হয়ে যায়।
- predict_proba()-এর আউটপুটকে নিখুঁত Calibrated Probability হিসেবে বিশ্বাস করা, যদিও Naive Bayes একটি ভালো Classifier হলেও একটি দুর্বল Probability Estimator।
- শক্তিশালী Feature Correlation থাকা সত্ত্বেও (যেমন Height ও Weight) Naive Independence Assumption উপেক্ষা করে ফলাফল ব্যাখ্যা করা।

সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)

Introduction

এই সেকশনে একটি ছোট Categorical ডেটাসেট ব্যবহার করে সম্পূর্ণ Naive Bayes Classifier হাতে-কলমে সমাধান করা হবে — Prior Probability, Likelihood, এবং Bayes' Theorem ব্যবহার করে Posterior Probability গণনা করে নতুন স্যাম্পলের ক্লাস প্রেডিক্ট করা হবে।

Dataset

Sample	Weather	Wind	Play (Label)
1	Sunny	Weak	No
2	Sunny	Strong	No
3	Rainy	Weak	Yes
4	Rainy	Strong	Yes
5	Sunny	Weak	Yes

Prior Probability

$P(\text{Class}) = (\text{Class-এর স্যাম্পল সংখ্যা}) / (\text{মোট স্যাম্পল সংখ্যা})$

মোট স্যাম্পল $n = 5$ | Yes = 3টি, No = 2টি।

$P(\text{Yes}) = 3/5 = 3/5 = 0.6000$

$P(\text{No}) = 2/5 = 2/5 = 0.4000$

Likelihood

$P(\text{Feature} = \text{value} \mid \text{Class}) = (\text{ঐ Class-এর মধ্যে যে সংখ্যক স্যাম্পলে Feature} = \text{value}) / (\text{ঐ Class-এর মোট স্যাম্পল})$

আমরা যে নতুন স্যাম্পলের ক্লাস প্রেডিক্ট করবো তা হলো: Weather = "Sunny", Wind = "Weak"।
তাই নিচের Likelihood-গুলো গণনা করতে হবে: $P(\text{Sunny}|\text{Yes})$, $P(\text{Weak}|\text{Yes})$, $P(\text{Sunny}|\text{No})$, $P(\text{Weak}|\text{No})$ ।

Class = Yes (মোট 3টি স্যাম্পল: [('Rainy', 'Weak'), ('Rainy', 'Strong'), ('Sunny', 'Weak')]):

$P(\text{Weather}=\text{Sunny} \mid \text{Yes}) = 1/3 = 1/3 = 0.3333$

$$P(\text{Wind=Weak} \mid \text{Yes}) = 2/3 = 2/3 = 0.6667$$

Class = No (মোট ২টি স্যাম্পল: [('Sunny', 'Weak'), ('Sunny', 'Strong')]):

$$P(\text{Weather=Sunny} \mid \text{No}) = 2/2 = 1 = 1.0000$$

$$P(\text{Wind=Weak} \mid \text{No}) = 1/2 = 1/2 = 0.5000$$

Probability Table (Likelihood Summary)

Class	P(Weather=Sunny Class)	P(Wind=Weak Class)
Yes	1/3 = 0.3333	2/3 = 0.6667
No	1 = 1.0000	1/2 = 0.5000

Conditional Probability ও Bayes' Theorem

$$P(\text{Class} \mid X) = [P(X \mid \text{Class}) \cdot P(\text{Class})] / P(X)$$

Naive Bayes-এর মূল অনুমান (Naive Assumption) হলো ফিচারগুলো একে অপরের থেকে স্বাধীন (Conditionally Independent), তাই:

$$P(X \mid \text{Class}) = P(\text{Weather} \mid \text{Class}) \cdot P(\text{Wind} \mid \text{Class})$$

যেহেতু $P(X)$ সব ক্লাসের জন্য একই ধ্রুবক (constant), তাই ক্লাস তুলনা করার সময় শুধু Numerator ($P(X|\text{Class}) \cdot P(\text{Class})$) গণনা করলেই চলে — একে বলা হয় Unnormalized Joint Probability বা "Score"।

Posterior Probability গণনা

$$\text{Score(Yes)} = P(\text{Yes}) \times P(\text{Sunny}|\text{Yes}) \times P(\text{Weak}|\text{Yes}) = 3/5 \times 1/3 \times 2/3 = 2/15 = 0.1333$$

$$\text{Score(No)} = P(\text{No}) \times P(\text{Sunny}|\text{No}) \times P(\text{Weak}|\text{No}) = 2/5 \times 1 \times 1/2 = 1/5 = 0.2000$$

$$\text{Normalize করার জন্য: } P(X) = \text{Score(Yes)} + \text{Score(No)} = 2/15 + 1/5 = 1/3 = 0.3333$$

$$P(\text{Yes} \mid X) = 2/15 / 1/3 = 2/5 = 0.4000$$

$$P(\text{No} \mid X) = 1/5 / 1/3 = 3/5 = 0.6000$$

Prediction

যেহেতু $P(\text{Yes} \mid X) = 0.4000 > P(\text{No} \mid X) = 0.6000$, তাই চূড়ান্ত Prediction = No।

Interpretation

লক্ষণীয় বিষয় হলো, যদিও Prior Probability Yes-এর পক্ষে বেশি ($P(\text{Yes})=0.6$ বনাম $P(\text{No})=0.4$), তবুও চূড়ান্ত Posterior Probability-তে No জিতে যায়। এর কারণ হলো Likelihood-এর প্রভাব — Training ডেটাতে যত No স্যাম্পল ছিল তার সবগুলোই Sunny ($P(\text{Sunny}|\text{No})=1.0$), যেখানে Yes স্যাম্পলগুলোর মধ্যে মাত্র $1/3$ Sunny ছিল ($P(\text{Sunny}|\text{Yes})=0.33$)। এটি দেখায় কীভাবে Naive Bayes শুধু Prior নয়, বরং প্রতিটি Feature-এর Class-ভিত্তিক বিতরণকেও ওজন দিয়ে চূড়ান্ত সিদ্ধান্তে পৌঁছায়।

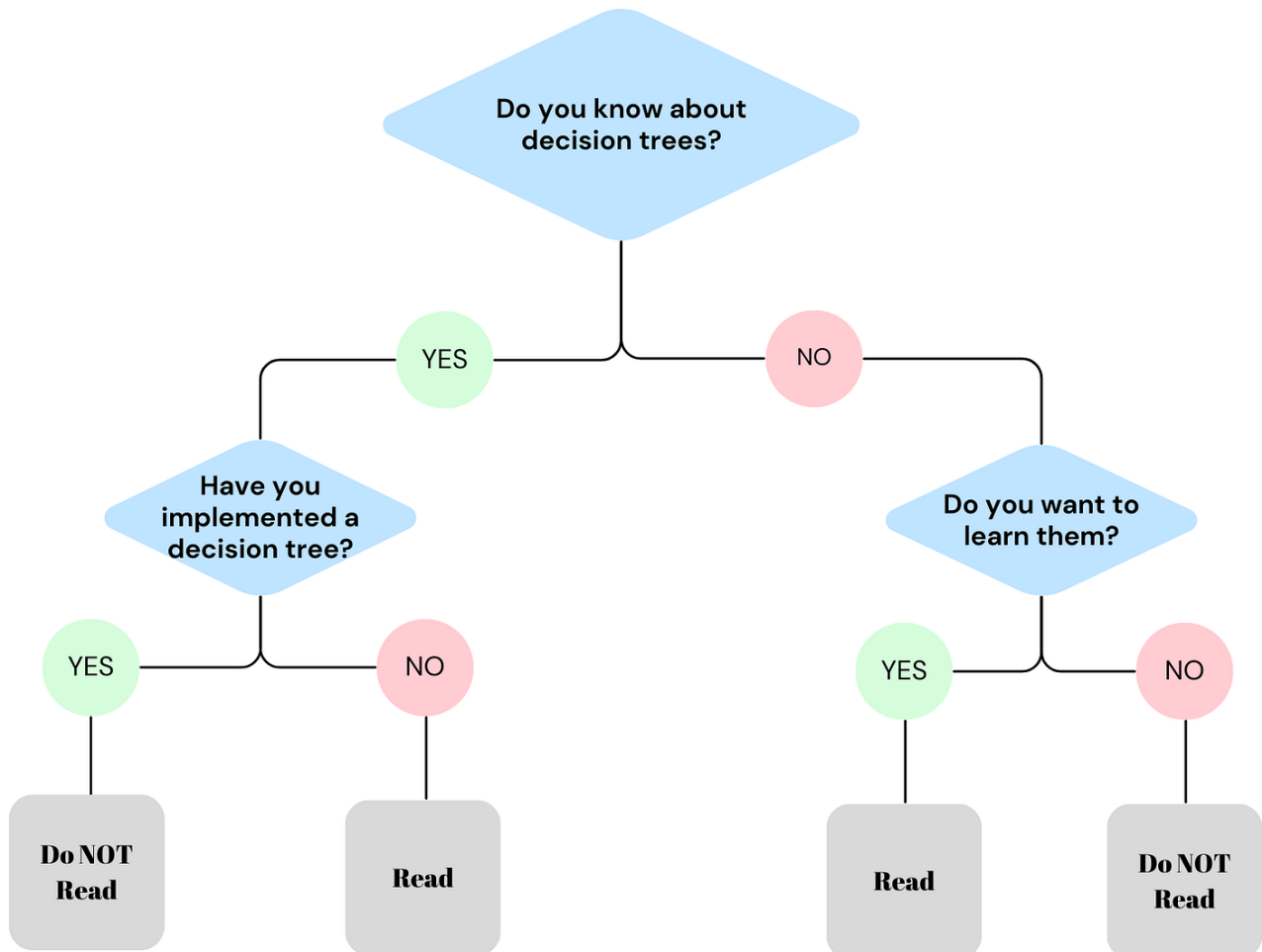
Decision Tree

১. ভূমিকা (Introduction)

Decision Tree (ডিসিশন ট্রি) হলো মেশিন লার্নিংয়ের একটি অত্যন্ত জনপ্রিয়, সহজ এবং শক্তিশালী Supervised Learning Algorithm, যা ক্লাসিফিকেশন (Classification) এবং রিগ্রেশন (Regression) উভয় ধরনের সমস্যার সমাধানে ব্যবহৃত হয়।

সহজ কথায়, এটি দেখতে একটি উল্টানো গাছের মতো, যার ওপরের দিকে থাকে মূল রুট বা কাণ্ড এবং নিচের দিকে ডালপালা বেয়ে চূড়ান্ত পাতা বা সিদ্ধান্তে পৌঁছানো যায়।

DECISION TREE



২. কেন এই অ্যালগরিদম তৈরি হয়েছিল (Why This Algorithm Was Developed)

প্রাথমিক দিকের Statistical Model-গুলো (যেমন Logistic Regression) একটি Linear Decision Boundary অনুমান করত, যা বাস্তব জীবনের অনেক জটিল, শর্তসাপেক্ষ (Conditional) সিদ্ধান্ত-গ্রহণ প্রক্রিয়া প্রকাশ করতে পারত না — যেমন "যদি আয় বেশি হয় এবং শিক্ষার্থী না হয়, তাহলে..."। ১৯৮০-এর দশকে Breiman, Quinlan-দের মতো গবেষকরা এমন একটি Model তৈরি করেন যা মানুষের প্রাকৃতিক সিদ্ধান্ত-গ্রহণ প্রক্রিয়া (একের পর এক প্রশ্নের মাধ্যমে) সরাসরি অনুকরণ করে — এটাই Decision Tree, যা কোনো Linear Assumption ছাড়াই জটিল, Non-Linear সম্পর্ক ধারণ করতে সক্ষম।

৩. বাস্তব জীবনের প্রেরণা (Real-life Motivation)

ডাক্তাররা প্রায়ই রোগ নির্ণয়ের সময় একটি ধারাবাহিক প্রশ্নমালা অনুসরণ করেন — "জ্বর আছে কি? থাকলে, কাশি আছে কি? থাকলে, শ্বাসকষ্ট আছে কি?" — প্রতিটি উত্তর পরবর্তী প্রশ্ন নির্ধারণ করে, যতক্ষণ না একটি চূড়ান্ত সিদ্ধান্তে পৌঁছানো যায়। Decision Tree ঠিক এই প্রাকৃতিক প্রক্রিয়াটিকে গাণিতিকভাবে প্রয়োগ করে, যার ফলে এটি মানুষের কাছে সবচেয়ে সহজে ব্যাখ্যাযোগ্য Machine Learning Model-গুলোর একটি।

৪. স্বভাৱ (Intuition)

উদাহরণ (Intuition)

ধরুন, ছুটির দিনে আপনি বন্ধুদের সাথে বাইরে খেলতে যাবেন কি না, তা নিয়ে সিদ্ধান্ত নিতে চান। আপনার মনের ভেতর ঠিক নিচের মতো একটি চিন্তা কাজ করবে:

১. **প্রথম প্রশ্ন (রুট):** বাইরে কি আজ বৃষ্টি হচ্ছে?
 - যদি হ্যাঁ হয় → আপনি বাইরে যাবেন না (চূড়ান্ত সিদ্ধান্ত)।
 - যদি না হয় → আপনি পরবর্তী প্রশ্নে যাবেন।
২. **দ্বিতীয় প্রশ্ন:** বাইরে কি প্রচণ্ড গরম বা রোদ?
 - যদি হ্যাঁ হয় → আপনি বাইরে যাবেন না।
 - যদি না হয় → আপনি বাইরে খেলতে যাবেন (চূড়ান্ত সিদ্ধান্ত)।

মেশিন লার্নিংয়ে ডিসিশন ট্রি ঠিক এইভাবেই ডেটাসেটের ফিচারগুলোর ওপর ভিত্তি করে একটার পর একটা "হ্যাঁ/না" (Yes/No) বা "সত্য/মিথ্যা" (True/False) প্রশ্ন তৈরি করে ডেটাকে ছোট ছোট ভাগে ভাগ করে ফেলে।

৫. কার্যপ্রণালী (Working Principle)

ডিসিশন ট্রি-এর মূল গাঠনিক উপাদান (Core Structure)

একটি ডিসিশন ট্রি মূলত ৩টি প্রধান অংশ নিয়ে গঠিত হয়:

- Root Node (মূল নোড):** এটি গাছের একদম শুরুর বা শীর্ষের নোড। পুরো ডেটাসেটটি এখান থেকেই প্রথম বিভাজন (Split) শুরু করে। আমাদের উদাহরণে "বৃষ্টি হচ্ছে কি না" ছিল রুট নোড।
- Internal/Decision Node (অভ্যন্তরীণ নোড):** এগুলো হলো মাঝখানের ডালপালা বা প্রশ্নগুলোর নোড, যেখানে ডেটার কোনো একটি বৈশিষ্ট্যের ওপর ভিত্তি করে নতুন সিদ্ধান্ত নেওয়া হয় এবং ট্রি-টি আরও নিচে নেমে যায়।
- Leaf Node (পাতা বা চূড়ান্ত নোড):** এগুলো হলো গাছের একদম শেষ প্রান্তের নোড। এর নিচে আর কোনো ডালপালা গজায় না। এটিই মূলত আমাদের চূড়ান্ত প্রেডিকশন বা আউটপুট (যেমন: বাইরে যাব নাকি যাব না) নির্দেশ করে।

কেন এটি এত জনপ্রিয়? (Advantages)

- সহজে বোঝা যায় (Interpretable):** এটি কীভাবে সিদ্ধান্ত নিচ্ছে তা মানুষ খুব সহজেই বুঝতে পারে (অনেকটা হোয়াইট-বক্স মডেলের মতো)।
- ডেটা প্রিপারেশন কম লাগে:** এর জন্য খুব বেশি ফিচার স্কেলিং (যেমন: StandardScaler বা MinMaxScaler) করার প্রয়োজন হয় না, কারণ এটি ডেটার মানের আকারের চেয়ে থ্রেশহোল্ড স্প্লিটের ওপর বেশি নির্ভর করে।
- Non-linear ডেটায় দারুণ কাজ করে:** ডেটার ভেতরে যদি সরলরৈখিক সম্পর্ক না থেকে জটিল বা আঁকাবাঁকা সম্পর্ক থাকে, তবুও এটি চমৎকার প্যাটার্ন খুঁজে বের করতে পারে।

তবে ডিসিশন ট্রি-এর একটি বড় সীমাবদ্ধতা হলো এটি খুব দ্রুত ডেটা মুখস্থ করে ফেলে, যাকে মেশিন লার্নিংয়ের ভাষায় Overfitting (High Variance) বলা হয়। এই ওভারফিটিং দূর করার জন্য

পরবর্তীতে গাছের ডালপালা কেটে ছোট করা হয় (Pruning) অথবা একাধিক ডিসিশন ট্রি একসাথে মিলিয়ে Random Forest-এর মতো উন্নত অ্যালগরিদম ব্যবহার করা হয়।

ডিসিশন ট্রি (Decision Tree) Classification (ক্লাসিফিকেশন) এবং Regression (রিগ্রেশন) উভয় সমস্যার জন্যই ব্যবহার করা যায়। টার্গেট ভ্যারিয়েবলের বা আউটপুটের ধরনের ওপর ভিত্তি করে এটিকে দুই ভাগে ভাগ করা হয়:

১. Decision Tree Classifier (শ্রেণিবিভাগকারী)

- কখন ব্যবহার হয়: যখন আপনার আউটপুট বা টার্গেট ভ্যারিয়েবলটি কোনো ক্যাটাগরি, শ্রেণি বা ডিসক্রিট লেবেল (Discrete Label) হয়।
- কীভাবে প্রেডিক্ট করে: এটি গাছের শেষ প্রান্তে বা লিফ নোডে (Leaf Node) থাকা স্যাম্পলগুলোর মেজরিটি ভোটিং (Majority Voting) বা সংখ্যাগরিষ্ঠের ভোটের ভিত্তিতে ফাইনাল ক্লাস প্রেডিক্ট করে।
- বাস্তব উদাহরণ: কোনো ইমেইল 'Spam' নাকি 'Not Spam'।
 - একজন যাত্রী টাইটানিকে 'Survived' (বঁচে গেছেন) নাকি 'Not Survived' (মারা গেছেন)।

২. Decision Tree Regressor (মান অনুমানকারী)

- কখন ব্যবহার হয়: যখন আপনার আউটপুট বা টার্গেট ভ্যারিয়েবলটি কোনো কন্টিনিউয়াস বা ধারাবাহিক সংখ্যা (Continuous Numeric Value) হয়।
- কীভাবে প্রেডিক্ট করে: এটি লিফ নোডে (Leaf Node) পৌঁছানো ডেটা পয়েন্টগুলোর মানের গড় (Mean) বা মিডিয়ান (Median) হিসাব করে চূড়ান্ত সংখ্যাটি প্রেডিক্ট করে।
- বাস্তব উদাহরণ:
 - একটি বাড়ির আয়তন ও লোকেশন দেখে তার 'দাম' বা 'মূল্য' (Price) কত হবে তা অনুমান করা।
 - আবহাওয়ার ডেটা বিশ্লেষণ করে আগামীকালের 'তাপমাত্রা' (Temperature) কত ডিগ্রি হবে তা প্রেডিক্ট করা।

৬. গাণিতিক ভিত্তি (Mathematical Backend)

৭. সূত্র (Formula)

The Mathematical Split

গাছটি কীভাবে ডালপালা মেলবে বা ডেটাকে কোন পয়েন্টে কাটবে (Split), তার গাণিতিক পরিমাপ দুই ক্ষেত্রে ভিন্ন:

- **Classifier-এর ক্ষেত্রে:** ডেটার বিশুদ্ধতা পরিমাপ করার জন্য Gini Impurity (জিনি ইম্পিউরিটি) অথবা Information Gain / Entropy (এনট্রপি) ব্যবহার করা হয়। লক্ষ্য থাকে প্রতিটা স্প্লিটে ডেটাকে যতটা সম্ভব নিখুঁতভাবে আলাদা করা।
- **Regressor-এর ক্ষেত্রে:** ডেটার বিস্তৃতি বা ভুল পরিমাপের জন্য Variance Reduction (ভ্যারিয়েন্স রিডাকশন) অথবা Mean Squared Error (MSE) ব্যবহার করা হয়। লক্ষ্য থাকে এমনভাবে ডেটা ভাগ করা যাতে প্রতিটা ভাগের ভেতরের সংখ্যাগুলোর নিজেদের মধ্যকার দূরত্ব বা এরর সর্বনিম্ন হয়।

কোন ক্ষেত্রে সবচেয়ে বেশি ব্যবহার হয় এবং কেন?

উত্তর: Classification

- **কারণ ১ (সহজ সিদ্ধান্ত):** বাস্তব জীবনের বেশিরভাগ ব্যবসায়িক সিদ্ধান্ত বা শ্রেণিবিভাগ "হ্যাঁ/না" (Yes/No) বা নির্দিষ্ট কিছু ক্যাটাগরির ওপর ভিত্তি করে হয় (যেমন: কাস্টমার প্রোডাক্ট কিনবে নাকি কিনবে না)। ডিসিশন ট্রি-এর "IF-ELSE" লজিক এই ধরনের সমস্যা খুব নিখুঁতভাবে সমাধান করতে পারে।
- **রিগ্রেশনে সীমাবদ্ধতা:** রিগ্রেশনের ক্ষেত্রে (যেমন: বাড়ির দাম অনুমান করা) ডিসিশন ট্রি ডেটাকে ছোট ছোট বক্সে ভাগ করে প্রতিটা বক্সের গড় মান দেয়। এর ফলে আউটপুটের গ্রাফটি মসৃণ (Smooth) না হয়ে সিঁড়ির মতো ধাপযুক্ত (Step-like) হয়, যা অনেক সময় নিখুঁত সংখ্যাচক প্রেডিকশন দিতে পারে না। তাই রিগ্রেশনের জন্য মানুষ লিনিয়ার রিগ্রেশন বা অন্যান্য অ্যালগরিদম বেশি পছন্দ করে।

১৯. প্রয়োগক্ষেত্র (Applications)

কোন কোন সেক্টরে সবচেয়ে বেশি ব্যবহার হয়?

ডিসিশন ট্রি এবং এর উন্নত রূপ (যেমন: Random Forest, XGBoost) মূলত নিচের সেক্টরগুলোতে ব্যাপকভাবে ব্যবহৃত হয়:

১. ব্যাংকিং ও ফাইন্যান্স সেক্টর (Banking & Finance)

- **ক্রেডিট স্কোরিং (Credit Scoring):** একজন কাস্টমারকে লোন বা ক্রেডিট কার্ড দেওয়া নিরাপদ কি না (ল্যোন ডিফল্টার হবে কি না), তা ক্লাসিফাই করতে।
- **জালিয়াতি শনাক্তকরণ (Fraud Detection):** কোনো ক্রেডিট কার্ডের ট্রানজেকশন আসল নাকি জালিয়াতি (*Fraud*), তা তাৎক্ষণিকভাবে ধরে ফেলতে।

২. স্বাস্থ্যসেবা ও চিকিৎসা বিজ্ঞান (Healthcare & Medicine)

- **রোগ নির্ণয় (Disease Diagnosis):** রোগীর লক্ষণ (রক্তচাপ, সুগার লেভেল, বয়স) দেখে তার টিউমারটি ম্যালিগন্যান্ট (ক্যান্সার) নাকি বিনাইন (সাধারণ), তা শ্রেণিবদ্ধ করতে।
- **ঝুঁকি মূল্যায়ন (Risk Assessment):** রোগীর মেডিকেল হিস্ট্রি দেখে ভবিষ্যতে তার হার্ট অ্যাটাকের ঝুঁকি কতটা, তা বের করতে।

৩. ই-কমার্স ও মার্কেটিং (E-commerce & Marketing)

- **কাস্টমার চার্ন প্রেডিকশন (Customer Churn):** কোনো কাস্টমার আপনার সার্ভিস বা সাবস্ক্রিপশন ছেড়ে চলে যাবে কি না, তা আগে থেকে অনুমান করতে।
- **টার্গেটেড মার্কেটিং:** কাস্টমারের বয়স, লিঙ্গ ও ব্রাউজিং হিস্ট্রি দেখে সে কোন ক্যাটাগরির প্রোডাক্ট কিনতে পারে, তা ক্লাসিফাই করে সঠিক অফার পাঠানো।

৪. মানব সম্পদ বিভাগ (Human Resources - HR)

- **প্রার্থী বাছাই (Candidate Screening):** শত শত জীবনবৃত্তান্ত (CV) থেকে কোন প্রার্থী ইন্টারভিউয়ের জন্য যোগ্য (Shortlisted) আর কে যোগ্য নয়, তা ডিসিশন ট্রি ফিল্টারিংয়ের মাধ্যমে দ্রুত আলাদা করতে।

২০. পূর্ববর্তী অ্যালগরিদমের সাথে তুলনা (Comparison with Previous Algorithm)

Why we need Decision Tree instead of Linear Regression, Logistic Regression and KNN?

বাস্তব প্রজেক্টের কাজের অভিজ্ঞতার ওপর ভিত্তি করে ডিসিশন ট্রি-এর প্রয়োজনীয়তাকে ৪টি প্রধান সুবিধায় ভাগ করা যায়:

১. জটিল এবং নন-লিনিয়ার ডেটা হ্যান্ডলিং (Handling Complex Non-linear Relations)

- **লিনিয়ার ও লজিস্টিক রিগ্রেশনের সীমাবদ্ধতা:** এই দুটি মডেল ধরে নেয় যে ইনপুট ফিচার এবং টার্গেট ভ্যারিয়েবলের মধ্যে একটি সরলরৈখিক সম্পর্ক (Linear Relationship) বিদ্যমান। কিন্তু বাস্তব দুনিয়ার ডেটা (যেমন: টাইটানিক ডেটাসেট বা বাড়ির দাম) সাধারণত অত্যন্ত আঁকাবাঁকা বা নন-লিনিয়ার হয়। রিগ্রেশন মডেল দিয়ে জোর করে এই ডেটা ফিট করতে গেলে মডেলটির Bias হাই হয়ে যায় (Underfitting ঘটে)।
- **ডিসিশন ট্রি-এর সুবিধা:** ডিসিশন ট্রি ডেটার মধ্যে কোনো সরলরৈখিক সম্পর্ক খোঁজে না। এটি ডেটাসেটকে টুকরো টুকরো বক্সে ভাগ (Feature Space Partitioning) করে কাজ করে। ফলে ডেটার সম্পর্ক যত জটিল বা আঁকাবাঁকাই হোক না কেন, এটি খুব সহজেই সেই প্যাটার্ন ক্যাপচার করতে পারে।

২. ফিচার স্কেলিং এবং আউটলায়ার থেকে মুক্তি (No Need for Feature Scaling & Outlier Robustness)

- **KNN এবং রিগ্রেশনের সীমাবদ্ধতা:** KNN সম্পূর্ণভাবে দূরত্বের (Distance Metrics) (যেমন: Euclidean Distance) ওপর ভিত্তি করে কাজ করে। ফলে কোনো ফিচারের স্কেল যদি বড় হয় (যেমন: Salary ০-১,০০০,০০০ বনাম Age ০-১০০), তবে বড় ফিচারটি পুরো দূরত্বকে ডমিনেন্ট করে। তাই KNN এবং রিগ্রেশন ব্যবহারের আগে StandardScaler বা MinMaxScaler দিয়ে ডেটা স্কেল করা বাধ্যতামূলক। এছাড়া এগুলো আউটলায়ারের প্রতি অত্যন্ত সংবেদনশীল (Sensitive)।
- **ডিসিশন ট্রি-এর সুবিধা:** ডিসিশন ট্রি একটি Scale-Independent অ্যালগরিদম। এটি দূরত্বের হিসাব করে না, বরং প্রতিটা ফিচারের জন্য কাস্টম থ্রেশহোল্ড (যেমন: Age > 30 কিনা) চেক করে। তাই আপনার ডেটাতে ফিচার স্কেলিং না করলেও এটি নিখুঁত কাজ করে এবং চরম আউটলায়ার থাকলেও গাছের স্প্লিটিং মেকানিজমে কোনো নেতিবাচক প্রভাব পড়ে না।

৩. চরম ব্যাখ্যাযোগ্যতা বা হোয়াইট-বক্স মেকানিজম (Extreme Interpretability)

- **লজিস্টিক রিগ্রেশন ও KNN-এর সীমাবদ্ধতা:** KNN-কে "Lazy Learner" বলা হয়, কারণ এটি ব্যাকগ্রাউন্ডে কোনো গাণিতিক মডেল বা সূত্র তৈরি করে না, শুধু ডেটা মেমোরিতে জমিয়ে রাখে। বড় প্রজেক্টে বা ইন্টারভিউতে মডেল কীভাবে প্রেডিক্ট করছে তা KNN বা লজিস্টিক রিগ্রেশনের জটিল সহগ (Coefficients) থেকে সাধারণ মানুষের পক্ষে বোঝা কঠিন।

- **ডিসিশন ট্রি-এর সুবিধা:** ডিসিশন ট্রি-কে White-Box Model বলা হয়। এটি একটি স্পষ্ট ভিজ্যুয়াল ফ্লোচার্ট বা "IF-ELSE" নিয়মের সেট তৈরি করে সিদ্ধান্ত নেয়। কোনো ব্যাংকের ম্যানেজার বা ডাক্তার খুব সহজেই গাছের ডালপালা দেখে বুঝতে পারেন যে কেন একজন কাস্টমারকে লোন দেওয়া হলো না কিংবা কেন একজন রোগীকে উচ্চ ঝুঁকিপূর্ণ বলা হলো।

8. ক্যাটাগরিক্যাল ফিচারের স্বাভাবিক হ্যান্ডলিং (Natural Handling of Categorical Data)

- **অন্যান্য মডেলের সীমাবদ্ধতা:** লিনিয়ার/লজিস্টিক রিগ্রেশন এবং KNN-এ ক্যাটাগরিক্যাল ডেটা (যেমন: Sex বা Embarked) সরাসরি পাস করলে মডেল এরর দেয়। এদের জন্য ওয়ান-হট এনকোডিং (One-Hot Encoding) করা বাধ্যতামূলক, যা হাই-কার্ডিনালিটি ডেটার ক্ষেত্রে কলামের সংখ্যা বহুগুণ বাড়িয়ে দেয় (Column Explosion ঘটে)।
- **ডিসিশন ট্রি-এর সুবিধা:** ডিসিশন ট্রি ক্যাটাগরিক্যাল এবং নিউমেরিক্যাল উভয় ধরনের ডেটা একসাথে খুব স্বাভাবিকভাবে সামলাতে পারে। এটি সরাসরি টেক্সট বা ক্যাটাগরির ওপর ভিত্তি করে ডালপালা মেলতে পারে (যেমন: Sex == 'male' হলে বামে যাও, female হলে ডানে যাও), ফলে অতিরিক্ত ওয়ান-হট এনকোডিংয়ের ওপর এর নির্ভরশীলতা অনেক কম।

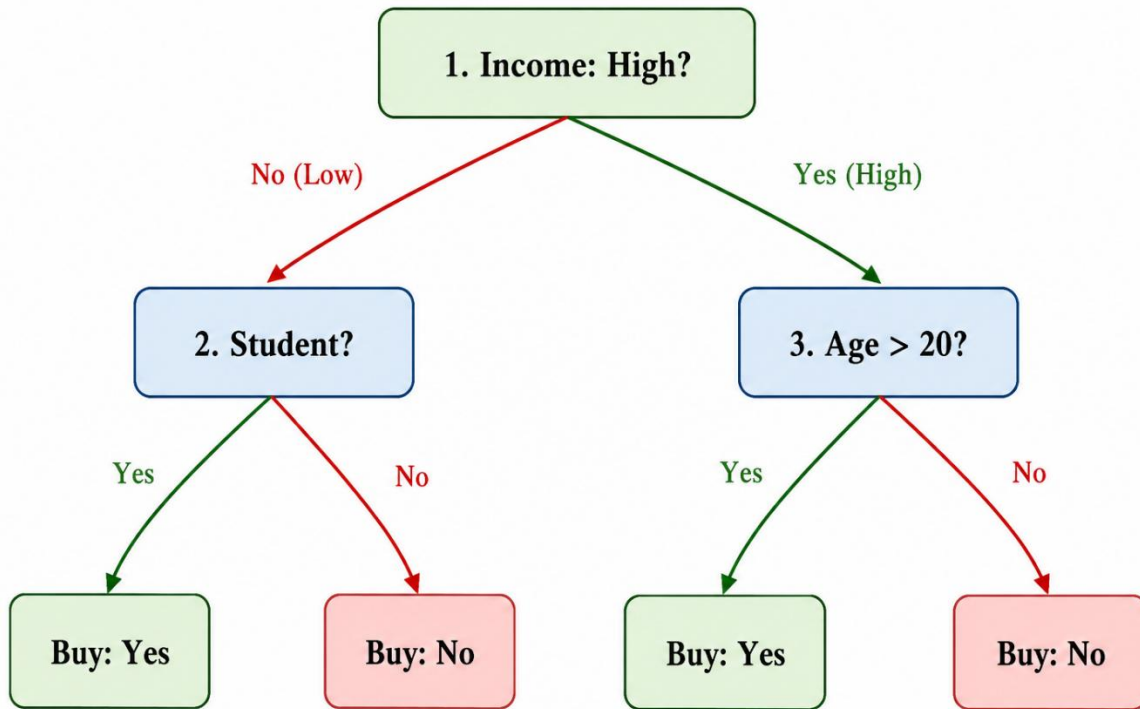
Example over a small dataset:

Customer	Age	Income	Student	Buy (Target)
C1	22	Low	Yes	Yes
C2	45	Low	No	No
C3	28	High	No	Yes
C4	35	High	Yes	Yes
C5	19	High	No	No

ডিসিশন ট্রি-এর লজিক্যাল গঠন (The Tree Structure)

আমরা যদি এই ডেটার ওপর একটি ডিসিশন ট্রি ট্রেন করি, তবে সে ব্যাকঅ্যান্ডে ইনফরমেশন গেইন (Information Gain) হিসাব করে নিচের মতো একটি গাছের কাঠামো তৈরি করবে:

1. **রুট নোড (Income):** আয় কি High?
 - যদি **না (Low)** হয় → সে ২ নম্বর প্রশ্ন বা সাব-নোডে যাবে।
 - যদি **হ্যাঁ (High)** হয় → সে ৩ নম্বর প্রশ্ন বা সাব-নোডে যাবে।
2. **সাব-নোড ১ (Student):** সে কি একজন শিক্ষার্থী?
 - হ্যাঁ → **Buy: Yes**
 - না → **Buy: No**
3. **সাব-নোড ২ (Age):** বয়স কি ২০-এর ওপরে?
 - হ্যাঁ → **Buy: Yes**
 - না → **Buy: No**



ডিসিশন ট্রির ব্যাকগ্রাউন্ডে গাছ তৈরির গাণিতিক প্রক্রিয়া

১. মূল লক্ষ্য: বিশুদ্ধতা (Purity) অর্জন

ডিসিশন ট্রি (Decision Tree) অ্যালগরিদমের প্রধান উদ্দেশ্য হলো ডেটাসেটকে এমনভাবে ধারাবাহিকভাবে বিভক্ত করা, যাতে প্রতিটি চূড়ান্ত নোডে (Leaf Node) একই শ্রেণির (Class) ডেটা থাকে। অর্থাৎ, প্রতিটি বিভাজনের মাধ্যমে ডেটার বিশৃঙ্খলা (Impurity) কমিয়ে আনা এবং যতটা সম্ভব বিশুদ্ধ (Pure) নোড তৈরি করাই এই অ্যালগরিদমের মূল লক্ষ্য।

অবিশুদ্ধ নোড (Impure Node)

যদি কোনো নোডে একাধিক শ্রেণির ডেটা একসঙ্গে উপস্থিত থাকে, তবে সেটিকে অবিশুদ্ধ নোড বলা হয়। উদাহরণস্বরূপ, একটি নোডে যদি সমান সংখ্যক **Yes** এবং **No** শ্রেণির ডেটা থাকে, তবে সেই নোডে অনিশ্চয়তা সর্বাধিক থাকে। ফলে শুধুমাত্র সেই নোডের তথ্যের ভিত্তিতে সঠিক সিদ্ধান্ত গ্রহণ করা সম্ভব হয় না।

বিশুদ্ধ নোড (Pure Node)

অন্যদিকে, যদি কোনো নোডে কেবলমাত্র একটি শ্রেণির ডেটা উপস্থিত থাকে, যেমন সবগুলোই **Yes** অথবা সবগুলোই **No**, তাহলে সেই নোডকে সম্পূর্ণ বিশুদ্ধ (Pure Node) বলা হয়। এ ধরনের

নোডে আর কোনো বিভাজনের প্রয়োজন হয় না এবং এটিই ডিসিশন ট্রির চূড়ান্ত পাতা বা **Leaf Node** হিসেবে বিবেচিত হয়।

২. অবিশুদ্ধতা পরিমাপের গাণিতিক পদ্ধতি (Mathematical Metrics)

৮. সূত্রের ডেরিভেশন (Formula Derivation)

ডিসিশন ট্রি কোন ফিচারের ভিত্তিতে ডেটা ভাগ করবে তা নির্ধারণ করার জন্য বিভিন্ন গাণিতিক পরিমাপক (Metric) ব্যবহার করে। এই পরিমাপকগুলোর মাধ্যমে প্রতিটি সম্ভাব্য বিভাজনের গুণগত মান মূল্যায়ন করা হয়। সর্বাধিক ব্যবহৃত তিনটি পদ্ধতি হলো:

- Entropy
- Information Gain
- Gini Impurity

২.১ Entropy (এনট্রপি): বিশৃঙ্খলার পরিমাপ

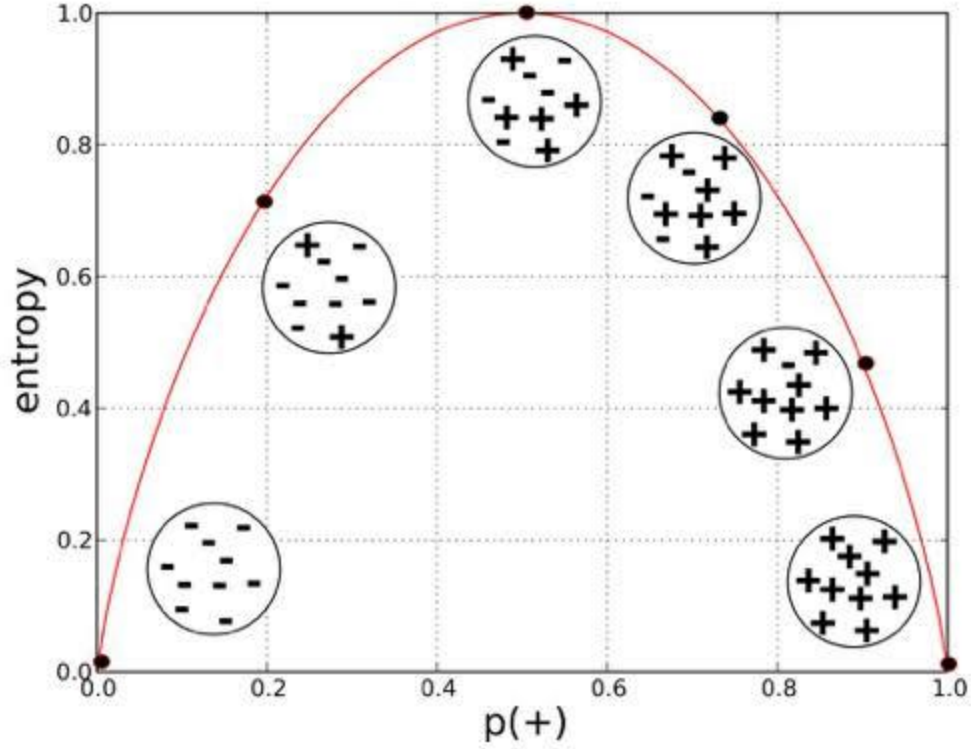
Entropy হলো কোনো নোডের অনিশ্চয়তা বা বিশৃঙ্খলার (Disorder) পরিমাপ। একটি নোডে বিভিন্ন শ্রেণির ডেটা যত বেশি মিশ্রিত থাকবে, এনট্রপির মান তত বেশি হবে। বিপরীতভাবে, নোড যত বেশি বিশুদ্ধ হবে, এনট্রপির মান তত কম হবে।

এর গাণিতিক সূত্র হলো,

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

এখানে,

- S হলো বর্তমান নোডের ডেটাসেট।
- c হলো মোট শ্রেণির সংখ্যা।
- p_i হলো i -তম শ্রেণির ডেটার অনুপাত (Probability)।



এনট্রপির মানের ব্যাখ্যা

- যদি নোডে ৫০% Yes এবং ৫০% No থাকে, তাহলে অনিশ্চয়তা সর্বাধিক হয় এবং Entropy-এর মান হয় 1.0।
- যদি নোডে ১০০% একই শ্রেণির ডেটা থাকে, তাহলে কোনো অনিশ্চয়তা থাকে না এবং Entropy-এর মান হয় 0।

অতএব, ডিসিশন ট্রির লক্ষ্য হলো প্রতিটি বিভাজনের মাধ্যমে Entropy-এর মান ক্রমান্বয়ে কমিয়ে আনা।

২.২ Information Gain (ইনফরমেশন গেইন): তথ্য অর্জনের পরিমাণ

Entropy শুধুমাত্র বর্তমান নোডের বিশৃঙ্খলা পরিমাপ করে। কিন্তু কোন ফিচার ব্যবহার করলে সেই বিশৃঙ্খলা সবচেয়ে বেশি কমবে, তা নির্ধারণ করার জন্য Information Gain ব্যবহার করা হয়।

প্রথমে প্রতিটি সম্ভাব্য ফিচার ব্যবহার করে ডেটাকে বিভক্ত করা হয়। এরপর বিভাজনের আগে ও পরে Entropy-এর পরিবর্তন নির্ণয় করা হয়। এই পরিবর্তনের পরিমাণই Information Gain নামে পরিচিত।

এর গাণিতিক সূত্র হলো,

$$Information\ Gain = Entropy(Parent) - Weighted\ Average\ Entropy(Children)$$

এখানে,

- **Parent Entropy** হলো বিভাজনের পূর্বের নোডের Entropy।
- **Children Entropy** হলো বিভাজনের পর সৃষ্ট সকল সাব-নোডের ওজনযুক্ত (Weighted) গড় Entropy।

Information Gain-এর সিদ্ধান্ত

ডিসিশন ট্রি প্রতিটি ফিচারের জন্য Information Gain গণনা করে।

যে ফিচারের Information Gain সর্বাধিক হয়, অর্থাৎ যে ফিচারটি ডেটার বিশৃঙ্খলা সবচেয়ে বেশি কমাতে সক্ষম হয়, সেই ফিচারকেই পরবর্তী Decision Node অথবা Root Node হিসেবে নির্বাচন করা হয়।

২.৩ Gini Impurity (জিনি ইম্পিউরিটি): দ্রুত বিশুদ্ধতা পরিমাপ

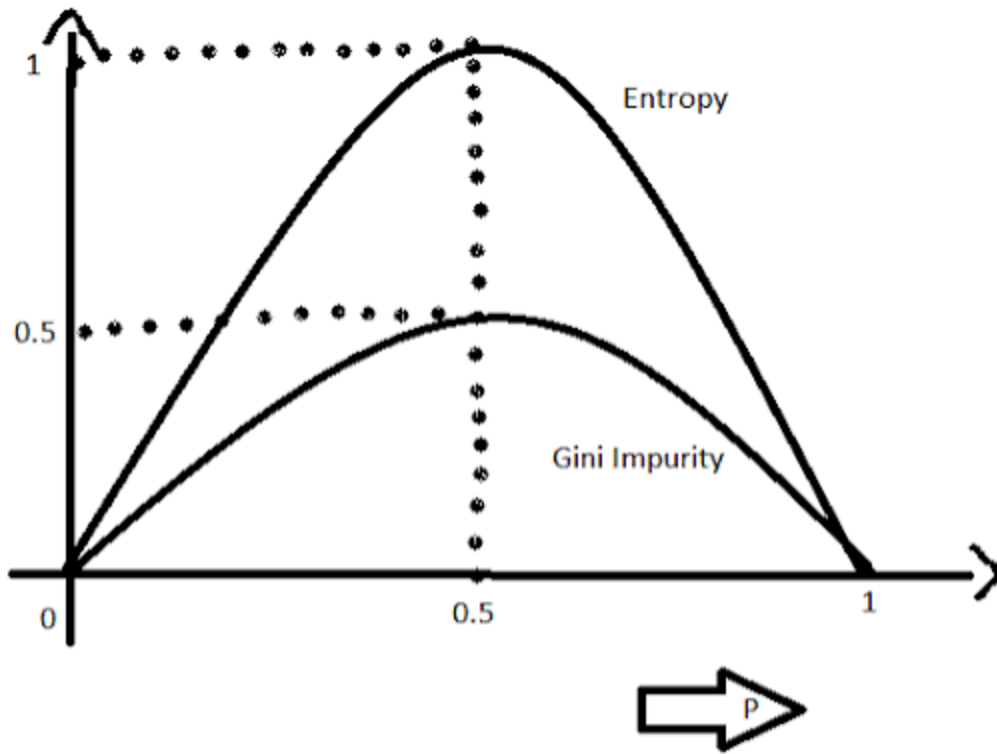
Entropy গণনার সময় লগারিদমিক ($\log_2$) অপারেশন ব্যবহৃত হয়, যা তুলনামূলকভাবে বেশি কম্পিউটেশনাল সময় গ্রহণ করে। এই কারণে বাস্তব প্রয়োগে অনেক লাইব্রেরি, বিশেষ করে **Scikit-Learn**, ডিফল্টভাবে Entropy-এর পরিবর্তে **Gini Impurity** ব্যবহার করে।

Gini Impurity-এর সূত্র হলো,

$$Gini = 1 - \sum_{i=1}^c (p_i)^2$$

এখানে,

- p_i হলো প্রতিটি শ্রেণির Probability।



$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

$$Gini(E) = 1 - \sum_{j=1}^c p_j^2$$

Gini Impurity-এর মানের ব্যাখ্যা

- সম্পূর্ণ বিশুদ্ধ নোডের Gini মান 0।
- দুটি শ্রেণির সমান অনুপাত থাকলে Gini-এর মান 0.5, যা সর্বোচ্চ অবিশুদ্ধতার নির্দেশক।

ডিসিশন ট্রি সর্বদা এমন বিভাজন নির্বাচন করে যাতে সাব-নোডগুলোর Gini Impurity যত সম্ভব কম হয়।

৩. ডিসিশন ট্রির ব্যাকএন্ড ওয়ার্কফ্লো

৯. প্রতিটি চিহ্নের অর্থ (Meaning of Every Symbol)

Symbol	Meaning (অর্থ)
S	বর্তমান নোডের ডেটাসেট
c	মোট শ্রেণি (Class) সংখ্যা
p_i	i-তম শ্রেণির ডেটার অনুপাত (Probability)
Entropy(S)	নোড S-এর অনিশ্চয়তা/বিশৃঙ্খলার পরিমাপ
Gini(S)	নোড S-এর অবিশুদ্ধতার (Impurity) পরিমাপ
IG(S,F)	Feature F দিয়ে Split করলে প্রাপ্ত Information Gain
S_v	Feature F-এর মান v অনুযায়ী তৈরি হওয়া উপসেট

১০. সূত্রের ধাপে-ধাপে ব্যাখ্যা (Step-by-Step Formula Explanation)

model.fit() মেথড কল করার পর ডিসিশন ট্রি একটি ধারাবাহিক গাণিতিক প্রক্রিয়ার মাধ্যমে নিজস্ব কাঠামো তৈরি করে। এই প্রক্রিয়াটি নিম্নলিখিত ধাপগুলো অনুসরণ করে।

ধাপ ১: Parent Node-এর বিশুদ্ধতা নির্ণয়

প্রথমে সম্পূর্ণ Training Dataset-কে একটি Parent Node হিসেবে বিবেচনা করা হয়। এরপর ওই নোডের Entropy অথবা Gini Impurity গণনা করা হয়।

ধাপ ২: সমস্ত ফিচার ও সম্ভাব্য Threshold পরীক্ষা করা

এরপর প্রতিটি Feature একে একে বিশ্লেষণ করা হয়।

যদি Feature সংখ্যাসূচক (Numerical) হয়, তাহলে সম্ভাব্য বিভিন্ন Threshold পরীক্ষা করা হয়।

যেমন,

- Age > 20
- Age > 25
- Age > 30

আবার যদি Feature শ্রেণিভিত্তিক (Categorical) হয়, তাহলে প্রতিটি Category অনুযায়ী সম্ভাব্য বিভাজন পরীক্ষা করা হয়।

প্রতিটি ক্ষেত্রেই ডেটাকে সাময়িকভাবে (Temporarily) দুই ভাগে বিভক্ত করা হয়।

ধাপ ৩: প্রতিটি বিভাজনের গুণগত মান মূল্যায়ন

প্রতিটি সম্ভাব্য বিভাজনের পরে সৃষ্ট সাব-নোডগুলোর Entropy অথবা Gini Impurity পুনরায় গণনা করা হয়।

এরপর Information Gain নির্ণয় করা হয় এবং দেখা হয়, কোন বিভাজন Parent Node-এর তুলনায় সবচেয়ে বেশি বিশৃঙ্খলা কমাতে সক্ষম হয়েছে।

ধাপ ৪: সর্বোত্তম বিভাজন নির্বাচন

যে Feature এবং Threshold সর্বোচ্চ Information Gain প্রদান করে অথবা সর্বনিম্ন Gini Impurity সৃষ্টি করে, সেটিকে চূড়ান্ত Split হিসেবে নির্বাচন করা হয়।

এরপর Parent Node-কে স্থায়ীভাবে (Permanently) সেই নিয়ম অনুযায়ী দুই বা ততোধিক সাব-নোডে বিভক্ত করা হয়।

ধাপ ৫: পুনরাবৃত্তিমূলক (Recursive) প্রক্রিয়া

একবার প্রথম বিভাজন সম্পন্ন হলে একই প্রক্রিয়া প্রতিটি নতুন সাব-নোডের জন্য পুনরায় চালানো হয়।

অর্থাৎ,

- পুনরায় Entropy বা Gini গণনা করা হয়।
- সম্ভাব্য সকল Feature পরীক্ষা করা হয়।
- সর্বোত্তম Split নির্বাচন করা হয়।
- নতুন Sub-node তৈরি করা হয়।

এই Recursive প্রক্রিয়া বারবার চলতে থাকে যতক্ষণ না অ্যালগরিদমের থামার কোনো শর্ত পূরণ হয়।

৪. ডিসিশন ট্রি কখন বৃদ্ধি বন্ধ করে? (Stopping Criteria)

যদি ডিসিশন ট্রিকে কোনো সীমাবদ্ধতা ছাড়া প্রশিক্ষণ দেওয়া হয়, তাহলে এটি প্রতিটি Leaf Node সম্পূর্ণ বিশুদ্ধ না হওয়া পর্যন্ত বিভাজন চালিয়ে যেতে পারে। এর ফলে মডেলটি শুধুমাত্র প্রকৃত প্যাটার্ন নয়, বরং Training Dataset-এর Noise এবং ব্যতিক্রমী তথ্যও শিখে ফেলে। এই সমস্যাকে **Overfitting** বলা হয়।

Overfitting প্রতিরোধ করার জন্য বিভিন্ন Hyperparameter ব্যবহার করা হয়।

max_depth

গাছ সর্বোচ্চ কত স্তর পর্যন্ত বৃদ্ধি পাবে তা নির্ধারণ করে।

উদাহরণস্বরূপ,

max_depth = 3

দেওয়া হলে গাছ তিন স্তরের বেশি বৃদ্ধি পাবে না।

min_samples_split

কোনো নোডকে নতুনভাবে বিভক্ত করার জন্য সেখানে ন্যূনতম কতটি Sample থাকতে হবে, তা নির্ধারণ করে।

min_samples_leaf

প্রতিটি Leaf Node-এ সর্বনিম্ন কতটি Sample থাকতে হবে, তা নির্ধারণ করে। এর ফলে খুব ছোট বা অপ্রয়োজনীয় Leaf তৈরি হওয়ার সম্ভাবনা কমে যায়।

১১. সংখ্যাগত উদাহরণ (Numerical Example)

Example of Decision Tree over a Categorical Dataset:

ধরা যাক, একটি দোকান জানতে চায় কোনো গ্রাহক পণ্য কিনবে কিনা। এজন্য নিচের ডেটাসেটটি সংগ্রহ করা হয়েছে।

ID	Income	Student	Buy
1	High	Yes	Yes
2	High	Yes	Yes
3	High	No	No
4	High	No	No
5	Low	Yes	Yes
6	Low	Yes	Yes

7	Low	No	No
8	Low	No	Yes

এখানে,

- **Features**

- Income
- Student

- **Target**

- Buy

ধাপ ১: Parent Node-এর Entropy গণনা

প্রথমে সম্পূর্ণ ডেটাসেটকে Parent Node ধরা হবে।

মোট ডেটা = 8

Buy = Yes = 5

Buy = No = 3

অতএব,

$$P(Yes) = \frac{5}{8} = 0.625$$

$$P(No) = \frac{3}{8} = 0.375$$

Entropy-এর সূত্র,

$$Entropy(S) = -\sum p_i \log_2(p_i)$$

সুতরাং,

$$Entropy(S) = -\left(\frac{5}{8} \log_2 \frac{5}{8} + \frac{3}{8} \log_2 \frac{3}{8}\right)$$

গণনা করলে,

$$\log_2(0.625) = -0.678$$

$$\log_2(0.375) = -1.415$$

অতএব,

$$\begin{aligned} Entropy &= -(0.625 \times -0.678 + 0.375 \times -1.415) \\ &= 0.954 \end{aligned}$$

অতএব,

$$Entropy(Parent) = 0.954$$

ধাপ ২: Feature = Income পরীক্ষা করা

Income-এর দুটি মান আছে।

- High
- Low

Income = High

Buy
Yes
Yes
No
No

Yes = 2

No = 2

অতএব,

$$Entropy(High) = -\left(\frac{2}{4}\log_2\frac{2}{4} + \frac{2}{4}\log_2\frac{2}{4}\right)$$

$$= 1.0$$

Income = Low

Buy
Yes
Yes
No
Yes

Yes = 3

No = 1

$$Entropy(Low) = -\left(\frac{3}{4}\log_2\frac{3}{4} + \frac{1}{4}\log_2\frac{1}{4}\right)$$

$$= 0.811$$

Weighted Entropy

$$= \frac{4}{8} \times 1 + \frac{4}{8} \times 0.811$$

$$= 0.9055$$

Information Gain (Income)

$$IG = 0.954 - 0.9055$$

$$= 0.0485$$

ধাপ ৩: Feature = Student পরীক্ষা করা

Student-এর দুটি মান।

- Yes
- No

Student = Yes

Buy
Yes
Yes
Yes
Yes

Yes = 4

No = 0

$$Entropy = 0$$

Student = No

Buy
No
No
No
Yes

Yes = 1

No = 3

$$Entropy = -\left(\frac{1}{4}\log_2\frac{1}{4} + \frac{3}{4}\log_2\frac{3}{4}\right)$$
$$= 0.811$$

Weighted Entropy

$$= \frac{4}{8} \times 0 + \frac{4}{8} \times 0.811$$
$$= 0.4055$$

Information Gain

$$IG = 0.954 - 0.4055 \\ = 0.5485$$

ধাপ ৪: Information Gain তুলনা

Feature Information Gain

Income 0.0485

Student 0.5485

স্পষ্টভাবে দেখা যাচ্ছে,

$$0.5485 > 0.0485$$

অতএব,

Student Feature সর্বাধিক Information Gain প্রদান করেছে।

সুতরাং,

Student হবে Root Node।

ধাপ ৫: প্রথম Split

Student?
/
Yes No

\

Student = Yes

ডেটা

Buy
Yes
Yes

Yes
Yes

সবগুলো একই ক্লাস।

Entropy = 0

এটি একটি **Leaf Node**।

Student = Yes



Buy = Yes

Student = No

ডেটা

Income	Buy
High	No
High	No
Low	No
Low	Yes

এখানে

Yes = 1

No = 3

এখন এই নোডটি বিশুদ্ধ নয়।

অতএব, আবার Feature নির্বাচন করতে হবে।

এখানে অবশিষ্ট Feature হলো

Income

ধাপ ৬: Student = No নোডে Income দিয়ে Split

Income = High

Buy
No
No

Entropy = 0

Income = Low

Buy
No
Yes

Entropy

$$= -\left(\frac{1}{2}\log_2 \frac{1}{2} + \frac{1}{2}\log_2 \frac{1}{2}\right) = 1$$

Weighted Entropy

$$= \frac{2}{4} \times 0 + \frac{2}{4} \times 1 = 0.5$$

বর্তমান Parent Entropy

$$0.811$$

Information Gain

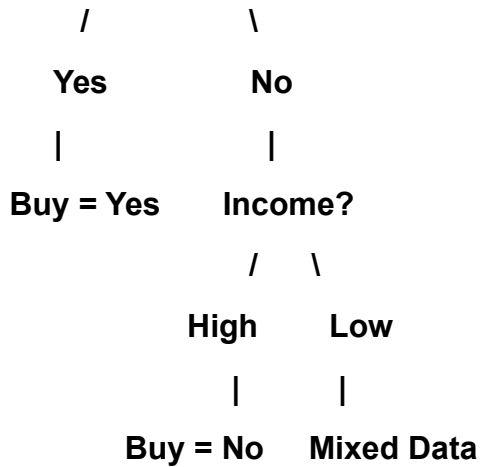
$$= 0.811 - 0.5 = 0.311$$

অতএব,

Income দিয়ে আবার Split করা হবে।

Final Decision Tree

Student?



এখানে Income = Low শাখায় এখনও দুটি ভিন্ন ক্লাস (Yes এবং No) রয়েছে। যেহেতু আর কোনো Feature অবশিষ্ট নেই, তাই বাস্তব অ্যালগরিদম সাধারণত Majority Class নির্বাচন করে অথবা নির্ধারিত Stopping Criteria অনুযায়ী সেই নোডকে Leaf Node বানিয়ে দেয়।

Dataset

ID	Income	Student	Buy
1	High	Yes	Yes
2	High	Yes	Yes
3	High	No	No
4	High	No	No
5	Low	Yes	Yes
6	Low	Yes	Yes
7	Low	No	No
8	Low	No	Yes

ধাপ ১: Parent Gini

মোট ৮টি ডেটা

- Yes = 5
- No = 3

$$Gini = 1 - (P_{Yes}^2 + P_{No}^2)$$

$$\begin{aligned}
 &= 1 - \left(\left(\frac{5}{8} \right)^2 + \left(\frac{3}{8} \right)^2 \right) \\
 &= 1 - (0.3906 + 0.1406) \\
 &= \boxed{0.4688}
 \end{aligned}$$

ধাপ ২: Income Feature

Income = High

Yes = 2, No = 2

$$Gini = 1 - (0.5^2 + 0.5^2) = 0.5$$

Income = Low

Yes = 3, No = 1

$$Gini = 1 - \left(\left(\frac{3}{4} \right)^2 + \left(\frac{1}{4} \right)^2 \right) = 0.375$$

Weighted Gini

$$= \frac{4}{8}(0.5) + \frac{4}{8}(0.375) = 0.4375$$

Gini Gain (Reduction)

$$0.4688 - 0.4375 = \boxed{0.0313}$$

ধাপ ৩: Student Feature

Student = Yes

Yes = 4, No = 0

$$Gini = 0$$

Student = No

Yes = 1, No = 3

$$Gini = 1 - \left(\left(\frac{1}{4} \right)^2 + \left(\frac{3}{4} \right)^2 \right) = 0.375$$

Weighted Gini

$$= \frac{4}{8}(0) + \frac{4}{8}(0.375) = 0.1875$$

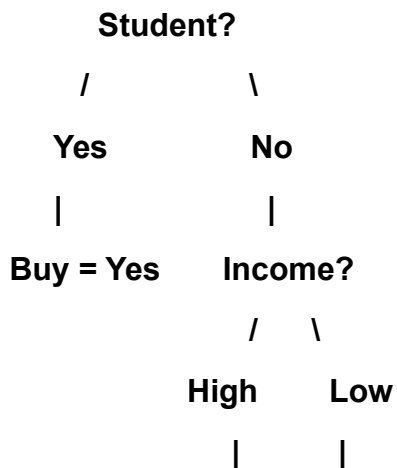
Gini Gain (Reduction)

$$0.4688 - 0.1875 = \boxed{0.2813}$$

ধাপ ৪: সিদ্ধান্ত

Feature	Weighted Gini	Gini Reduction
Income	0.4375	0.0313
Student	0.1875	0.2813

যেহেতু Student Feature-এর Weighted Gini সবচেয়ে কম (0.1875) এবং Gini Reduction সবচেয়ে বেশি (0.2813), তাই Student-কেই Root Node হিসেবে নির্বাচন করা হবে।



Buy = No Mixed Data

Classification-এ আমরা Entropy বা Gini ব্যবহার করি, কারণ সেখানে লক্ষ্য থাকে ক্লাস (Yes/No) আলাদা করা। কিন্তু Regression Tree-তে লক্ষ্য থাকে Continuous Value (যেমন: Price, Salary, House Price) পূর্বাভাস দেওয়া। তাই এখানে Variance বা Mean Squared Error (MSE) ব্যবহার করা হয়।

Dataset

ধরা যাক, আমরা পড়ার সময় (Study Hours) দেখে পরীক্ষার নম্বর (Marks) পূর্বাভাস দিতে চাই।

ID	Study Hours	Marks
1	1	40
2	2	45
3	3	50
4	4	80
5	5	85
6	6	90

Target = **Marks**

ধাপ ১: সম্ভাব্য Split নির্বাচন

ধরি,

$$\text{Split} = \text{Study Hours} \leq 3$$

তাহলে Dataset দুই ভাগে বিভক্ত হবে।

Left Node

Hours	Marks
1	40
2	45
3	50

Mean

$$= \frac{40 + 45 + 50}{3} = 45$$

Variance

$$= \frac{(40 - 45)^2 + (45 - 45)^2 + (50 - 45)^2}{3}$$

$$= \frac{25 + 0 + 25}{3} = 16.67$$

Right Node

Hours	Marks
4	80
5	85
6	90

Mean

$$= \frac{80 + 85 + 90}{3} = 85$$

Variance

$$= \frac{(80 - 85)^2 + (85 - 85)^2 + (90 - 85)^2}{3}$$

$$= \frac{25 + 0 + 25}{3} = 16.67$$

ধাপ ২: Weighted Variance

Left-এ ৩টি Sample

Right-এ ৩টি Sample

$$Weighted\ Variance = \frac{3}{6}(16.67) + \frac{3}{6}(16.67) = 16.67$$

ধাপ ৩: অন্য একটি Split পরীক্ষা

ধরি,

Study Hours ≤ 4

Left Node

Marks

40, 45, 50, 80

Mean

$$= \frac{40 + 45 + 50 + 80}{4} = 53.75$$

Variance

$$= 242.19$$

Right Node

Marks

85, 90

Mean

$$= 87.5$$

Variance

$$= 6.25$$

Weighted Variance

$$\begin{aligned} &= \frac{4}{6}(242.19) + \frac{2}{6}(6.25) \\ &= 163.54 \end{aligned}$$

ধাপ ৪: সিদ্ধান্ত

Split Weighted Variance

Study Hours ≤ 3 **16.67**

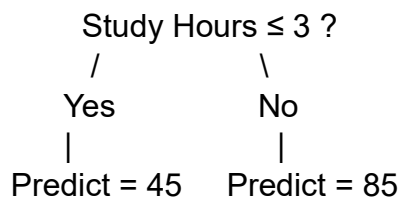
Study Hours ≤ 4 163.54

যেহেতু

16.67 < 163.54

তাই **Study Hours ≤ 3** হবে **Best Split**।

Decision Tree



Prediction

Study Hours	Prediction
2	45
3	45
5	85
6	85

এখানে প্রতিটি Leaf Node কোনো Class Label সংরক্ষণ করে না। বরং সেই Leaf-এর সকল Target Value-এর Mean (Average) সংরক্ষণ করে।

১২. ভিজ্যুয়ালাইজেশন (Visualization)

উপরে দেখানো Tree Diagram-গুলো (Student? → Yes/No শাখা) লক্ষ্য করুন — এটিই Decision Tree ভিজ্যুয়ালাইজ করার সবচেয়ে স্বাভাবিক উপায়। প্রতিটি Internal Node-এ একটি প্রশ্ন (Feature Condition) থাকে, প্রতিটি শাখা (Branch) একটি সম্ভাব্য উত্তর প্রকাশ করে, এবং প্রতিটি Leaf Node একটি চূড়ান্ত সিদ্ধান্ত ধারণ করে। বাস্তব প্রজেক্টে `sklearn.tree.plot_tree()` ফাংশন ব্যবহার করে এই একই কাঠামো স্বয়ংক্রিয়ভাবে আঁকা যায়, যেখানে প্রতিটি Node-এ Gini/Entropy মান, Sample সংখ্যা এবং শ্রেণি বিতরণও দেখানো থাকে।

ডিসিশন ট্রি ক্লাসিফায়ার এবং ডিসিশন ট্রি রিগ্রেসরের মধ্যে পার্থক্য

বিষয়	Decision Tree Classifier	Decision Tree Regressor
পূর্বাভাসের ধরন	শ্রেণিভিত্তিক (Categorical) মান পূর্বাভাস দেয়	ধারাবাহিক সংখ্যাসূচক (Continuous Numerical) মান পূর্বাভাস দেয়
Target Variable-এর ধরন	Categorical (যেমন: Yes/No, Spam/Not Spam)	Numerical (যেমন: Price, Salary, Temperature)
মূল উদ্দেশ্য	ডেটাকে বিভিন্ন শ্রেণিতে (Class) ভাগ করা	কোনো সংখ্যাসূচক মান (Value) পূর্বাভাস দেওয়া
Split করার মানদণ্ড	Entropy, Information Gain অথবা Gini Impurity	Variance Reduction অথবা Mean Squared Error (MSE) Reduction
Impurity Measure	বিভিন্ন Class কতটা মিশ্রিত আছে তা পরিমাপ করে	সংখ্যাগুলোর বিচ্ছুরণ (Variation) কতটা তা পরিমাপ করে
Leaf Node-এর আউটপুট	একটি Class Label (যেমন: Yes বা No) সংরক্ষণ করে	Target Value-এর গড় (Mean/Average) সংরক্ষণ করে
সেরা Split নির্বাচন	সর্বোচ্চ Information Gain অথবা সর্বনিম্ন Gini বিশিষ্ট Split নির্বাচন করে	সর্বনিম্ন Variance অথবা সর্বনিম্ন MSE বিশিষ্ট Split নির্বাচন করে
Decision Rule	ভিন্ন ভিন্ন Class আলাদা করার চেষ্টা করে	কাছাকাছি সংখ্যাসূচক মানগুলোকে একই দলে রাখার চেষ্টা করে
আউটপুটের উদাহরণ	Buy = Yes	House Price = 25,00,000 টাকা

Evaluation Metrics	Accuracy, Precision, Recall, F1-score, ROC-AUC	MAE, MSE, RMSE, R ² Score
Scikit-Learn Class	DecisionTreeClassifier	DecisionTreeRegressor
সাধারণ ব্যবহার	Spam Detection, Disease Classification, Customer Churn Prediction	House Price Prediction, Salary Prediction, Sales Forecasting

১৩. হাইপারপ্যারামিটার (Hyperparameters)

Scikit-Learn-এর DecisionTreeClassifier Hyperparameter:

১. মোস্ট ইম্পোর্টেন্ট প্যারামিটারসমূহ (The Heavy Hitters)

গাছের সাইজ নিয়ন্ত্রণ করতে, ওভারফিটিং রোধ করতে এবং ইন্টারভিউতে সবচেয়ে বেশি ফোকাস করা হয় এই প্যারামিটারগুলোর ওপর।

max_depth: None (default)

- **কাজ কী:** এটি আপনার তৈরি হওয়া গাছের সর্বোচ্চ গভীরতা বা লেভেল (Level) নির্ধারণ করে।
- **কখন 'None' (Default) রাখব:** ডেটাসেট যদি খুব ছোট হয় এবং আপনি যদি দেখতে চান গাছটি সর্বোচ্চ কতদূর পর্যন্ত যেতে পারে, তখন এটি ডিফল্ট রাখা যায়। None থাকলে গাছটি বাড়তেই থাকবে যতক্ষণ না সব পাতা ১০০% বিশুদ্ধ হচ্ছে বা min_samples_split-এর চেয়ে কম স্যাম্পল থাকছে।
- **ঝুঁকি কী:** এটি ডিফল্ট রেখে দিলে গাছটি একদম শেষ সীমানা পর্যন্ত মুখস্থ করতে থাকবে এবং মারাত্মক **Overfitting (High Variance)** ঘটবে।
- **অন্যান্য অপশন ও কখন ব্যবহার করব:** এখানে যেকোনো পূর্ণসংখ্যা (যেমন: 3, 5, 10) দেওয়া যায়। বাস্তব প্রজেক্টে ওভারফিটিং বন্ধ করার সবচেয়ে প্রথম হাতিয়ার হলো এটি। সবসময় ছোট মান (যেমন: max_depth=3 বা 5) দিয়ে শুরু করা ভালো, যাতে মডেলটি জেনারেলাইজড বা সুষম থাকে।

criterion: "gini" (default)

- **কাজ কী:** প্রতিটা ধাপে ডালপালা মেলানোর জন্য বা ডেটা ভাগ (Split) করার কোয়ালিটি মাপার জন্য ব্যাকগ্রাউন্ডে কোন গাণিতিক সূত্র ব্যবহার করা হবে, তা এটি নির্ধারণ করে।

- **কখন 'gini' (Default) ব্যবহার করব:** বাস্তব প্রজেক্টে দ্রুত ট্রেনিং শেষ করার জন্য এটি ব্যবহার করা হয়। জিনি ইম্পিউরিটির গাণিতিক সূত্রে কোনো লগারিদম ($\log_2$) হিসাব করতে হয় না, তাই এটি কম্পিউটেশনালি অত্যন্ত ফাস্ট এবং দ্রুত রেজাল্ট দেয়।
- **অন্যান্য অপশন:** "entropy" অথবা "log_loss" (উভয়ই শ্যানন ইনফরমেশন গেইনের ওপর ভিত্তি করে কাজ করে)।
- **কখন এবং কেন 'entropy' ব্যবহার করব:** যখন আপনি অত্যন্ত সূক্ষ্ম এবং সুষমভাবে ডেটা ভাগ করতে চান। এনট্রপি নোডের ভেতরের বিশৃঙ্খলাকে আরও নিখুঁতভাবে মাপে, তবে এর সূত্রে $\log_2$ থাকায় কম্পিউটারের প্রসেসিং টাইম সামান্য বেশি লাগে। বড় ডেটাসেটে জিনি আর এনট্রপির এক্যুরেসির মধ্যে খুব বেশি তফাৎ পাওয়া যায় না, তাই ডিফল্ট 'gini' রাখাই স্ট্যান্ডার্ড।

min_samples_split: 2 (default)

- **কাজ কী:** মাঝখানের কোনো একটি নোডকে (Internal Node) ভেঙে নতুন ডালপালা তৈরি করার জন্য ওই নোডে নূন্যতম কয়টি ডেটা স্যাম্পল থাকতে হবে, তা এটি ঠিক করে।
- **কখন '2' (Default) রাখব:** যখন আপনি গাছের স্প্লিটিং প্রক্রিয়ায় কোনো বাধা দিতে চান না। তবে ২ রাখা মানে নোডে মাত্র ২টি স্যাম্পল থাকলেও গাছ সেটিকে ভেঙে আরও নিচে নামবে, যা ওভারফিটিংয়ের ঝুঁকি বাড়ায়।
- **অন্যান্য অপশন ও কখন ব্যবহার করব:** এখানে পূর্ণসংখ্যা (যেমন: 10, 50) অথবা ফ্লোট ভগ্নাংশ (যেমন: 0.05 অর্থাৎ মোট ডেটার ৫%) দেওয়া যায়।
- **কেন পরিবর্তন করব:** ওভারফিটিং কমানোর জন্য। যদি আপনি min_samples_split=50 দেন, তবে কোনো নোডে ৫০টির কম ডেটা থাকলে গাছ সেটিকে আর ভাঙবে না, সরাসরি ওখানেই থামিয়ে দেবে।

min_samples_leaf: 1 (default)

- **কাজ কী:** একটি চূড়ান্ত পাতা বা লিফ নোডে (Leaf Node) নূন্যতম কয়টি স্যাম্পল থাকতেই হবে, তা এটি ফিক্স করে।
- **কখন '1' (Default) রাখব:** ডিফল্ট ১ রাখা মানে গাছের শেষ পাতায় মাত্র ১টি স্যাম্পল থাকলেও মডেল খুশি। এটি মডেলকে চরম মুখস্থবিদ্যার দিকে ঠেলে দেয়।
- **অন্যান্য অপশন ও কখন ব্যবহার করব:** এখানেও পূর্ণসংখ্যা বা ফ্লোট ভগ্নাংশ দেওয়া যায় (যেমন: 5, 10 বা 0.02)।

- **কেন পরিবর্তন করব:** এটি মডেলকে স্মুথ (Smooth) করতে সাহায্য করে। যদি `min_samples_leaf=10` দেওয়া হয়, তবে কোনো স্প্লিট করার কারণে যদি বাম বা ডান ডালের কোনো একটিতে ১০টির কম স্যাম্পল চলে যাওয়ার পরিস্থিতি তৈরি হয়, তবে ডিসিশন ট্রি ওই স্প্লিটটি বাতিল করে দেবে। এটি ওভারফিটিং কমানোর অত্যন্ত শক্তিশালী একটি প্যারামিটার।

২. মিডিয়াম ইম্পর্টেন্ট প্যারামিটারসমূহ (Advanced Tuning)

মডেলের পারফরম্যান্স আরও নিখুঁত করতে এবং ডেটার বিশেষ পরিস্থিতি সামলাতে এগুলো ব্যবহার করা হয়।

class_weight: None (default)

- **কাজ কী:** ইমব্যালেন্সড ডেটাসেটের (Imbalanced Dataset) ক্ষেত্রে মাইনোরিটি ক্লাসকে বা ছোট ক্লাসকে অতিরিক্ত গুরুত্ব বা ওজন দেওয়ার জন্য এটি ব্যবহৃত হয়।
- **কখন 'None' (Default) রাখব:** যখন আপনার ডেটাসেটে সব ক্লাসের অনুপাত প্রায় সমান থাকে (যেমন: ৫০% Yes, ৫০% No)।
- **অন্যান্য অপশন:** "balanced" অথবা কাস্টম ডিকশনারি {0: 1, 1: 5}।
- **কখন এবং কেন 'balanced' ব্যবহার করব:** যখন ডেটা প্রবলভাবে ইমব্যালেন্সড (যেমন: টাইটানিকে মারা গেছে ৮০%, বেঁচেছে মাত্র ২০%)। `class_weight='balanced'` দিলে মডেলটি স্বয়ংক্রিয়ভাবে ছোট ক্লাসকে বেশি গুরুত্ব দেয়, ফলে মডেলের রিকল (Recall) এবং এফ১-স্কোর (F1-score) উন্নত হয়।

max_features: None (default)

- **কাজ কী:** প্রতিটা স্প্লিটে সেরা শর্তটি খোঁজার সময় ডিসিশন ট্রি সর্বোচ্চ কয়টি ফিচার স্ক্যান করবে, তা এটি নিয়ন্ত্রণ করে।
- **কখন 'None' (Default) রাখব:** যখন ডেটাসেটে ফিচারের সংখ্যা কম (যেমন: ১০-১৫টি কলাম) এবং আপনি চান মডেলটি প্রতিবার সব ফিচার দেখে সেরা সিদ্ধান্তটি নিক।
- **অন্যান্য অপশন:** "sqrt" (ফিচার সংখ্যার বর্গমূল), "log2" (ফিচার সংখ্যার লগ বেস ২) অথবা সরাসরি কোনো সংখ্যা বা ভগ্নাংশ।
- **কখন এবং কেন পরিবর্তন করব:** যখন ডেটাসেটে ফিচারের সংখ্যা বিশাল (যেমন: ৫০০ বা ১০০০ কলাম)। তখন প্রতিটা নোডে সব ফিচার চেক করতে গেলে মডেল স্লো হয়ে যায়।

`max_features='sqrt'` দিলে সে প্রতি স্প্লিটে র্যান্ডমলি কিছু ফিচার বেছে নিয়ে তার মধ্য থেকে সেরাটি খোঁজে। এটি মূলত **Random Forest** অ্যালগরিদমের মূল ভিত্তি।

ccp_alpha: 0.0 (default)

- **কাজ কী:** এটি গাছের পোস্ট-প্রুনিং (Post-Pruning) বা বড় হয়ে যাওয়া গাছের অপ্রয়োজনীয় ডালপালা কেটে ছেঁটে ফেলার জটিল গাণিতিক প্যারামিটার।
- **কখন '0.0' (Default) রাখব:** ডিফল্ট শূন্য রাখা মানে মডেলটি কোনো ডালপালা কাটবে না, গাছ যেভাবে বাড়ে সেভাবেই থাকবে।
- **অন্যান্য অপশন ও কখন ব্যবহার করব:** এখানে যেকোনো অ-ঋণাত্মক ফ্লোট মান (যেমন: 0.01, 0.015) দেওয়া যায়। যখন গাছটি অলরেডি সম্পূর্ণ বড় হয়ে ওভারফিট করে ফেলেছে, তখন কস্ট-কমপ্লেক্সিটি অ্যালগরিদম দিয়ে দুর্বল ডালগুলো কেটে মডেলকে জেনারেলাইজড করতে এটি টিউন করা হয়।

random_state: None (default)

- **কাজ কী:** মডেলের শাফলিং এবং র্যান্ডম অপারেশনগুলোকে নিয়ন্ত্রণ করে কোডের আউটপুটকে প্রতিবার একই রকম বা রিপ্ৰোডিউসিবল (*Reproducible*) রাখে।
- **কখন 'None' (Default) রাখব:** যদি আপনি কোড প্রতিবার রান করার সময় সামান্য ভিন্ন ভিন্ন গাছ বা আউটপুট দেখতে চান (রিসার্চের সময় সাধারণত রাখা হয় না)।
- **অন্যান্য অপশন ও কখন ব্যবহার করব:** এখানে যেকোনো ফিক্সড পূর্ণসংখ্যা (কনভেনশন হিসেবে 42 বা 0) দেওয়া উচিত। অ্যাসাইনমেন্ট, পরীক্ষা বা টিম প্রজেক্টে যেন আপনার কোড অন্য কম্পিউটারে রান করলেও অবিকল একই গাছ এবং একই এক্যুরেসির রেজাল্ট দেয়, তা নিশ্চিত করতে এটি ফিক্স করা বাধ্যতামূলক।

৩. লেস ইম্পোর্টেন্ট প্যারামিটারসমূহ (Rarely Used)

সাধারণ মডেলিংয়ে এগুলো খুব একটা পরিবর্তন করার প্রয়োজন হয় না, ডিফল্ট মানই চমৎকার কাজ করে।

splitter: "best" (default)

- **কাজ কী:** নোড ভাগ করার সময় ফিচারগুলোর মধ্য থেকে থ্রেসহোল্ড বেছে নেওয়ার স্ট্র্যাটেজি।
- **কখন 'best' (Default) রাখব:** সবসময়। এটি গাণিতিকভাবে সবচেয়ে নিখুঁত এবং সর্বোত্তম স্প্লিটিং পয়েন্টটি খুঁজে বের করে।

- **অন্যান্য অপশন:** "random"।
- **কখন এবং কেন 'random' ব্যবহার করব:** যখন ডেটাসেট অনেক বিশাল এবং আপনি চান মডেলটি বেস্ট পয়েন্ট না খুঁজে র্যান্ডমলি একটি ভালো পয়েন্ট নিয়ে দ্রুত ট্রেনিং শেষ করুক (কম্পিউটেশনাল টাইম বাঁচাতে)।

max_leaf_nodes: None (default)

- **কাজ কী:** একটি গাছে সর্বোচ্চ কয়টি শেষ পাতা বা লিফ নোড থাকতে পারবে তার সংখ্যা বেঁধে দেওয়া।
- **কখন 'None' (Default) রাখব:** যখন আপনি max_depth বা অন্যান্য প্যারামিটার দিয়ে অলরেডি গাছ নিয়ন্ত্রণ করছেন। None মানে অসীম সংখ্যক লিফ নোড হতে পারে।
- **অন্যান্য অপশন ও কখন ব্যবহার করব:** যেকোনো পূর্ণসংখ্যা। এটিও ওভারফিটিং কমানোর আরেকটি বিকল্প রাস্তা, তবে সাধারণত max_depth টিউন করলে এটির আলাদা প্রয়োজন পড়ে না।

min_impurity_decrease: 0.0 (default)

- **কাজ কী:** একটি নোডকে তখনই ভাঙা হবে, যদি সেই স্প্লিটের কারণে ইম্পিউরিটি বা অবিশুদ্ধতার হ্রাস পাওয়ার মান এই থ্রেশহোল্ডের চেয়ে সমান বা বড় হয়।
- **কখন '0.0' (Default) রাখব:** ডিফল্ট ০.০ রাখা মানে ইম্পিউরিটি সামান্য কমলেও (অর্থাৎ সামান্য তথ্য লাভ হলেও) গাছটি স্প্লিট করবে। সাধারণত এটি ডিফল্ট রেখেই বাকি প্যারামিটার টিউন করা হয়।

monotonic_cst: None (default)

- **কাজ কী:** ফিচারের ওপর মোনোটোনিক বা একমুখী শর্তারোপ করা (যেমন: বয়স বাড়লে ইন্সুরেন্সের প্রিমিয়াম শুধু বাড়বেই, কখনো কমবে না—এমন লজিক এনফোর্স করা)। এটি বাইনারি ক্লাসিফিকেশনের ক্ষেত্রে পজিটিভ ক্লাসের প্রোবাবিলিটির ওপর কাজ করে। সাধারণ প্রজেক্টে এটি None রাখা হয়।

min_weight_fraction_leaf: 0.0 (default)

- **কাজ কী:** একটি লিফ নোডে থাকার জন্য স্যাম্পলগুলোর ওজনের নূন্যতম যোগফলের অনুপাত। যদি স্যাম্পল ওয়েট (sample_weight) ব্যবহার না করা হয়, তবে সব স্যাম্পলের ওজন সমান থাকে এবং এটি min_samples_leaf-এর মতোই কাজ করে। সাধারণত এটি পরিবর্তন করার প্রয়োজন হয় না।

বাকি থাকা কম ব্যবহৃত প্যারামিটারসমূহ (Remaining Parameters)

এগুলো সাধারণত অ্যাডভান্সড রিসার্চ বা কাস্টম প্রজেক্ট ছাড়া খুব একটা পরিবর্তন করতে হয় না।

min_weight_fraction_leaf: 0.0 (default)

- **কাজ কী:** এটি min_samples_leaf-এর মতোই কাজ করে, তবে এটি সরাসরি স্যাম্পলের সংখ্যার বদলে স্যাম্পলগুলোর ওজনের (Weights) নূন্যতম অনুপাত হিসাব করে।
- **কখন এবং কেন ব্যবহার করব:** যদি আপনি মডেল ডট ফিট (model.fit) করার সময় sample_weight প্যারামিটার দিয়ে কিছু ডেটা রো-কে বেশি বা কম গুরুত্ব দিয়ে থাকেন। তখন এই নোডের ওজন সামগ্রিক ওজনের নির্দিষ্ট শতাংশ (যেমন: 0.05 বা ৫%) এর নিচে নেমে গেলে গাছ সেখানে ডালপালা মেলানো বন্ধ করে দেবে। যদি কোনো ওজনের তারতম্য না থাকে, তবে সব স্যাম্পলের ওজন সমান (1) ধরা হয় এবং এটি ডিফল্ট 0.0 রাখাই স্ট্যান্ডার্ড।

min_impurity_decrease: 0.0 (default)

- **কাজ কী:** একটি নোডকে তখনই স্প্লিট বা ভাগ করা হবে, যদি সেই ভাগের কারণে ইম্পিউরিটি (Gini বা Entropy) হ্রাস পাওয়ার মান এই থ্রেশহোল্ডের চেয়ে সমান বা বড় হয়।
- **গাণিতিক ব্যাকএন্ড সূত্র:**

$$\Delta i = \frac{N_t}{N} \times \left(\text{impurity} - \frac{N_{tR}}{N_t} \times \text{right_impurity} - \frac{N_{tL}}{N_t} \times \text{left_impurity} \right)$$

(এখানে N হলো মোট স্যাম্পল, N_t হলো বর্তমান নোডের স্যাম্পল, এবং L/R হলো বাম ও ডান চাইল্ড নোডের স্যাম্পল)

- **কখন এবং কেন পরিবর্তন করব:** যখন আপনি চান গাছটি ছোটখাটো বা অর্থহীন ফিচারের জন্য ডালপালা না মেলুক। যদি আপনি min_impurity_decrease=0.01 সেট করেন, তবে যে স্প্লিটটি ডেটার বিশুদ্ধতা অন্তত 0.01 পরিমাণ উন্নত করতে পারবে না, ডিসিশন ট্রি সেই স্প্লিটটি বাতিল করে দেবে। এটিও ওভারফিটিং নিয়ন্ত্রণে সাহায্য করে।

monotonic_cst: None (default)

- **কাজ কী:** এটি ফিচারের ওপর মোনোটোনিক বা একমুখী গাণিতিক শর্ত (Monotonicity Constraint) জোরপূর্বক চাপিয়ে দিতে ব্যবহৃত হয় (Scikit-Learn ভার্সন ১.৪-এ নতুন যুক্ত হয়েছে)।
- **অপশনসমূহ:** প্রতিটা ফিচারের জন্য একটি অ্যারে আকারে 1 (Monotonic Increase), -1 (Monotonic Decrease) অথবা 0 (No Constraint) দেওয়া যায়।

- **কখন এবং কেন ব্যবহার করব:** ধরুন আপনি একটি ক্রেডিট কার্ড লোন প্রজেক্ট করছেন, যেখানে আপনি নিশ্চিত করতে চান যে "কাস্টমারের আয় বাড়লে লোন পাওয়ার প্রোবাবিলিটি সবসময় বাড়বেই, কখনো কমবে না"। তখন আয়ের কলামের জন্য 1 সেট করা হয়। মনে রাখুন, এটি মাল্টি-ক্লাস ক্লাসিফিকেশন (Classes > 2) সাপোর্ট করে না, শুধু বাইনারি ক্লাসিফিকেশনের পজিটিভ ক্লাসের প্রোবাবিলিটির ওপর কাজ করে।

ডিসিশন ট্রির মূল অ্যাট্রিবিউটসমূহ (Important Attributes)

মডেল ডট ফিট (model.fit(X_train, y_train)) করার পর, মডেলটি ব্যাকএন্ডের লার্নিং থেকে কী কী তথ্য মেমোরিতে জমা করে রেখেছে, তা আমরা এই অ্যাট্রিবিউটগুলো দিয়ে দেখতে পারি। এদের শেষে সবসময় একটি আন্ডারস্কোর () থাকে।

feature_importances_

- **কাজ কী:** এটি একটি অ্যারে ব্যাক করে যা দেখায় যে, আপনার দেওয়া ফিচারগুলোর মধ্যে কোন কলামটি টার্গেট ভ্যারিয়েবল প্রেডিক্ট করতে সবচেয়ে বেশি ভূমিকা পালন করেছে।
- **ব্যাকএন্ড মেকানিজম:** যে ফিচারটি গাছের যত ওপরের দিকে (রুট নোডের কাছাকাছি) স্প্লিট করতে ব্যবহৃত হয়েছে এবং যার কারণে জিনি বা এনট্রপি সবচেয়ে বেশি কমেছে, তার ইম্পোর্টেন্স তত বেশি হয়। সব ফিচারের গুরুত্বের যোগফল সবসময় 1.0 হয়। এটি **Feature Selection**-এর জন্য সবচেয়ে জনপ্রিয় একটি টুল।

tree_

- **কাজ কী:** এটি ডিসিশন ট্রির আসল ব্যাকএন্ড অবজেক্ট (Underlying Tree Object)।
- **কেন প্রয়োজন:** গাছটি ভেতরে ভেতরে কীভাবে তৈরি হলো—তার নোড সংখ্যা কত, কোন নোডের বামে বা ডানে কী আছে, তা যদি আপনি পাইথন কোড দিয়ে স্ক্র্যাপ বা কাস্টমাইজ করতে চান, তবে model.tree_ ব্যবহার করে একদম র লেভেলের আর্কিটেকচার অ্যাক্সেস করা যায়।

classes_

- **কাজ কী:** আপনার টার্গেট ভ্যারিয়েবলে (y) আসলে কী কী শ্রেণি বা লেবেল ছিল, তার একটি ইউনিক অ্যারে দেখায়।

- **উদাহরণ:** টাইটানিক ডেটাসেটে `model.classes_` রান করলে আউটপুট আসবে `array([0, 1])`।

n_features_in_

- **কাজ কী:** মডেলটি ট্রেন করার সময় ইনপুট ম্যাট্রিক্সে (`X_train`) সর্বমোট কতগুলো ফিচার বা কলাম দেখেছিল, তার সুনির্দিষ্ট সংখ্যা জানায়।

feature_names_in_

- **কাজ কী:** ট্রেনিং ডেটায় থাকা সব কলামের অফিশিয়াল নামের একটি অ্যারে দেখায় (এটি তখনই কাজ করে যখন আপনার ইনপুট ডেটা একটি `pandas DataFrame` বা স্ট্রিং কলামযুক্ত হয়)।

n_classes_

- **কাজ কী:** টার্গেট কলামে মোট কয়টি ক্লাস বা ক্যাটাগরি আছে তার সংখ্যা দেয়। বাইনারি ক্লাসিফিকেশনে এর মান হবে 2।

n_outputs_

- **কাজ কী:** মডেলটি ফিট করার সময় কয়টি আউটপুট ভ্যারিয়েবল প্রেডিক্ট করার জন্য ট্রেন করা হয়েছিল তার সংখ্যা। সাধারণ সিঙ্গেল-আউটপুট প্রজেক্টে এর মান সবসময় 1 হয়।

DecisionTreeRegressor() Attributes

feature_importances_

Default: Created after `fit()`

প্রতিটি Feature-এর Importance Score সংরক্ষণ করে। যেসব Feature Prediction-এর জন্য বেশি গুরুত্বপূর্ণ, তাদের Score বেশি হয়।

tree_

Default: Created after `fit()`

Decision Tree-এর সম্পূর্ণ Internal Structure সংরক্ষণ করে। এর মধ্যে প্রতিটি Node-এর Threshold, Split Feature, Child Node, Sample সংখ্যা এবং Prediction Value থাকে।

max_features_

Default: Created after `fit()`

প্রতিটি Split-এর সময় সর্বোচ্চ কতটি Feature বিবেচনা করা হয়েছে তা সংরক্ষণ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় ব্যবহৃত মোট Input Feature-এর সংখ্যা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

n_outputs_

Default: Created after fit()

Model মোট কতটি Target Output Predict করছে তা সংরক্ষণ করে। সাধারণ Regression-এ এটি সাধারণত 1 হয়।

১৪. স্কিট-লার্ন ইম্পোর্ট (Scikit-learn Import)

Decision Tree Code using Sci-Kit Learn:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

default_pipeline = Pipeline(
    steps=[
        ('model', DecisionTreeClassifier(random_state=42))
    ]
)

default_pipeline.fit(X_train, y_train)

y_pred_default = default_pipeline.predict(X_test)
print("--- Default Model Performance ---")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred_default):.4f}")
print(classification_report(y_test, y_pred_default, target_names=iris.target_names))
```

Using Grid Search CV:

```
from sklearn.model_selection import GridSearchCV

tuning_pipeline = Pipeline(
    steps=[
        ('model', DecisionTreeClassifier(random_state=42))
    ]
)

param_grid = {
    'model__criterion': ['gini', 'entropy'],
    'model__max_depth': [3, 4, 5, None],
    'model__min_samples_split': [2, 5, 10],
    'model__min_samples_leaf': [1, 2, 4]
}

grid_search = GridSearchCV(
    estimator=tuning_pipeline,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("--- GridSearchCV Tuning Results (No Scaling) ---")
print("Best Parameters Found:", grid_search.best_params_)
print(f"Best CV Accuracy: {grid_search.best_score_:.4f}")

best_model_pipeline = grid_search.best_estimator_
y_pred_tuned = best_model_pipeline.predict(X_test)

print("\n--- Tuned Model Performance on Test Set ---")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred_tuned):.4f}")
print(classification_report(y_test, y_pred_tuned, target_names=iris.target_names))

importances = best_model_pipeline.named_steps['model'].feature_importances_
for name, importance in zip(iris.feature_names, importances):
    print(f"{name:20s}: {importance:.4f}")
```

১৫. পাইথন উদাহরণ — From-Scratch Implementation (Python Example)

নিচে একটি সরলীকৃত From-Scratch Decision Tree (Entropy ও Information Gain ব্যবহার করে) দেখানো হলো, যা এই অধ্যায়ের ম্যানুয়াল গণনার সাথে হুবহু মেলে:

```
import numpy as np
from collections import Counter

def entropy(labels):
    counts = Counter(labels)
    n = len(labels)
    return -sum((c/n) * np.log2(c/n) for c in counts.values())

def info_gain(parent, left, right):
    n = len(parent)
    weighted = (len(left)/n)*entropy(left) + (len(right)/n)*entropy(right)
    return entropy(parent) - weighted

def best_split(X, y, feature_idx):
    best_gain, best_val = -1, None
    values = set(row[feature_idx] for row in X)
    for v in values:
        left_y = [y[i] for i in range(len(X)) if X[i][feature_idx] == v]
        right_y = [y[i] for i in range(len(X)) if X[i][feature_idx] != v]
        if not left_y or not right_y:
            continue
        gain = info_gain(y, left_y, right_y)
        if gain > best_gain:
            best_gain, best_val = gain, v
    return best_val, best_gain

# Example usage
X = [['Sunny', 'Hot'], ['Sunny', 'Cool'], ['Rainy', 'Hot'],
      ['Rainy', 'Cool'], ['Sunny', 'Cool']]
y = ['No', 'Yes', 'No', 'Yes', 'No']
val, gain = best_split(X, y, feature_idx=1) # Temperature column
print('Best split value:', val, '| Information Gain:', round(gain, 4))
```

১৬. প্রেডিকশন প্রসেস (Prediction Process)

Training শেষে (গাছ সম্পূর্ণ তৈরি হয়ে গেলে), নতুন স্যাম্পলের Prediction হয় খুব সহজে: Root Node থেকে শুরু করে, প্রতিটি Internal Node-এর প্রশ্নের উত্তর (নতুন স্যাম্পলের ফিচার মান অনুযায়ী) অনুসরণ করে নিচের দিকে নামতে হয়, যতক্ষণ না একটি Leaf Node-এ পৌঁছানো যায়। সেই Leaf Node-এ সংরক্ষিত ক্লাস (Classification) বা গড় মান (Regression)-ই চূড়ান্ত Prediction।

এই প্রক্রিয়া অত্যন্ত দ্রুত, কারণ এতে মাত্র কয়েকটি Comparison প্রয়োজন হয় (গাছের গভীরতা অনুযায়ী)।

২২. সাক্ষাৎকারের প্রশ্ন (Interview Questions)

প্রশ্ন ১: Decision Tree-তে Gini Impurity এবং Entropy-এর মধ্যে প্রধান পার্থক্য কী? Scikit-Learn কেন ডিফল্ট হিসেবে criterion="gini" ব্যবহার করে?

উত্তর:

Gini Impurity এবং Entropy উভয়ই Decision Tree-এর প্রতিটি নোডের বিশুদ্ধতা (Purity) বা অবিশুদ্ধতা (Impurity) পরিমাপের জন্য ব্যবহৃত হয়। Entropy তথ্য তত্ত্ব (Information Theory)-এর উপর ভিত্তি করে কাজ করে এবং Information Gain ব্যবহার করে সর্বোত্তম Split নির্বাচন করে। এর গণনায় লগারিদম ($\log_2$) ব্যবহৃত হওয়ায় এটি তুলনামূলকভাবে বেশি কম্পিউটেশনাল সময় প্রয়োজন করে।

অন্যদিকে, Gini Impurity-এর সূত্র তুলনামূলকভাবে সহজ এবং এতে কোনো লগারিদমিক গণনা প্রয়োজন হয় না। ফলে এটি দ্রুত গণনা করা যায়। এই কারণেই Scikit-Learn ডিফল্টভাবে criterion="gini" ব্যবহার করে। তবে বাস্তবে Gini এবং Entropy উভয়ের Performance সাধারণত খুব কাছাকাছি হয়।

প্রশ্ন ২: Decision Tree-তে Overfitting কেন ঘটে এবং এটি প্রতিরোধ করার প্রধান উপায়গুলো কী?

উত্তর:

Decision Tree-কে যদি কোনো সীমাবদ্ধতা ছাড়া প্রশিক্ষণ দেওয়া হয়, তাহলে এটি যতক্ষণ সম্ভব Data Split করতে থাকে এবং প্রতিটি Leaf Node-কে যতটা সম্ভব বিশুদ্ধ করার চেষ্টা করে। এর ফলে মডেলটি শুধুমাত্র প্রকৃত Pattern নয়, বরং Training Data-এর Noise এবং ব্যতিক্রমী তথ্যও শিখে ফেলে। এই অবস্থাকে Overfitting বলা হয়।

Overfitting প্রতিরোধের জন্য সাধারণত নিচের Hyperparameter-গুলো ব্যবহার করা হয়:

- `max_depth` : Tree-এর সর্বোচ্চ গভীরতা নির্ধারণ করে।
- `min_samples_split` : একটি Node Split করার জন্য ন্যূনতম Sample সংখ্যা নির্ধারণ করে।

- `min_samples_leaf` : প্রতিটি Leaf Node-এ ন্যূনতম কতটি Sample থাকতে হবে তা নির্ধারণ করে।
- `ccp_alpha` : Cost-Complexity Pruning-এর মাধ্যমে অপ্ৰয়োজনীয় Branch অপসারণ করে।

প্রশ্ন ৩: যদি `max_depth=None` এবং `min_samples_split=2` একসাথে ব্যবহার করা হয়, তাহলে Decision Tree-এর আচরণ কেমন হবে?

উত্তর:

যখন `max_depth=None` এবং `min_samples_split=2` একসাথে ব্যবহার করা হয়, তখন Decision Tree-এর গভীরতার ওপর কোনো সীমা থাকে না। মডেলটি যতক্ষণ Node বিশুদ্ধ না হয়, Split করার জন্য পর্যাপ্ত Sample থাকে এবং একটি বৈধ Split পাওয়া যায়, ততক্ষণ পর্যন্ত Tree বৃদ্ধি পেতে থাকে।

ফলে Tree অত্যন্ত গভীর হয়ে যেতে পারে এবং Training Data-এর Noise পর্যন্ত শিখে ফেলতে পারে। এর ফলে Training Accuracy অনেক বেশি হলেও Test Data বা নতুন Data-এর উপর Performance কমে যেতে পারে, যা Overfitting-এর একটি সাধারণ লক্ষণ।

প্রশ্ন ৪: `feature_importances_` অ্যাট্রিবিউট কীভাবে কাজ করে এবং এটি কেন গুরুত্বপূর্ণ?

উত্তর:

`feature_importances_` প্রতিটি Feature Decision Tree তৈরির সময় মোট কতটুকু Impurity কমাতে সাহায্য করেছে, তার একটি পরিমাপ প্রদান করে। কোনো Feature যদি বারবার গুরুত্বপূর্ণ Split তৈরি করে এবং প্রতিবার উল্লেখযোগ্য পরিমাণে Gini Impurity বা Entropy কমায়, তাহলে তার Importance বেশি হয়।

সমস্ত Feature Importance-এর যোগফল সর্বদা **1.0** হয়।

এই Attribute ব্যবহার করে গুরুত্বপূর্ণ Feature শনাক্ত করা, অপ্ৰয়োজনীয় Feature বাদ দেওয়া এবং Feature Selection সম্পাদন করা সহজ হয়। ফলে Model আরও কার্যকর এবং ব্যাখ্যাযোগ্য (Interpretable) হয়।

প্রশ্ন ৫: Decision Tree-কে কেন একটি White-Box Model বলা হয়? কোন পরিস্থিতিতে এটি Logistic Regression-এর তুলনায় বেশি কার্যকর?

উত্তর:

Decision Tree-কে একটি **White-Box Model** বলা হয়, কারণ এর সিদ্ধান্ত গ্রহণের প্রতিটি ধাপ সহজেই দেখা, বোঝা এবং ব্যাখ্যা করা যায়। এটি মূলত ধারাবাহিক **IF-ELSE** নিয়মের মাধ্যমে সিদ্ধান্ত গ্রহণ করে, ফলে কোনো Prediction কীভাবে এসেছে তা সহজেই অনুসরণ করা সম্ভব।

যখন Feature এবং Target Variable-এর মধ্যে জটিল (Complex) এবং Non-linear সম্পর্ক থাকে, তখন Decision Tree সাধারণত Logistic Regression-এর তুলনায় ভালো ফলাফল প্রদান করতে পারে। কারণ Decision Tree সহজেই Non-linear Decision Boundary শিখতে পারে, যেখানে Logistic Regression মূলত Linear Decision Boundary-এর উপর নির্ভর করে।

২৩. সারাংশ (Summary)

Quick Revision

১. মূল ধারণা (Core Intuition)

- **Type:** এটি একটি অত্যন্ত জনপ্রিয় Supervised Machine Learning Algorithm, যা ক্লাসিফিকেশন ও রিগ্রেশন উভয় ক্ষেত্রে কাজ করে।
- **Mechanism:** এটি মানুষের চিন্তাভাবনার মতো ডেটার বিভিন্ন বৈশিষ্ট্যের ওপর ভিত্তি করে একটার পর একটা "IF-ELSE" বা "Yes/No" কন্ডিশন তৈরি করে পুরো ডেটাসেটকে ছোট ছোট ভাগে ভাগ করে ফেলে।
- **White-Box Model:** এর সিদ্ধান্ত নেওয়ার পুরো প্রক্রিয়াটি একটি ফ্লোচার্ট বা গাছের ডালপালার মতো দৃশ্যমান হওয়ায় একে হোয়াইট-বক্স মডেল বলা হয় (Highly Interpretable)।

২. গাঠনিক উপাদান (Tree Anatomy)

- **Root Node (মূল নোড):** গাছের একদম শীর্ষের নোড, যেখান থেকে প্রথম স্প্লিট বা বিভাজন শুরু হয়।
- **Internal / Decision Node:** মাঝখানের নোডগুলো, যা কোনো ফিচারের শর্ত নির্দেশ করে এবং ডেটাকে আরও নিচে ভাগ করে দেয়।
- **Leaf Node (চূড়ান্ত পাতা):** গাছের একদম শেষ প্রান্তের নোড। এর নিচে আর কোনো স্প্লিট হয় না, এটিই মূলত চূড়ান্ত প্রেডিকশন বা আউটপুট।

৩. ব্যাকএন্ডের গাণিতিক পরিমাপ (Mathematical Formulation)

মডেলটি কোন ফিচারের কোন মান দিয়ে নোড ভাগ করবে, তা নির্ধারণ করতে নিচের ৩টি ডিফল্ট ম্যাট্রিক্স ব্যবহার করে:

- **Entropy (এনট্রপি):** ডেটার মধ্যকার এলোমেলো অবস্থা বা বিশৃঙ্খলার পরিমাপ। সম্পূর্ণ অবিশুদ্ধ নোডে এর মান 1.0 এবং সম্পূর্ণ বিশুদ্ধ নোডে এর মান 0 হয়।

$$Entropy(S) = - \sum p_i \log_2 p_i$$

- **Information Gain (তথ্য লাভ):** কোনো ফিচার দিয়ে ডেটা ভাগ করলে এনট্রপি বা বিশৃঙ্খলা কতটুকু কমছে তার পরিমাপ। ডিসিশন ট্রি সবসময় সর্বোচ্চ Information Gain থাকা ফিচারটিকে স্প্লিট করার জন্য বেছে নেয়।
- **Gini Impurity (জিনি ইম্পিউরিটি):** Scikit-Learn-এর ডিফল্ট মেকানিজম। এটি কম্পিউটেশনালি অত্যন্ত ফাস্ট কারণ এতে কোনো লগারিদম হিসাব করতে হয় না। সম্পূর্ণ বিশুদ্ধ নোডের জিনি ইম্পিউরিটি হয় 0।

$$Gini = 1 - \sum (p_i)^2$$

৪. প্রধান হাইপারপ্যারামিটার ও ওভারফিটিং নিয়ন্ত্রণ (Top Hyperparameters)

ডিসিশন ট্রিকে বাধা না দিলে সে শেষ পর্যন্ত ডেটা মুখস্থ করে Overfitting (High Variance) ঘটায়। এটি বন্ধ করার মূল ব্রেকগুলো হলো:

- **max_depth:** গাছের সর্বোচ্চ গভীরতা বা লেভেল ফিক্সড করে দেওয়া। ওভারফিটিং কমানোর প্রধান হাতিয়ার।
- **min_samples_split:** একটি নোডকে ভাগের জন্য নূন্যতম কয়টি স্যাম্পল থাকতে হবে (ডিফল্ট: ২)। এর মান বাড়ালে ওভারফিটিং কমে।
- **min_samples_leaf:** একটি চূড়ান্ত পাতায় নূন্যতম কয়টি স্যাম্পল থাকতেই হবে (ডিফল্ট: ১)। এর মান বাড়ালে মডেল স্মুথ হয়।
- **max_features:** প্রতি স্প্লিটে সেরা শর্ত খোঁজার জন্য সর্বোচ্চ কয়টি ফিচার স্ক্যান করা হবে (যেমন: "sqrt", "log2")।
- **class_weight:** ইমব্যালেন্সড ডেটাসেটে ছোট ক্লাসকে বেশি গুরুত্ব দিতে "balanced" মোড ব্যবহার করা হয়।

১৭-১৮. সুবিধা ও অসুবিধা (Advantages & Disadvantages)

৫. সুবিধা ও সীমাবদ্ধতা (Pros & Cons)

Advantages (সুবিধাসমূহ)	Disadvantages / Limitations
সহজে বোঝা এবং ব্যাখ্যা করা যায় (Interpretability)	খুব দ্রুত Overfitting বা ডেটা মুখস্থ করার প্রবণতা দেখায়
ফিচার স্কেলিং বাধ্যতামূলক নয় (Scale-Independent)	ডেটার সামান্য পরিবর্তনে পুরো গাছের গঠন বদলে যায় (Unstable)
আউটলায়ারের প্রতি অত্যন্ত Robust (প্রভাব পড়ে না)	রিগ্রেশনের ক্ষেত্রে আউটপুট মসৃণ না হয়ে সিঁড়ির মতো ধাপযুক্ত হয়
ক্যাটাগরিক্যাল ও নিউমেরিক্যাল ডেটা একসাথে সামলাতে পারে	ক্যাটাগরির সংখ্যা অনেক বেশি হলে (High Cardinality) গাছ জটিল হয়

২১. সাধারণ ভুলক্রটি (Common Mistakes)

- `max_depth` বা কোনো Pruning Parameter না দিয়ে গাছকে সম্পূর্ণ অবাধে বাড়তে দেওয়া — এতে Severe Overfitting ঘটে (গাছ Training Data-র Noise পর্যন্ত মুখস্থ করে ফেলে)।
- Gini ও Entropy-এর মধ্যে পার্থক্যকে অতিরিক্ত গুরুত্ব দেওয়া — বাস্তবে দুটি প্রায় একই রকম ফলাফল দেয়, শুধু Gini কম্পিউটেশনালি একটু দ্রুত।
- Imbalanced Dataset-এ `class_weight='balanced'` ব্যবহার না করা, যার ফলে গাছ শুধু Majority Class-কে প্রাধান্য দেয়।
- একটি একক Decision Tree-কে চূড়ান্ত Production Model হিসেবে ব্যবহার করা — বাস্তবে একক ট্রি প্রায়ই Unstable (ডেটার সামান্য পরিবর্তনেও সম্পূর্ণ ভিন্ন গাছ তৈরি হতে পারে); Random Forest-এর মতো Ensemble পদ্ধতি এই সমস্যা সমাধান করে।

সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)

Introduction

এই সেকশনে একটি ৫-স্যাম্পলের ছোট ডেটাসেট নিয়ে সম্পূর্ণ Decision Tree ম্যানুয়ালি তৈরি করা হবে — Entropy, Gini এবং Information Gain হাতে-কলমে গণনা করে, প্রতিটি সম্ভাব্য Split তুলনা করে, এবং ধাপে ধাপে সম্পূর্ণ ট্রি নির্মাণ করে।

Dataset

Sample	Outlook	Temperature	Play (Label)
1	Sunny	Hot	No
2	Sunny	Cool	Yes
3	Rainy	Hot	No
4	Rainy	Cool	Yes
5	Sunny	Cool	No

Possible Splits

দুইটি Feature আছে — Outlook এবং Temperature। প্রতিটির জন্য আলাদাভাবে Split করে Entropy, Gini ও Information Gain গণনা করা হবে।

Entropy Calculation (Root Node / পুরো ডেটাসেট)

$$\text{Entropy}(S) = - \sum p_i \cdot \log_2(p_i)$$

পুরো ডেটাসেটে Yes = 2টি, No = 3টি, মোট $n = 5$ ।

$$\text{Entropy}(S) = -[(0.4000) \cdot \log_2(0.4000) + (0.6000) \cdot \log_2(0.6000)] = -[-0.5288 + -0.4422] = 0.9710$$

$$\text{Gini}(S) = 1 - \sum p_i^2$$

$$\text{Gini}(S) = 1 - [(0.4000)^2 + (0.6000)^2] = 1 - [0.1600 + 0.3600] = 0.4800$$

Information Gain Calculation — প্রতিটি Candidate Split তুলনা

$$\text{IG}(S, F) = \text{Entropy}(S) - \sum (|S_v|/|S|) \cdot \text{Entropy}(S_v)$$

Split by Outlook:

Outlook = Rainy: লেবেল = ['No', 'Yes'], কাউন্ট = {'No': 1, 'Yes': 1}, Entropy = 1.0000, Gini = 0.5000, গুণন = $2/5 = 0.40$

Outlook = Sunny: লেবেল = ['No', 'Yes', 'No'], কাউন্ট = {'No': 2, 'Yes': 1}, Entropy = 0.9183, Gini = 0.4444, গুণন = $3/5 = 0.60$

গুণন Entropy = 0.9510 $\rightarrow$ Information Gain = $0.9710 - 0.9510 = 0.0200$

গুণন Gini = 0.4667 $\rightarrow$ Gini হ্রাস = $0.4800 - 0.4667 = 0.0133$

Split by Temperature:

Temperature = Cool: লেবেল = ['Yes', 'Yes', 'No'], কাউন্ট = {'Yes': 2, 'No': 1}, Entropy = 0.9183, Gini = 0.4444, গুণন = $3/5 = 0.60$

Temperature = Hot: লেবেল = ['No', 'No'], কাউন্ট = {'No': 2}, Entropy = 0.0000, Gini = 0.0000, গুণন = $2/5 = 0.40$

গুণন Entropy = 0.5510 $\rightarrow$ Information Gain = $0.9710 - 0.5510 = 0.4200$

গুণন Gini = 0.2667 $\rightarrow$ Gini হ্রাস = $0.4800 - 0.2667 = 0.2133$

Feature (Split Candidate)	Weighted Entropy	Information Gain
Outlook	0.9510	0.0200
Temperature	0.5510	0.4200

Choose the Best Split (Level 1 / Root)

যেহেতু Information Gain(Temperature) = 0.4200, যা Outlook-এর Gain (0.0200)-এর চেয়ে অনেক বেশি, তাই Root Node-এ প্রথম Split হবে "Temperature" ফিচার দিয়ে।

Repeat for the Next Level

Temperature = Hot শাখা: [('Sunny', 'No'), ('Rainy', 'No')] — উভয় লেবেলই "No", তাই Entropy = 0 এবং এটি সরাসরি একটি Pure Leaf Node: প্রেডিকশন = No।

Temperature = Cool শাখা: [('Sunny', 'Yes'), ('Rainy', 'Yes'), ('Sunny', 'No')] — এখানে Yes ও No উভয়ই আছে, তাই এই শাখাকে আরও ভাগ করতে হবে। এখানে বাকি একমাত্র ফিচার হলো Outlook।

Entropy(Cool subset) = 0.9183 (আগেই গণনা করা হয়েছে)।

Outlook = Rainy (Cool subset-এর মধ্যে): লেবেল = ['Yes'], Entropy = 0.0000, ওজন = $1/3 = 0.3333$

Outlook = Sunny (Cool subset-এর মধ্যে): লেবেল = ['Yes', 'No'], Entropy = 1.0000, ওজন = $2/3 = 0.6667$

$IG(\text{Cool subset, Outlook}) = 0.9183 - 0.6667 = 0.2516$ — যেহেতু এটিই একমাত্র বাকি ফিচার, এই Split গ্রহণ করা হলো।

Outlook = Rainy (Cool subset-এর মধ্যে): [('Rainy', 'Yes')] — Pure, সব "Yes", তাই Leaf Node: প্রেডিকশন = Yes।

Outlook = Sunny (Cool subset-এর মধ্যে): [('Sunny', 'Yes'), ('Sunny', 'No')] — এখানে একটি Yes ও একটি No আছে (Tie), এবং আর কোনো ফিচার অবশিষ্ট নেই বিভক্ত করার জন্য। তাই এই Node-এ পুরো Training Dataset-এর সংখ্যাগরিষ্ঠ ক্লাস (No = 3, Yes = 2) অনুযায়ী Leaf প্রেডিকশন ধরা হলো: No।

Construct the Complete Tree Manually

সম্পূর্ণ Decision Tree:

Root: Temperature?

├── Hot → Leaf: No

└── Cool → Outlook?

 ├── Rainy → Leaf: Yes

 └── Sunny → Leaf: No (Tie-break, parent majority class অনুযায়ী)

Training Progress Table (Split Comparison Table)

Node	Feature Tried	Information Gain	Decision
Root	Outlook	0.0200	বাতিল (কম Gain)
Root	Temperature	0.4200	নির্বাচিত (Best Split)
Cool subset	Outlook	0.2516	নির্বাচিত (একমাত্র বিকল্প)

Prediction on a New Sample

নতুন স্যাম্পল: Outlook = Sunny, Temperature = Cool।

দ্রি অনুসরণ করে: Root-এ Temperature=Cool → Cool শাখায় যাও → সেখানে Outlook=Sunny → Leaf Node "No"-তে পৌঁছাও। তাই Final Prediction = No।

Interpretation

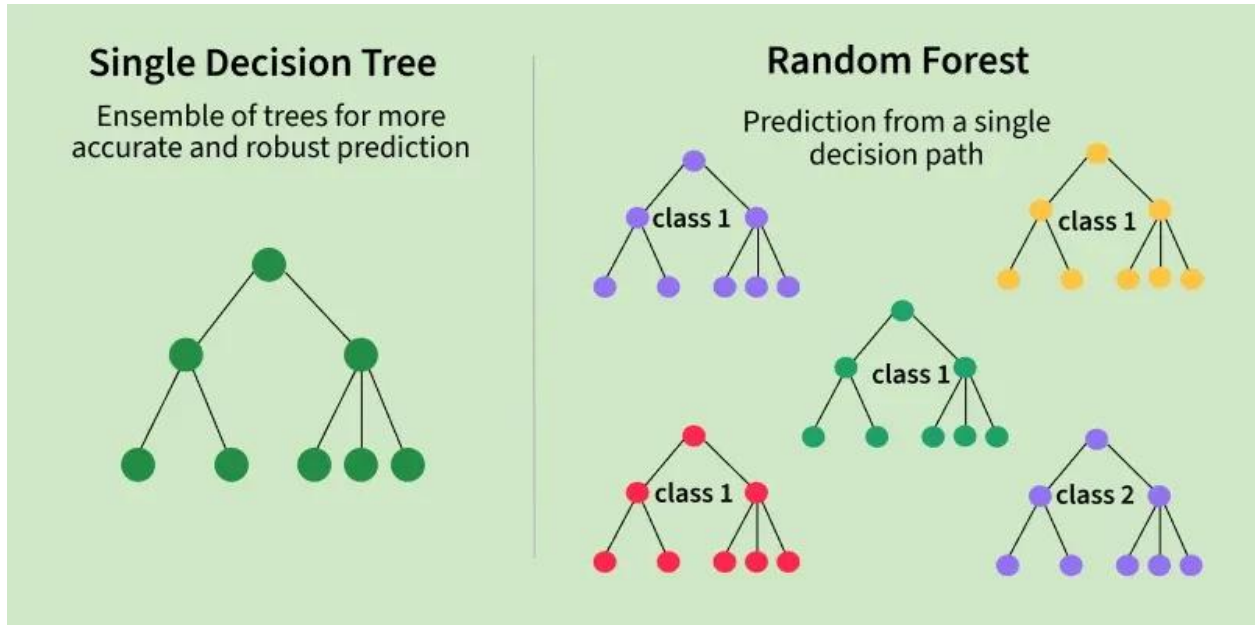
এই উদাহরণ দেখায় কেন Temperature-কে Root Split হিসেবে বেছে নেওয়া হলো — কারণ এটি Outlook-এর তুলনায় ডেটাকে অনেক বেশি বিশুদ্ধ (Purer) উপসেট-এ ভাগ করে (Information Gain 0.42 বনাম 0.02)। তবে লক্ষণীয় যে Sunny+Cool শাখায় শুধুমাত্র ২টি স্যাম্পল অবশিষ্ট ছিল এবং তাদের লেবেল ভিন্ন ছিল (Tie) — বাস্তব জীবনে এত ছোট Leaf Node-এ Overfitting-এর ঝুঁকি থাকে, যা max_depth বা min_samples_leaf-এর মতো Hyperparameter দিয়ে নিয়ন্ত্রণ করা হয়।

Random Forest

১. ভূমিকা (Introduction)

Random Forest (র্যান্ডম ফরেস্ট) হলো মেশিন লার্নিংয়ের অত্যন্ত শক্তিশালী, জনপ্রিয় এবং বহুমুখী একটি Supervised Learning Algorithm, যা ক্লাসিফিকেশন (Classification) এবং রিগ্রেশন (Regression) দুই ধরনের সমস্যার সমাধানই নিখুঁতভাবে করতে পারে।

সহজ কথায়, অনেকগুলো ডিসিশন ট্রি (Decision Tree) বা সিদ্ধান্ত-গাছ একসাথে মিলে যে জঙ্গল বা ফরেস্ট তৈরি করে, সেটিই হলো র্যান্ডম ফরেস্ট। এটি একটি Ensemble Learning মেথড, যার মূল মন্ত্র হলো—"একটি একক গাছের সিদ্ধান্তের চেয়ে পুরো জঙ্গলের সম্মিলিত সিদ্ধান্ত অনেক বেশি নিখুঁত ও শক্তিশালী।"



২. কেন এই অ্যালগরিদম তৈরি হয়েছিল (Why This Algorithm Was Developed)

একক Decision Tree-এর সবচেয়ে বড় সমস্যা ছিল High Variance — Training Data সামান্য পরিবর্তন হলেই সম্পূর্ণ ভিন্ন একটি গাছ তৈরি হয়ে যেত, যা Model-কে অস্থিতিশীল করে তুলত। ১৯৯৫ সালে Tin Kam Ho এবং পরে ২০০১ সালে Leo Breiman এই সমস্যার সমাধান হিসেবে Random Forest প্রস্তাব করেন — একাধিক Decision Tree-কে ভিন্ন ভিন্ন Random Data Subset ও Feature Subset দিয়ে Train করে তাদের ফলাফল একত্রিত (Ensemble) করার মাধ্যমে। পরিসংখ্যানের "Law of Large Numbers" নীতি অনুযায়ী, অনেকগুলো স্বাধীন, বৈচিত্র্যময় গাছের গড় ফলাফল একক গাছের চেয়ে অনেক বেশি স্থিতিশীল ও নির্ভরযোগ্য হয়।

৩. বাস্তব জীবনের প্রেরণা (Real-life Motivation)

ধরুন, আপনি ছুটিতে কোথাও ঘুরতে যাবেন এবং একটি সুন্দর জায়গা সিলেক্ট করতে চান।

- **ডিসিশন ট্রির পদ্ধতি:** আপনি আপনার একজন বন্ধুকে জিজ্ঞেস করলেন। সে তার ব্যক্তিগত অভিজ্ঞতা অনুযায়ী আপনাকে কক্সবাজার যাওয়ার পরামর্শ দিল। এখানে ভুল হওয়ার সম্ভাবনা বা বায়াসড (Biased) হওয়ার চান্স বেশি, কারণ এটি মাত্র একজন মানুষের মত।
- **র‍্যান্ডম ফরেস্টের পদ্ধতি:** আপনি ১ জনের কাছে না গিয়ে আপনার ২০ জন বন্ধুকে আলাদা আলাদাভাবে জিজ্ঞেস করলেন। কেউ বলল সিলেট, কেউ বলল সাজেক, আবার বেশিরভাগ বন্ধু বলল কক্সবাজার। এবার আপনি সবার মতামত নিয়ে ভোটিং (Majority Voting) করলেন এবং দেখলেন যে ২০ জনের মধ্যে ১৪ জনই কক্সবাজারের পক্ষে। আপনি মেজরিটি ভোটের ভিত্তিতে সিদ্ধান্ত নিলেন কক্সবাজার যাবেন। র‍্যান্ডম ফরেস্ট ঠিক এইভাবেই কাজ করে।

৪. স্বজ্ঞা (Intuition)

সহজভাবে বললে, Random Forest হলো "বুদ্ধিমান ভিড়ের প্রজ্ঞা" (Wisdom of the Crowd)-এর গাণিতিক রূপ। প্রতিটি একক গাছ কিছুটা ভুল করতে পারে, কিন্তু যদি গাছগুলো একে অপরের থেকে যথেষ্ট স্বাধীন (বিভিন্ন Data ও Feature দেখে Train হওয়া) হয়, তাহলে তাদের ভুলগুলো এলোমেলো দিকে ছড়িয়ে থাকে এবং Majority Voting বা Averaging করার সময় একে অপরকে কাটাকাটি করে ফেলে — ফলে সামগ্রিক ফলাফল একক গাছের চেয়ে বেশি নির্ভুল হয়।

৫. কার্যপ্রণালী (Working Principle)

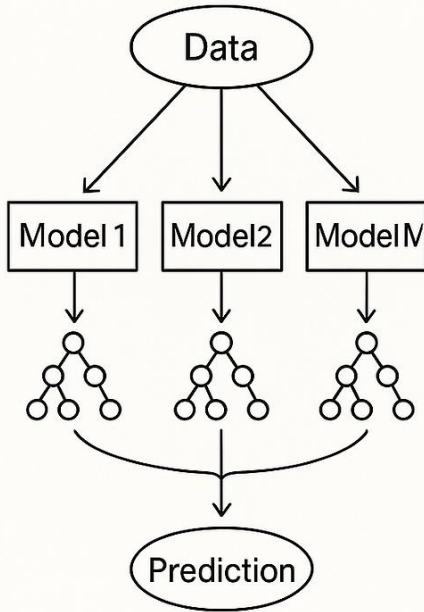
র‍্যান্ডম ফরেস্টের ব্যাকএন্ড মেকানিজম

র‍্যান্ডম ফরেস্ট মূলত দুটি প্রধান স্তরের ওপর দাঁড়িয়ে কাজ করে:

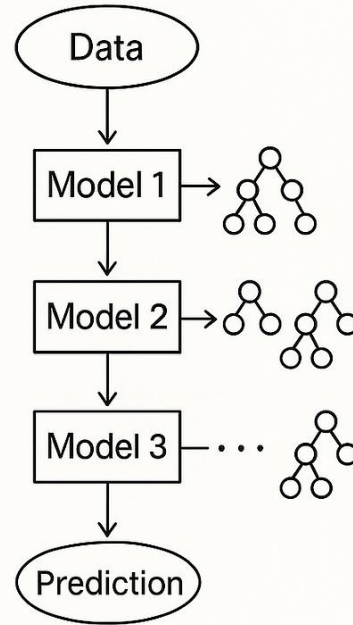
১. Bagging বা Bootstrap Aggregating (বুটস্ট্র্যাপ অ্যাগ্রিগেটিং)

আপনার কাছে যদি একটি মূল ডেটাসেট থাকে, র‍্যান্ডম ফরেস্ট সব গাছকে হুবহু এক ডেটা দেয় না। সে মূল ডেটাসেট থেকে এলোমেলোভাবে (Row Sampling with replacement) ডেটা তুলে ছোট ছোট সাব-ডেটাসেট তৈরি করে এবং এক একটি ডিসিশন ট্রি-কে ট্রেন করার জন্য দেয়। একে বলে Bootstrapping।

Bagging



Boosting



২. Feature Randomness (ফিচারের র্যান্ডমনেস)

শুধু ডেটা রো (Row) নয়, প্রতিটা ডিসিশন ট্রি নোড স্প্লিট করার সময় ডেটাসেটের সব কলাম বা ফিচার দেখতে পায় না। সে সম্পূর্ণ র্যান্ডমলি কিছু ফিচার (যেমন: মোট ১০টি ফিচারের মধ্যে ৩টি) বেছে নিয়ে তার মধ্য থেকে সেটা স্প্লিট খুঁজে নেয় (max_features='sqrt')। এর ফলে জঙ্গলের প্রতিটা গাছ একে অপরের থেকে সম্পূর্ণ আলাদা ও স্বাধীনভাবে গড়ে ওঠে।

৩. Aggregation (একত্রীকরণ বা চূড়ান্ত সিদ্ধান্ত)

সবগুলো গাছ যখন ট্রেন হয়ে যায়, তখন নতুন কোনো ডেটা প্রেডিক্ট করার জন্য সেটিকে জঙ্গলের সব গাছের মধ্য দিয়ে পাস করানো হয়:

- **Classification-এর ক্ষেত্রে:** সব গাছের আউটপুটের মধ্যে Majority Voting বা সংখ্যাগরিষ্ঠের ভোটের ভিত্তিতে ফাইনাল ক্লাস নির্ধারণ করা হয়।
- **Regression-এর ক্ষেত্রে:** প্রতিটি গাছের দেওয়া প্রেডিকশনের গড় বা গড় মান (Average /Mean) হিসাব করে চূড়ান্ত সংখ্যাটি আউটপুট দেওয়া হয়।

১৭-১৮. সুবিধা ও অসুবিধা (Advantages & Disadvantages)

র্যান্ডম ফরেস্টের প্রধান সুবিধাসমূহ (Advantages)

1. **ওভারফিটিং মুক্ত (Robust to Overfitting):** ডিসিশন ট্রির সবচেয়ে বড় দুর্বলতা হলো ওভারফিটিং। র্যান্ডম ফরেস্ট অনেকগুলো গাছের ফলাফলের গড় বা ভোট নেয় বলে এতে ওভারফিটিংয়ের সম্ভাবনা প্রায় থাকে না বললেই চলে।
2. **মিসিং ভ্যালু হ্যান্ডলিং:** ডেটাসেটে যদি বড় অঙ্কের মিসিং ভ্যালু থাকে, তবুও এটি তার ব্যাকঅ্যান্ড প্রক্সিমিটি অ্যালগরিদম দিয়ে চমৎকার এক্যুরেসী ধরে রাখতে পারে।
3. **ফিচার ইম্পোর্টেন্স (Feature Importance):** ডিসিশন ট্রির মতো এটিও ট্রেনিং শেষে `feature_importances_` প্রপার্টি ব্যবহার করে ডেটাসেটের কোন কলামটি প্রেডিকশনের জন্য সবচেয়ে বেশি গুরুত্বপূর্ণ, তা নিখুঁতভাবে জানিয়ে দেয়।
4. **স্কেলিংয়ের প্রয়োজন নেই:** ডিসিশন ট্রির মতো র্যান্ডম ফরেস্টও কোনো প্রকার ফিচার স্কেলিং (StandardScaler বা MinMaxScaler) করার প্রয়োজন পড়ে না।

সীমাবদ্ধতা (Limitations)

- **ব্ল্যাক-বক্স মডেল (Black-box Nature):** একক ডিসিশন ট্রিকে খুব সহজে `plot_tree` দিয়ে ভিজুয়ালাইজ করে সাধারণ মানুষকে বোঝানো যায় (White-box)। কিন্তু র্যান্ডম ফরেস্ট যখন ৫০০টি গাছ একসাথে কাজ করে, তখন ঠিক কোন লজিকে সিদ্ধান্ত এসেছে তা ট্র্যাক করা অসম্ভব হয়ে পড়ে। তাই এটি একটি ব্ল্যাক-বক্স মডেল।
- **ধীরগতি (Slow to Predict):** র্যান্ডম ফরেস্ট ট্রেনিংয়ের সময় সমান্তরালভাবে (Parallely) কাজ করলেও, রিয়েল-টাইম প্রোডাকশনে প্রতিটি গাছের মধ্য দিয়ে ডেটা পাস করে ভোট গণনা করতে লিনিয়ার মডেলের চেয়ে সামান্য বেশি সময় নিতে পারে।

র্যান্ডম ফরেস্ট (Random Forest) ক্লাসিফিকেশন এবং রিগ্রেশন দুই ধরনের সমস্যার সমাধান করে।

১. Classification (শ্রেণিবিভাগ) এর ক্ষেত্রে:

- **মেকানিজম:** ক্লাসিফিকেশন সমস্যার জন্য র্যান্ডম ফরেস্ট তার জঙ্গলের প্রতিটি ডিসিশন ট্রি-কে আলাদা আলাদাভাবে ডেটা প্রেডিক্ট করতে দেয়।
- **চূড়ান্ত সিদ্ধান্ত:** সব গাছ থেকে প্রেডিকশন পাওয়ার পর, মডেলটি **Majority Voting** (সংখ্যার ভোট) গণনা করে। যে ক্লাস বা ক্যাটাগরিটি সবচেয়ে বেশি গাছ প্রেডিক্ট

করেছে (যেমন: ১০০টি গাছের মধ্যে ৭০টি গাছ বলল 'Yes' আর ৩০টি গাছ বলল 'No'), মেজরিটি ভোটের ভিত্তিতে চূড়ান্ত আউটপুট হবে 'Yes'।

২. Regression (মান অনুমান) এর ক্ষেত্রে:

- **মেকানিজম:** রিগ্রেশন সমস্যার ক্ষেত্রে (যেখানে আউটপুট কোনো সংখ্যা হয়), জঙ্গলের প্রতিটি ডিসিশন ট্রি স্বাধীনভাবে একটি সংখ্যা বা মান প্রেডিক্ট করে।
- **চূড়ান্ত সিদ্ধান্ত:** সব গাছের দেওয়া সংখ্যাচক প্রেডিকশনগুলোর গড় বা গড় মান (Average/Mean) হিসাব করা হয়। এই গড় মানটিই হয় মডেলের চূড়ান্ত প্রেডিকশন (যেমন: ৫টি গাছ একটি বাড়ির দাম বলল যথাক্রমে ৫০, ৫২, ৪৮, ৫১ এবং ৪৯ লাখ টাকা; তাহলে র‍্যাশম ফরেস্ট এদের গড় করে চূড়ান্ত দাম বলবে ৫০.২ লাখ টাকা)।

১৯. প্রয়োগক্ষেত্র (Applications)

- **ব্যাংকিং:** ক্রেডিট স্কোরিং ও ঋণ Default ঝুঁকি নির্ধারণে, যেখানে অনেকগুলো Feature (আয়, ইতিহাস, বয়স) বিবেচনা করতে হয়।
- **স্বাস্থ্যসেবা:** জিনগত ডেটা বা মেডিকেল রিপোর্ট থেকে রোগ নির্ণয়ে, যেখানে High-Dimensional Feature Space থাকে।
- **ই-কমার্স:** Product Recommendation ও Customer Segmentation-এ।
- **Finance:** Stock Market Trend Prediction ও Fraud Detection-এ, যেখানে Robustness অত্যন্ত গুরুত্বপূর্ণ।

Decision Tree থেকে Random Forest-এ রূপান্তর এবং এর প্রয়োজনীয়তা

Bias (বায়াস)

- **Core Concept:** একটি মেশিন লার্নিং মডেলের সরল অনুমানের কারণে যে ভুলের সৃষ্টি হয়, তাকে বায়াস বলে। মডেল যদি ডেটার ভেতরের আসল এবং জটিল প্যাটার্ন ধরতে না পারে, তবে তাকে High Bias বলা হয়। এর ফলে মডেলটি ট্রেনিং ডেটা এবং টেস্ট ডেটা-উভয় ক্ষেত্রেই খুব খারাপ পারফর্ম করে, যাকে মেশিন লার্নিংয়ের ভাষায় Underfitting বলা হয় (যেমন: একটি জটিল লিনিয়ার-নয় এমন ডেটার ওপর

সাধারণ Linear Regression ব্যবহার করা)। অন্যদিকে, মডেল যদি ডেটার প্যাটার্ন খুব ভালোভাবে ধরতে পারে, তবে তাকে Low Bias বলা হয়।

Variance (ভ্যারিয়েন্স)

- Core Concept: ট্রেনিং ডেটার সামান্য পরিবর্তনের কারণে মডেলের প্রেডিকশনে যে পরিমাণ ওলটপালট বা বিচ্যুতির সৃষ্টি হয়, তাকে ভ্যারিয়েন্স বলে। মডেল যদি ট্রেনিং ডেটার প্রতিটি নয়েজ (Noise) ও ব্যতিক্রমী পয়েন্ট মুখস্থ করে ফেলে, তবে তাকে High Variance বলা হয়। এর ফলে মডেলটি ট্রেনিং সেটে ১০০% এক্যুরেসী দিলেও নতুন কোনো টেস্ট ডেটার সামনে গেলেই সম্পূর্ণ ফেইল করে, যাকে Overfitting বলা হয় (যেমন: গভীর ডালপালা বিশিষ্ট একটি আনফ্রন্ড Decision Tree)। আর মডেল যদি নতুন ডেটাতেও স্থিতিশীল পারফর্ম করে, তবে তাকে Low Variance বলা হয়।

2. The 4 Quadrants Matrix

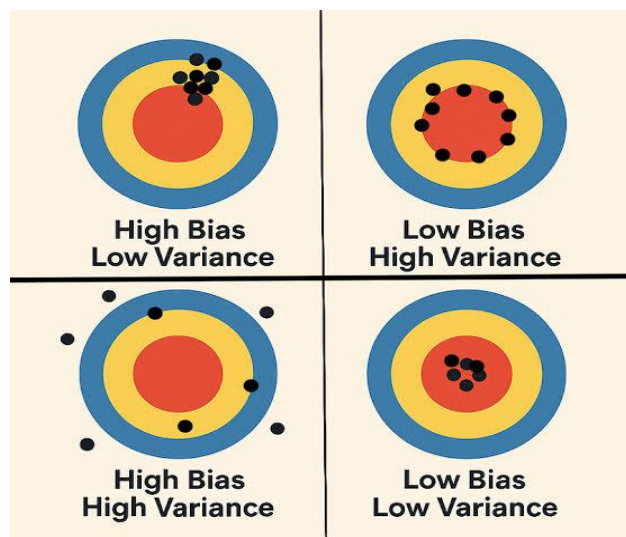
মেশিন লার্নিং মডেলের কার্যক্ষমতা বুঝতে একটি জনপ্রিয় বুলস-আই (Bulls-eye) বা লক্ষ্যভেদের ডার্টবোর্ডের উদাহরণ ব্যবহার করা হয়, যেখানে বোর্ডের একদম কেন্দ্রটি (Center) হলো আমাদের আসল বা সঠিক উত্তর:

Low Bias, Low Variance (আদর্শ মডেল)

- ব্যাখ্যা: এই মডেলের প্রেডিকশনগুলো একদম নিখুঁত এবং কেন্দ্রের খুব কাছাকাছি থাকে (Low Bias)। একই সাথে ডেটা পরিবর্তন করলেও এর সিদ্ধান্ত বদলে যায় না, সবগুলো শট এক জায়গাতেই পড়ে (Low Variance)। এটিই মেশিন লার্নিংয়ের সবচেয়ে কাঙ্ক্ষিত ও সুষম মডেল।

Low Bias, High Variance (ওভারফিটিং)

- ব্যাখ্যা: এই মডেলটি গড়ে সঠিক উত্তরের কাছাকাছি পৌঁছাতে পারে (Low Bias)। কিন্তু এর প্রেডিকশনগুলোর মধ্যে ছড়ানো-ছিটানো ভাব বা অস্থিরতা অনেক বেশি (High Variance)। অর্থাৎ, ট্রেনিং



ডেটা সামান্য পরিবর্তন হলেই এর প্রেডিকশনগুলো চারদিকে ছড়িয়ে পড়ে। এটি Overfitting-এর স্পষ্ট লক্ষণ।

High Bias, Low Variance (আন্ডারফিটিং)

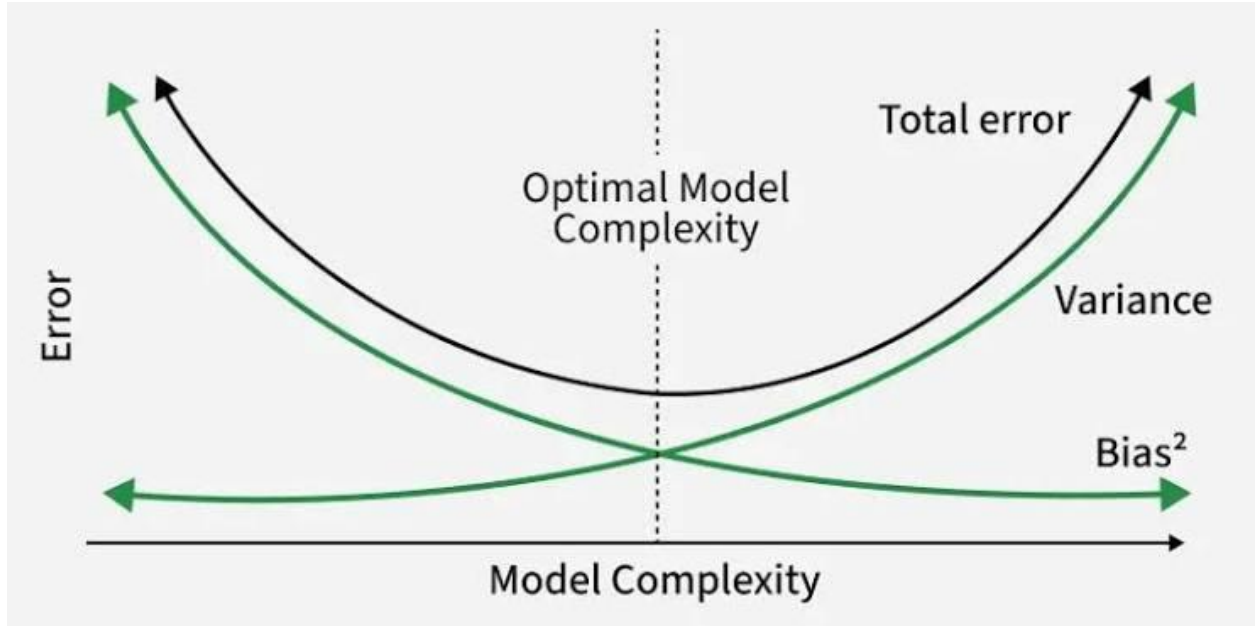
- ব্যাখ্যা: এই মডেলের সবগুলো প্রেডিকশন খুব কাছাকাছি এক জায়গাতেই পড়ে (Low Variance), কিন্তু সেগুলো আসল লক্ষ্য বা কেন্দ্র থেকে অনেক দূরে অবস্থান করে (High Bias)। মডেলটি ভুল লজিকে অনড় থাকে। এটি Underfitting-এর উদাহরণ।

High Bias, High Variance (সবচেয়ে বাজে মডেল)

- ব্যাখ্যা: এই মডেলটির প্রেডিকশনগুলো যেমন লক্ষ্যভ্রষ্ট বা কেন্দ্র থেকে দূরে (High Bias), তেমনই সেগুলো একে অপরের থেকে চরমভাবে ছড়ানো-ছিটানো (High Variance)। এটি সবচেয়ে অস্থিতিশীল এবং অকেজো মডেল।

3. The Bias-Variance Trade-off

- The Conflict: গাণিতিক ও বাস্তবিকভাবে একই সাথে বায়াস এবং ভ্যারিয়েন্স—উভয়কেই একসাথে শূন্যে নামিয়ে আনা অসম্ভব। আপনি যদি মডেলকে খুব জটিল (Complex) বানাতে যান (যেমন: Deep Neural Networks বা Deep Decision Trees), তবে বায়াস কমবে কিন্তু ভ্যারিয়েন্স আশঙ্কাজনকভাবে বেড়ে যাবে। আবার মডেলকে খুব সরল (Simple) রাখলে ভ্যারিয়েন্স কম থাকবে ঠিকই, কিন্তু বায়াস বেড়ে যাবে।



- The Goal: মেশিন লার্নিং ইঞ্জিনিয়ারদের মূল লক্ষ্য হলো মডেলের জটিলতার এমন একটি সর্বোত্তম বিন্দু বা সুইট স্পট (Sweet Spot) খুঁজে বের করা, যেখানে Total Error (যা বায়াস এবং ভ্যারিয়েন্সের বর্গের যোগফল) সর্বনিম্ন হয়। র্যান্ডম ফরেস্ট (Random Forest) অ্যালগরিদমটি মূলত প্রতিটি গাছের লো-বায়াস ক্ষমতাকে ধরে রেখে, ব্যাগিং প্রক্রিয়ার মাধ্যমে হাই-ভ্যারিয়েন্সকে টেনে নিচে নামিয়ে এনে এই চমৎকার ভারসাম্যটি বজায় রাখে।

The Evolution

- **মূল সমস্যা:** একটি একক ডিসিশন ট্রি ট্রেনিং ডেটাসেটের শেষ সীমা পর্যন্ত ডালপালা গজাতে থাকে এবং ডেটাকে ১০০% বিশুদ্ধ (Pure) করার চেষ্টা করে। এর ফলে এটি ডেটার ভেতরের নয়েজ (Noise) বা ছোটখাটো ভুল প্যাটার্নও মুখস্থ করে ফেলে এবং মডেলটিতে হাই ভ্যারিয়েন্স বা ওভারফিটিং (Overfitting) দেখা দেয়। এমন মডেল নতুন কোনো টেস্ট ডেটার মুখোমুখি হলে সঠিক সিদ্ধান্ত দিতে পারে না।
- **সমাধান:** ডিসিশন ট্রির জটিল নন-লিনিয়ার সম্পর্ক ধরার ক্ষমতাকে অক্ষুণ্ণ রেখে ওভারফিটিং দূর করার জন্য র্যান্ডম ফরেস্টের আগমন ঘটে। এখানে একটিমাত্র গভীর ও জটিল গাছের ওপর নির্ভর না করে, শত শত ভিন্ন ভিন্ন স্বাধীন ডিসিশন ট্রি (একটি জঙ্গল) তৈরি করা হয়। প্রতিটি গাছের নিজস্ব ভুল বা ত্রুটিগুলো যখন একত্রিত করে গড় করা হয়, তখন সামগ্রিক জঙ্গলের সিদ্ধান্তটি অত্যন্ত সুসম ও নিখুঁত হয়ে ওঠে।

Why We Moved to Random Forest

- **ওভারফিটিং ও ভ্যারিয়েন্স নিয়ন্ত্রণ (Reduction in Variance):** একটি একক ডিসিশন ট্রিতে লো বায়াস (Low Bias) কিন্তু হাই ভ্যারিয়েন্স (High Variance) থাকে। র্যান্ডম ফরেস্ট শত শত গাছের ফলাফলের গড় বা ভোট নেয় বলে এই হাই ভ্যারিয়েন্স বা ওভারফিটিংয়ের ঝুঁকি স্বয়ংক্রিয়ভাবে কমে যায়। কোনো একটি গাছের ভুল সিদ্ধান্ত বাকি গাছগুলোর সঠিক সিদ্ধান্তের ভিড়ে ঢাকা পড়ে যায়।
- **ফিচারের র্যান্ডমনেস (Feature Randomness):** একটি সাধারণ ডিসিশন ট্রি প্রতিটি নোড স্প্লিট করার সময় ডেটাসেটের উপলব্ধ সবকটি কলাম স্ক্যান করে সেরা কলামটি বেছে নেয়, যার ফলে শক্তিশালী ফিচারগুলোই বারবার ঘুরেফিরে আসে। কিন্তু র্যান্ডম ফরেস্ট প্রতি স্প্লিটে সম্পূর্ণ র্যান্ডমলি নির্দিষ্ট সংখ্যক ফিচারের একটি সাবসেট (*max_features*) বেছে নেয়। এর ফলে জঙ্গলের গাছগুলোর মধ্যে চরম গাঠনিক বৈচিত্র্য তৈরি হয় এবং ডেটার ভেতরে লুকিয়ে থাকা দুর্বল ফিচারের গুরুত্বপূর্ণ প্যাটার্নও উন্মোচিত হয়।
- **গাঠনিক স্থিতিশীলতা (Structural Stability):** একটি একক ডিসিশন ট্রি অত্যন্ত সংবেদনশীল বা আনস্টেবল। ট্রেনিং ডেটার সামান্য পরিবর্তন বা দু-একটি নতুন রো

যুক্ত হলে এর রুট নোডের স্প্লিট বদলে যেতে পারে, যা পুরো গাছের গঠন ও সিদ্ধান্ত ওলটপালট করে দেয়। র্যান্ডম ফরেস্ট বুটস্ট্রাপ স্যাম্পলিং ব্যবহারের মাধ্যমে প্রতিটি গাছকে ডেটার ভিন্ন ভিন্ন অংশ দিয়ে ট্রেন করে, ফলে ডেটার আংশিক পরিবর্তনে পুরো ফরেস্টের ওপর কোনো নেতিবাচক প্রভাব পড়ে না।

- **ব্যখ্যাযোগ্যতা হ্রাস পাওয়া (Loss of Full Interpretability):** একটি একক গাছ থেকে যখন আমরা শত শত গাছের জঙ্গলে স্থানান্তরিত হই, তখন মডেলটি একটি স্পষ্ট হোয়াইট-বক্স (White-Box) থেকে জটিল ব্ল্যাক-বক্স (Black-Box) সিস্টেমে পরিণত হয়। একসাথে ৫০০টি গাছের হাজার হাজার কন্ডিশনাল পাথ (if-else রুলস) ট্র্যাক করা মানুষের পক্ষে অসম্ভব হয়ে পড়ে। উচ্চমাত্রার এক্যুরেসী অর্জনের জন্য এটিই হলো র্যান্ডম ফরেস্টের প্রধান ট্রেড-অফ (Trade-off)।

২০. পূর্ববর্তী অ্যালগরিদমের সাথে তুলনা (Comparison with Previous Algorithm)

ডিসিশন ট্রি বনাম র্যান্ডম ফরেস্ট (Decision Tree vs Random Forest)

বৈশিষ্ট্য	Decision Tree	Random Forest
গঠন	মাত্র একটি একক গাছ দিয়ে গঠিত হয়।	শত শত ডিসিশন ট্রির সমন্বয়ে গঠিত জঙ্গল।
ওভারফিটিং ঝুঁকি	অত্যন্ত বেশি (High Variance)। খুব দ্রুত ডেটা মুখস্থ করে ফেলে।	অত্যন্ত কম। অনেক গাছের গড় নেওয়ার ফলে ওভারফিটিং স্বয়ংক্রিয়ভাবে দূর হয়।
এক্যুরেসী	মঝঝরি বা তুলনামূলক কম।	সাধারণত অত্যন্ত নিখুঁত এবং উচ্চ এক্যুরেসী দেয়।
কম্পিউটেশন টাইম	খুব দ্রুত ট্রেন হয়।	ট্রেনিং হতে কিছুটা বেশি সময় ও মেমোরি লাগে।

এনসেম্বল লার্নিংয়ের প্রধান দুটি পদ্ধতি

১. Bagging (Bootstrap Aggregating) বা সমান্তরাল পদ্ধতি

- **কাজের প্রক্রিয়া:** এই পদ্ধতিতে মূল ডেটাসেট থেকে এলোমেলোভাবে প্রতিস্থাপনের মাধ্যমে (With replacement) একাধিক স্বাধীন সাব-ডেটাসেট বা বুটস্ট্র্যাপ স্যাম্পল তৈরি করা হয়। এরপর প্রতিটি স্যাম্পলের ওপর আলাদা আলাদা মডেলকে (যেমন: ডিসিশন ট্রি) সম্পূর্ণ স্বাধীন এবং সমান্তরালভাবে (Parallely) ট্রেন করা হয়।
- **চূড়ান্ত সিদ্ধান্ত:** সবকটি মডেলের ট্রেনিং শেষে তাদের আউটপুটগুলোকে একত্রিত করা হয়। ক্লাসিফিকেশনের ক্ষেত্রে মেজরিটি ভোটিং (Majority Voting) এবং রিগ্রেশনের ক্ষেত্রে সব মডেলের প্রেডিকশনের গড় (Averaging) হিসাব করে চূড়ান্ত সিদ্ধান্ত নেওয়া হয়।
- **প্রধান লক্ষ্য:** এই পদ্ধতির মূল লক্ষ্য হলো মডেলের ভ্যারিয়েন্স (Variance) বা ওভারফিটিংয়ের ঝুঁকি কমিয়ে আনা।
- **উদাহরণ:** Random Forest।

২. Boosting বা ধারাবাহিক পদ্ধতি

- **কাজের প্রক্রিয়া:** বুস্টিং পদ্ধতিতে মডেলগুলো সমান্তরালভাবে কাজ করে না, বরং সিকোয়েন্সিয়াল বা ধারাবাহিকভাবে (One after another) কাজ করে। এখানে প্রথমে একটি দুর্বল বেস মডেল (Weak Learner) দিয়ে ট্রেনিং শুরু করা হয়। প্রথম মডেলটি ট্রেন করার পর যে ডেটা পয়েন্টগুলোতে ভুল প্রেডিকশন বা ত্রুটি (Errors) থেকে যায়, পরবর্তী মডেলটি তৈরি করার সময় সেই ভুল ডেটাগুলোর ওপর বেশি গুরুত্ব বা ওয়েট (Weight) দেওয়া হয়।
- **ভুল থেকে শিক্ষা:** এভাবে প্রতিটি নতুন মডেল তার আগের মডেলের করা ভুলগুলো থেকে শিক্ষা নিয়ে নিজে থেকে ক্রমান্বয়ে উন্নত করতে থাকে। অর্থাৎ, আগের মডেলের দুর্বলতাকে পরের মডেলটি এসে সংশোধন করে।
- **চূড়ান্ত সিদ্ধান্ত:** সবকটি মডেল ধারাবাহিকভাবে ট্রেন হওয়ার পর, তাদের ওজনের ওপর ভিত্তি করে একটি ভরযুক্ত যোগফল (Weighted Average/Sum) তৈরি করে চূড়ান্ত প্রেডিকশন দেওয়া হয়।
- **প্রধান লক্ষ্য:** এই পদ্ধতির মূল লক্ষ্য হলো মডেলের বায়াস (Bias) বা ভুলের পরিমাণ কমিয়ে আনা এবং মডেলের শক্তি বৃদ্ধি করা। তবে এটি ব্যাগিংয়ের চেয়ে ওভারফিট করার ক্ষেত্রে কিছুটা বেশি সংবেদনশীল।
- **উদাহরণ:** XGBoost, AdaBoost, Gradient Boosting (GBM)।

৬. গাণিতিক ভিত্তি (Mathematical Backend)

৭. সূত্র (Formula)

Mathematical Intuition

- Core Philosophy:** র্যান্ডম ফরেস্ট গাণিতিকভাবে বৈচিত্র্যময় ও স্বাধীন ডিসিশন ট্রি তৈরির জন্য Law of Large Numbers-এর নীতি ব্যবহার করে। প্রতিটি একক গাছের ভুল বা নয়েজ একে অপরের থেকে আলাদা হওয়ায়, যখন শত শত গাছের ফলাফলকে একত্রিত করা হয়, তখন সেই ত্রুটিগুলো পরস্পরকে নাকচ (Cancel out) করে দেয় এবং সামগ্রিক ভ্যারিয়েন্স শূন্যের কাছাকাছি নেমে আসে।

Gini Index / Entropy

- Impurity Measurement:** জঙ্গলের ভেতরের প্রতিটি ডিসিশন ট্রি নোড স্প্লিট করার সময় নিচের যেকোনো একটি গাণিতিক সূত্র দিয়ে তথ্যের বিশুদ্ধতা পরিমাপ করে:
- Gini Impurity:**

$$Gini = 1 - \sum_{i=1}^c (p_i)^2$$

(এখানে p_i হলো কোনো নোডে একটি নির্দিষ্ট ক্লাসের ডেটার অনুপাত। সম্পূর্ণ বিশুদ্ধ নোডের জিনি মান হয় 0 এবং সম্পূর্ণ অবিশুদ্ধ বা সমান অনুপাতের নোডের মান হয় 0.5)

- Entropy:**

$$Entropy = - \sum_{i=1}^c p_i \log_2 p_i$$

(বিশুদ্ধতার গাণিতিক পরিমাপ। সম্পূর্ণ বিশুদ্ধ নোডে এর মান 0 এবং সর্বোচ্চ অবিশুদ্ধ নোডে এর মান 1.0 হয়)

Information Gain

- Optimization Metric:** একটি নোডকে ভাঙার পর জিনি ইম্পিউরিটি বা এনট্রপি কতটা হ্রাস পেল, তা ইনফরমেশন গেইন দিয়ে পরিমাপ করা হয়। প্রতিটি গাছ এমন ফিচার এবং থ্রেশহোল্ড বেছে নেয় যার জন্য নিচের এই সমীকরণের মান সর্বোচ্চ হয়:

$$Information\ Gain = Impurity(Parent) - \sum \left(\frac{N_{Child}}{N_{Parent}} \times Impurity(Child) \right)$$

৮. সূত্রের ডেরিভেশন (Formula Derivation)

Random Feature Selection

- **Feature Subsetting:** সাধারণ ডিসিশন ট্রি নোড ভাগার সময় সব ফিচার (d) স্ক্যান করে। কিন্তু র্যান্ডম ফরেস্ট প্রতিটি স্প্লিটে র্যান্ডমলি m সংখ্যক ফিচারের একটি সাবসেট বেছে নেয়।
- **Standard Convention:** ক্লাসিফিকেশনের ক্ষেত্রে সাধারণত $m = \sqrt{d}$ এবং রিগ্রেশনের ক্ষেত্রে $m = d/3$ কলাম নেওয়া হয়। গাণিতিকভাবে এটি গাছগুলোর মধ্যকার কোরিলেশন বা পারস্পরিক মিল কমিয়ে দেয়, যা জঙ্গলকে আরও শক্তিশালী করে।

৯. প্রতিটি চিহ্নের অর্থ (Meaning of Every Symbol)

Symbol	Meaning (অর্থ)
N	মোট Decision Tree (n estimators)-এর সংখ্যা
m	প্রতিটি Split-এ বিবেচিত Random Feature-এর সংখ্যা
p	মোট Feature-এর সংখ্যা
Bootstrap Sample	Replacement সহ Randomly নির্বাচিত Training Subset
Majority Voting	Classification-এ সব গাছের সবচেয়ে বেশিবার আসা প্রেডিকশন
Averaging	Regression-এ সব গাছের প্রেডিকশনের গাণিতিক গড়

১০. সূত্রের ধাপে-ধাপে ব্যাখ্যা (Step-by-Step Formula Explanation)

Training Algorithm

- **Step 1 (Bootstrapping):** মূল ডেটাসেট থেকে প্রতিস্থাপনের নীতি মেনে (With replacement) $n_{estimators}$ সংখ্যক ভিন্ন ভিন্ন বুটস্ট্র্যাপ স্যাম্পল তৈরি করা হয়।
- **Step 2 (Tree Growth):** প্রতিটি স্যাম্পল ডেটার ওপর একটি করে ডিসিশন ট্রি ট্রেন করা হয়। গাছ বৃদ্ধির সময় প্রতিটি নোডে শুধুমাত্র র্যান্ডমলি সিলেক্টেড $max_features$ কলামের মধ্য থেকে সেরা স্প্লিটটি বেছে নেওয়া হয়। কোনো প্রকার প্রুনিং (Pruning) ছাড়াই গাছগুলোকে সর্বোচ্চ গভীরতা পর্যন্ত বাড়তে দেওয়া হয়।
- **Step 3 (Ensemble):** এই প্রক্রিয়াটি লুপের মতো চলতে থাকে যতক্ষণ না নির্ধারিত সংখ্যক (T) স্বাধীন গাছ তৈরি সম্পন্ন হচ্ছে।

Prediction Algorithm

- **Step 1 (Individual Prediction):** নতুন কোনো টেস্ট ডেটা রো (x) ইনপুট হিসেবে আসলে, সেটিকে জঙ্গলের প্রতিটি ট্রেইনড গাছের মধ্য দিয়ে পাস করানো হয়। প্রতিটি গাছ সম্পূর্ণ স্বাধীনভাবে নিজের একটি আউটপুট $(h_t(x))$ প্রদান করে।
- **Step 2 (Classification Aggregation):** ক্লাসিফিকেশনের ক্ষেত্রে ফাইনাল প্রেডিকশন হলো মেজরিটি ভোট:

$$\hat{Y} = \text{mode}\{h_1(x), h_2(x), \dots, h_T(x)\}$$

- **Step 3 (Regression Aggregation):** রিগ্রেশনের ক্ষেত্রে ফাইনাল প্রেডিকশন হলো সব গাছের আউটপুটের গাণিতিক গড়:

$$\hat{Y} = \frac{1}{T} \sum_{t=1}^T h_t(x)$$

Time & Space Complexity

- **Training Time Complexity:**

$$O(T \cdot m \cdot N \log N)$$

(এখানে T = number of trees, N = number of samples, এবং m = number of random features যেহেতু প্রতিটি গাছ সমান্তরালভাবে বা মাল্টি-থ্রেডিং ব্যবহার করে ট্রেইন করা যায়, তাই বড় ডেটাসেটেও এটি বেশ দ্রুত কাজ করে)

- **Prediction Time Complexity:**

$$O(T \cdot \text{depth})$$

(টেস্ট টাইমে প্রতিটি গাছের গভীরতা বা depth বরাবর ডেটা ট্রাভার্স করতে হয়, যা রিয়েল-টাইম প্রেডিকশনের জন্য অত্যন্ত ফাস্ট)

- **Space Complexity:**

$$O(T \cdot \text{number of nodes})$$

(যেহেতু শত শত গভীর ডিসিশন ট্রির প্রতিটি নোডের শর্ত ও মান মেমোরিতে ধরে রাখতে হয়, তাই র্যান্ডম ফরেস্টের মডেল সাইজ বা মেমোরি কস্ট লিনিয়ার মডেলের চেয়ে অনেক বেশি হয়)

Number of Trees (n_estimators): 3

Maximum Features (max_features): 2

ID	Study Hours	Attendance	Assignment	Pass (Target)
1	2	Low	No	No
2	3	Medium	Yes	No
3	5	High	Yes	Yes
4	6	High	Yes	Yes
5	4	Medium	No	Yes

Total instances (N) = 5

Class distribution: Pass = Yes → {3, 4, 5} (3টি) | Pass = No → {1, 2} (2টি)

Bootstrap Sampling

Random Forest এ প্রতিটি Decision Tree কে আলাদা Bootstrap Sample দিয়ে train করা হয়। Bootstrap Sampling হলো sampling with replacement — অর্থাৎ একটি instance একাধিকবার সিলেক্ট হতে পারে। প্রতিটি sample এ মোট N = 5টি row থাকবে, কিন্তু কিছু row repeat হবে এবং কিছু বাদ পড়বে (Out-of-Bag)।

Bootstrap Sample Size = N = 5 (original dataset এর সমান)

Sampling: with replacement → প্রতিবার random index $\in \{1, 2, 3, 4, 5\}$ থেকে বাছাই

BOOTSTRAP SAMPLE 1 — TREE 1

Row	ID	Pass
1	1	No
2	3	Yes
3	3	Yes
4	4	Yes
5	5	Yes

Yes=4, No=1 | OOB: ID {2}

BOOTSTRAP SAMPLE 2 — TREE 2

Row	ID	Pass
1	2	No
2	2	No
3	3	Yes
4	4	Yes
5	5	Yes

Yes=3, No=2 | OOB: ID {1}

BOOTSTRAP SAMPLE 3 — TREE 3

Row	ID	Pass
1	1	No
2	2	No
3	4	Yes
4	4	Yes
5	5	Yes

Yes=3, No=2 | OOB: ID {3}

Random Feature Selection ($\text{max_features} = 2$)

Random Forest এর একটি মূল বৈশিষ্ট্য হলো – প্রতিটি split এর সময় সব feature ব্যবহার না করে randomly একটি subset বাছাই করা হয়। এখানে মোট 3টি feature আছে (Study Hours, Attendance, Assignment) এবং $\text{max_features} = 2$ । অর্থাৎ প্রতি split এ যেকোনো 2টি feature থেকে সেরাটি বেছে নেওয়া হবে।

All features: {Study Hours, Attendance, Assignment} $\rightarrow$ total = 3
max_features = 2 $\rightarrow$ প্রতিটি split এ randomly 2টি বাছাই করতে হবে

Tree 1 — Feature Selection

✓ Study Hours ✓ Attendance ✗ Assignment

Random selection: **Study Hours** এবং **Attendance** chosen $\rightarrow$ **Assignment** এই split এ বিবেচিত হবে না।

Tree 2 — Feature Selection

✗ Study Hours ✓ Attendance ✓ Assignment

Random selection: **Attendance** এবং **Assignment** chosen $\rightarrow$ **Study Hours** এই split এ বিবেচিত হবে না।

Tree 3 — Feature Selection

✓ Study Hours ✗ Attendance ✓ Assignment

Random selection: **Study Hours** এবং **Assignment** chosen $\rightarrow$ **Attendance** এই split এ বিবেচিত হবে না।

Gini Impurity – সূত্র ও ধারণা

Decision Tree এ সেরা split খুঁজে বের করতে Gini Impurity ব্যবহার করা হয়। Gini Impurity একটি node এর অশুদ্ধতা (impurity) পরিমাপ করে। $\text{Gini} = 0$ মানে node সম্পূর্ণ pure (সব একই class), $\text{Gini} = 0.5$ মানে সর্বোচ্চ impurity (binary case)।

Gini(node) = 1 - Σ p_i²

যেখানে p_i = node এ class i এর proportion

Weighted Gini (split এর জন্য):

Gini_split = (|Left| / |Total|) × Gini(Left) + (|Right| / |Total|) × Gini(Right)

Information Gain = Gini(Parent) - Gini_split

→ যে split এ Information Gain সবচেয়ে বেশি, সেটিই সেরা split।

Tree 1 — সম্পূর্ণ Gini Calculation

Bootstrap Sample 1 (features: Study Hours, Attendance)

Row	ID	Study Hours	Attendance	Pass
1	1	2	Low	No
2	3	5	High	Yes
3	3	5	High	Yes
4	4	6	High	Yes
5	5	4	Medium	Yes

Step 1 — Root Node Gini Impurity

Total = 5 | Yes = 4 | No = 1

p(Yes) = 4/5 = 0.80 | p(No) = 1/5 = 0.20

Gini(Root) = 1 - (0.80² + 0.20²) = 1 - (0.64 + 0.04) = 1 - 0.68 = 0.3200

Step 2 — Feature: Attendance (Low/Medium vs High)

Split: Attendance = High (Right) vs Non-High (Left)

Left (Low or Medium): Row {1, 5} → Yes=1 (ID5), No=1 (ID1) → n=2

$$p(\text{Yes})=1/2=0.5, p(\text{No})=1/2=0.5$$

$$\text{Gini}(\text{Left}) = 1 - (0.5^2 + 0.5^2) = 1 - 0.5 = \mathbf{0.5000}$$

Right (High): Row {2,3,4} → Yes=3, No=0 → n=3

$$p(\text{Yes})=3/3=1.0, p(\text{No})=0$$

$$\text{Gini}(\text{Right}) = 1 - (1^2 + 0^2) = 1 - 1 = \mathbf{0.0000} \text{ (pure!)}$$

$$\text{Weighted Gini} = (2/5) \times 0.5 + (3/5) \times 0.0 = 0.20 + 0.0 = \mathbf{0.2000}$$

$$\text{Information Gain} = 0.3200 - 0.2000 = \mathbf{0.1200}$$

Step 3 — Feature: Study Hours (threshold = 3.5 → ≤ 3.5 vs > 3.5)

Split: Study Hours ≤ 3.5 (Left) vs Study Hours > 3.5 (Right)

Left (≤ 3.5): Row {1} (Study=2) → Yes=0, No=1 → n=1

$$\text{Gini}(\text{Left}) = 1 - (0^2 + 1^2) = 0 \text{ (pure)}$$

Right (> 3.5): Row {2,3,4,5} → Yes=4, No=0 → n=4

$$\text{Gini}(\text{Right}) = 1 - (1^2 + 0^2) = 0 \text{ (pure)}$$

$$\text{Weighted Gini} = (1/5) \times 0 + (4/5) \times 0 = \mathbf{0.0000}$$

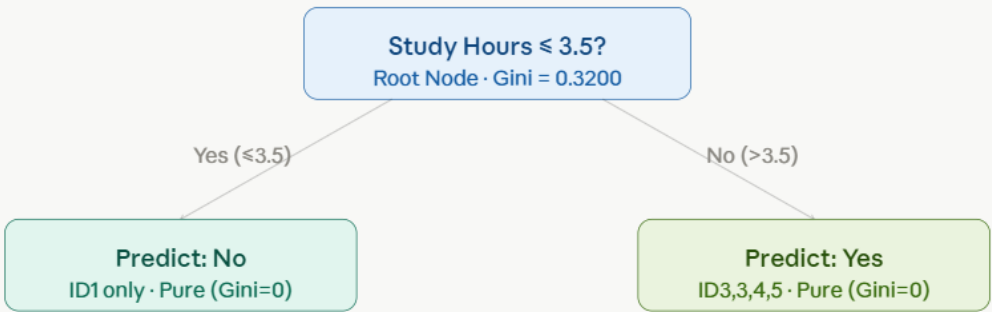
$$\text{Information Gain} = 0.3200 - 0.0000 = \mathbf{0.3200}$$

Tree 1 Root Split Decision:

Study Hours (IG = 0.3200) > Attendance (IG = 0.1200)

→ **Best Split: Study Hours ≤ 3.5**

Tree 1 — Structure



Tree 1 — max depth reached, both leaves are pure

Tree 2 — সম্পূর্ণ Gini Calculation

Bootstrap Sample 2 (features: Attendance, Assignment)

Row	ID	Attendance	Assignment	Pass
1	2	Medium	Yes	No
2	2	Medium	Yes	No
3	3	High	Yes	Yes
4	4	High	Yes	Yes
5	5	Medium	No	Yes

Step 1 — Root Node Gini Impurity

Total=5 | Yes=3 | No=2

$p(\text{Yes})=3/5=0.60$ | $p(\text{No})=2/5=0.40$

$\text{Gini}(\text{Root}) = 1 - (0.60^2 + 0.40^2) = 1 - (0.36 + 0.16) = 1 - 0.52 = 0.4800$

Step 2 — Feature: Attendance (High vs Non-High)

Left (Non-High = Medium): Row {1,2,5} → Yes=1(ID5), No=2(ID2,2) → n=3

$$p(\text{Yes})=1/3\approx 0.333, p(\text{No})=2/3\approx 0.667$$

$$\text{Gini}(\text{Left}) = 1 - (0.333^2 + 0.667^2) = 1 - (0.111 + 0.444) = 1 - 0.556 = \mathbf{0.4444}$$

Right (High): Row {3,4} → Yes=2, No=0 → n=2

$$\text{Gini}(\text{Right}) = 1 - (1^2 + 0^2) = \mathbf{0.0000} \text{ (pure)}$$

$$\text{Weighted Gini} = (3/5) \times 0.4444 + (2/5) \times 0.0 = 0.2667 + 0 = \mathbf{0.2667}$$

$$\text{Information Gain} = 0.4800 - 0.2667 = \mathbf{0.2133}$$

Step 3 — Feature: Assignment (Yes vs No)

Left (Assignment=No): Row {5} → Yes=1, No=0 → n=1

$$\text{Gini}(\text{Left}) = 0 \text{ (pure)}$$

Right (Assignment=Yes): Row {1,2,3,4} → Yes=2(ID3,4), No=2(ID2,2) → n=4

$$p(\text{Yes})=2/4=0.5, p(\text{No})=2/4=0.5$$

$$\text{Gini}(\text{Right}) = 1 - (0.5^2 + 0.5^2) = 1 - 0.5 = \mathbf{0.5000}$$

$$\text{Weighted Gini} = (1/5) \times 0 + (4/5) \times 0.5 = 0 + 0.4 = \mathbf{0.4000}$$

$$\text{Information Gain} = 0.4800 - 0.4000 = \mathbf{0.0800}$$

Tree 2 Root Split Decision:

Attendance (IG = 0.2133) > Assignment (IG = 0.0800)

→ **Best Split: Attendance = High**

Tree 2 — Second Level Split (Left child: Non-High Attendance)

Left node এ 3টি row আছে (ID2, ID2, ID5) — Yes=1, No=2। এটি impure। এখন Assignment দিয়ে split করব।

Step 4 — Left Child: Assignment split (only feature left)

$Gini(\text{Left-node}) = 0.4444$

Assignment=Yes: Row {1,2} (ID2,ID2) $\rightarrow$ Yes=0, No=2 $\rightarrow n=2$

$Gini = 1 - (0^2 + 1^2) = 0$ (pure)

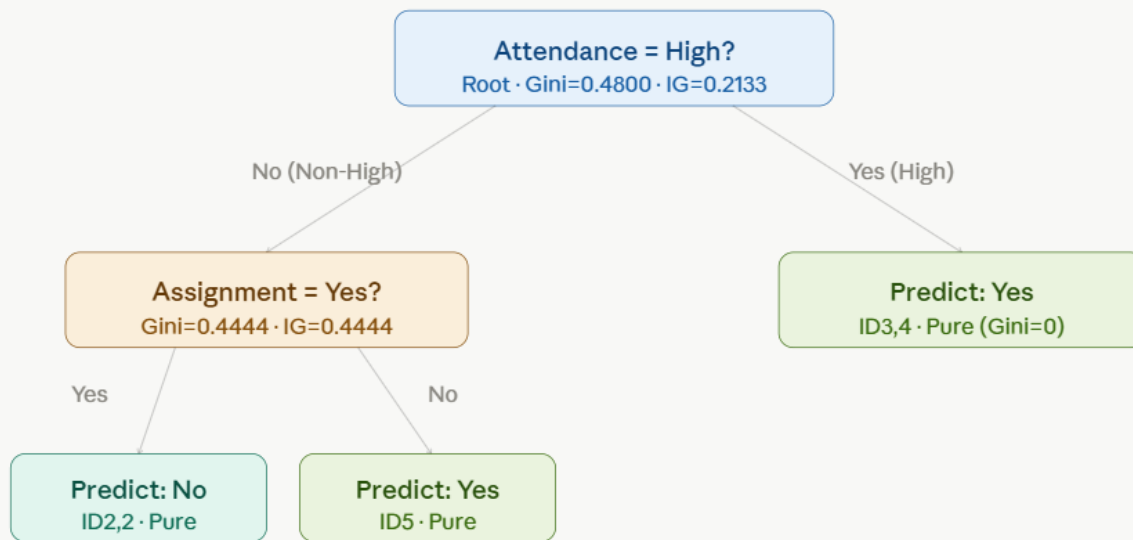
Assignment=No: Row {5} $\rightarrow$ Yes=1, No=0 $\rightarrow n=1$

$Gini = 0$ (pure)

Weighted Gini = $(2/3) \times 0 + (1/3) \times 0 = 0.0000$

IG = $0.4444 - 0 = 0.4444 \rightarrow$ Split করা!

Tree 2 — Structure



Tree 2 — depth 2, all leaves pure

Tree 3 — সম্পূর্ণ Gini Calculation

Bootstrap Sample 3 (features: Study Hours, Assignment)

Row	ID	Study Hours	Assignment	Pass
1	1	2	No	No
2	2	3	Yes	No
3	4	6	Yes	Yes
4	4	6	Yes	Yes
5	5	4	No	Yes

Step 1 — Root Node Gini Impurity

Total=5 | Yes=3 | No=2

$p(\text{Yes})=0.60$, $p(\text{No})=0.40$

Gini(Root) = $1 - (0.36 + 0.16) = 0.4800$

Step 2 — Feature: Study Hours (threshold = 3.5)

Left (≤ 3.5): Row {1,2} $\rightarrow$ Yes=0, No=2 $\rightarrow$ n=2

Gini(Left) = $1 - (0^2 + 1^2) = 0$ (pure)

Right (> 3.5): Row {3,4,5} $\rightarrow$ Yes=3 (ID 4,4,5), No=0 $\rightarrow$ n=3

Gini(Right) = $1 - (1^2 + 0^2) = 0$ (pure)

Weighted Gini = 0

Information Gain = $0.4800 - 0 = 0.4800$

Step 3 — Feature: Assignment (Yes vs No)

Left (No): Row {1,5} → Yes=1(ID5), No=1(ID1) → n=2

$$\text{Gini(Left)} = 1 - (0.5^2 + 0.5^2) = \mathbf{0.5000}$$

Right (Yes): Row {2,3,4} → Yes=2(ID4,4), No=1(ID2) → n=3

$$p(\text{Yes}) = 2/3 \approx 0.667, p(\text{No}) = 1/3 \approx 0.333$$

$$\text{Gini(Right)} = 1 - (0.667^2 + 0.333^2) = 1 - (0.444 + 0.111) = \mathbf{0.4444}$$

$$\text{Weighted Gini} = (2/5) \times 0.5 + (3/5) \times 0.4444 = 0.2 + 0.2667 = \mathbf{0.4667}$$

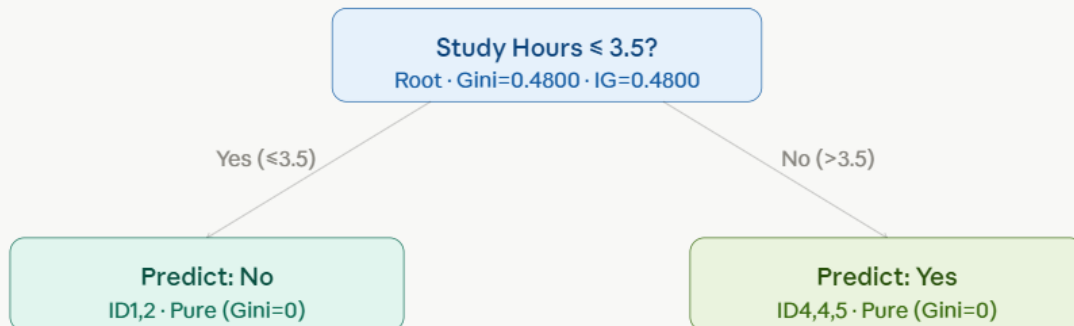
$$\text{Information Gain} = 0.4800 - 0.4667 = \mathbf{0.0133}$$

Tree 3 Root Split Decision:

Study Hours (IG = 0.4800) >> Assignment (IG = 0.0133)

→ **Best Split: Study Hours ≤ 3.5**

Tree 3 — Structure



Tree 3 — depth 1, both leaves pure (perfect separation)

নতুন Instance — Prediction

নতুন data point:

Study Hours = 4 | Attendance = High | Assignment = Yes

Tree 1 এ Traverse

Root: Study Hours ≤ 3.5 ? $\rightarrow 4 > 3.5 \rightarrow$ Right branch

Right leaf: Predict = **Yes**

Tree 1 Prediction $\rightarrow$ Yes

Tree 2 এ Traverse

Root: Attendance = High? $\rightarrow$ High = Yes $\rightarrow$ Right branch

Right leaf: Predict = **Yes**

Tree 2 Prediction $\rightarrow$ Yes

Tree 3 এ Traverse

Root: Study Hours ≤ 3.5 ? $\rightarrow 4 > 3.5 \rightarrow$ Right branch

Right leaf: Predict = **Yes**

Tree 3 Prediction $\rightarrow$ Yes

§ 9

Majority Voting — চূড়ান্ত সিদ্ধান্ত

Random Forest এ সমস্ত tree এর individual prediction collect করে **Majority Vote** নেওয়া হয়। Classification task এ যে class বেশি vote পায়, সেটিই final prediction।

Tree 1 $\rightarrow$ Yes

+

Tree 2 $\rightarrow$ Yes

+

Tree 3 $\rightarrow$ Yes

=

Yes: 3 votes

Vote Count:

Yes = 3 votes | No = 0 votes

Majority Class = Yes (3 out of 3 trees agree)

FINAL PREDICTION

✓ **Pass = Yes**

Study Hours=4, Attendance=High, Assignment=Yes → এই student টি Pass করবে।

Confidence: 3/3 trees → 100% vote unanimity

১১. সংখ্যাগত উদাহরণ (Numerical Example)

মানদণ্ড: Squared Error (MSE) | n_estimators = 3 | max_features = 2

ধাপ ১: মূল ডেটাসেট

ID	Study Hours	Sleep Hours	Attendance	Marks (Target)
1	2	8	Low	40
2	3	7	Medium	50
3	4	7	Medium	60
4	5	6	High	75
5	6	6	High	90

ধাপ ২: Bootstrap Sampling

র‍্যান্ডম ফরেস্টে প্রতিটি ট্রিকে একটি Bootstrap Sample দিয়ে প্রশিক্ষণ দেওয়া হয়। এটি হলো মূল ডেটাসেট থেকে পুনরাবৃত্তিসহ (with replacement) র‍্যান্ডমভাবে n সংখ্যক নমুনা নেওয়া (এখানে n = 5)।

Bootstrap Sample 1 (Tree 1 এর জন্য)

নির্বাচিত সারি (পুনরাবৃত্তিসহ): ID → 1, 2, 2, 4, 5

ID	Study Hours	Sleep Hours	Attendance	Marks
1	2	8	Low	40
2	3	7	Medium	50
2	3	7	Medium	50
4	5	6	High	75
5	6	6	High	90

Bootstrap Sample 2 (Tree 2 এর জন্য)

নির্বাচিত সারি (পুনরাবৃত্তিসহ): ID → 1, 3, 3, 4, 5

ID	Study Hours	Sleep Hours	Attendance	Marks
1	2	8	Low	40
3	4	7	Medium	60
3	4	7	Medium	60
4	5	6	High	75
5	6	6	High	90

Bootstrap Sample 3 (Tree 3 এর জন্য)

নির্বাচিত সারি (পুনরাবৃত্তিসহ): ID → 2, 3, 4, 4, 5

ID	Study Hours	Sleep Hours	Attendance	Marks
2	3	7	Medium	50
3	4	7	Medium	60
4	5	6	High	75
4	5	6	High	75
5	6	6	High	90

ধাপ ৩: র্যান্ডম ফিচার নির্বাচন ($\text{max_features} = 2$)

প্রতিটি নোড বিভাজনের সময় ৩টি উপলব্ধ ফিচার থেকে মাত্র ২টি ফিচার র্যান্ডমভাবে বেছে নেওয়া হয়।

- Tree 1: Study Hours, Attendance
- Tree 2: Study Hours, Sleep Hours
- Tree 3: Study Hours, Attendance

ধাপ ৪: মূল সূত্র — Squared Error (MSE)

একটি নোডে $y_1, y_2, \dots, y_n$ মান থাকলে Mean Squared Error হবে:

$$MSE = (1/n) \times \sum (y_i - \bar{y})^2$$

যেখানে $\bar{y}$ = সেই নোডের সকল টার্গেট মানের গড়।

একটি বিভাজনের Weighted MSE:

$$MSE_split = (n_left / n) \times MSE_left + (n_right / n) \times MSE_right$$

MSE হ্রাস (Information Gain):

$$Gain = MSE_parent - MSE_split$$

যে বিভাজনে Gain সর্বোচ্চ (অর্থাৎ Weighted MSE সর্বনিম্ন), সেটিই নির্বাচন করা হয়।

ধাপ ৫: Tree 1 – সম্পূর্ণ গণনা

Bootstrap Sample 1 এর ডেটা:

Study Hours	Attendance	Marks
2	Low	40
3	Medium	50
3	Medium	50
5	High	75
6	High	90

নির্বাচিত ফিচার: Study Hours, Attendance

৫.১ – Root Node MSE (Parent)

$$\bar{y}_{parent} = (40 + 50 + 50 + 75 + 90) / 5 = 305 / 5 = 61$$

$$MSE_parent = [(40-61)^2 + (50-61)^2 + (50-61)^2 + (75-61)^2 + (90-61)^2] / 5$$

$$= [(-21)^2 + (-11)^2 + (-11)^2 + (14)^2 + (29)^2] / 5$$

$$= [441 + 121 + 121 + 196 + 841] / 5$$

$$= 1720 / 5 = 344$$

$$\text{MSE}_{\text{parent}} = 344$$

৫.২ – ফিচার: Study Hours এর উপর বিভাজন পরীক্ষা

সম্ভাব্য Split Points (পরপর unique মানের মধ্যবিন্দু): 2.5, 4.0, 5.5

Split A: Study Hours ≤ 2.5

বাম দিক (Left): {40} $\rightarrow \bar{y}_L = 40$, $\text{MSE}_L = 0$

ডান দিক (Right): {50, 50, 75, 90} $\rightarrow \bar{y}_R = (50+50+75+90)/4 = 265/4 = 66.25$

$$\text{MSE}_R = [(50-66.25)^2 + (50-66.25)^2 + (75-66.25)^2 + (90-66.25)^2] / 4$$

$$= [(-16.25)^2 + (-16.25)^2 + (8.75)^2 + (23.75)^2] / 4$$

$$= [264.0625 + 264.0625 + 76.5625 + 564.0625] / 4$$

$$= 1168.75 / 4 = 292.1875$$

$$\text{MSE}_{\text{split}} = (1/5) \times 0 + (4/5) \times 292.1875 = 233.75$$

$$\text{Gain} = 344 - 233.75 = 110.25$$

Split B: Study Hours ≤ 4.0

বাম দিক: {40, 50, 50} $\rightarrow \bar{y}_L = 140/3 = 46.67$

$$\text{MSE}_L = [(40-46.67)^2 + (50-46.67)^2 + (50-46.67)^2] / 3$$

$$= [(-6.67)^2 + (3.33)^2 + (3.33)^2] / 3$$

$$= [44.49 + 11.09 + 11.09] / 3$$

$$= 66.67 / 3 = 22.22$$

ডান দিক: {75, 90} $\rightarrow \bar{y}_R = 165/2 = 82.5$

$$MSE_R = [(75-82.5)^2 + (90-82.5)^2] / 2$$

$$= [(-7.5)^2 + (7.5)^2] / 2$$

$$= [56.25 + 56.25] / 2 = \mathbf{56.25}$$

$$MSE_{split} = (3/5) \times 22.22 + (2/5) \times 56.25$$

$$= 13.33 + 22.50 = \mathbf{35.83}$$

$$\mathbf{Gain = 344 - 35.83 = 308.17} \leftarrow \text{এখন পর্যন্ত সর্বোচ্চ}$$

Split C: Study Hours ≤ 5.5

বাম দিক: {40, 50, 50, 75} $\rightarrow \bar{y}_L = 215/4 = 53.75$

$$MSE_L = [(40-53.75)^2 + (50-53.75)^2 + (50-53.75)^2 + (75-53.75)^2] / 4$$

$$= [(-13.75)^2 + (-3.75)^2 + (-3.75)^2 + (21.25)^2] / 4$$

$$= [189.0625 + 14.0625 + 14.0625 + 451.5625] / 4$$

$$= 668.75 / 4 = \mathbf{167.1875}$$

ডান দিক: {90} $\rightarrow \bar{y}_R = 90, MSE_R = 0$

$$MSE_{split} = (4/5) \times 167.1875 + (1/5) \times 0 = \mathbf{133.75}$$

$$\mathbf{Gain = 344 - 133.75 = 210.25}$$

৫.৩ – ফিচার: Attendance এর উপর বিভাজন পরীক্ষা

Split D: Attendance = Low বনাম {Medium, High}

বাম দিক (Low): {40} $\rightarrow \bar{y}_L = 40, MSE_L = 0$

ডান দিক (Medium, High): {50, 50, 75, 90} $\rightarrow \bar{y}_R = 66.25, MSE_R = 292.1875$

$$MSE_{split} = (1/5) \times 0 + (4/5) \times 292.1875 = \mathbf{233.75}$$

$$\text{Gain} = 344 - 233.75 = 110.25$$

Split E: Attendance $\in \{\text{Low, Medium}\}$ বনাম $\{\text{High}\}$

বাম দিক (Low, Medium): $\{40, 50, 50\} \rightarrow \bar{y}_L = 46.67, \text{MSE}_L = 22.22$

ডান দিক (High): $\{75, 90\} \rightarrow \bar{y}_R = 82.5, \text{MSE}_R = 56.25$

$$\text{MSE}_{\text{split}} = (3/5) \times 22.22 + (2/5) \times 56.25 = 13.33 + 22.50 = 35.83$$

$$\text{Gain} = 344 - 35.83 = 308.17 \leftarrow \text{Split B এর সমান}$$

৫.৪ – Tree 1 এর সেরা বিভাজন

Split	ফিচার	শর্ত	Gain
A	Study Hours	≤ 2.5	110.25
B	Study Hours	≤ 4.0	308.17
C	Study Hours	≤ 5.5	210.25
D	Attendance	Low বনাম বাকি	110.25
E	Attendance	$\{\text{Low, Med}\}$ বনাম High	308.17

Split B ও Split E উভয়ের Gain = 308.17 সমান। আমরা **Study Hours ≤ 4.0** নির্বাচন করি।

Root Split: Study Hours ≤ 4.0

৫.৫ – Tree 1 দ্বিতীয় স্তর: বাম নোড (Study Hours ≤ 4.0)

নমুনা: $\{40, 50, 50\} \rightarrow \bar{y} = 46.67, \text{MSE} = 22.22$

Study Hours ≤ 2.5 বিভাজন চেষ্টা:

বাম: {40} $\rightarrow$ MSE = 0

ডান: {50, 50} $\rightarrow \bar{y} = 50$, MSE = 0

MSE_split = $(1/3) \times 0 + (2/3) \times 0 = 0$

Gain = $22.22 - 0 = 22.22 \rightarrow$ নিখুঁত বিভাজন

সেরা বিভাজন: Study Hours ≤ 2.5

৫.৬ – Tree 1 দ্বিতীয় স্তর: ডান নোড (Study Hours > 4.0)

নমুনা: {75, 90} $\rightarrow \bar{y} = 82.5$, MSE = 56.25

Study Hours ≤ 5.5 বিভাজন চেষ্টা:

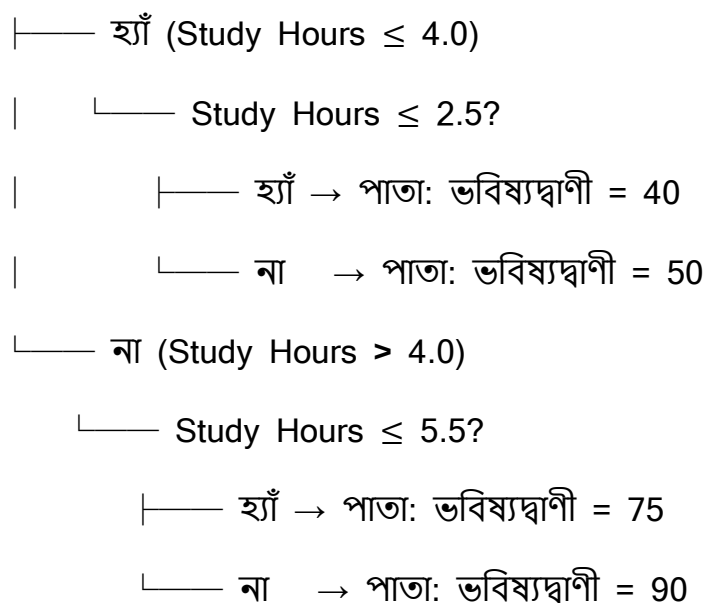
বাম: {75} $\rightarrow$ MSE = 0

ডান: {90} $\rightarrow$ MSE = 0

MSE_split = 0, Gain = **56.25** $\rightarrow$ নিখুঁত বিভাজন

৫.৭ – Tree 1: চূড়ান্ত কাঠামো

Root: Study Hours ≤ 4.0 ?



ধাপ ৬: Tree 2 – সম্পূর্ণ গণনা

Bootstrap Sample 2 এর ডেটা:

Study Hours	Sleep Hours	Marks
2	8	40
4	7	60
4	7	60
5	6	75
6	6	90

নির্বাচিত ফিচার: Study Hours, Sleep Hours

৬.১ – Root Node MSE (Parent)

$$\bar{y}_{\text{parent}} = (40 + 60 + 60 + 75 + 90) / 5 = 325 / 5 = 65$$

$$\begin{aligned}\text{MSE}_{\text{parent}} &= [(40-65)^2 + (60-65)^2 + (60-65)^2 + (75-65)^2 + (90-65)^2] / 5 \\ &= [(-25)^2 + (-5)^2 + (-5)^2 + (10)^2 + (25)^2] / 5 \\ &= [625 + 25 + 25 + 100 + 625] / 5 \\ &= 1400 / 5 = 280\end{aligned}$$

৬.২ – ফিচার: Study Hours এর উপর বিভাজন পরীক্ষা

Split Points: 3.0, 4.5, 5.5

Split A: Study Hours ≤ 3.0

বাম: {40} $\rightarrow \text{MSE}_L = 0$

ডান: {60, 60, 75, 90} $\rightarrow \bar{y}_R = 285/4 = 71.25$

$$\begin{aligned}\text{MSE}_R &= [(60-71.25)^2 + (60-71.25)^2 + (75-71.25)^2 + (90-71.25)^2] / 4 \\ &= [(-11.25)^2 + (-11.25)^2 + (3.75)^2 + (18.75)^2] / 4 \\ &= [126.5625 + 126.5625 + 14.0625 + 351.5625] / 4 \\ &= 618.75 / 4 = 154.6875\end{aligned}$$

$$\text{MSE}_{\text{split}} = (1/5) \times 0 + (4/5) \times 154.6875 = 123.75$$

$$\text{Gain} = 280 - 123.75 = 156.25$$

Split B: Study Hours ≤ 4.5

বাম: {40, 60, 60} $\rightarrow \bar{y}_L = 160/3 = 53.33$

$$\begin{aligned}\text{MSE}_L &= [(40-53.33)^2 + (60-53.33)^2 + (60-53.33)^2] / 3 \\ &= [(-13.33)^2 + (6.67)^2 + (6.67)^2] / 3\end{aligned}$$

$$= [177.69 + 44.49 + 44.49] / 3$$

$$= 266.67 / 3 = \mathbf{88.89}$$

$$\text{ডান: } \{75, 90\} \rightarrow \bar{y}_R = 82.5, \text{MSE}_R = 56.25$$

$$\text{MSE}_{\text{split}} = (3/5) \times 88.89 + (2/5) \times 56.25$$

$$= 53.33 + 22.50 = \mathbf{75.83}$$

$$\mathbf{Gain = 280 - 75.83 = 204.17} \leftarrow \text{এখন পর্যন্ত সর্বোচ্চ}$$

Split C: Study Hours ≤ 5.5

$$\text{বাম: } \{40, 60, 60, 75\} \rightarrow \bar{y}_L = 235/4 = 58.75$$

$$\text{MSE}_L = [(40-58.75)^2 + (60-58.75)^2 + (60-58.75)^2 + (75-58.75)^2] / 4$$

$$= [(-18.75)^2 + (1.25)^2 + (1.25)^2 + (16.25)^2] / 4$$

$$= [351.5625 + 1.5625 + 1.5625 + 264.0625] / 4$$

$$= 618.75 / 4 = \mathbf{154.6875}$$

$$\text{ডান: } \{90\} \rightarrow \text{MSE}_R = 0$$

$$\text{MSE}_{\text{split}} = (4/5) \times 154.6875 + (1/5) \times 0 = \mathbf{123.75}$$

$$\mathbf{Gain = 280 - 123.75 = 156.25}$$

৬.৩ – ফিচার: Sleep Hours এর উপর বিভাজন পরীক্ষা

Split Points: 6.5, 7.5

Split D: Sleep Hours ≤ 6.5

$$\text{বাম (Sleep } \leq 6.5): \{75, 90\} \rightarrow \bar{y}_L = 82.5, \text{MSE}_L = 56.25$$

$$\text{ডান (Sleep } > 6.5): \{40, 60, 60\} \rightarrow \bar{y}_R = 53.33, \text{MSE}_R = 88.89$$

$$\begin{aligned}\text{MSE}_{\text{split}} &= (2/5) \times 56.25 + (3/5) \times 88.89 \\ &= 22.50 + 53.33 = \mathbf{75.83}\end{aligned}$$

$$\text{Gain} = 280 - 75.83 = 204.17 \leftarrow \text{Split B এর সমান}$$

Split E: Sleep Hours ≤ 7.5

বাম (Sleep ≤ 7.5): {60, 60, 75, 90} $\rightarrow \bar{y}_L = 71.25$, $\text{MSE}_L = 154.6875$

ডান (Sleep > 7.5): {40} $\rightarrow \text{MSE}_R = 0$

$$\text{MSE}_{\text{split}} = (4/5) \times 154.6875 + (1/5) \times 0 = \mathbf{123.75}$$

$$\text{Gain} = 280 - 123.75 = 156.25$$

৬.৪ – Tree 2 এর সেরা বিভাজন

Split	ফিচার	শর্ত	Gain
A	Study Hours	≤ 3.0	156.25
B	Study Hours	≤ 4.5	204.17
C	Study Hours	≤ 5.5	156.25
D	Sleep Hours	≤ 6.5	204.17
E	Sleep Hours	≤ 7.5	156.25

নির্বাচিত বিভাজন: Study Hours ≤ 4.5

৬.৫ – Tree 2 দ্বিতীয় স্তর: বাম নোড (Study Hours $\leq$ 4.5)

নমুনা: {40, 60, 60} $\rightarrow \bar{y} = 53.33$, MSE = 88.89

Study Hours $\leq$ 3.0 বিভাজন:

বাম: {40} $\rightarrow$ MSE = 0 | ডান: {60, 60} $\rightarrow$ MSE = 0

MSE_split = 0, Gain = 88.89 $\rightarrow$ নিখুঁত বিভাজন

৬.৬ – Tree 2 দ্বিতীয় স্তর: ডান নোড (Study Hours $>$ 4.5)

নমুনা: {75, 90} $\rightarrow \bar{y} = 82.5$, MSE = 56.25

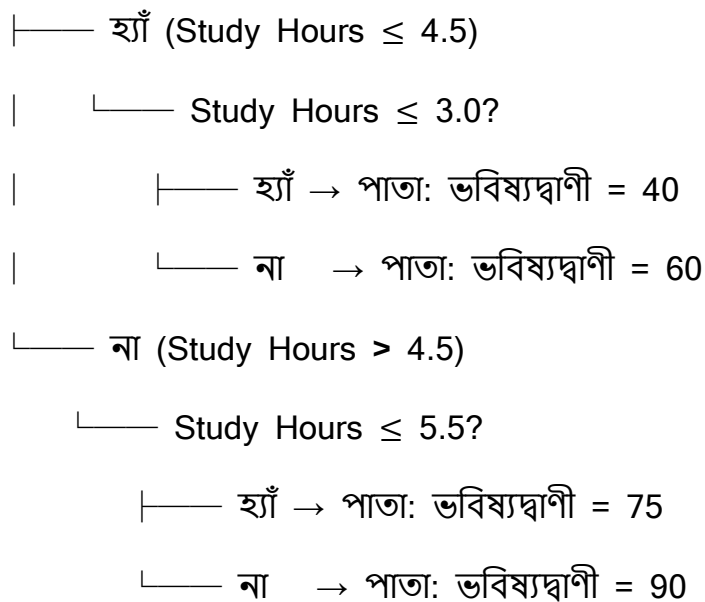
Study Hours $\leq$ 5.5 বিভাজন:

বাম: {75} $\rightarrow$ MSE = 0 | ডান: {90} $\rightarrow$ MSE = 0

MSE_split = 0, Gain = 56.25 $\rightarrow$ নিখুঁত বিভাজন

৬.৭ – Tree 2: চূড়ান্ত কাঠামো

Root: Study Hours $\leq$ 4.5?



ধাপ ৭: Tree 3 – সম্পূর্ণ গণনা

Bootstrap Sample 3 এর ডেটা:

Study Hours	Attendance	Marks
3	Medium	50
4	Medium	60
5	High	75
5	High	75
6	High	90

নির্বাচিত ফিচার: Study Hours, Attendance

৭.১ – Root Node MSE (Parent)

$$\bar{y}_{\text{parent}} = (50 + 60 + 75 + 75 + 90) / 5 = 350 / 5 = 70$$

$$\begin{aligned}\text{MSE}_{\text{parent}} &= [(50-70)^2 + (60-70)^2 + (75-70)^2 + (75-70)^2 + (90-70)^2] / 5 \\ &= [(-20)^2 + (-10)^2 + (5)^2 + (5)^2 + (20)^2] / 5 \\ &= [400 + 100 + 25 + 25 + 400] / 5 \\ &= 950 / 5 = 190\end{aligned}$$

৭.২ – ফিচার: Study Hours এর উপর বিভাজন পরীক্ষা

Split Points: 3.5, 4.5, 5.5

Split A: Study Hours $\leq$ 3.5

বাম: {50} $\rightarrow$ MSE_L = 0

ডান: {60, 75, 75, 90} $\rightarrow \bar{y}_R = 300/4 = 75$

$$\begin{aligned} \text{MSE}_R &= [(60-75)^2 + (75-75)^2 + (75-75)^2 + (90-75)^2] / 4 \\ &= [225 + 0 + 0 + 225] / 4 \\ &= 450 / 4 = \mathbf{112.5} \end{aligned}$$

$$\text{MSE}_{\text{split}} = (1/5) \times 0 + (4/5) \times 112.5 = \mathbf{90}$$

$$\mathbf{\text{Gain} = 190 - 90 = 100}$$

Split B: Study Hours ≤ 4.5

$$\text{বাম: } \{50, 60\} \rightarrow \bar{y}_L = 55$$

$$\text{MSE}_L = [(50-55)^2 + (60-55)^2] / 2 = [25 + 25] / 2 = \mathbf{25}$$

$$\text{ডান: } \{75, 75, 90\} \rightarrow \bar{y}_R = 240/3 = 80$$

$$\begin{aligned} \text{MSE}_R &= [(75-80)^2 + (75-80)^2 + (90-80)^2] / 3 \\ &= [25 + 25 + 100] / 3 \\ &= 150 / 3 = \mathbf{50} \end{aligned}$$

$$\text{MSE}_{\text{split}} = (2/5) \times 25 + (3/5) \times 50 = 10 + 30 = \mathbf{40}$$

$$\mathbf{\text{Gain} = 190 - 40 = 150} \leftarrow \text{এখন পর্যন্ত সর্বোচ্চ}$$

Split C: Study Hours ≤ 5.5

$$\text{বাম: } \{50, 60, 75, 75\} \rightarrow \bar{y}_L = 260/4 = 65$$

$$\begin{aligned} \text{MSE}_L &= [(50-65)^2 + (60-65)^2 + (75-65)^2 + (75-65)^2] / 4 \\ &= [225 + 25 + 100 + 100] / 4 \\ &= 450 / 4 = \mathbf{112.5} \end{aligned}$$

$$\text{ডান: } \{90\} \rightarrow \text{MSE}_R = 0$$

$$\text{MSE}_{\text{split}} = (4/5) \times 112.5 + (1/5) \times 0 = \mathbf{90}$$

$$\text{Gain} = 190 - 90 = 100$$

৭.৩ – ফিচার: Attendance এর উপর বিভাজন পরীক্ষা

Split D: Attendance = Medium বনাম High

বাম (Medium): {50, 60} $\rightarrow \bar{y}_L = 55$, $\text{MSE}_L = 25$

ডান (High): {75, 75, 90} $\rightarrow \bar{y}_R = 80$, $\text{MSE}_R = 50$

$$\text{MSE}_{\text{split}} = (2/5) \times 25 + (3/5) \times 50 = 10 + 30 = 40$$

$$\text{Gain} = 190 - 40 = 150 \leftarrow \text{Split B এর সমান}$$

৭.৪ – Tree 3 এর সেরা বিভাজন

Split	ফিচার	শর্ত	Gain
A	Study Hours	≤ 3.5	100
B	Study Hours	≤ 4.5	150
C	Study Hours	≤ 5.5	100
D	Attendance	Medium বনাম High	150

নির্বাচিত বিভাজন: Study Hours ≤ 4.5

৭.৫ – Tree 3 দ্বিতীয় স্তর: বাম নোড (Study Hours ≤ 4.5)

নমুনা: {50, 60} $\rightarrow \bar{y} = 55$, $\text{MSE} = 25$

Study Hours ≤ 3.5 বিভাজন:

বাম: {50} $\rightarrow \text{MSE} = 0$ | ডান: {60} $\rightarrow \text{MSE} = 0$

$\text{MSE}_{\text{split}} = 0$, $\text{Gain} = 25 \rightarrow$ নিখুঁত বিভাজন

৭.৬ – Tree 3 দ্বিতীয় স্তর: ডান নোড (Study Hours > 4.5)

নমুনা: {75, 75, 90} $\rightarrow \bar{y} = 80$, MSE = 50

Study Hours ≤ 5.5 বিভাজন:

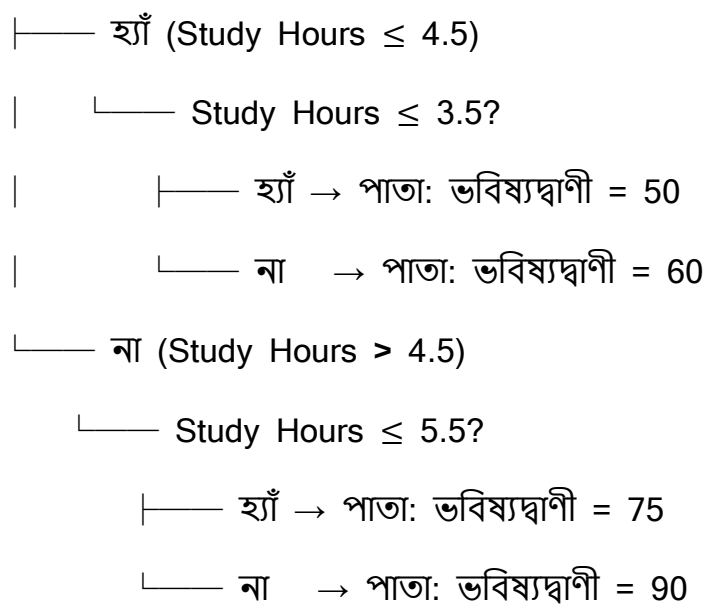
বাম: {75, 75} $\rightarrow \bar{y} = 75$, MSE = 0

ডান: {90} $\rightarrow$ MSE = 0

MSE_split = 0, Gain = 50 $\rightarrow$ নিখুঁত বিভাজন

৭.৭ – Tree 3: চূড়ান্ত কাঠামো

Root: Study Hours ≤ 4.5 ?



ধাপ ৮: নতুন ডেটার জন্য ভবিষ্যদ্বাণী

নতুন ইনপুট: Study Hours = 4, Sleep Hours = 7 (Tree 2 এর জন্য ধরা হয়েছে),
Attendance = High

Tree 1 এর ভবিষ্যদ্বাণী (ফিচার: Study Hours, Attendance)

- Root: Study Hours ≤ 4.0 ? $\rightarrow 4 \leq 4.0 \rightarrow$ হ্যাঁ, বাম দিকে যাও
- নোড: Study Hours ≤ 2.5 ? $\rightarrow 4 \leq 2.5 \rightarrow$ না, ডান দিকে যাও
- পাতা: ভবিষ্যদ্বাণী = 50

Tree 2 এর ভবিষ্যদ্বাণী (ফিচার: Study Hours, Sleep Hours)

- Root: Study Hours ≤ 4.5 ? $\rightarrow 4 \leq 4.5 \rightarrow$ হ্যাঁ, বাম দিকে যাও
- নোড: Study Hours ≤ 3.0 ? $\rightarrow 4 \leq 3.0 \rightarrow$ না, ডান দিকে যাও
- পাতা: ভবিষ্যদ্বাণী = 60

Tree 3 এর ভবিষ্যদ্বাণী (ফিচার: Study Hours, Attendance)

- Root: Study Hours ≤ 4.5 ? $\rightarrow 4 \leq 4.5 \rightarrow$ হ্যাঁ, বাম দিকে যাও
- নোড: Study Hours ≤ 3.5 ? $\rightarrow 4 \leq 3.5 \rightarrow$ না, ডান দিকে যাও
- পাতা: ভবিষ্যদ্বাণী = 60

ধাপ ৯: চূড়ান্ত ভবিষ্যদ্বাণী – গড় গণনা (Averaging)

গুরুত্বপূর্ণ নোট: Regressor এর ক্ষেত্রে Random Forest **Majority Voting** ব্যবহার করে না (সেটি Classifier এর জন্য)। Regression এ সকল ট্রির ভবিষ্যদ্বাণীর গড় নেওয়া হয়।

ট্রি	ভবিষ্যদ্বাণী
Tree 1	50
Tree 2	60

Tree 3	60
--------	----

চূড়ান্ত ভবিষ্যদ্বাণী = $(50 + 60 + 60) / 3 = 170 / 3 \approx 56.67$

নতুন শিক্ষার্থীর প্রাপ্ত নম্বরের ভবিষ্যদ্বাণী ≈ 56.67

১২. ভিজ্যুয়ালাইজেশন (Visualization)

Random Forest-কে কল্পনা করা যায় একটি "ভোটের বোর্ড" হিসেবে — প্রতিটি গাছ তার নিজের রায় দেয় এবং শেষে সবচেয়ে বেশি ভোট পাওয়া উত্তরটি জেতে। আরেকটি জনপ্রিয় ভিজ্যুয়ালাইজেশন হলো feature_importances_ Bar Chart, যেখানে প্রতিটি Feature-এর গুরুত্ব একটি বার (Bar) দিয়ে দেখানো হয় — যত লম্বা বার, সেই Feature Prediction-এ তত বেশি অবদান রাখে। উপরে দেখানো Tree 1, Tree 2, Tree 3-এর পৃথক পৃথক Structure লক্ষ্য করলে বোঝা যায়, একই ডেটাসেট থেকে Bootstrap Sampling ও Feature Randomness কীভাবে সম্পূর্ণ ভিন্ন ভিন্ন গাছ তৈরি করে।

১৩. হাইপারপ্যারামিটার (Hyperparameters)

Classifier:

n_estimators

- **Default:** ১০০
- **কাজ কী:** র্যান্ডম ফরেস্ট মডেলের ভেতরে মোট কতগুলো ডিসিশন ট্রি (Decision Tree) বা সিদ্ধান্ত-গাছ তৈরি হবে, তা এটি ঠিক করে।
- **অন্যান্য অপশন:** যেকোনো পজিটিভ পূর্ণসংখ্যা (যেমন: ৫০, ২০০, ৫০০)।
- **কখন কোনটি ব্যবহার করব:** যদি ডেটাসেট অনেক ছোট হয় বা কম্পিউটারের র‍্যাম/প্রসেসর কম শক্তির হয়, তবে এটি কমিয়ে ৫০ করা যায়। আর যদি ডেটাসেট অনেক বড় হয় এবং আপনার সর্বোচ্চ এক্যুরেসী বা স্থায়িত্বের প্রয়োজন পড়ে, তবে এটি বাড়িয়ে ৩০০ বা ৫০০ করা উচিত।

max_features

- **Default:** "sqrt" (মোট ফিচারের সংখ্যার বর্গমূল)
- **কাজ কী:** গাছের প্রতিটা নোড স্প্লিট বা ভাগ করার সময় র্যান্ডমলি সর্বোচ্চ কয়টি ফিচার (কলাম) স্ক্যান করা হবে, তা এটি নিয়ন্ত্রণ করে।
- **অন্যান্য অপশন:** "log2" (লগ বেস ২), কোনো নির্দিষ্ট সংখ্যা (যেমন: ৩, ৫) বা ভগ্নাংশ (যেমন: ০.৫ অর্থাৎ মোট ফিচারের ৫০%)। এছাড়া None দিলে প্রতিবার সব ফিচার স্ক্যান করে।

- **কখন কোনটি ব্যবহার করব:** ডেটাসেটে যদি শত শত কলাম (High Dimensionality) থাকে, তবে ডিফল্ট "sqrt" বা "log2" রাখাই সবচেয়ে বেস্ট, কারণ এটি গাছগুলোর মধ্যে বৈচিত্র্য বাড়ায়। কিন্তু যদি কলামের সংখ্যা খুব কম হয় (যেমন: মাত্র ৩-৪টি কলাম), তখন এটি None রাখা ভালো যাতে মডেল সব কলাম দেখার সুযোগ পায়।

max_depth

- **Default:** None
- **কাজ কী:** জঙ্গলের প্রতিটি একক গাছের সর্বোচ্চ গভীরতা বা লেভেল কতটুকু হতে পারবে, তা এটি বেঁধে দেয়। None থাকলে গাছগুলো একদম শেষ সীমা পর্যন্ত বাড়তে থাকে।
- **অন্যান্য অপশন:** যেকোনো পূর্ণসংখ্যা (যেমন: ৫, ১০, ১৫)।
- **কখন কোনটি ব্যবহার করব:** যদি দেখেন মডেলটি ট্রেনিং ডেটায় চমৎকার রেজাল্ট দিলেও টেস্ট ডেটায় ফেইল করছে (ওভারফিটিং হচ্ছে), তখন গাছের এই অনিয়ন্ত্রিত বৃদ্ধি থামাতে এখানে একটি নির্দিষ্ট গভীরতা (যেমন: max_depth=10 বা 12) সেট করতে হবে।

min_samples_split

- **Default:** ২
- **কাজ কী:** একটি নোডকে ভেঙে নতুন দুটি ডাল তৈরি করার জন্য ওই নোডে নূন্যতম কয়টি ডেটা স্যাম্পল থাকতে হবে, তা এটি ঠিক করে।
- **অন্যান্য অপশন:** যেকোনো পূর্ণসংখ্যা (যেমন: ৫, ১০, ৫০) বা ভগ্নাংশ।
- **কখন কোনটি ব্যবহার করব:** যখন মডেলের ওভারফিটিং বন্ধ করার প্রয়োজন পড়ে। যদি আপনি এখানে min_samples_split=10 দেন, তবে কোনো নোডে ১০টির

কম ডেটা থাকলে গাছ সেটিকে আর ভাঙবে না, ওখানেই থামিয়ে দেবে। বড় এবং নয়েজি ডেটাসেটে এর মান বাড়িয়ে দেওয়া উচিত।

min_samples_leaf

- **Default:** ১
- **কাজ কী:** একটি গাছের একদম শেষ পাতা বা লিফ নোডে (Leaf Node) নূন্যতম কয়টি ডেটা স্যাম্পল থাকতেই হবে, তা ফিক্স করে।
- **অন্যান্য অপশন:** যেকোনো পূর্ণসংখ্যা (যেমন: ২, ৫, ১০) বা ভগ্নাংশ।
- **কখন কোনটি ব্যবহার করব:** এটিও ওভারফিটিং কমানোর এবং মডেলের আউটপুটকে মসৃণ বা স্মুথ করার জন্য ব্যবহৃত হয়। যদি ডেটাসেটে অনেক আউটলায়ার বা ভুল ডেটা থাকে, তবে এর মান বাড়িয়ে ৫ বা ১০ করে দিলে ক্ষুদ্রাতিক্ষুদ্র ও অর্থহীন প্যাটার্নের জন্য নতুন কোনো পাতা তৈরি হওয়া বন্ধ হয়ে যায়।

bootstrap

- **Default:** True
- **কাজ কী:** প্রতিটি গাছ ট্রেন করার সময় বুটস্ট্রাপ স্যাম্পলিং (Row sampling with replacement) পদ্ধতি ব্যবহার করা হবে কি না, তা এটি নির্ধারণ করে।
- **অন্যান্য অপশন:** False
- **কখন কোনটি ব্যবহার করব:** এটি সবসময় True রাখাই র্যান্ডম ফরেস্টের মূল নিয়ম। তবে যদি আপনার ডেটাসেট অত্যন্ত নিখুঁত হয় এবং আপনি চান প্রতিটি গাছ পুরো ডেটাসেটটি দেখেই তৈরি হোক (কোনো রো বাদ না পড়ুক), শুধুমাত্র তখনই এটি False করা হয়।

oob_score

- **Default:** False

- **কাজ কী:** বুটস্ট্রাপ স্যাম্পলিংয়ের সময় যে ৩৬.৮% ডেটা ট্রেনিং থেকে বাদ পড়ে যায়, সেগুলো ব্যবহার করে মডেলের ট্রেনিং চলাকালীনই একটি ইন্টারনাল ভ্যালিডেশন স্কোর হিসাব করা হবে কি না, তা এটি ঠিক করে।
- **অন্যান্য অপশন:** True
- **কখন কোনটি ব্যবহার করব:** যখন আপনার ডেটাসেট ছোট এবং আপনি ভ্যালিডেশনের জন্য আলাদা করে মূল্যবান ডেটা নষ্ট করতে চান না (যেমন: train_test_split বা K-Fold না করে)। তখন oob_score=True করে দিলে আলাদা টেস্ট সেট ছাড়াই মডেলের রিয়েল এক্যুরেসীর ধারণা পাওয়া যায়।

n_jobs

- **Default:** None (কম্পিউটারের মাত্র ১টি প্রসেসর কোর ব্যবহার করে)
- **কাজ কী:** মডেল ট্রেনিং ও প্রেডিকশনের সময় ব্যাকগ্রাউন্ডে সিপিইউ-এর কয়টি কোর সমান্তরালভাবে কাজ করবে, তা এটি নিয়ন্ত্রণ করে।
- **অন্যান্য অপশন:** যেকোনো পূর্ণসংখ্যা বা -1।
- **কখন কোনটি ব্যবহার করব:** বাস্তব জীবনের বড় প্রজেক্টে সবসময় এটি -1 ব্যবহার করা উচিত। n_jobs=-1 দিলে এটি কম্পিউটারের সবগুলো প্রসেসর কোরকে একসাথে কাজে লাগিয়ে শত শত গাছ ট্রেন করার প্রক্রিয়াটিকে সুপার-ফাস্ট করে তোলে।

random_state

- **Default:** None
- **কাজ কী:** বুটস্ট্রাপ স্যাম্পলিং এবং ফিচার সিলেকশনের র্যান্ডম বা এলোমেলো প্রক্রিয়াটিকে একটি নির্দিষ্ট গাণিতিক বীজ দিয়ে লক করে দেয়।
- **অন্যান্য অপশন:** যেকোনো ফিক্সড পূর্ণসংখ্যা (যেমন: 0, 42)।
- **কখন কোনটি ব্যবহার করব:** টিম প্রজেক্ট, অ্যাসাইনমেন্ট বা কোড পরীক্ষার সময় এটি যেকোনো একটি ফিক্সড সংখ্যায় সেট করা বাধ্যতামূলক। এটি নিশ্চিত করে যে

কোডটি আপনার কম্পিউটারে বা অন্য কারো কম্পিউটারে যতবারই রান করা হোক না কেন, প্রতিবার অবিকল একই গাছ এবং একই এক্যুরেসী স্কোর পাওয়া যাবে।

class_weight

- **Default:** None (সব ক্লাসকে সমান গুরুত্ব দেয়)
- **কাজ কী:** ইমব্যালেন্সড ডেটাসেটের ক্ষেত্রে ছোট বা মাইনোরিটি ক্লাসকে অতিরিক্ত গাণিতিক গুরুত্ব বা ওজন দেওয়ার জন্য এটি ব্যবহৃত হয়।
- **অন্যান্য অপশন:** "balanced" অথবা কাস্টম ডিকশনারি।
- **কখন কোনটি ব্যবহার করব:** যখন ডেটা প্রবলভাবে ইমব্যালেন্সড (যেমন: ক্রেডিট কার্ড জালিয়াতির ডেটা মাত্র ১% আর নরমাল ডেটা ৯৯%)। তখন `class_weight='balanced'` দিলে মডেল স্বয়ংক্রিয়ভাবে ওই ১% ছোট ক্লাসকে বেশি প্রাধান্য দেয়, যার ফলে মডেলের প্রেডিকশন কোয়ালিটি বা রিকল (*Recall*) উন্নত হয়।

Regressor:

criterion

- **Default:** "squared_error"
- **কাজ কী:** রিগ্রেশন সমস্যার ক্ষেত্রে প্রতিটি ডিসিশন ট্রির নোড ভাগ বা স্প্লিট করার কোয়ালিটি মাপার জন্য ব্যাকগ্রাউন্ডে কোন গাণিতিক লস ফাংশন (Loss Function) বা পরিমাপক ব্যবহার করা হবে, তা এটি নির্ধারণ করে।
- **অন্যান্য অপশন:** "absolute_error", "friedman_mse", "poisson"
- **কখন কোনটি ব্যবহার করব:**
 - **"squared_error" (Mean Squared Error - MSE):** এটি আপনার ডেটাসেটে থাকা বড় বড় ভুলগুলোকে (Large Errors) বেশি গাণিতিক শাস্তি বা গুরুত্ব দেয়। ডেটা যদি সাধারণ বা নরমাল ডিস্ট্রিবিউশন মেনে চলে, তবে এটি সবচেয়ে দ্রুত এবং সেরা ফলাফল দেয়।

- **"absolute_error" (Mean Absolute Error - MAE):** এটি প্রতিটি ভুলের পরম মান হিসাব করে। আপনার ডেটাসেটে যদি প্রচুর আউটলায়ার (Outliers) বা অস্বাভাবিক বড়/ছোট মান থাকে, তখন এই অপশনটি ব্যবহার করা উচিত। কারণ এটি আউটলায়ারের প্রতি অত্যন্ত প্রতিরোধী (Robust)। তবে ব্যাকগ্রাউন্ডে এটি প্রসেস হতে MSE-এর চেয়ে অনেক বেশি সময় নেয়।
- **"friedman_mse":** এটি মূলত জিবিএম (Gradient Boosting) অ্যালগরিদমের উদ্ভাবক জেরোম ফ্রিডম্যানের তৈরি একটি বিশেষ পরিমাপক। এটি সাধারণ MSE-এর ওপর ভিত্তি করেই কাজ করে, তবে কিছু বিশেষ ক্ষেত্রে এটি ডালপালা গজানোর জন্য আরও নিখুঁত থ্রেশহোল্ড খুঁজে বের করতে পারে।
- **"poisson":** যখন আপনার টার্গেট কলামের ডেটা কোনো কাউন্ট বা গণনা নির্দেশ করে (যেমন: "আজকে দোকানে কতজন কাস্টমার আসবে" বা "এই রাস্তায় প্রতি ঘণ্টায় কয়টি গাড়ি চলে") এবং ডেটাটি পয়সঁ ডিস্ট্রিবিউশন (Poisson Distribution) মেনে চলে, তখন এই ক্রাইটেরিয়ানটি ব্যবহার করলে লস সবচেয়ে কম হয়।

max_features

- **Default:** 1.0 (অর্থাৎ, মোট ফিচারের ১০০% বা সবগুলোই স্ক্যান করবে)
- **কাজ কী:** রিগ্রেশন গাছের প্রতিটা নোড স্প্লিট বা ভাগ করার সময় র্যান্ডমলি সর্বোচ্চ কয়টি কলাম বা ফিচার বিবেচনা করা হবে, তা এটি নিয়ন্ত্রণ করে।
- **অন্যান্য অপশন:** "sqrt", "log2", কোনো নির্দিষ্ট সংখ্যা (int) বা ভগ্নাংশ (float)।
- **কখন কোনটি ব্যবহার করব:** ক্লাসিফায়ারের ডিফল্ট ছিল "sqrt", কিন্তু রিগ্রেশনের ডিফল্ট হলো 1.0। এর মানে হলো ডিফল্ট অবস্থায় রিগ্রেশনের প্রতিটি গাছ নোড ভাগার সময় সব ফিচার চেক করার সুযোগ পায়। তবে আপনার ডেটাসেটে যদি ফিচারের সংখ্যা অনেক বেশি হয় এবং গাছগুলোর মধ্যে বৈচিত্র্য (Diversity) বাড়িয়ে ওভারফিটিং কমাতে চান, তবে এটি পরিবর্তন করে কাস্টম ফ্লোট মান (যেমন: 0.3 বা ৩০%) অথবা ক্লাসিফায়ারের মতো "sqrt" সেট করা অত্যন্ত কার্যকর।

RandomForestClassifier() Attributes

feature_importances_

Default: Created after fit()

সবগুলো Decision Tree-এর Feature Importance-এর Average মান সংরক্ষণ করে। এটি দেখায় কোন Feature Prediction করার ক্ষেত্রে সবচেয়ে বেশি গুরুত্বপূর্ণ। সব Importance Score-এর যোগফল 1 হয়।

estimators_

Default: Created after fit()

Forest-এর ভিতরে থাকা প্রতিটি Trained Decision Tree-এর List সংরক্ষণ করে। প্রতিটি Element একটি আলাদা DecisionTreeClassifier Object।

oob_score_

Default: Created after fit() *(Only when oob_score=True)*

Out-of-Bag (OOB) Sample ব্যবহার করে Model-এর Accuracy Score সংরক্ষণ করে। এটি আলাদা Validation Set ছাড়াই Model-এর Performance মূল্যায়ন করতে সাহায্য করে।

oob_decision_function_

Default: Created after fit() *(Only when oob_score=True)*

প্রতিটি Training Sample-এর জন্য Out-of-Bag Sample ব্যবহার করে Predicted Probability সংরক্ষণ করে। এটি শুধুমাত্র Classification-এর ক্ষেত্রে পাওয়া যায়।

classes_

Default: Created after fit()

Training Data-তে থাকা সকল Target Class-এর List সংরক্ষণ করে।

n_classes_

Default: Created after fit()

Training Data-তে মোট কতটি Target Class রয়েছে তা সংরক্ষণ করে।

estimators_samples_

Default: Created after fit()

প্রতিটি Decision Tree কোন কোন Training Sample (Bootstrap Sample) ব্যবহার করে Train হয়েছে, তাদের Index সংরক্ষণ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় Model মোট কতটি Input Feature ব্যবহার করেছে তা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

n_outputs_

Default: Created after fit()

Model মোট কতটি Target Output Predict করেছে তা সংরক্ষণ করে। সাধারণ Classification-এ এটি সাধারণত 1 হয়।

RandomForestRegressor() Attributes

feature_importances_

Default: Created after fit()

সবগুলো Decision Tree-এর Feature Importance-এর Average মান সংরক্ষণ করে। এটি দেখায় কোন Feature Regression Prediction-এর জন্য সবচেয়ে বেশি গুরুত্বপূর্ণ।

estimators_

Default: Created after fit()

Forest-এর ভিতরে থাকা প্রতিটি Trained Decision Tree-এর List সংরক্ষণ করে। প্রতিটি Element একটি DecisionTreeRegressor Object।

oob_score_

Default: Created after fit() (*Only when oob_score=True*)

Out-of-Bag (OOB) Sample ব্যবহার করে Model-এর R^2 Score সংরক্ষণ করে। এটি Model-এর Generalization Performance মূল্যায়ন করতে সাহায্য করে।

estimators_samples_

Default: Created after fit()

প্রতিটি Decision Tree কোন কোন Bootstrap Sample ব্যবহার করে Train হয়েছে, তাদের Index সংরক্ষণ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় ব্যবহৃত মোট Input Feature-এর সংখ্যা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

n_outputs_

Default: Created after fit()

Model মোট কতটি Target Output Predict করছে তা সংরক্ষণ করে। সাধারণ Regression-এ এটি সাধারণত 1 হয়।

১৪. স্কিট-লার্ন ইম্পোর্ট (Scikit-learn Import)

Code:

Classification:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay

digits = load_digits()
X = pd.DataFrame(digits.data)
y = pd.Series(digits.target)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

rf_pipeline = Pipeline(
    steps=[
        ('model', RandomForestClassifier(random_state=42, n_jobs=-1))
    ]
)
```

```

param_grid = {
    'model__n_estimators': [50, 100, 200],
    'model__max_features': ['sqrt', 'log2'],
    'model__max_depth': [10, 20, None],
    'model__min_samples_split': [2, 5]
}

grid_search = GridSearchCV(
    estimator=rf_pipeline,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("--- GridSearchCV Tuning Results ---")
print("Best Parameters Found:", grid_search.best_params_)
print(f"Best CV Accuracy: {grid_search.best_score_:.4f}\n")

best_rf_pipeline = grid_search.best_estimator_
y_pred = best_rf_pipeline.predict(X_test)

print("--- Final Model Performance on Test Set ---")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred):.4f}\n")
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=digits.target_names)
disp.plot(cmap=plt.cm.Blues)
plt.title("Random Forest Confusion Matrix - Digits Dataset")
plt.show()

```

Code Regressor:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

housing = fetch_california_housing()
X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = pd.Series(housing.target)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

rf_reg_pipeline = Pipeline(
    steps=[
        ('model', RandomForestRegressor(random_state=42, n_jobs=-1))
    ]
)
```

```

param_grid = {
    'model__n_estimators': [50, 100],
    'model__criterion': ['squared_error', 'absolute_error'],
    'model__max_features': [0.5, 'sqrt'],
    'model__max_depth': [10, None]
}

grid_search = GridSearchCV(
    estimator=rf_reg_pipeline,
    param_grid=param_grid,
    cv=3,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("--- GridSearchCV Tuning Results ---")
print("Best Parameters Found:", grid_search.best_params_)

best_rf_reg_pipeline = grid_search.best_estimator_
y_pred = best_rf_reg_pipeline.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("--- Final Model Performance on Test Set ---")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"R-squared Score (R2 Score): {r2:.4f}\n")

plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.3, color='crimson')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2)
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')
plt.title('Actual vs Predicted Prices - California Housing')
plt.show()

```

১৫. পাইথন উদাহরণ — From-Scratch Implementation (Python Example)

নিচে একটি সরলীকৃত From-Scratch Random Forest দেখানো হলো, যা এই অধ্যায়ের Bootstrap Sampling ও Majority Voting-এর সাথে হুবহু মেলে:

```
import numpy as np
import random
from collections import Counter

def bootstrap_sample(X, y):
    n = len(X)
    idx = [random.randrange(n) for _ in range(n)]
    return [X[i] for i in idx], [y[i] for i in idx]

def majority_vote(predictions):
    return Counter(predictions).most_common(1)[0][0]

class RandomForestScratch:
    def __init__(self, n_trees=3, tree_fn=None):
        self.n_trees = n_trees
        self.tree_fn = tree_fn # a simple decision-stump function
        self.trees = []

    def fit(self, X, y):
        self.trees = []
        for _ in range(self.n_trees):
            X_boot, y_boot = bootstrap_sample(X, y)
            self.trees.append(self.tree_fn(X_boot, y_boot))

    def predict_one(self, x):
        preds = [tree.predict(x) for tree in self.trees]
        return majority_vote(preds)
```

১৬. প্রেডিকশন প্রসেস (Prediction Process)

নতুন স্যাম্পলের Prediction তিন ধাপে হয়: (১) স্যাম্পলটি জঙ্গলের প্রতিটি গাছের মধ্য দিয়ে আলাদাভাবে পাঠানো হয় এবং প্রতিটি গাছ তার নিজস্ব Prediction দেয়, (২) Classification-এর ক্ষেত্রে সব গাছের Prediction-এর মধ্যে Majority Vote নেওয়া হয় (Regression-এর ক্ষেত্রে সবগুলোর গড় নেওয়া হয়), এবং (৩) সেই চূড়ান্ত ভোট/গড়-ই Random Forest-এর Output হিসেবে দেওয়া হয়। এই সম্পূর্ণ প্রক্রিয়া প্রতিটি গাছের জন্য সমান্তরালভাবে (Parallel) চালানো যায়, যা Prediction দ্রুত করতে সাহায্য করে।

২২. সাক্ষাৎকারের প্রশ্ন (Interview Questions)

Interview Questions

Q1: Out-of-Bag (OOB) কী এবং এটি ব্যবহারের সুবিধা কী?

- **Answer:** র্যান্ডম ফরেস্টে বুটস্ট্র্যাপ স্যাম্পলিং করার সময় প্রতিস্থাপনের নীতির কারণে প্রতিটি গাছের ট্রেনিং সেট থেকে গড়ে প্রায় ৩৬.৮% ডেটা বাদ পড়ে যায়। এই বাদ পড়া ডেটাপয়েন্টগুলোকে **Out-of-Bag (OOB)** স্যাম্পল বলা হয়। মডেল ট্রেনিং শেষ হওয়ার পর, প্রতিটি গাছকে তার নিজস্ব ওওবি স্যাম্পল দিয়ে টেস্ট করে যে গড় স্কোর পাওয়া যায়, সেটিই ওওবি স্কোর। এর বড় সুবিধা হলো—এটি ব্যবহার করলে আলাদা করে কোনো টেস্ট সেট বা ক্রস-ভ্যালিডেশন (K-Fold CV) ছাড়াই মডেলের জেনারেলাইজেশন ক্ষমতা বা রিয়েল-টাইম এক্যুরেসী নিখুঁতভাবে পরিমাপ করা যায়।

Q2: Random Forest-কে কেন একটি 'Black-Box' মডেল বলা হয়?

- **Answer:** একটি একক ডিসিশন ট্রিকে খুব সহজে গ্রাফ বা ডায়াগ্রামের সাহায্যে ভিজ্যুয়ালাইজ করা যায় এবং এর প্রতিটি শর্ত (if-else রুলস) সহজে ট্র্যাকিং করা যায় (White-Box)। কিন্তু র্যান্ডম ফরেস্ট যখন ১০০ থেকে ৫০০টি গভীর গাছ একসাথে কাজ করে, তখন প্রতিটি গাছের নিজস্ব র্যান্ডম ফিচার ও র্যান্ডম ডেটা স্যাম্পল থাকে। কোনো একটি নির্দিষ্ট প্রেডিকশনের জন্য ব্যাকএন্ডে ৫০০টি গাছের হাজার হাজার জটিল শর্ত ও ভোটিং মেকানিজম একসাথে ট্র্যাক করা মানুষের পক্ষে অসম্ভব হয়ে পড়ে। গাণিতিক লজিক স্পষ্টভাবে ব্যাখ্যা করা যায় না বলেই একে **Black-Box** মডেল বলা হয়।

Q3: Random Forest-এর দুটি গাছ কি কখনো ছবছ এক হতে পারে? যদি না হয়, তবে কেন?

- **Answer:** তাত্ত্বিকভাবে বা বাস্তবে শত শত গাছের মধ্যে দুটি গাছ ছবছ এক হওয়া প্রায় অসম্ভব। এর পেছনে দুটি মূল কারণ রয়েছে: প্রথমত, **Bootstrap Sampling**-এর কারণে প্রতিটি গাছ সম্পূর্ণ আলাদা ও এলোমেলো ডেটার সাবসেট পায়। দ্বিতীয়ত এবং সবচেয়ে বড় কারণ হলো **Feature Randomness**; প্রতিটি নোড ভাগার সময় গাছগুলো সব কলাম দেখতে পায় না, বরং সম্পূর্ণ র্যান্ডমলি সিলেক্টেড কিছু কলাম (যেমন: $max_features = \sqrt{d}$) থেকে সেরা স্প্লিট বেছে নেয়। এই দ্বিমুখী র্যান্ডমনেসের কারণে জঙ্গলের প্রতিটি গাছের গঠন একে অপরের থেকে সম্পূর্ণ ভিন্ন ও স্বাধীন হয়।

Q4: আপনার ডেটাসেটে যদি প্রচুর আউটলায়ার (Outliers) থাকে, তবে রিগ্রেশনের ক্ষেত্রে আপনি কোন Criterion ব্যবহার করবেন এবং কেন?

- **Answer:** ডেটাসেটে প্রচুর আউটলায়ার থাকলে আমি criterion='absolute_error' (Mean Absolute Error - MAE) ব্যবহার করব। ডিফল্ট ক্রাইটেরিয়ন squared_error (MSE) ভুলের মানকে বর্গ (Square) করে হিসাব করে, যার ফলে একটি বড় আউটলায়ারের ভুল পুরো মডেলের ওপর বিশাল নেতিবাচক প্রভাব ফেলে। অন্যদিকে, MAE ভুলের পরম মান (Absolute value) নেয়, যা আউটলায়ারের প্রতি অত্যন্ত প্রতিরোধী (**Robust to outliers**) এবং মডেলের চূড়ান্ত গড় সিদ্ধান্তকে সহজে বিচ্যুত হতে দেয় না।

Q5: Random Forest কি ওভারফিট করতে পারে? যদি করে, তবে তা কীভাবে সমাধান করবেন?

- **Answer:** সাধারণ ডিসিশন ট্রির মতো র্যান্ডম ফরেস্ট সহজে ওভারফিট করে না, কারণ ব্যাগিং বা এভারেজিং প্রক্রিয়াটি ভ্যারিয়েন্স কমিয়ে আনে। তবে ডেটাসেটে যদি চরম মাত্রায় নয়েজ থাকে এবং হাইপারপ্যারামিটার নিয়ন্ত্রণ না করা হয়, তবে এটিও ওভারফিট করতে পারে। এর প্রধান সমাধানগুলো হলো:
- max_depth-এর মান একটি নির্দিষ্ট সংখ্যায় (যেমন: ১০ বা ১৫) বেঁধে দেওয়া।
- min_samples_leaf-এর মান বাড়িয়ে (যেমন: ৫ বা ১০) শেষ পাতার নূন্যতম ডেটা ফিল্ল করা।
- n_estimators বা জঙ্গলে গাছের সংখ্যা বাড়িয়ে মডেলকে আরও সুষম করা।

২৩. সারাংশ (Summary)

Summary

- **Core Concept:** র্যান্ডম ফরেস্ট হলো একটি অত্যন্ত শক্তিশালী এবং স্থিতিশীল এনসেম্বল লার্নিং অ্যালগরিদম, যা বুটস্ট্র্যাপ স্যাম্পলিং এবং ফিচারের র্যান্ডমনেস ব্যবহার করে শত শত ডিসিশন ট্রির সমন্বয়ে জঙ্গল তৈরি করে।

- **Mechanism:** এটি ক্লাসিফিকেশনের জন্য মেজরিটি ভোটিং (Majority Voting) এবং রিগ্রেশনের জন্য গড় (Averaging) মেথড ব্যবহার করে চূড়ান্ত প্রেডিকশন দেয়।
- **Strength:** এর সবচেয়ে বড় শক্তি হলো এটি কোনো প্রকার ফিচার স্কেলিং ছাড়াই র-ডেটাসেটে চমৎকার কাজ করে এবং ওভারফিটিংয়ের হাত থেকে মডেলকে স্বয়ংক্রিয়ভাবে রক্ষা করে।
- **Trade-off:** উচ্চমাত্রার এক্যুরেসী এবং স্ট্যাবিলিটি দিলেও, এটি মেমোরিতে অনেক বেশি জায়গা নেয় এবং এর ব্ল্যাক-বক্স প্রকৃতির কারণে মডেলের ভেতরের লজিক পুরোপুরি ব্যাখ্যা করা যায় না।

২১. সাধারণ ভুলক্রটি (Common Mistakes)

- খুব কম `n_estimators` (যেমন ১০টির কম) ব্যবহার করা — এতে Ensemble-এর সুবিধা পুরোপুরি পাওয়া যায় না; সাধারণত ১০০ বা তার বেশি গাছ ভালো ফলাফল দেয়।
- প্রতিটি একক গাছকে খুব গভীর (`max_depth=None`) হতে দেওয়া এবং ধরে নেওয়া যে Ensemble নিজে থেকেই Overfitting সামলে নেবে — বড় Dataset-এ এটি এখনও Memory ও Time-এর অপচয় ঘটাতে পারে।
- `feature_importances_`-কে সরাসরি Causal সম্পর্ক হিসেবে ব্যাখ্যা করা, যদিও এটি শুধু Correlation/Predictive Power দেখায়, Causation নয়।
- `oob_score` ব্যবহার করার সুযোগ থাকা সত্ত্বেও আলাদা Validation Set তৈরি করে ডেটা নষ্ট করা।

সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)

Introduction

Random Forest হলো অনেকগুলো Decision Tree-এর Ensemble। এই সেকশনে Decision Tree অধ্যায়ে ব্যবহৃত একই ৫-স্যাম্পল ডেটাসেট নিয়ে Bootstrap Sampling-এর মাধ্যমে ৩টি ভিন্ন ট্রি তৈরি করে, প্রতিটির প্রেডিকশন নিয়ে Majority Voting দিয়ে চূড়ান্ত সিদ্ধান্ত নেওয়া হবে।

Dataset (Decision Tree অধ্যায়ের মতোই একই ডেটাসেট)

Sample	Outlook	Temperature	Play (Label)
1	Sunny	Hot	No
2	Sunny	Cool	Yes
3	Rainy	Hot	No
4	Rainy	Cool	Yes
5	Sunny	Cool	No

Bootstrap Sampling

Bootstrap Sampling মানে হলো মূল ৫টি স্যাম্পল থেকে Replacement সহ (একই স্যাম্পল একাধিকবার নেওয়া যেতে পারে) র‍্যান্ডমভাবে ৫টি স্যাম্পল বেছে নিয়ে একটি নতুন Training Set তৈরি করা। প্রতিটি ট্রির জন্য আলাদা Bootstrap Set তৈরি করা হয়েছে (random_state=42 দিয়ে Reproducible রাখা হয়েছে):

Tree 1

Bootstrap Sample: [('Sunny', 'Hot', 'No'), ('Sunny', 'Hot', 'No'), ('Rainy', 'Hot', 'No'), ('Sunny', 'Cool', 'Yes'), ('Sunny', 'Cool', 'Yes')]

এই স্যাম্পলের Entropy(S) = 0.9710। এখন Outlook ও Temperature উভয়ের জন্য Information Gain গণনা করে সবচেয়ে ভালো Split ফিচার বাছাই করা হলো।

Outlook দিয়ে Split → Outlook=Sunny: ['No', 'No', 'Yes', 'Yes'], Entropy=1.0000, weight=0.80; Outlook=Rainy: ['No'], Entropy=0.0000, weight=0.20 → Weighted Entropy = 0.8000 → IG = 0.9710 - 0.8000 = 0.1710

Temperature দিয়ে Split → Temperature=Cool: ['Yes', 'Yes'], Entropy=0.0000, weight=0.40; Temperature=Hot: ['No', 'No', 'No'], Entropy=0.0000, weight=0.60 → Weighted Entropy = 0.0000 → IG = 0.9710 - 0.0000 = 0.9710

সর্বোচ্চ Information Gain পাওয়া গেছে "Temperature" ফিচারে ($IG = 0.9710$), তাই এই ট্রি Temperature দিয়ে Split করবে (Decision Stump — এক-স্তরের ট্রি, ছোট ডেটাসেটের কারণে)।

নতুন স্যাম্পলের Temperature = "Cool", তাই এই Bootstrap Set-এর মধ্যে Temperature = "Cool" সাবসেটের সংখ্যাগরিষ্ঠ ক্লাস নেওয়া হলো → Tree 1-এর প্রেডিকশন = Yes।

Tree 2

Bootstrap Sample: [('Sunny', 'Cool', 'Yes'), ('Sunny', 'Hot', 'No'), ('Sunny', 'Cool', 'No'), ('Sunny', 'Hot', 'No'), ('Sunny', 'Cool', 'No')]

এই স্যাম্পলের Entropy(S) = 0.7219। এখন Outlook ও Temperature উভয়ের জন্য Information Gain গণনা করে সবচেয়ে ভালো Split ফিচার বাছাই করা হলো।

Outlook দিয়ে Split → Outlook=Sunny: ['Yes', 'No', 'No', 'No', 'No'], Entropy=0.7219, weight=1.00 → Weighted Entropy = 0.7219 → $IG = 0.7219 - 0.7219 = 0.0000$

Temperature দিয়ে Split → Temperature=Cool: ['Yes', 'No', 'No'], Entropy=0.9183, weight=0.60; Temperature=Hot: ['No', 'No'], Entropy=0.0000, weight=0.40 → Weighted Entropy = 0.5510 → $IG = 0.7219 - 0.5510 = 0.1710$

সর্বোচ্চ Information Gain পাওয়া গেছে "Temperature" ফিচারে ($IG = 0.1710$), তাই এই ট্রি Temperature দিয়ে Split করবে (Decision Stump — এক-স্তরের ট্রি, ছোট ডেটাসেটের কারণে)।

নতুন স্যাম্পলের Temperature = "Cool", তাই এই Bootstrap Set-এর মধ্যে Temperature = "Cool" সাবসেটের সংখ্যাগরিষ্ঠ ক্লাস নেওয়া হলো → Tree 2-এর প্রেডিকশন = No।

Tree 3

Bootstrap Sample: [('Rainy', 'Cool', 'Yes'), ('Sunny', 'Hot', 'No'), ('Sunny', 'Hot', 'No'), ('Sunny', 'Hot', 'No'), ('Sunny', 'Cool', 'Yes')]

এই স্যাম্পলের Entropy(S) = 0.9710। এখন Outlook ও Temperature উভয়ের জন্য Information Gain গণনা করে সবচেয়ে ভালো Split ফিচার বাছাই করা হলো।

Outlook দিয়ে Split → Outlook=Sunny: ['No', 'No', 'No', 'Yes'], Entropy=0.8113, weight=0.80; Outlook=Rainy: ['Yes'], Entropy=0.0000, weight=0.20 → Weighted Entropy = 0.6490 → $IG = 0.9710 - 0.6490 = 0.3219$

Temperature দিয়ে Split → Temperature=Cool: ['Yes', 'Yes'], Entropy=0.0000, weight=0.40; Temperature=Hot: ['No', 'No', 'No'], Entropy=0.0000, weight=0.60 → Weighted Entropy = 0.0000 → $IG = 0.9710 - 0.0000 = 0.9710$

সর্বোচ্চ Information Gain পাওয়া গেছে "Temperature" ফিচারে (IG = 0.9710), তাই এই ট্রি Temperature দিয়ে Split করবে (Decision Stump — এক-স্তরের ট্রি, ছোট ডেটাসেটের কারণে)।

নতুন স্যাম্পলের Temperature = "Cool", তাই এই Bootstrap Set-এর মধ্যে Temperature = "Cool" সাবসেটের সংখ্যাগরিষ্ঠ ক্লাস নেওয়া হলো → Tree 3-এর প্রেডিকশন = Yes।

Decision of Every Tree

Tree	Split Feature Used	Prediction for (Sunny, Cool)
Tree 1	—	Yes
Tree 2	—	No
Tree 3	—	Yes

Majority Voting

তিনটি ট্রির প্রেডিকশন: ['Yes', 'No', 'Yes']। ভোট গণনা: Yes = 2 ভোট, No = 1 ভোট।

Final Prediction

যেহেতু সংখ্যাগরিষ্ঠ ট্রি "Yes" ভোট দিয়েছে, তাই Random Forest-এর চূড়ান্ত প্রেডিকশন (Outlook=Sunny, Temperature=Cool) = Yes।

Interpretation — কেন Random Forest একক Decision Tree থেকে বেশি Robust?

লক্ষ্য করুন, একক Decision Tree অধ্যায়ে একই ইনপুট (Sunny, Cool)-এর জন্য প্রেডিকশন ছিল "No" (Tie-break-এর মাধ্যমে), কিন্তু Random Forest-এ Bootstrap Sampling-এর কারণে বিভিন্ন ট্রি বিভিন্ন সাবসেট দেখেছে এবং ফলাফলে ভিন্নতা এসেছে, চূড়ান্ত ফলাফল হয়েছে "Yes"। এটিই Random Forest-এর মূল শক্তি — একটি একক ট্রি একটি নির্দিষ্ট Sample বা Noise-এর প্রতি অতি সংবেদনশীল (High Variance) হতে পারে, কিন্তু অনেকগুলো ট্রির গড় (Majority Vote) নেওয়ার ফলে এলোমেলো ত্রুটিগুলো একে অপরকে কাটাকাটি (cancel out) করে ফেলে, ফলে সামগ্রিক Prediction অনেক বেশি স্থিতিশীল (Robust) ও কম Overfit-prone হয়।

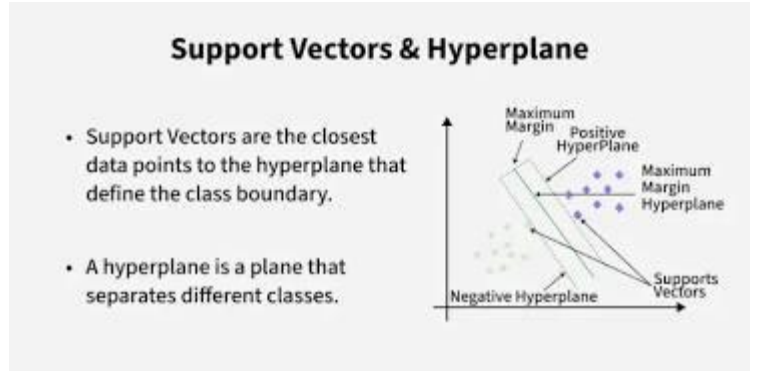
Support Vector Machine

১. ভূমিকা (Introduction)

সাপোর্ট ভেক্টর মেশিন (SVM) হলো একটি অত্যন্ত শক্তিশালী ও গাণিতিকভাবে সুসংহত সুপারভাইজড মেশিন লার্নিং অ্যালগরিদম। এর প্রধান উদ্দেশ্য হলো উচ্চ-মাত্রা বিশিষ্ট উপাত্ত ক্ষেত্রে (High-Dimensional Space) এমন একটি সর্বোত্তম সিদ্ধান্ত সীমানা বা Hyperplane নির্ধারণ করা, যা বিভিন্ন শ্রেণীভুক্ত উপাত্ত বিন্দুগুলোকে (Data Points) পরস্পরের থেকে সর্বোচ্চ দূরত্বে বা Maximum Margin-এ পৃথক করতে পারে।

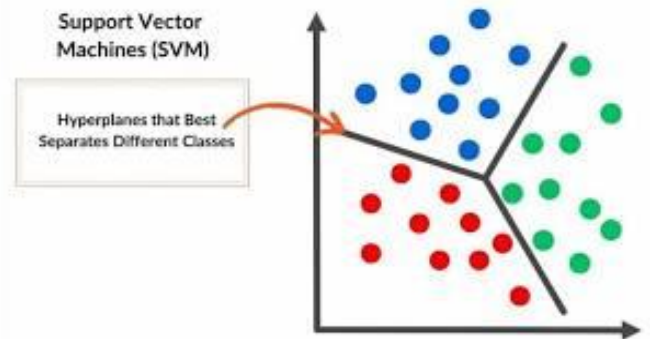
সাপোর্ট ভেক্টর (Support Vectors):

হাইপারপ্লেনের অবস্থান ও অভিযোজন (Orientation) নির্ধারণের জন্য যে নির্দিষ্ট উপাত্ত বিন্দুগুলো সরাসরি দায়ী, সেগুলোকে সাপোর্ট ভেক্টর বলা হয়। এই বিন্দুগুলো সিদ্ধান্ত সীমানার সবচেয়ে নিকটে অবস্থান করে। যদি এই বিশেষ বিন্দুগুলোকে ডেটাসেট থেকে অপসারণ করা হয়, তবে সমগ্র হাইপারপ্লেনের অবস্থান পরিবর্তিত হয়ে যাবে।



মার্জিন (Margin): হাইপারপ্লেন থেকে উভয় পাশের নিকটবর্তী সাপোর্ট ভেক্টরগুলোর লম্ব দূরত্বের সমষ্টিকে মার্জিন বলা হয়। SVM-এর গাণিতিক লক্ষ্য হলো এই মার্জিনের মান সর্বোচ্চ (Maximize) করা, যাতে মডেলটির সাধারণীকরণ ক্ষমতা (Generalization Power) বৃদ্ধি পায় এবং টেস্ট ডেটায় ভুলের পরিমাণ সর্বনিম্ন হয়।

- **শ্রেণীবিভাগ (Classification):** এটি দ্বিমাত্রিক (Binary) এবং বহুমাত্রিক (Multi-class) উভয় প্রকার জটিল শ্রেণীবিভাগ মেলাতে সক্ষম।
- **রিগ্রেশন (Regression):** সংখ্যাচক মান অনুমানের জন্য এর একটি বিশেষ সংস্করণ রয়েছে, যা **Support**



Vector Regression (SVR) নামে পরিচিত।

- **উচ্চ-মাত্রিক উপাত্ত (High-Dimensional Data):** যখন উপাত্ত সেটে নমুনার সংখ্যার (Rows) তুলনায় বৈশিষ্ট্যের সংখ্যা (Columns/Features) অনেক বেশি থাকে (যেমন: বায়োইনফরমেটিক্স, জিনোম সিকোয়েন্সিং বা টেক্সট ক্লাসিফিকেশন), তখন SVM অনন্য কর্মদক্ষতা প্রদর্শন করে।
- **অরৈখিক উপাত্ত বিন্যাস (Non-Linear Data):** উপাত্ত যখন সরলরেখায় বিভাজনযোগ্য হয় না, তখন SVM তার বিখ্যাত **Kernel Trick** ব্যবহার করে উপাত্তগুলোকে একটি উচ্চতর মাত্রার ক্ষেত্রে (Higher Dimensional Space) রূপান্তরিত করে এবং একটি রৈখিক হাইপারপ্লেন দ্বারা পৃথক করে।

২. কেন এই অ্যালগরিদম তৈরি হয়েছিল (Why This Algorithm Was Developed)

১৯৯০-এর দশকে Vladimir Vapnik ও তার সহকর্মীরা লক্ষ্য করেন, Logistic Regression-এর মতো মডেল শুধুমাত্র একটি "যথেষ্ট ভালো" Decision Boundary খুঁজে বের করে — যেকোনো Boundary যা ক্লাসগুলো আলাদা করতে পারে, তা গ্রহণযোগ্য বলে বিবেচিত হয়, ভবিষ্যতের নতুন ডেটার জন্য সেটি কতটা নিরাপদ (Safe) তা বিবেচনা না করেই। SVM তৈরি হয়েছিল একটি স্পষ্ট গাণিতিক প্রশ্নের উত্তর হিসেবে: "অসংখ্য সম্ভাব্য Decision Boundary-র মধ্যে কোনটি সবচেয়ে নিরাপদ, অর্থাৎ উভয় ক্লাস থেকে সর্বোচ্চ দূরত্বে অবস্থিত?" — এই প্রশ্নের উত্তর খুঁজতে গিয়েই Maximum Margin Classifier-এর ধারণা জন্ম নেয়।

৩. বাস্তব জীবনের প্রেরণা (Real-life Motivation)

দুই দেশের মাঝে সীমান্ত রেখা টানার কথা ভাবুন। শুধু একটি লাইন টানাই যথেষ্ট নয় — বাস্তবে একটি নিরাপদ "বাফার জোন" (Buffer Zone) রাখা হয় যাতে সীমান্তের দুই পাশের এলাকা যথেষ্ট দূরত্বে থাকে এবং সামান্য অনুপ্রবেশেও কোনো বিরোধ না বাধে। SVM ঠিক এই বাফার জোন ধারণাটিই প্রয়োগ করে — এটি শুধু ক্লাস আলাদা করে না, বরং দুই ক্লাসের মাঝে সম্ভাব্য সবচেয়ে চওড়া "নিরাপত্তা অঞ্চল" (Margin) তৈরি করে।

৪. স্বভাৱ (Intuition)

সহজভাবে বললে, অসংখ্য লাইন দুটি ক্লাসকে সঠিকভাবে আলাদা করতে পারে, কিন্তু SVM সেই একটি নির্দিষ্ট লাইন (বা হাইপারপ্লেন) খুঁজে বের করে যেটি উভয় ক্লাসের সবচেয়ে কাছের বিন্দুগুলো (Support Vectors) থেকে সর্বোচ্চ দূরত্বে অবস্থিত। শুধুমাত্র এই কাছের বিন্দুগুলোই Decision Boundary নির্ধারণ করে — বাকি সব বিন্দু তারা যতই দূরে থাকুক, সিদ্ধান্তে কোনো প্রভাব ফেলে না।

মেশিন লার্নিং শাস্ত্রে SLR, MLR, Logistic Regression, Decision Tree, Random Forest এবং KNN-এর মতো প্রতিষ্ঠিত মডেল থাকা সত্ত্বেও নিম্নলিখিত কাঠামোগত সুবিধার কারণে SVM অপরিস্রব:

- **Logistic Regression বনাম SVM:** লজিস্টিক রিগ্রেশন ক্লাসগুলোকে পৃথক করার জন্য যেকোনো একটি সাধারণ রৈখিক রেখা টানে, যা মূলত ট্রেনিং লস কমাতে সাহায্য করে। বিপরীতে, SVM খোঁজে Optimal Hyperplane, যা দুই ক্লাসের মধ্যকার দূরত্ব সর্বোচ্চ রাখে। এর ফলে নতুন ও অজানা উপাত্তের ওপর SVM-এর নির্ভুলতা লজিস্টিক রিগ্রেশনের চেয়ে অনেক বেশি স্থিতিশীল হয়।
- **KNN বনাম SVM:** KNN একটি অলস শিক্ষণ পদ্ধতি (Lazy Learner)। এটি টেস্ট টাইমে প্রতিটি নতুন নমুনার জন্য সম্পূর্ণ ডেটাসেটের দূরত্বের গাণিতিক হিসাব করে, যা বৃহৎ ডেটাসেটে অত্যন্ত ধীরগতির এবং মেমোরি সাপেক্ষ। পক্ষান্তরে, SVM ট্রেনিং শেষে শুধুমাত্র সাপোর্ট ভেক্টরগুলোকে মেমোরিতে ধারণ করে। ফলে এর প্রেডিকশন স্পিড অত্যন্ত দ্রুত এবং মেমোরি সাশ্রয়ী।
- **Decision Tree ও Random Forest বনাম SVM:** বৃক্ষ-ভিত্তিক মডেলগুলো উপাত্ত ক্ষেত্রকে অক্ষ-বরাবর লম্বালম্বিভাবে (Axis-aligned splitting) খণ্ড বিখণ্ড করে। উপাত্তের বিন্যাস যদি বৃত্তাকার বা জটিল প্যাটার্নের হয়, তবে এই মডেলগুলো অতিরিক্ত ডালপালা গজিয়ে ওভারফিটিং (Overfitting) তৈরি করে। কিন্তু SVM তার RBF (Radial Basis Function) কার্নেল প্রয়োগ করে অতি সহজেই যেকোনো জটিল ও আঁকাবাঁকা বৃত্তাকার সীমানা নিখুঁতভাবে চিহ্নিত করতে পারে।
- **SLR/MLR বনাম SVM (SVR):** সাধারণ রৈখিক রিগ্রেশন মডেলগুলো আউটলায়ার (Outliers) বা অস্বাভাবিক মান দ্বারা তীব্রভাবে প্রভাবিত ও বিচ্যুত হয়। কিন্তু SVM রিগ্রেশন (SVR) তার মার্জিনাল বাউন্ডারির ভেতরের সুনির্দিষ্ট ক্রটিকে (epsilon) সম্পূর্ণ উপেক্ষা করে, যা মডেলটিকে আউটলায়ারের বিরুদ্ধে অত্যন্ত প্রতিরোধী (Robust) করে তোলে।

সাপোর্ট ভেক্টর মেশিন (SVM)-কে কি লজিস্টিক রিগ্রেশন (Logistic Regression)-এর একটি উন্নত সংস্করণ বা বর্ধিত রূপ (Enhanced Version) বলা চলে? উপযুক্ত গাণিতিক ও জ্যামিতিক যুক্তিসহ আলোচনা করো।

তাত্ত্বিক এবং গাণিতিক দৃষ্টিকোণ থেকে সাপোর্ট ভেক্টর মেশিন (SVM) এবং লজিস্টিক রিগ্রেশন (Logistic Regression) দুটি ভিন্ন নীতিমালার ওপর ভিত্তি করে প্রতিষ্ঠিত পৃথক ক্লাসিফিকেশন অ্যালগরিদম। লজিস্টিক রিগ্রেশন যেখানে সম্ভাব্যতা (Probability) এবং সর্বোচ্চ সম্ভাব্যতা অনুমানের (Maximum Likelihood Estimation) ওপর ভিত্তি করে কাজ করে, সেখানে SVM

জ্যামিতিক মার্জিন এবং অস্টিমাইজেশন তত্ত্বের ওপর ভিত্তি করে গড়ে উঠেছে। তবে, লজিস্টিক রিগ্রেশনের কিছু মৌলিক সীমাবদ্ধতা দূরীকরণ এবং ক্লাসিফিকেশন বাউন্ডারিকে জ্যামিতিকভাবে অধিক শক্তিশালী করার কারণে, লিনিয়ারলি সেপারেবল ও নন-লিনিয়ার ডেটা ক্লাসিফিকেশনের ক্ষেত্রে SVM-কে লজিস্টিক রিগ্রেশনের একটি "উন্নত বা বর্ধিত রূপ" (Enhanced Version) হিসেবে বিবেচনা করা যৌক্তিক।

নিচে প্রধান ৫টি গাঠনিক ও জ্যামিতিক রূপান্তরের মাধ্যমে এই উন্নয়ন প্রক্রিয়াটি ব্যাখ্যা করা হলো:

লস ফাংশনের উন্নয়ন (Loss Function Enhancement)

লজিস্টিক রিগ্রেশন এবং SVM-এর কার্যপদ্ধতির মূল পার্থক্য নিহিত রয়েছে এদের লস ফাংশনের (Loss Function) মধ্যে:

- লজিস্টিক রিগ্রেশন (Cross-Entropy Loss):** এটি লিনিয়ার কম্বিনেশন $z = wx + b$ কে সিগময়েড ফাংশনে রূপান্তর করে সম্ভাবনা হিসাব করে। এর লস ফাংশন (Log Loss) প্রতিটি ডেটা পয়েন্টের দূরত্বের ওপর ভিত্তি করে ক্রমাগত পরিবর্তিত হয়। ফলে এটি সঠিক ও ভুল—উভয় প্রকার ক্লাসিফাইড পয়েন্টের আত্মবিশ্বাসের মাত্রা নিয়ে কাজ করে এবং প্রতিটি বিন্দুর জন্য কম-বেশি লস গণনা করে।
- SVM (Hinge Loss):** SVM-এ লগ-লসকে প্রতিস্থাপন করা হয় হিঞ্জ লস (Hinge Loss) দ্বারা, যার গাণিতিক রূপ:

$$\text{Hinge Loss} = \max\{0, 1 - y_i(wx + b)\}$$

এই লস ফাংশনের অন্যতম বৈশিষ্ট্য হলো, যদি কোনো ডেটা পয়েন্ট সঠিকভাবে ক্লাসিফাইড হয় এবং মার্জিনের বাইরে থাকে ($y_i(wx + b) \geq 1$), তবে তার জন্য লসের মান সরাসরি **০ (Zero)** হয়ে যায়। এটি মডেলটিকে কেবল বাউন্ডারির নিকটবর্তী জটিল পয়েন্টগুলোর ওপর ফোকাস করতে বাধ্য করে।

মার্জিন ম্যাক্সিমাইজেশন এবং জ্যামিতিক বাউন্ডারি (Margin Maximization)

- লজিস্টিক রিগ্রেশন:** লজিস্টিক রিগ্রেশনের মূল লক্ষ্য হলো এমন একটি ডিসিশন বাউন্ডারি ($wx + b = 0$) খুঁজে বের করা যা ডেটাসেটকে আলাদা করতে পারে। কিন্তু জ্যামিতিকভাবে এই বাউন্ডারিটি ডেটা পয়েন্ট থেকে কতটা দূরে থাকবে, তার কোনো সুনির্দিষ্ট নিয়ন্ত্রণ বা লক্ষ্য এতে থাকে না। এর ফলে বাউন্ডারিটি কোনো নির্দিষ্ট ক্লাসের অতি নিকটে চলে যেতে পারে, যা নতুন টেস্ট ডেটার ক্ষেত্রে ভুল সিদ্ধান্তের (Overfitting) ঝুঁকি বাড়ায়।

- **SVM (Maximum Margin Classifier):** SVM এই ধারণাকে উন্নত করে ডিসিশন বাউন্ডারির দুই পাশে একটি 'নিরাপদ অঞ্চল' বা **মার্জিন (Margin)** তৈরি করে। এর উদ্দেশ্য হলো মার্জিনের দূরত্ব $M = \frac{2}{|w|}$ কে সর্বোচ্চ (Maximize) করা। এই ম্যাক্সিমাম মার্জিন বৈশিষ্ট্যের কারণে মডেলটির সাধারণীকরণ ক্ষমতা (Generalization Ability) বহুগুণ বৃদ্ধি পায়।

আউটলায়ার প্রতিরোধ ক্ষমতা (Robustness to Outliers)

- **লজিস্টিক রিগ্রেশন:** লজিস্টিক রিগ্রেশনের লস ফাংশন পুরো ডেটাসেটের সব বিন্দুর ওপর নির্ভরশীল। ফলে ডিসিশন বাউন্ডারি থেকে অনেক দূরে অবস্থিত কোনো আউটলায়ার (Outlier) বা ভুল ডেটার সামান্য পরিবর্তনেও সম্পূর্ণ ডিসিশন বাউন্ডারিটি স্থানচ্যুত বা প্রভাবিত হতে পারে।
- **SVM:** SVM-এর ডিসিশন বাউন্ডারি কেবলমাত্র মার্জিনের ওপর বা মার্জিনের ভেতরে অবস্থিত নির্দিষ্ট কিছু বিন্দুর ওপর নির্ভর করে নির্ধারিত হয়, যাদের সাপোর্ট ভেক্টর (Support Vectors) বলা হয়। মার্জিনের বাইরে অবস্থিত অন্য হাজারটি বিন্দু পরিবর্তন বা অপসারণ করলেও ডিসিশন বাউন্ডারির কোনো পরিবর্তন হয় না। এই কারণে SVM আউটলায়ারের বিরুদ্ধে অত্যন্ত শক্তিশালী (Robust to Outliers)।

কার্নেল ট্রিক এবং নন-লিনিয়ার ডেটা ব্যবস্থাপন (The Kernel Trick)

- **লজিস্টিক রিগ্রেশন:** জটিল এবং বাস্তব জীবনের নন-লিনিয়ার ডেটা ক্লাসিফাই করার জন্য লজিস্টিক রিগ্রেশনে ম্যানুয়ালি উচ্চ-মাত্রার পলিনোমিয়াল ফিচার তৈরি করতে হয়, যা অত্যন্ত জটিল এবং কম্পিউটেশনালি ব্যয়বহুল।
- **SVM:** SVM-এ **কার্নেল ট্রিক (Kernel Trick)** নামক একটি উন্নত গাণিতিক কৌশল ব্যবহার করা হয়। এর মাধ্যমে মূল ইনপুট স্পেসের (Input Space) নন-লিনিয়ার ডেটাকে কোনো রূপান্তর ছাড়াই উচ্চ মাত্রার ফিচার স্পেসে (Feature Space) রূপান্তর করা যায় এবং সেখানে একটি লিনিয়ার হাইপারপ্লেন দ্বারা ডেটা আলাদা করা সম্ভব হয়। Linear, Polynomial, এবং RBF (Gaussian) কার্নেল ব্যবহারের এই সুবিধা ক্লাসিফিকেশনের দক্ষতাকে লজিস্টিক রিগ্রেশনের তুলনায় বহুগুণ বাড়িয়ে দেয়।

সফট মার্জিন ও হাইপারপ্যারামিটার নিয়ন্ত্রণ (C)

লজিস্টিক রিগ্রেশনে ভুল ক্লাসিফিকেশনের ওপর জ্যামিতিক কঠোরতা নিয়ন্ত্রণের সরাসরি কোনো প্যারামিটার নেই। পক্ষান্তরে, SVM-এ হাইপারপ্যারামিটার C (Penalty Parameter) ব্যবহারের মাধ্যমে 'হার্ড মার্জিন' (যেখানে কোনো ভুল গ্রহণযোগ্য নয়) এবং 'সফট মার্জিন' (যেখানে কিছু ভুল

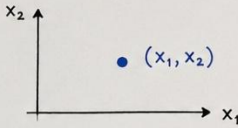
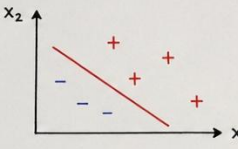
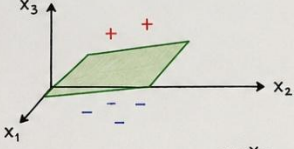
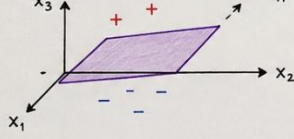
বা ওভারল্যাপিং অনুমোদন করা হয়)-এর মধ্যে একটি চমৎকার ভারসাম্য রক্ষা করা যায়। এটি মডেলের ওভারফিটিং এবং আন্ডারফিটিং সমস্যা খুব নিখুঁতভাবে নিয়ন্ত্রণ করতে পারে।

পরিশেষে বলা যায় যে, যদিও লজিস্টিক রিগ্রেশন এবং SVM-এর তাত্ত্বিক সূত্রপাত ভিন্ন, তবুও লজিস্টিক রিগ্রেশনের ডিসিশন বাউন্ডারিকে জ্যামিতিক মার্জিন দ্বারা সুসংহত করা, হিঞ্জ লসের মাধ্যমে অপ্ৰয়োজনীয় বিন্দুর প্রভাব মুক্ত করা, এবং কার্নেল ট্রিকের সাহায্যে জটিল নন-লিনিয়ার ডেটা প্রক্রিয়াকরণের ক্ষমতা যোগ করার মাধ্যমে SVM ক্লাসিফিকেশন প্রক্রিয়াকে এক নতুন উচ্চতায় নিয়ে গেছে। অতএব, ব্যবহারিক ও কার্যকারিতার দিক থেকে **SVM-কে লজিস্টিক রিগ্রেশনের একটি অত্যন্ত শক্তিশালী এবং উন্নত সংস্করণ (Enhanced Version) বলা সম্পূর্ণ যুক্তিযুক্ত।**

৫. কার্যপ্রণালী (Working Principle)

মেশিন লার্নিং এবং সাপোর্ট ভেক্টর মেশিন (SVM) বোঝার জন্য ডেটার জ্যামিতিক রূপ জানা অত্যন্ত জরুরি। নিচে মাত্রার (Dimension) ওপর ভিত্তি করে প্রধান ৫টি জ্যামিতিক উপাদান সংক্ষেপে আলোচনা করা হলো:

Geometric Concepts in SVM

1. <u>Point</u> A point represents a single data instance.		In n-dimensional space: $x = [x_1, x_2, \dots, x_n]^T$
2. <u>Line</u> A line in 2D space separates the two classes.		Linear Equation: $w_1 x_1 + w_2 x_2 + b = 0$ This is a line.
3. <u>Plane</u> A plane in 3D space separates the two classes.		Linear Equation: $w_1 x_1 + w_2 x_2 + w_3 x_3 + b = 0$ This is a plane.
4. <u>Hyperplane</u> A hyperplane in n-dimensional space separates the two classes.		Linear Equation: $w^T x + b = 0$ This is a hyperplane.
5. <u>Linear Equation (General)</u> The general form of linear equation in n-dimensional space.	$w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b = 0$ or $w^T x + b = 0$	This equation represents a hyperplane.

১. বিন্দু (Point)

- বহুমাত্রিক স্থানে একটি বিন্দু মূলত একটি একক ডেটা ইনস্ট্যান্স বা রেকর্ডকে নির্দেশ করে।
- গাণিতিক রূপ: n -ডাইমেনশনাল স্পেসে একে একটি ভেক্টর হিসেবে প্রকাশ করা হয়:

$$x = [x_1, x_2, \dots, x_n]^T$$

২. রেখা (Line)

- দ্বিমাত্রিক (2D) স্থানে দুটি ভিন্ন ক্লাস বা ডেটা দলকে আলাদা করার জন্য ডিসিশন বাউন্ডারি হিসেবে একটি সরলরেখা ব্যবহার করা হয়।
- লিনিয়ার সমীকরণ (Linear Equation):

$$w_1 x_1 + w_2 x_2 + b = 0$$

৩. সমতল (Plane)

- ত্রিমাত্রিক (3D) স্থানে দুটি ক্লাসকে পৃথক করার বাউন্ডারি হিসেবে একটি চ্যাপ্টা সমতল পৃষ্ঠ বা প্লেন ব্যবহৃত হয়।
- **লিনিয়ার সমীকরণ (Linear Equation):**

$$w_1x_1 + w_2x_2 + w_3x_3 + b = 0$$

৪. হাইপারপ্লেন (Hyperplane)

- যখন ডেটার মাত্রা n -ডাইমেনশনে রূপ নেয়, তখন ক্লাস দুটিকে আলাদা করার বাউন্ডারিকে **হাইপারপ্লেন** বলা হয়।
- **লিনিয়ার সমীকরণ (Linear Equation):**

$$w^T x + b = 0$$

৫. সাধারণ লিনিয়ার সমীকরণ (Linear Equation - General)

- এটি হলো n -ডাইমেনশনাল স্পেসে যেকোনো ডিসিশন বাউন্ডারি প্রকাশের একটি সাধারণ গাণিতিক রূপ। এই সমীকরণটি মূলত একটি **হাইপারপ্লেন**-কে নির্দেশ করে।
- **সাধারণ রূপ:**

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = 0 \quad \text{অথবা} \quad w^T x + b = 0$$

ম্যাক্সিমাম মার্জিন ক্লাসিফায়ার (Maximum Margin Classifier)

SVM-কে একটি ম্যাক্সিমাম মার্জিন ক্লাসিফায়ার বলা হয়। এটি কেবল দুই ক্লাসের ডেটাকে আলাদাই করে না, বরং উভয় ক্লাসের নিকটবর্তী ডেটা পয়েন্ট থেকে সমান দূরত্ব বজায় রেখে সম্ভাব্য সবচেয়ে চওড়া রাস্তা বা সর্বোচ্চ মার্জিন (Maximum Margin) তৈরি করে সিদ্ধান্ত রেখা নির্ধারণ করে।

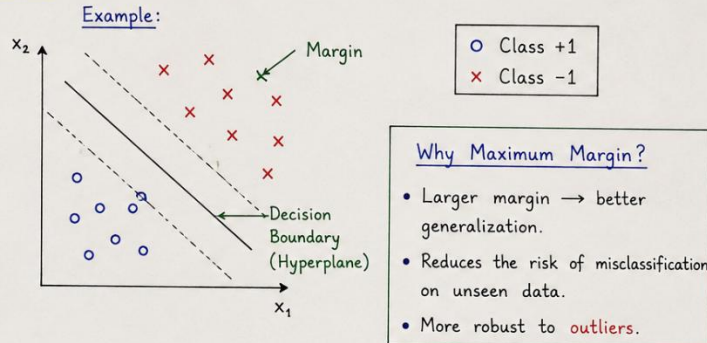
ম্যাক্সিমাম মার্জিন কেন গুরুত্বপূর্ণ (Why Maximum Margin is important)

- **Better Generalization:** মার্জিনের মান যত বড় হবে ($M = \frac{2}{|w|}$), মডেলটির নতুন ও অজানা ডেটার ওপর সঠিক প্রেডিকশন দেওয়ার ক্ষমতা (Generalization Ability) তত বেশি বৃদ্ধি পায়।

Overfitting রোধ: মার্জিন ছোট হলে ডিসিশন বাউন্ডারি কোনো একটি ক্লাসের খুব কাছাকাছি চলে যায়, যা ওভারফিটিংয়ের ঝুঁকি বাড়ায়। ম্যাক্সিমাম মার্জিন মডেলকে এই ঝুঁকি থেকে মুক্ত রাখে।

3.1 What is Support Vector Machine (SVM)?

- SVM is a supervised learning algorithm used for classification (mainly binary classification).
- The main idea of SVM is to find the optimal decision boundary (hyperplane) that separates the classes with the maximum margin.
- It is a Maximum Margin Classifier.
- SVM works well even when there are outliers in the data.



3.2 Components of SVM

① Decision Boundary / Hyperplane

The boundary that separates the classes.

② Margin

The distance between the hyperplane and the nearest data points from both classes.

③ Support Vectors

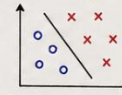
The data points that lie closest to the decision boundary. These points define the position of the boundary.

④ Generalized Model

SVM finds the best boundary that works well on new, unseen data.

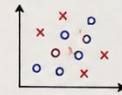
3.3 Types of Dataset

① Linearly Separable Data



Classes can be separated perfectly by a straight line / hyperplane.

② Non-Linearly Separable Data



Classes cannot be separated by a straight line / hyperplane. (Need Kernel Trick)

আউটলায়ারের বিরুদ্ধে প্রতিরোধ ক্ষমতা (Robustness against Outliers)

SVM আউটলায়ারের বিরুদ্ধে অত্যন্ত শক্তিশালী বা প্রতিরোধী (Robust to Outliers)। এর ডিসিশন বাউন্ডারি ডেটাসেটের সমস্ত বিন্দুর ওপর নির্ভর করে না; বরং এটি কেবল মার্জিনের প্রান্তে থাকা নির্দিষ্ট কিছু পয়েন্টের ওপর ভিত্তি করে তৈরি হয়। ফলে বাউন্ডারি থেকে দূরে থাকা কোনো আউটলায়ারের সামান্য পরিবর্তনে মূল সিদ্ধান্ত রেখার কোনো পরিবর্তন হয় listen না।

SVM-এর উপাদানসমূহ (Components of SVM)

ডিসিশন বাউন্ডারি (Decision Boundary)

দুই বা বহুমাত্রিক স্থানে বিভিন্ন ক্লাসের ডেটাকে পৃথক করার জন্য যে জ্যামিতিক রেখা, সমতল বা হাইপারপ্লেন ব্যবহার করা হয়, তাকে ডিসিশন বাউন্ডারি বলে। গাণিতিকভাবে একে $\pi = wx + b = 0$ সমীকরণ দ্বারা প্রকাশ করা হয়।

মার্জিন (Margin)

ডিসিশন বাউন্ডারি বা কেন্দ্রীয় হাইপারপ্লেন থেকে উভয় পাশের ক্লাসের নিকটবর্তী ডেটা পয়েন্টের মধ্যবর্তী মোট দূরত্বকে মার্জিন বলে ($M = d_1 + d_2$)। এটি মূলত একটি সুরক্ষিত অঞ্চল (Safe Zone) হিসেবে কাজ করে।

সাপোর্ট ভেক্টরস (Support Vectors)

ডেটাসেটের যে নির্দিষ্ট বিন্দু বা ডেটা ইনস্ট্যান্সগুলো মার্জিনের সীমানার ওপর অবস্থান করে এবং ডিসিশন বাউন্ডারির অবস্থান ও আকৃতি নির্ধারণে সরাসরি ভূমিকা রাখে, সেগুলোকে সাপোর্ট ভেক্টর বলে। এই ভেক্টরগুলো সরিয়ে নিলে পুরো মডেলের বাউন্ডারি পরিবর্তন হয়ে যায়।

জেনারেলাইজড মডেল (Generalized Model)

সাপোর্ট ভেক্টর ও ম্যাক্সিমাম মার্জিন ব্যবহারের মাধ্যমে তৈরি ডিসিশন বাউন্ডারিটি একটি জেনারেলাইজড বা সাধারণীকরণ মডেল তৈরি করে। এটি ট্রেনিং ডেটার পাশাপাশি সম্পূর্ণ নতুন বা টেস্ট ডেটার ওপরেও সমানভাবে নির্ভুল ফলাফল দিতে সক্ষম।

ডেটাসেটের প্রকারভেদ (Types of Dataset)

লিনিয়ারলি সেপারেবল ডেটা (Linearly Separable Data)

যেসব ডেটাসেটের ক্লাসগুলোকে একটিমাত্র সোজা সরলরেখা (2D-তে), সমতল (3D-তে) বা লিনিয়ার হাইপারপ্লেন দ্বারা সম্পূর্ণ আলাদা করা সম্ভব, সেগুলোকে লিনিয়ারলি সেপারেবল ডেটা বলে। এই ক্ষেত্রে কোনো ভুল বা ওভারল্যাপিং ছাড়া "Hard Margin" ব্যবহার করে নিখুঁত ক্লাসিফিকেশন করা যায়।

নন-লিনিয়ারলি সেপারেবল ডেটা (Non-Linearly Separable Data)

বাস্তব জীবনের যেসব জটিল ডেটাসেটকে কোনো সোজা লাইন বা সমতল হাইপারপ্লেন দিয়ে আলাদা করা যায় না, সেগুলোকে নন-লিনিয়ার ডেটা বলে। এই ধরনের ডেটা ক্লাসিফাই করার জন্য SVM-এ "Kernel Trick" ব্যবহার করে ডেটাকে উচ্চ মাত্রার ফিচার স্পেসে রূপান্তর করে আলাদা করা হয়।

৬. গাণিতিক ভিত্তি (Mathematical Backend)

৭. সূত্র (Formula)

হাইপারপ্লেনের প্রাথমিক সমীকরণসমূহ

ধরা যাক, একটি বহুমাত্রিক স্পেসে আমাদের কাছে দুই ধরনের ক্লাস রয়েছে—পজিটিভ (+1) এবং নেগেটিভ (-1)। এই দুই ক্লাসের মধ্যবর্তী স্থানে আমরা তিনটি সমান্তরাল হাইপারপ্লেন বা ডিসিশন বাউন্ডারি বিবেচনা করি:

1. কেন্দ্রীয় সিদ্ধান্ত রেখা (Optimal Hyperplane):

$$w^T x + b = 0$$

2. পজিটিভ মার্জিনাল হাইপারপ্লেন (π_+):

$$w^T x + b = +1$$

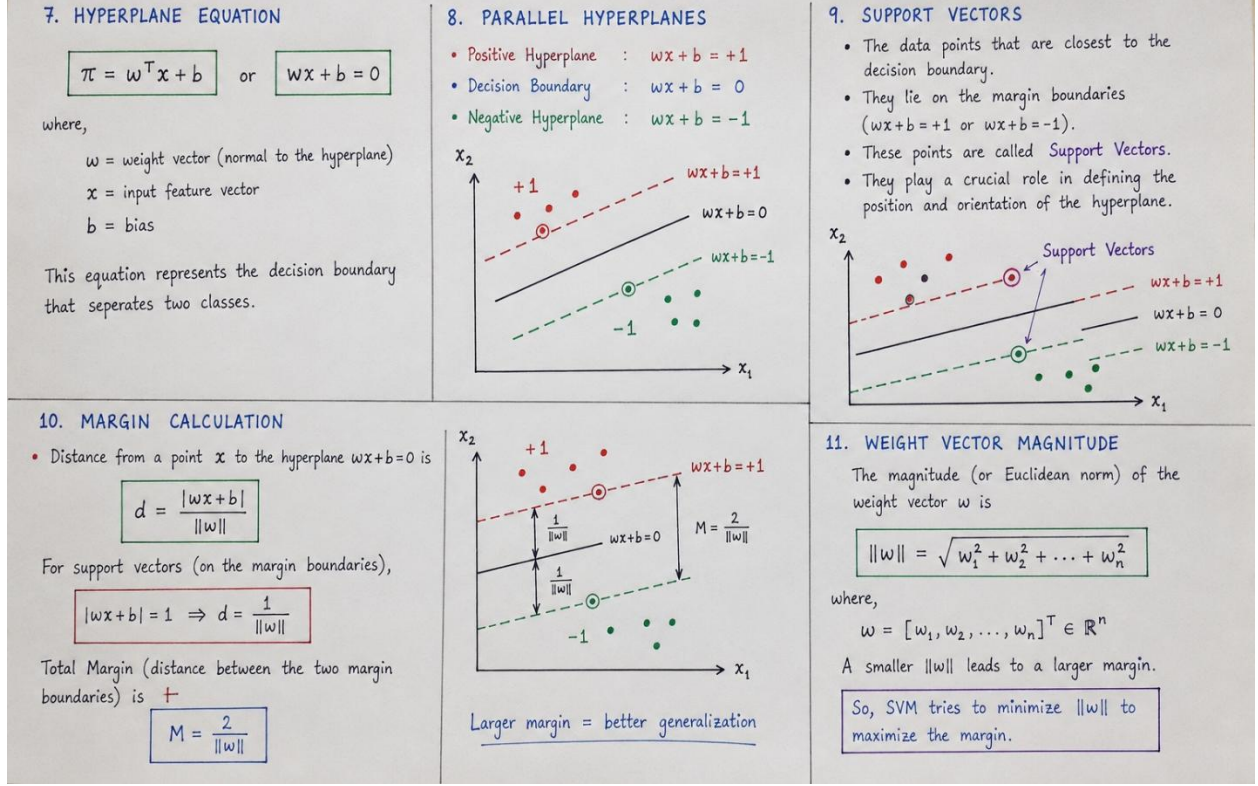
3. নেগেটিভ মার্জিনাল হাইপারপ্লেন (π_-):

$$w^T x + b = -1$$

এখানে, w হলো লম্ব ভেক্টর বা ওয়েট ভেক্টর (Weight Vector), x হলো ইনপুট ফিচার ভেক্টর এবং b হলো বায়াস (Bias)।

এখানে, $w = [w_1, w_2, w_3, \dots, w_n]$ হলো ওয়েট ম্যাট্রিক্স বা ভেক্টর এবং এর মান বা ম্যাগনিটিউড বের করার সূত্রটি:

$$|w| = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$$



৮. সূত্রের ডেরিভেশন (Formula Derivation)

ডিসিশন বাউন্ডারি থেকে দুই পাশের মার্জিনাল লাইনের দূরত্ব পরস্পর সমান, অর্থাৎ:

$$d_1 = d_2$$

এখানে, কেন্দ্রীয় লাইন থেকে যেকোনো একপাশের দূরত্বের জ্যামিতিক সূত্রটি হলো:

$$\text{distance} = \frac{w x + b}{\|w\|}$$

এখন, পজিটিভ মার্জিনাল লাইনের সীমানার ওপর থাকা বিন্দুর জন্য স্কের $w x + b = +1$ । সুতরাং, কেন্দ্রীয় লাইন থেকে পজিটিভ লাইনের দূরত্ব:

$$d_1 = \frac{1}{\|w\|}$$

যেহেতু দুই পাশের দূরত্ব সমান ($d_1 = d_2$), তাই অপর পাশের দূরত্বটিও হবে:

$$d_2 = \frac{1}{\|w\|}$$

নোটের চিত্র ও সমীকরণ অনুযায়ী, মোট মার্জিন (M) হলো এই দুই দূরত্বের সমষ্টি:

$$\text{Margin}(M) = d_1 + d_2$$

$$M = 2 \times d_1$$

এখন d_1 -এর মান বসিয়ে আমরা পাই:

$$M = 2 \times \frac{1}{\|w\|} = \frac{2}{\|w\|}$$

- " $\|w\|$ যত ছোট হবে, M তত বড় হবে"
- "Target Margin maximize করা"

গাণিতিক নিয়ম অনুযায়ী, ভগ্নাংশের হর ($\|w\|$) যত ছোট বা ডিক্রিজ করা যাবে, সম্পূর্ণ ভগ্নাংশের মান অর্থাৎ মার্জিন (M) তত বড় বা ম্যাক্সিমাইজ হবে। যেহেতু SVM-এর প্রধান লক্ষ্যই হলো দুই ক্লাসের মধ্যকার দূরত্ব বা মার্জিন সর্বোচ্চ করা (Target Margin Maximize), তাই মডেলটি সর্বদা $\|w\|$ -এর মানকে সর্বনিম্ন (Minimize) করার চেষ্টা করে।

মার্জিন সর্বোচ্চকরণ (Maximize Margin)

সাপোর্ট ভেক্টর মেশিনের প্রধান গাণিতিক লক্ষ্য হলো পজিটিভ এবং নেগেটিভ ক্লাসের মধ্যবর্তী দূরত্ব বা মার্জিন (M) সর্বোচ্চ করা। জ্যামিতিক প্রতিপাদন থেকে প্রাপ্ত মার্জিনের গাণিতিক রূপটি হলো:

$$M = \frac{2}{\|w\|}$$

সমতুল্য সর্বনিম্নকরণ সমস্যা (Equivalent Minimization Problem)

বীজগণিতীয় নিয়ম অনুযায়ী, কোনো ভগ্নাংশের মানকে সর্বোচ্চ (Maximize) করতে হলে তার হর-এর মানকে সর্বনিম্ন করতে হয়। অর্থাৎ, মার্জিন $M = \frac{2}{\|w\|}$ কে সর্বোচ্চ করার অর্থ হলো ওয়েট ভেক্টরের ম্যাগনিটিউড $\|w\|$ কে সর্বনিম্ন (Minimize) করা।

ক্যালকুলাস এবং ডিফারেনশিয়েশনের সুবিধার্থে (বিশেষ করে বর্গমূলের জটিলতা এড়াতে) এই সর্বোচ্চকরণের সমস্যাটিকে একটি সমতুল্য সর্বনিম্নকরণ সমস্যা হিসেবে নিচে রূপান্তর করা হয়:

$$\min \frac{1}{2} \|w\|^2$$

৯. প্রতিটি চিহ্নের অর্থ (Meaning of Every Symbol)

Symbol	Meaning (অর্থ)
w	Weight Vector — হাইপারপ্লেনের দিক নির্ধারণ করে
b	Bias — হাইপারপ্লেনের অবস্থান নির্ধারণ করে
x	ইনপুট Feature Vector
y	প্রকৃত ক্লাস লেবেল (+1 বা -1)
$\ w\ $	Weight Vector-এর Magnitude (দৈর্ঘ্য)
Margin ($2/\ w\ $)	দুই ক্লাসের মধ্যবর্তী নিরাপদ অঞ্চলের প্রস্থ

Support Vectors	Margin-এর সীমানায় থাকা নিকটতম ডেটা পয়েন্টসমূহ
C	Soft Margin-এ ভুল ক্লাসিফিকেশনের জন্য পেনাল্টি নিয়ন্ত্রণকারী Hyperparameter
$\xi(x_i)$	Slack Variable — একটি পয়েন্ট কতটা Margin লঙ্ঘন করেছে তার পরিমাপ
$\phi(x)$	Kernel Feature Mapping Function

১০. সূত্রের ধাপে-ধাপে ব্যাখ্যা (Step-by-Step Formula Explanation)

হিঞ্জ লস (Hinge Loss)

বাস্তব জীবনের ডেটাসেটে সম্পূর্ণ নিখুঁতভাবে ডেটা আলাদা করা সবসময় সম্ভব হয় না। মডেলের এই ত্রুটি বা ভুলের পরিমাণ গাণিতিকভাবে গণনা করার জন্য Hinge Loss ফাংশন ব্যবহার করা হয়। এর গাণিতিক সূত্রটি নিম্নরূপ:

$$\text{Hinge Loss} = \max\{0, 1 - y_i(wx + b)\}$$

এখানে, y_i হলো ডেটার প্রকৃত ক্লাস বা লেবেল (+1 অথবা -1) এবং $(wx + b)$ হলো মডেল দ্বারা প্রাপ্ত প্রেডিকশন স্কোর।

বিভিন্ন ক্ষেত্রে হিঞ্জ লসের মান (Different Hinge Loss Cases)

ডিসিশন বাউন্ডারির সাপেক্ষে কোনো ডেটা পয়েন্ট কোথায় অবস্থান করেছে, তার ওপর ভিত্তি করে হিঞ্জ লসের মানকে ৩টি সুনির্দিষ্ট ক্ষেত্রে বিভক্ত করা যায়:

১১. সংখ্যাগত উদাহরণ (Numerical Example)

ক. সঠিক ক্লাসিফিকেশন (Correct Classification)

যখন কোনো ডেটা পয়েন্ট সম্পূর্ণ সঠিকভাবে মার্জিনের সুরক্ষিত অঞ্চলের বাইরে ক্লাসিফাই হয় (অর্থাৎ, $y_i(wx + b) \geq 1$), তখন সমীকরণের ভেতরের মানটি ঋণাত্মক বা শূন্য হয়ে যায়।

- গাণিতিক উদাহরণ: যদি $y_i(wx + b) = 2$ হয়, তবে $1 - 2 = -1$ ।
- লস গণনা: $\max(0, -1) = 0$ ।
- সিদ্ধান্ত: সঠিকভাবে ক্লাসিফাইড এবং মার্জিনের বাইরের বিন্দুর জন্য মডেলে কোনো পেনাল্টি বা লস যুক্ত হয় না।

খ. সঠিক কিন্তু মার্জিনের ভেতরে (Correct but Inside Margin)

যখন কোনো পয়েন্ট সঠিকভাবে সঠিক ক্লাসে অবস্থান করে, কিন্তু সেটি মার্জিনের সীমানার ভেতরে ঢুকে পড়ে (অর্থাৎ, $0 \leq y_i(wx + b) < 1$), তখন মডেলের জন্য পেনাল্টি তৈরি হয়।

- গাণিতিক উদাহরণ: যদি $y_i(wx + b) = 0.5$ হয়, তবে $1 - 0.5 = 0.5$ ।
- লস গণনা: $\max(0, 0.5) = 0.5$ ।

- **সিদ্ধান্ত:** যদিও পয়েন্টটি সঠিক ক্লাসেই আছে, তবুও মার্জিন লঙ্ঘন করার অপরাধে মডেলটিকে কিছুটা লস বা পেনাল্টি গুনতে হয়।

গ. ভুল ক্লাসিফিকেশন (Misclassified)

যখন কোনো ডেটা পয়েন্ট সম্পূর্ণ ভুল ক্লাসে চলে যায় এবং ডিসিশন বাউন্ডারি পার হয়ে অন্য পাশে অবস্থান করে (অর্থাৎ, $y_i(wx + b) < 0$), তখন লসের মান ধনাত্মকভাবে বৃদ্ধি পায়।

- **গাণিতিক উদাহরণ:** যদি $y_i(wx + b) = -0.5$ হয়, তবে $1 - (-0.5) = 1 + 0.5 = 1.5$ ।
- **লস গণনা:** $\max(0, 1.5) = 1.5$ ।
- **সিদ্ধান্ত:** ভুল ক্লাসিফিকেশনের জন্য মডেলকে সর্বোচ্চ পেনাল্টি দেওয়া হয়। ডিসিশন বাউন্ডারি থেকে পয়েন্টটি যত দূরে ভুল দিকে যাবে, লসের পরিমাণও তত রৈখিকভাবে বাড়তে থাকবে।

চূড়ান্ত কস্ট ফাংশন (Final Cost Function)

SVM-এর মূল উদ্দেশ্য (মার্জিন সর্বোচ্চ করা) এবং হিঞ্জ লস (ভুলের পেনাল্টি সর্বনিম্ন করা)—এই দুটি ভিন্ন লক্ষ্যকে একটি একক সমীকরণে যুক্ত করে চূড়ান্ত কস্ট ফাংশন J গঠন করা হয়:

$$J = \frac{1}{2} \|w\|^2 + \sum \text{Hinge Loss}$$

এই সমীকরণের প্রথম অংশ ($\frac{1}{2} \|w\|^2$) মার্জিনকে বড় করার চেষ্টা করে (যা রেগুলারাইজেশন হিসেবে কাজ করে) এবং দ্বিতীয় অংশ ($\sum \text{Hinge Loss}$) মডেলের ক্লাসিফিকেশন ভুলগুলোকে দমন করার চেষ্টা করে। একটি আদর্শ মডেল এই দুটি বিষয়ের মধ্যে একটি নিখুঁত গাণিতিক ভারসাম্য বজায় রাখে।

কোয়াদ্রেটিক প্রোগ্রামিং (Quadratic Programming - QP)

অপটিমাইজেশন টেকনিক (Optimization Technique)

চূড়ান্ত কস্ট ফাংশন J -এর মান সর্বনিম্ন করার জন্য এবং নির্দিষ্ট শর্তসাপেক্ষে (Constraints) অপটিমাল প্যারামিটার খোঁজার জন্য যে গাণিতিক ও কম্পিউটেশনাল পদ্ধতি ব্যবহার করা হয়, তাকে কোয়াদ্রেটিক প্রোগ্রামিং (QP) বলা হয়।

উত্তল অপটিমাইজেশন (Convex Optimization)

যেহেতু SVM-এর এই কস্ট ফাংশনটি একটি **উত্তল ফাংশন** বা **Convex Function**, সেহেতু এর জ্যামিতিক গ্রাফে কেবল একটিমাত্র সর্বনিম্ন বিন্দু বা **Global Minima** থাকে। কোয়াদ্রেটিক প্রোগ্রামিং টেকনিক ব্যবহারের মাধ্যমে কোনো লোকাল মিনিমামে (Local Minima) না আটকে সরাসরি বৈশ্বিক বা নিখুঁত অপটিমাল সলিউশন খুঁজে পাওয়া নিশ্চিত করা সম্ভব হয়।

হার্ড মার্জিন ও সফট মার্জিন (Hard Margin & Soft Margin)

বাস্তব জীবনের ডেটাসেটগুলোর গঠন এবং জটিলতার ওপর ভিত্তি করে সাপোর্ট ভেক্টর মেশিনের ডিসিশন বাউন্ডারি নির্ধারণের পদ্ধতিকে প্রধানত দুটি ভাগে ভাগ করা যায়। নিচে এই দুটি মার্জিন পদ্ধতি এবং তাদের নিয়ন্ত্রণকারী প্যারামিটার বিস্তারিতভাবে আলোচনা করা হলো:

হার্ড মার্জিন এসভিএম (Hard Margin SVM)

কোনো ভুল ক্লাসিফিকেশন নয় (No Misclassification)

হার্ড মার্জিন এসভিএম-এর প্রধান বৈশিষ্ট্য হলো এটি ডিসিশন বাউন্ডারি বা মার্জিনের ভেতরে কোনো ধরনের ভুল ক্লাসিফিকেশন বা ডেটা পয়েন্টের অনুপ্রবেশ অনুমোদন করে না। অর্থাৎ, প্রতিটি ডেটা পয়েন্টকে অবশ্যই মার্জিনের সঠিক পাশে এবং বাইরে অবস্থান করতে হবে।

নিখুঁতভাবে পৃথকীকরণযোগ্য ডেটা (Perfectly Separable Data)

এই পদ্ধতিটি কেবল তখনই সফলভাবে কাজ করে, যখন ডেটাসেটটি লিনিয়ারলি নিখুঁতভাবে পৃথকীকরণযোগ্য (Perfectly Separable Data) হয়। যদি ডেটার মধ্যে কোনো নয়েজ (Noise) বা ওভারল্যাপিং থাকে, তবে হার্ড মার্জিন মডেল কোনো গাণিতিক সমাধানে পৌঁছাতে পারে না এবং ব্যর্থ হয়।

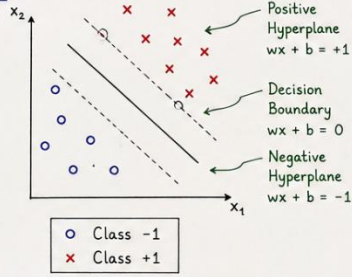
সফট মার্জিন এসভিএম (Soft Margin SVM)

ভুল ক্লাসিফিকেশন অনুমোদন (Allows Errors)

বাস্তবসম্মত সমাধানের জন্য সফট মার্জিন এসভিএম ডিসিশন বাউন্ডারি নির্ধারণের ক্ষেত্রে কিছুটা শিথিলতা প্রদর্শন করে। এই পদ্ধতিতে মডেলটি মার্জিনের ভেতরে বা ভুল পাশে কিছু সীমিত সংখ্যক ডেটা পয়েন্টের উপস্থিতি বা ত্রুটি অনুমোদন (Allows Errors) করে।

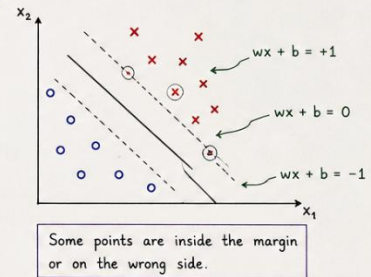
5.1 Hard Margin SVM

- Used when data is linearly separable.
- No misclassification allowed.
- Finds the maximum margin hyperplane that separates the two classes perfectly.



5.2 Soft Margin SVM

- Used when data is not perfectly linearly separable.
- Allows some misclassifications.
- More flexible and better generalization to real-world data.



5.3 Penalty Parameter (C)

In soft margin SVM, we use a penalty parameter C to control the trade-off between margin size and classification error.

Soft Margin SVM Objective:

$$\text{Minimize } J = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

- $\frac{1}{2} \|w\|^2 \rightarrow$ Maximizes the margin
- $C \sum_{i=1}^n \xi_i \rightarrow$ Penalty for misclassification
- $\xi_i \rightarrow$ Slack variable ($\xi_i \geq 0$)

5.4 Effect of C (Penalty Parameter)

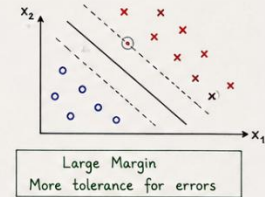
Large C ($C \uparrow$)

- High penalty on misclassification
- Tries to classify all points correctly
- Margin becomes smaller
- Behaves like Hard Margin



Small C ($C \downarrow$)

- Low penalty on misclassification
- Allows more errors
- Margin becomes larger
- Behaves like Soft Margin



Summary

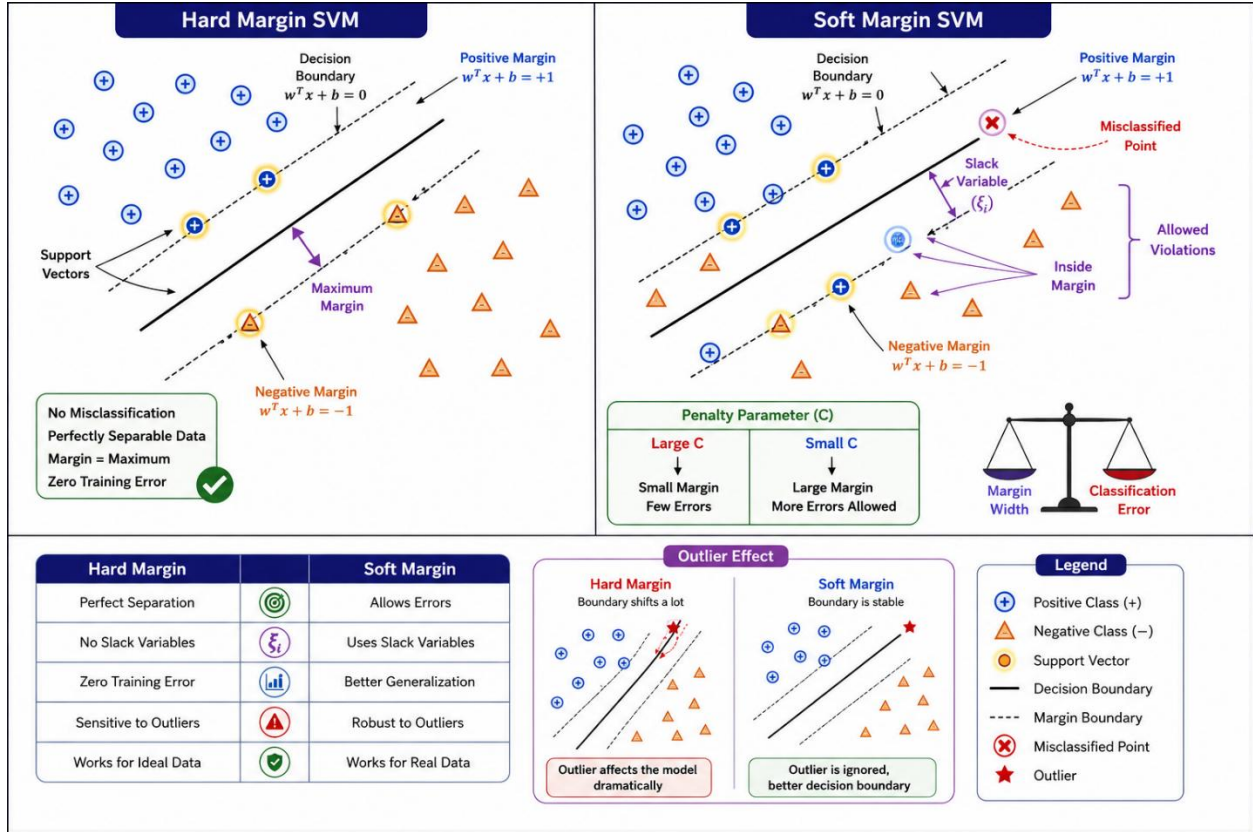
- Hard Margin $\rightarrow$ No error, smaller flexibility.
- Soft Margin $\rightarrow$ Allows error, better generalization.
- C controls the balance between margin size and training error.

Key Points

- C is a very important hyperparameter in SVM.
- Proper choice of C improves the performance of the model.
- Use cross-validation to select the best C .

উন্নত সাধারণীকরণ ক্ষমতা (Better Generalization)

কিছু ভুল বা ওভারফিটিং মেনে নেওয়ার কারণে সফট মার্জিন এসভিএম-এর ডিসিশন বাউন্ডারিটি খুব বেশি জটিল বা ওভারফিটেড হয় না। ফলে, এটি নতুন এবং অজানা টেস্ট ডেটার ওপর অনেক উন্নত সাধারণীকরণ ক্ষমতা (Better Generalization) বা নির্ভুল ফলাফল প্রদর্শন করতে পারে।



বাস্তব জীবনের ডেটা (Real World Data)

বাস্তব জীবনের প্রায় প্রতিটি ডেটাসেটই নয়াজ, আউটলায়ার এবং জটিল ওভারল্যাপিংয়ে পূর্ণ থাকে। এই ধরনের বাস্তবসম্মত ডেটা (Real World Data) অত্যন্ত সফলভাবে পরিচালনা করার জন্য সফট মার্জিন এসভিএম একটি আদর্শ ও অপরিহার্য পদ্ধতি।

পেনাল্টি প্যারামিটার (Penalty Parameter - C)

সফট মার্জিন এসভিএম-এ মডেলটি কতটা ভুল ক্লাসিফিকেশন মেনে নেবে, তা একটি নির্দিষ্ট হাইপারপ্যারামিটার দ্বারা নিয়ন্ত্রিত হয়, যাকে পেনাল্টি প্যারামিটার বা C বোলা হয়।

বড় সি-এর প্রভাব (Large C)

- কম ভুল ক্লাসিফিকেশন (Less Misclassification):** C -এর মান অনেক বড় হলে মডেলটি ভুলের জন্য উচ্চ পেনাল্টি বা শাস্তি নির্ধারণ করে। ফলে মডেলটি যেকোনো মূল্যে ভুল ক্লাসিফিকেশন এড়ানোর চেষ্টা করে এবং হার্ড মার্জিনের মতো আচরণ করে।

- **ছোট মার্জিন (Small Margin):** প্রতিটি বিন্দুকে নিখুঁতভাবে ক্লাসিফাই করার প্রয়াসে ডিসিশন বাউন্ডারির দুই পাশের মার্জিনটি অত্যন্ত সংকুচিত বা ছোট (Small Margin) হয়ে যায়।

ছোট সি-এর প্রভাব (Small C)

- **অধিক সহনশীলতা (More Tolerance):** C -এর মান ছোট হলে ভুল ক্লাসিফিকেশনের ওপর পেনাল্টির কঠোরতা অনেক কমে যায়। এর ফলে মডেলটি মার্জিনের ভেতরে বা ভুল পাশে ডেটা পয়েন্টের উপস্থিতির প্রতি অধিক সহনশীলতা (More Tolerance) প্রদর্শন করে।

বড় মার্জিন (Large Margin): ব্যক্তিগত বা বিচ্ছিন্ন কিছু ভুলকে উপেক্ষা করার কারণে মডেলটি দুই ক্লাসের মাঝে একটি সুপারিসর এবং বড় মার্জিন (Large Margin) তৈরি করতে সক্ষম হয়, যা মডেলের সাধারণীকরণ ক্ষমতাকে আরও মজবুত করে।

১২. ভিজ্যুয়ালাইজেশন (Visualization)

উপরে দেখানো Input Space $\rightarrow$ Feature Space ডায়াগ্রামটি লক্ষ্য করুন — এটি Kernel Trick-এর সবচেয়ে গুরুত্বপূর্ণ ভিজ্যুয়াল ধারণা। কল্পনা করুন 2D স্পেসে দুটি ক্লাস একটি বৃত্তাকার প্যাটার্নে সাজানো (একটি ক্লাস কেন্দ্রে, আরেকটি চারপাশে) — কোনো সরলরেখা দিয়ে এদের আলাদা করা অসম্ভব। কিন্তু এই একই ডেটাকে 3D-তে ম্যাপ করলে (যেমন উচ্চতা = দূরত্ব থেকে কেন্দ্র), হঠাৎ একটি সমতল (Plane) দিয়েই দুই ক্লাস সহজে আলাদা হয়ে যায়। এভাবেই RBF Kernel নন-লিনিয়ার ডেটাকে উচ্চ-মাত্রিক স্পেসে নিয়ে গিয়ে Linear ভাবে বিভাজনযোগ্য করে তোলে।

কার্নেল ট্রিক (Kernel Trick)

কার্নেলের প্রয়োজনীয়তা (Need for Kernel)

নন-লিনিয়ার ডেটা (Non-linear Data)

বাস্তব ক্ষেত্রের অধিকাংশ ডেটাসেটই সরলরৈখিকভাবে বিন্যস্ত থাকে না। যখন বিভিন্ন ক্লাসের ডেটা পয়েন্টগুলো একে অপরের সাথে এমনভাবে মিশ্রিত বা আবর্তিত অবস্থায় থাকে যে তাদেরকে কোনো সোজা সরলরেখা বা সমতল হাইপারপ্লেন দ্বারা আলাদা করা অসম্ভব হয়ে পড়ে, তখন তাকে নন-লিনিয়ার ডেটা বলা হয়। এই ধরনের ডেটা ক্লাসিফাই করার জন্য সাধারণ লিনিয়ার এসভিএম (Linear SVM) কার্যকর হয় না।

উচ্চ-মাত্রার ম্যাপিং (High-dimensional Mapping)

নন-লিনিয়ার ডেটার এই জটিলতা দূর করার একটি জ্যামিতিক উপায় হলো ডেটা পয়েন্টগুলোকে বর্তমান মাত্রা থেকে আরও উচ্চ মাত্রার (Higher Dimension) কোনো স্পেসে স্থানান্তরিত বা ম্যাপিং করা। কম মাত্রার স্পেসে যে ডেটা লিনিয়ারলি সেপারেবল নয়, উচ্চ মাত্রার স্পেসে ম্যাপিং করার পর সেখানে একটি সরলরৈখিক হাইপারপ্লেন দ্বারা তাদেরকে খুব সহজেই আলাদা করা সম্ভব হয়।

ফিচার ম্যাপিং (Feature Mapping)

ফিচার ম্যাপিং হলো নিম্ন-মাত্রার ইনপুট স্পেস থেকে উচ্চ-মাত্রার ফিচার স্পেসে ডেটা রূপান্তরের একটি গাণিতিক প্রক্রিয়া।

$$\begin{array}{ccc} [\text{ইনপুট স্পেস}] & \text{-----} (\text{ফিচার ম্যাপিং: } \phi(x)) \text{-----} & [\text{ফিচার স্পেস}] \\ (\text{নন-লিনিয়ার ডেটা}) & & (\text{লিনিয়ারলি সেপারেবল ডেটা}) \end{array}$$

ইনপুট স্পেস (Input Space)

এটি হলো ডেটার মূল বা প্রাথমিক ডাইমেনশন, যেখানে ফিচারগুলো সরাসরি ইনপুট হিসেবে অবস্থান করে। এই স্পেসে ডেটা পয়েন্টগুলো সাধারণত নন-লিনিয়ার বা গোলকধাঁধার মতো বিন্যাসে থাকে।

ফিচার স্পেস (Feature Space)

ইনপুট স্পেসের ডেটাকে গাণিতিক রূপান্তরের মাধ্যমে যখন একটি নতুন এবং উচ্চ-মাত্রার ডাইমেনশনে নিয়ে যাওয়া হয়, তখন সেই নতুন স্থানটিকে ফিচার স্পেস বলা হয়। এই স্পেসে জটিল ডেটাও লিনিয়ার ডিসিশন বাউন্ডারি দ্বারা বিভাজনযোগ্য হয়ে ওঠে।

$\phi(x)$ ফাংশন

ইনপুট স্পেসের একটি সাধারণ ভেক্টর x -কে উচ্চ-মাত্রার ফিচার স্পেসে রূপান্তরিত করার জন্য যে ম্যাপিং ফাংশন ব্যবহার করা হয়, তাকে $\phi(x)$ দ্বারা প্রকাশ করা হয়। তবে সরাসরি উচ্চ-মাত্রার ফিচার গণনা করা কম্পিউটেশনালি অত্যন্ত ব্যয়বহুল। এসভিএম-এ এই উচ্চ-মাত্রার রূপান্তরের ডট গুণফলকে সরাসরি এবং সহজে গণনা করার গাণিতিক কৌশলকেই মূলত **কার্নেল ট্রিক (Kernel Trick)** বলা হয়।

বিভিন্ন প্রকার কার্নেল (Types of Kernels)

লিনিয়ার কার্নেল (Linear Kernel)

যখন ডেটা পয়েন্টগুলো ইতিমধ্যে সরলরৈখিকভাবে পৃথকীকরণযোগ্য অবস্থায় থাকে, তখন লিনিয়ার কার্নেল ব্যবহার করা হয়। এটি ডেটার ওপর কোনো উচ্চ-মাত্রার রূপান্তর প্রয়োগ না করে সরাসরি মূল ইনপুট স্পেসেই ডিসিশন বাউন্ডারি তৈরি করে।

পলিনোমিয়াল কার্নেল (Polynomial Kernel)

পলিনোমিয়াল কার্নেল ইনপুট ফিচারের বিভিন্ন ঘাত বা শক্তির সংমিশ্রণ তৈরি করে ডেটাকে উচ্চ মাত্রায় নিয়ে যায়। এটি একটি নির্দিষ্ট সীমার মধ্যে ফিচারগুলোর মধ্যকার পারস্পরিক সম্পর্ক বা ইন্টারঅ্যাকশন বিবেচনা করতে সাহায্য করে।

- **হাইপারপ্যারামিটার - Degree:** পলিনোমিয়াল কার্নেলের প্রধান হাইপারপ্যারামিটার হলো এর ডিগ্রি (Degree)। ডিগ্রির মান নির্ধারণ করে যে ডেটাকে কত উচ্চ-মাত্রার পলিনোমিয়াল স্পেসে রূপান্তর করা হবে। ডিগ্রির মান যত বেশি হবে, ডিসিশন বাউন্ডারি তত বেশি জটিল আকার ধারণ করবে।

গাউসিয়ান / আরবিএফ কার্নেল (Gaussian / RBF Kernel)

রেডিয়াল বেসিস ফাংশন (RBF) বা গাউসিয়ান কার্নেল হলো এসভিএম-এর সবচেয়ে জনপ্রিয় এবং শক্তিশালী কার্নেলগুলোর একটি। এটি ডেটা পয়েন্টগুলোকে একটি অসীম-মাত্রার (Infinite-dimensional) ফিচার স্পেসে ম্যাপিং করতে পারে, যার ফলে অত্যন্ত জটিল এবং বৃত্তাকার ডেটা বিন্যাসকেও নিখুঁতভাবে আলাদা করা সম্ভব হয়।

- **হাইপারপ্যারামিটার - Gamma:** আরবিএফ কার্নেলের আচরণ নিয়ন্ত্রণ করার প্রধান হাইপারপ্যারামিটার হলো গামা (γ)। গামার মান নির্ধারণ করে একটি একক ট্রেনিং পয়েন্টের প্রভাব ডিসিশন বাউন্ডারির ওপর কতদূর পর্যন্ত বিস্তৃত থাকবে।

অধ্যায়: হাইপারপ্যারামিটার টিউনিং (Hyperparameter Tuning)

১৩. হাইপারপ্যারামিটার (Hyperparameters)

সাপোর্ট ভেক্টর ক্লাসিফায়ারের প্যারামিটারসমূহ (Support Vector Classifier Parameters)

একটি সাপোর্ট ভেক্টর ক্লাসিফায়ার (SVC) মডেলের কার্যকারিতা এবং বাউন্ডারির আকৃতি মূলত ৪টি প্রধান হাইপারপ্যারামিটারের ওপর নির্ভর করে:

- **C:** এটি হলো সফট মার্জিনের পেনাল্টি প্যারামিটার, যা ক্লাসিফিকেশন ভুল এবং মার্জিনের আকারের মধ্যে ভারসাম্য বজায় রাখে।
- **Kernel:** এটি মডেলের জন্য উপযুক্ত কার্নেল টাইপ (যেমন: 'linear', 'poly', 'rbf') নির্বাচন করে জটিল ডেটা প্রসেস করার পথ নির্ধারণ করে।
- **Gamma:** এটি মূলত RBF কার্নেলের জন্য ব্যবহৃত প্যারামিটার, যা ডিসিশন বাউন্ডারির বক্রতা ও স্পর্শকাতরতা নিয়ন্ত্রণ করে।

- **Degree:** এটি পলিনোমিয়াল কার্নেলের জন্য প্রযোজ্য প্যারামিটার, যা রূপান্তরকারী পলিনোমিয়ালের সর্বোচ্চ ঘাত নির্ধারণ করে।

অধ্যায়: ব্যবহারিক সারাংশ (Practical Summary)

মডেল নির্বাচন (Model Selection)

- **Linear Kernel কখন ব্যবহার করবেন:** যখন ডেটাসেটে ফিচারের সংখ্যা নমুনার সংখ্যার তুলনায় অনেক বেশি থাকে (যেমন: টেক্সট ক্লাসিফিকেশন) অথবা ডেটা যখন স্বভাবগতভাবেই সরলরৈখিক বিন্যাসে থাকে।
- **Polynomial Kernel কখন ব্যবহার করবেন:** যখন ডেটার ফিচারগুলোর মধ্যে একটি সুনির্দিষ্ট এবং সীমিত মাত্রার ঘাতভিত্তিক সম্পর্ক বিদ্যমান থাকে।
- **RBF Kernel কখন ব্যবহার করবেন:** বাস্তব জীবনের অধিকাংশ জটিল, ওভারল্যাপড এবং সম্পূর্ণ নন-লিনিয়ার ডেটাসেটের জন্য RBF কার্নেলকে প্রথম পছন্দ হিসেবে বিবেচনা করা হয়।

হাইপারপ্যারামিটার নির্বাচন (Choosing Hyperparameters)

Large C বনাম Small C

- **Large C (উচ্চ পেনাল্টি):** মডেলটি কোনো ভুল ক্লাসিফিকেশন মেনে নিতে চায় না, ফলে মার্জিন ছোট হয়ে যায়। এটি ট্রেনিং ডেটাকে নিখুঁতভাবে ফিট করলেও মডেলটিতে ওভারফিটিং (Overfitting)-এর ঝুঁকি তৈরি হয়।
- **Small C (কম পেনাল্টি):** মডেলটি কিছু ভুল ক্লাসিফিকেশন উপেক্ষা করে একটি বড় মার্জিন তৈরি করে। এর ফলে মডেলটির সাধারণীকরণ ক্ষমতা বৃদ্ধি পায় এবং ওভারফিটিং রোধ হয়।

Large Gamma বনাম Small Gamma

- **Large Gamma:** গামার মান বড় হলে প্রতিটি ডেটা পয়েন্টের প্রভাবের ব্যাসার্ধ কমে যায়। ডিসিশন বাউন্ডারিটি কেবল তার খুব কাছের পয়েন্টগুলোর ওপর ভিত্তি করে তৈরি হয়, যা বাউন্ডারিকে অত্যন্ত আঁকাবাঁকা ও জটিল করে তোলে এবং ওভারফিটিং ঘটায়।
- **Small Gamma:** গামার মান ছোট হলে দূরবর্তী পয়েন্টগুলোর প্রভাবও বাউন্ডারি গণনায় অন্তর্ভুক্ত হয়। এর ফলে ডিসিশন বাউন্ডারিটি অনেক মসৃণ (Smooth) এবং বিস্তৃত হয়, যা মডেলের জেনারেলাইজেশন উন্নত করে।

ডিগ্রি নির্বাচন (Degree Selection)

পলিনোমিয়াল কার্ণেলের ক্ষেত্রে ডিগ্রির মান নির্বাচনের সময় সতর্কতা অবলম্বন করতে হয়। কম ডিগ্রি (যেমন: 2 বা 3) মডেলকে সরল ও কার্যকর রাখে। ডিগ্রির মান অতিরিক্ত বেশি হলে মডেলটি কম্পিউটেশনালি অত্যন্ত ধীরগতির হয়ে পড়ে এবং ট্রেনিং ডেটার গোলমালের প্রতি অতিরিক্ত সংবেদনশীল হয়ে ওভারফিট করে ফেলে।

হাইপারপ্যারামিটার বিশ্লেষণ (Hyperparameters)

C

- **Default:** 1.0
- **কাজ কী:** এটি হলো রেগুলারাইজেশন (Regularization) বা পেনাল্টি প্যারামিটার। মডেলটি ট্রেনিং ডেটায় কোনো ভুল বা মিস-ক্লাসিফিকেশন করলে তাকে কতটা কঠোর গাণিতিক শাস্তি (Penalty) দেওয়া হবে, তা এটি নিয়ন্ত্রণ করে।
- **অন্যান্য অপশন:** যেকোনো ধনাত্মক ফ্লোট সংখ্যা (যেমন: 0.1, 10, 100)।
- **কখন কোনটি ব্যবহার করব:**
 - **C-এর মান অনেক বেশি হলে (যেমন: ১০০):** মডেল কোনো ভুল সহ্য করবে না (Hard Margin-এর মতো কাজ করবে)। এটি ট্রেনিং ডেটার প্রতিটি পয়েন্ট নিখুঁতভাবে মেলানোর চেষ্টা করবে, যা মডেলকে জটিল করে এবং **Overfitting (High Variance)**-এর ঝুঁকি বাড়ায়।
 - **C-এর মান অনেক কম হলে (যেমন: 0.1):** মডেল কিছুটা ভুল বা মার্জিন ওভারল্যাপ করার অনুমতি দেবে (**Soft Margin**)। এটি মডেলকে সরল রাখে এবং **Underfitting (High Bias)**-এর দিকে নিয়ে যেতে পারে। ডেটাতে নয়েজ বেশি থাকলে C-এর মান কমিয়ে দেওয়া উচিত।

kernel

- **Default:** 'rbf' (Radial Basis Function / Gaussian Kernel)
- **কাজ কী:** ইনপুট ডেটাকে কোন গাণিতিক ফাংশনের মাধ্যমে উচ্চতর মাত্রায় (Higher Dimension) রূপান্তর করা হবে, তা এটি ঠিক করে।
- **অন্যান্য অপশন:** 'linear', 'poly', 'sigmoid', অথবা কাস্টম কোনো ফাংশন।
- **কখন কোনটি ব্যবহার করব:**
 - **'linear':** আপনার ডেটা যদি অলরেডি সরলরেখায় বিভাজনযোগ্য (Linearly Separable) হয়, তবে এটি ব্যবহার করা উচিত। এটি সবচেয়ে ফাস্ট কাজ করে।
 - **'poly':** উপাত্তের মধ্যে যদি বহুপদী বা বাঁকা কোনো প্যাটার্ন থাকে।
 - **'rbf':** বাস্তব জীবনের বেশিরভাগ জটিল এবং অরৈখিক (Non-linear) ডেটার জন্য এটি ডিফল্ট এবং সেরা চয়েস। এটি ডেটাকে অসীম মাত্রায় কল্পনা করে বৃত্তাকার বা গোলকধাঁধার মতো সীমানা তৈরি করতে পারে।

gamma

- **Default: 'scale'**
- **কাজ কী:** এটি শুধুমাত্র 'rbf', 'poly', এবং 'sigmoid' কার্নেলের ক্ষেত্রে কাজ করে। একটি একক সাপোর্ট ভেক্টরের প্রভাব বা বিস্তার কত দূর পর্যন্ত ছড়াবে, তা এটি নির্ধারণ করে।
- **অন্যান্য অপশন:** 'auto' অথবা যেকোনো ধনাত্মক দশমিক সংখ্যা বা ফ্লোট মান (যেমন: 0.01, 1.0)।
- **কখন কোনটি ব্যবহার করব:**
 - মান অনেক বেশি হলে (High Gamma): সাপোর্ট ভেক্টরের প্রভাব শুধু তার খুব কাছের সীমানায় সীমাবদ্ধ থাকে। ফলে সিদ্ধান্ত রেখাটি অত্যন্ত আঁকাবাঁকা ও জটিল হয়, যা মডেলে Overfitting তৈরি করতে পারে।
 - মান অনেক কম হলে (Low Gamma): সাপোর্ট ভেক্টরের প্রভাব অনেক দূর পর্যন্ত বিস্তৃত হয়। এর ফলে সিদ্ধান্ত রেখাটি অনেক বেশি সোজা ও মসৃণ (Smooth) হয়ে যায়, যা ডেটার সূক্ষ্ম প্যাটার্ন মিস করে Underfitting ঘটাতে পারে।

epsilon (শুধুমাত্র SVR-এর জন্য)

- **Default: 0.1**
- **কাজ কী:** এটি সাপোর্ট ভেক্টর রিগ্রেশনের (SVR) একটি অত্যন্ত অনন্য প্যারামিটার। এটি হাইপারপ্লেনের উভয় পাশে একটি অদৃশ্য টিউব বা মার্জিন তৈরি করে। এই টিউবের ভেতরে থাকা কোনো ভুলের জন্য মডেল কোনো পেনাল্টি বা শাস্তি গণনা করে না।
- **অন্যান্য অপশন:** যেকোনো অ-ঋণাত্মক ফ্লোট সংখ্যা।
- **কখন কোনটি ব্যবহার করব:** যদি আপনি চান আপনার রিগ্রেশন মডেলটি ছোটখাটো নয়েজ বা ছোট ভুলগুলোকে সম্পূর্ণ ইগনোর করুক, তবে এর মান বাড়িয়ে দেওয়া যায়। আর যদি আপনি চান প্রতিটি ডেটা পয়েন্টের জন্য মডেল চরম নিখুঁত হোক, তবে এর মান কমিয়ে শূন্যের কাছাকাছি নিয়ে আসতে হবে।

degree

- **Default: 3**
- **কাজ কী:** এটি শুধুমাত্র তখনই কাজ করে যখন আপনি kernel='poly' (Polynomial) সিলেক্ট করেন। এটি বহুপদী সমীকরণের সর্বোচ্চ পাওয়ার বা ডিগ্রি নির্ধারণ করে।
- **অন্যান্য অপশন:** যেকোনো পূর্ণসংখ্যা (যেমন: 2, 4, 5)।

- **কখন কোনটি ব্যবহার করব:** যদি ডেটার বক্রতা বা প্যাটার্ন অনেক বেশি জটিল হয়, তবে ডিগ্রি বাড়িয়ে ৪ বা ৫ করা যায়। তবে ডিগ্রি অতিরিক্ত বাড়ালে কম্পিউটেশনাল কস্ট এবং ওভারফিটিংয়ের সম্ভাবনা মারাত্মকভাবে বেড়ে যায়।

coef0

- **Default:** 0.0
- **কাজ কী:** এটি শুধুমাত্র 'poly' এবং 'sigmoid' কার্ণেলের ক্ষেত্রে কাজ করে। এটি উচ্চ মাত্রার সমীকরণে একটি স্বাধীন ধ্রুবক বা বায়াস (Independent term) যোগ করে, যা উচ্চ মাত্রার ম্যাপিংকে কিছুটা শিফট বা নিয়ন্ত্রণ করতে সাহায্য করে।

shrinking

- **Default:** True
- **কাজ কী:** এটি একটি অস্টিমাইজেশন ট্রিক। ট্রেইনিং প্রক্রিয়ার শুরুতে যে ডেটা পয়েন্টগুলো সিদ্ধান্ত সীমানা থেকে অনেক দূরে থাকে এবং যেগুলো সাপোর্ট ভেক্টর হওয়ার কোনো সম্ভাবনাই নেই, সেগুলোকে ব্যাকএন্ডের বড় গাণিতিক হিসাব থেকে আগেই বাদ দিয়ে দেওয়া হয়। এর ফলে মডেল অনেক দ্রুত ট্রেইন হয়। এটি সাধারণত সর্বদা True রাখাই স্ট্যান্ডার্ড।

probability (SVC-এর জন্য)

- **Default:** 'deprecated' (পুরোনো সংস্করণে এটি False থাকত)
- **কাজ কী:** এটি True করা হলে মডেলটি শুধু সরাসরি ক্লাস (যেমন: +1 বা -1) প্রেডিক্ট না করে, প্রতিটি ক্লাসের নিখুঁত সম্ভাব্যতা বা প্রোবাবিলিটি (Probability Scores) হিসাব করতে পারে (যেমন: এই ডেটাটি পজিটিভ হওয়ার সম্ভাবনা ৮৫%)।
- **কখন ব্যবহার করব:** আপনার প্রজেক্টে যদি predict_proba() ফাংশন ব্যবহার করার দরকার পড়ে (যেমন: ROC-AUC কার্ভ আঁকার জন্য), তখন এটি True করতে হবে। তবে এটি অন করলে ব্যাকএন্ডে **Platt Scaling** নামের একটি ৫-ফোল্ড ক্রস-ভ্যালিডেশন লুপ চলে, যা মডেলের ট্রেইনিং টাইম মারাত্মকভাবে বাড়িয়ে দেয়।

tol

- **Default:** 0.001 (অর্থাৎ 10^{-3})
- **কাজ কী:** এটি হলো অস্টিমাইজেশন থামানোর জন্য সহনশীলতার মাত্রা বা Tolerance। ব্যাকএন্ডে লুপ চলার সময় ভুলের উন্নতির হার যদি এই মানের চেয়ে কম হয়ে যায়, তবে মডেল ধরে নেয় সে তার বেস্ট হাইপারপ্যারামেটার পেয়ে গেছে এবং ট্রেইনিং স্টপ করে দেয়।

cache_size

- **Default:** 200 (মেগাবাইট)
- **কাজ কী:** SVM ব্যাকএন্ডে প্রচুর ম্যাট্রিক্স ডট প্রোডাক্ট হিসাব করে। র‍্যামের (RAM) ঠিক কত মেগাবাইট জায়গা এই হিসাবের ক্যাশ মেমোরি হিসেবে লক করে রাখা হবে, তা এটি ঠিক করে। বড় ডেটাসেটে ট্রেনিং স্পিড বাড়াতে এটিকে বাড়িয়ে ৫০০ বা ১০০০ করা যেতে পারে।

class_weight (শুধুমাত্র SVC-এর জন্য)

- **Default:** None
- **কাজ কী:** প্রবল ইমব্যালেন্সড ডেটাসেটের ক্ষেত্রে ছোট ক্লাসকে অতিরিক্ত গাণিতিক গুরুত্ব দেওয়ার জন্য ব্যবহৃত হয়। class_weight='balanced' দিলে মডেল স্বয়ংক্রিয়ভাবে মাইনোরিটি ক্লাসের ভুলের জন্য পেনাল্টি বাড়িয়ে দেয়।

verbose

- **Default:** False
- **কাজ কী:** এটি True করে দিলে ব্যাকএন্ডে অপটিমাইজেশনের ক্যালোকেশনের লাইভ লগ বা টেক্সট টার্মিনালে প্রিন্ট করে দেখায়। সাধারণত এটি রিগ্রেশন বা ক্লাসিফিকেশনের সাধারণ প্রজেক্টে False-ই রাখা হয়।

max_iter

- **Default:** -1 (অর্থাৎ কোনো নির্দিষ্ট সীমা নেই, যতক্ষণ না অপটিমাইজেশন শেষ হচ্ছে চলতেই থাকবে)
- **কাজ কী:** ব্যাকএন্ডের কোয়ান্ট্রটিক প্রোগ্রামিং সলভার সর্বোচ্চ কতবার লুপ বা ইটারেশন চালাবে, তা বেঁধে দেওয়া। যদি মডেলটি বিশাল ডেটাসেটে আটকে গিয়ে অনন্তকাল ধরে চলতে থাকে, তবে এখানে একটি নির্দিষ্ট সংখ্যা (যেমন: ৫০০০) সেট করে ট্রেনিং ফোর্স-স্টপ করা যায়।

decision_function_shape (শুধুমাত্র SVC-এর জন্য)

- **Default:** 'ovr' (One-vs-Rest)
- **কাজ কী:** মাল্টি-ক্লাস ক্লাসিফিকেশনের ক্ষেত্রে (যেমন: ৩ বা তার বেশি ক্লাস থাকলে) মডেলটি ব্যাকএন্ডে কীভাবে কাজ করবে তা ঠিক করে। 'ovr' প্রতিটি ক্লাসের বিপরীতে বাকি সব ক্লাসকে ধরে আলাদা আলাদা বাইনারি মডেল বানায়। অন্য অপশনটি হলো 'ovo' (One-vs-One), যা প্রতি দুটি ক্লাসের জোড়া জোড়া নিয়ে আলাদা মডেল তৈরি করে (এটি কম্পিউটেশনালি অনেক ভারী)।

break_ties (শুধুমাত্র SVC-এর জন্য)

- **Default:** False
- **কাজ কী:** এটি শুধুমাত্র তখনই কাজ করে যখন decision_function_shape='ovr' থাকে এবং ক্লাসের সংখ্যা ২-এর বেশি হয়। যদি কোনো টেস্ট ডেটার ক্ষেত্রে দুটি ক্লাসের প্রোবাবিলিটি স্কোর হুবহু এক চলে আসে (টাই হয়), তখন True দেওয়া থাকলে মডেল তার ভেতরের গাণিতিক ডিসিশন ভ্যালুর সূক্ষ্ম ব্যবধান দেখে সেই টাই ভেঙে একটি চূড়ান্ত ক্লাস ঘোষণা করে।

random_state

- **Default:** None
- **কাজ কী:** যদিও SVM একটি ডিটারমিনিস্টিক (নির্দিষ্ট) মডেল, কিন্তু যখন probability=True করা হয়, তখন ব্যাকএন্ডের বুটস্ট্রাপ এবং প্ল্যাট স্কেলিং প্রক্রিয়ার র্যান্ডমনেসকে লক করার জন্য একটি নির্দিষ্ট গাণিতিক বীজ (Seed) বা সংখ্যা (যেমন: 42) এখানে ব্যবহার করা হয়, যাতে প্রতিবার অবিকল একই রেজাল্ট আসে।

SVC() Attributes

support_vectors_

Default: Created after fit()

Decision Boundary নির্ধারণ করার জন্য ব্যবহৃত প্রকৃত Support Vector Data Point-গুলো সংরক্ষণ করে। শুধুমাত্র এই Point-গুলোর উপর ভিত্তি করেই SVM Decision Boundary তৈরি করে।

support_

Default: Created after fit()

Training Data-এর মধ্যে কোন কোন Sample Support Vector হিসেবে নির্বাচিত হয়েছে, তাদের Index সংরক্ষণ করে।

dual_coef_

Default: Created after fit()

প্রতিটি Support Vector-এর Dual Coefficient ($\alpha \times y$) সংরক্ষণ করে। Decision Function গণনার সময় এই Coefficient ব্যবহার করা হয়।

n_support_

Default: Created after fit()

প্রতিটি Class-এ কতটি Support Vector রয়েছে তা সংরক্ষণ করে। Binary Classification-এ এটি সাধারণত দুইটি সংখ্যা ধারণ করে।

coef_

Default: Created after fit() *(Only when kernel='linear')*

Linear Kernel ব্যবহার করলে প্রতিটি Feature-এর Learned Weight (Coefficient) সংরক্ষণ করে। Non-linear Kernel-এর ক্ষেত্রে এই Attribute পাওয়া যায় না।

intercept_

Default: Created after fit()

Decision Function-এর Bias (Intercept) সংরক্ষণ করে।

classes_

Default: Created after fit()

Training Data-তে থাকা সকল Target Class-এর List সংরক্ষণ করে।

fit_status_

Default: Created after fit()

Model সফলভাবে Train হয়েছে কিনা তা নির্দেশ করে। 0 হলে Training সফল হয়েছে, অন্য মান হলে Training-এ সমস্যা হয়েছে।

shape_fit_

Default: Created after fit()

Training Data-এর Shape (n_samples, n_features) সংরক্ষণ করে।

probA_

Default: Created after fit() *(Only when probability=True)*

Probability Estimation-এর জন্য ব্যবহৃত Platt Scaling-এর Parameter A সংরক্ষণ করে।

probB_

Default: Created after fit() *(Only when probability=True)*

Probability Estimation-এর জন্য ব্যবহৃত Platt Scaling-এর Parameter B সংরক্ষণ করে।

class_weight_

Default: Created after fit() *(When class_weight is used)*

Training-এর সময় প্রতিটি Class-এর জন্য ব্যবহৃত Effective Weight সংরক্ষণ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় Model মোট কতটি Input Feature ব্যবহার করেছে তা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

SVR() Attributes

support_vectors_

Default: Created after fit()

Regression Model-এর Decision Function নির্ধারণ করার জন্য ব্যবহৃত প্রকৃত Support Vector-গুলো সংরক্ষণ করে।

support_

Default: Created after fit()

Training Data-এর মধ্যে কোন কোন Sample Support Vector হিসেবে নির্বাচিত হয়েছে, তাদের Index সংরক্ষণ করে।

dual_coef_

Default: Created after fit()

প্রতিটি Support Vector-এর Dual Coefficient সংরক্ষণ করে। Prediction করার সময় এই মানগুলো ব্যবহার করা হয়।

coef_

Default: Created after fit() (*Only when kernel='linear'*)

Linear Kernel ব্যবহার করলে প্রতিটি Feature-এর Learned Weight সংরক্ষণ করে। অন্য Kernel-এর ক্ষেত্রে এই Attribute পাওয়া যায় না।

intercept_

Default: Created after fit()

Regression Function-এর Bias (Intercept) সংরক্ষণ করে।

fit_status_

Default: Created after fit()

Model সফলভাবে Train হয়েছে কিনা তা নির্দেশ করে। 0 হলে Training সফল হয়েছে।

shape_fit_

Default: Created after fit()

Training Data-এর Shape (n_samples, n_features) সংরক্ষণ করে।

n_features_in_

Default: Created after fit()

Training-এর সময় ব্যবহৃত মোট Input Feature-এর সংখ্যা সংরক্ষণ করে।

feature_names_in_

Default: Created after fit() (*DataFrame input only*)

Training Data যদি Pandas DataFrame হয়, তাহলে Feature-গুলোর নাম সংরক্ষণ করে।

১৪. স্কিট-লার্ন ইম্পোর্ট (Scikit-learn Import)

Code:

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

cancer = load_breast_cancer()
X = pd.DataFrame(cancer.data, columns=cancer.feature_names)
y = pd.Series(cancer.target)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

numeric_features = X.columns.tolist()
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features)
    ]
)

svm_pipeline = Pipeline(
    steps=[
        ('preprocessor', preprocessor),
        ('model', SVC(random_state=42))
    ]
)

param_grid = {
    'model__C': [0.1, 1, 10],
    'model__kernel': ['linear', 'rbf'],
    'model__gamma': ['scale', 'auto']
}
```

```
grid_search = GridSearchCV(  
    estimator=svm_pipeline,  
    param_grid=param_grid,  
    cv=5,  
    scoring='accuracy',  
    n_jobs=-1  
)  
  
grid_search.fit(X_train, y_train)  
  
print("--- GridSearchCV Tuning Results ---")  
print("Best Parameters Found:", grid_search.best_params_)  
print(f"Best CV Accuracy: {grid_search.best_score_:.4f}\n")  
  
best_svm_model = grid_search.best_estimator_  
y_pred = best_svm_model.predict(X_test)  
  
print("--- Final Model Performance on Test Set ---")  
print(f"Test Accuracy: {accuracy_score(y_test, y_pred):.4f}\n")  
print(classification_report(y_test, y_pred))
```

Code:

```
wine = load_wine()
X = pd.DataFrame(wine.data, columns=wine.feature_names)
y = pd.Series(wine.target)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

numeric_features = X.columns.tolist()
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features)
    ]
)

svm_pipeline = Pipeline(
    steps=[
        ('preprocessor', preprocessor),
        ('model', SVC(random_state=42))
    ]
)
```

```
grid_param = [
    {
        "model__kernel": ['linear'],
        "model__C": [0.01, 0.1, 1, 10, 50, 100]
    },
    {
        "model__kernel": ['rbf'],
        "model__C": [0.01, 0.1, 1, 100],
        "model__gamma": [0.01, 0.1, 5, 10, 'scale', 'auto']
    },
    {
        "model__kernel": ['poly'],
        "model__C": [0.01, 0.1, 1, 100],
        "model__degree": [2, 3]
    }
]
```

```
grid_search = GridSearchCV(
    estimator=svm_pipeline,
    param_grid=grid_param,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("--- GridSearchCV Tuning Results ---")
print("Best Parameters Found:", grid_search.best_params_)
print(f"Best CV Accuracy: {grid_search.best_score_:.4f}\n")

best_svm_model = grid_search.best_estimator_
y_pred = best_svm_model.predict(X_test)

print("--- Final Model Performance on Test Set ---")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred):.4f}\n")
print(classification_report(y_test, y_pred, target_names=wine.target_names))
```

Code:

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ('num', StandardScaler(), X_train.select_dtypes(include=np.number).columns)  
    ]  
)  
  
pipeline = Pipeline(steps=[  
    ('preprocessor', preprocessor),  
    ('classifier', SVC())  
)  
  
param_grid = {  
    'classifier__C': [0.1, 0.5, 1, 5, 10],  
    #'classifier__penalty': ['l1', 'l2'],  
    'classifier__kernel': ['rbf', 'linear', 'poly', 'sigmoid'],  
    #'classifier__epsilon': [0.1, 0.2, 0.5, 1, 2, 5, 10],  
    'classifier__shrinking': [True, False]  
}  
  
grid_search = GridSearchCV(  
    estimator = pipeline,  
    param_grid = param_grid,  
    scoring = 'accuracy',  
    cv = 5,  
    n_jobs = -1  
)
```

```
grid_search.fit(X_train, y_train)  
grid_search.best_params_  
grid_search.best_score_  
best_model = grid_search.best_estimator_  
y_pred = best_model.predict(X_test)
```

১৫. পাইথন উদাহরণ — From-Scratch Implementation (Python Example)

নিচে একটি সরলীকৃত From-Scratch Linear SVM (Gradient Descent দিয়ে Hinge Loss মিনিমাইজ করে) দেখানো হলো:

```
import numpy as np

class LinearSVMScratch:
    def __init__(self, lr=0.001, C=1.0, n_iters=1000):
        self.lr = lr
        self.C = C
        self.n_iters = n_iters
        self.w = None
        self.b = 0.0

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.w = np.zeros(n_features)
        for _ in range(self.n_iters):
            for i, x_i in enumerate(X):
                condition = y[i] * (np.dot(x_i, self.w) + self.b) >= 1
                if condition:
                    dw = self.w
                    db = 0
                else:
                    dw = self.w - self.C * y[i] * x_i
                    db = -self.C * y[i]
                self.w -= self.lr * dw
                self.b -= self.lr * db

    def predict(self, X):
        return np.sign(np.dot(X, self.w) + self.b)

# Example usage
X = np.array([[4,3],[5,4],[6,3],[1,1],[2,1],[1,2]])
y = np.array([1,1,1,-1,-1,-1])
model = LinearSVMScratch(lr=0.001, C=1.0, n_iters=1000)
model.fit(X, y)
print(model.predict([[3,3]])) # -> [1.]
```

১৬. প্রেডিকশন প্রসেস (Prediction Process)

Training শেষে (w ও b জানা থাকলে), নতুন স্যাম্পলের Prediction মাত্র দুই ধাপে হয়: (১) Decision Function $f(x) = w \cdot x + b$ গণনা করা, এবং (২) $f(x) \geq 0$ হলে ক্লাস +1, নাহলে ক্লাস -1 হিসেবে চূড়ান্ত সিদ্ধান্ত নেওয়া। Kernel ব্যবহার করা হলে, প্রতিটি নতুন স্যাম্পলকে প্রথমে Kernel Function দিয়ে রূপান্তর করে তারপর Support Vector-গুলোর সাথে তুলনা করে Decision Function গণনা করা হয়।

২২. সাক্ষাৎকারের প্রশ্ন (Interview Questions)

Q1: SVM-এ ফিচার স্কেলিং (Feature Scaling) করা কেন বাধ্যতামূলক, যেখানে র‍্যান্ডম ফরেস্ট এর প্রয়োজন হয় না?

- **Answer:** SVM মূলত একটি জ্যামিতিক দূরত্ব-ভিত্তিক (Distance-based) অ্যালগরিদম। এটি বিভিন্ন ক্লাসের মধ্যকার দূরত্ব বা মার্জিন সর্বোচ্চ করার জন্য সাপোর্ট ভেক্টরগুলোর ইউক্লিডিয়ান দূরত্ব (Euclidean Distance) গণনা করে। যদি কোনো একটি ফিচারের রেঞ্জ অনেক বড় হয় (যেমন: Income = ৫০,০০০) এবং অন্যটির রেঞ্জ ছোট হয় (যেমন: Age = ২), তবে দূরত্বের হিসাবে বড় রেঞ্জের ফিচারটি পুরো মডেলকে ডমিনেট করবে এবং ছোট ফিচারের গুরুত্ব হারিয়ে যাবে। তাই সব ফিচারকে সমান গুরুত্ব দিতে স্কেলিং বাধ্যতামূলক। অন্যদিকে, র‍্যান্ডম ফরেস্ট ডিসিশন ট্রির রুলস মেনে নোড স্প্লিট করে, যা দূরত্বের ওপর নির্ভর করে না।

Q2: 'Kernel Trick' বলতে কী বোঝায়? এটি কি আসলেই ডেটার ডাইমেনশন বাড়িয়ে দেয়?

- **Answer:** যখন কোনো ডেটাসেটকে লিনিয়ার লাইনে আলাদা করা যায় না, তখন কার্নেল ট্রিক ব্যবহৃত হয়। তাত্ত্বিকভাবে এটি ডেটাকে একটি উচ্চতর মাত্রায় (Higher Dimension) ট্রান্সফর্ম করে যেন সেখানে একটি লিনিয়ার হাইপারপ্লেন দিয়ে ক্লাসগুলোকে আলাদা করা যায়। তবে এটি বাস্তবে বা ফিজিক্যালি ডেটার কোনো নতুন কলাম তৈরি করে না বা উচ্চ মাত্রায় ডেটা রিলোড করে না। এটি একটি গাণিতিক শর্টকাট ফাংশন, যা মূল লো-ডাইমেনশনে বসেই উচ্চ ডাইমেনশনের ডট প্রোডাক্টের $(x_i \cdot x_j)$ চূড়ান্ত ফলাফল সরাসরি গণনা করে দেয়। এর ফলে মেমোরি এবং কম্পিউটেশনাল সময় মারাত্মকভাবে বেঁচে যায়।

Q3: SVM-এর C প্যারামিটারটি কীভাবে Bias এবং Variance-এর ভারসাম্য (Trade-off) নিয়ন্ত্রণ করে?

- **Answer:** C প্যারামিটারটি মূলত রেগুলারাইজেশন এবং ট্রেইনিং এররের পেনাল্টি নিয়ন্ত্রণ করে:
 - **C -এর মান খুব বেশি হলে:** মডেল ট্রেইনিং সেটে কোনো ভুল করতে চায় না (Hard Margin)। এটি প্রতিটি নয়েজ পয়েন্টকে মেলানোর জন্য সিদ্ধান্ত সীমানাকে অনেক জটিল ও আঁকাবাঁকা করে তোলে। এর ফলে মডেলের **Bias কমে (Low Bias)** কিন্তু **Variance মারাত্মক বাড়ে (High Variance)**, যা ওভারফিটিং ঘটায়।
 - **C -এর মান খুব কম হলে:** মডেল কিছুটা ভুল বা মার্জিন ওভারল্যাপ করার ছাড় দেয় (Soft Margin)। সিদ্ধান্ত সীমানাটি খুব সরল বা মসৃণ হয়। এর ফলে মডেলের

Variance কমে (Low Variance) কিন্তু Bias বেড়ে যায় (High Bias), যা আন্ডারফিটিং ঘটাতে পারে।

Q4: SVM-এর প্রধান সীমাবদ্ধতা বা ডিসঅ্যাডভান্টেজগুলো কী কী?

- **Answer:** শক্তিশালী মডেল হলেও SVM-এর কিছু বড় সীমাবদ্ধতা রয়েছে:
 - **বৃহৎ ডেটাসেটে ধীরগতি:** ডেটার সংখ্যা (Rows) অনেক বেশি হলে (যেমন: ১ লাখের উপরে) এর ব্যাকএন্ডের কোয়ান্ট্রিটিক প্রোগ্রামিং হিসাব করতে প্রচুর সময় ও কম্পিউটেশনাল পাওয়ার লাগে।
 - **নয়েজি ডেটায় দুর্বলতা:** ডেটাসেটে যদি ক্লাসের সীমানাগুলো একে অপরের ভেতর অতিরিক্ত ঢুকে থাকে (High Overlapping Noise), তবে SVM ভালো মার্জিন তৈরি করতে পারে না।
 - **প্রোবাবিলিটি স্কোরের অভাব:** এটি লজিস্টিক রিগ্রেশনের মতো স্বয়ংক্রিয়ভাবে কোনো সম্ভাব্যতা স্কোর (predict_proba) দেয় না। এটি অন করতে গেলে প্ল্যাট স্কেলিং মেথড ব্যবহার করতে হয় যা মডেলকে আরও ধীরগতির করে তোলে।

Q5: 'Support Vectors' বলতে কী বোঝায় এবং মডেলের প্রেডিকশনে এদের ভূমিকা কী?

- **Answer:** ট্রেনিং ডেটাসেটের যে নির্দিষ্ট বিন্দু বা স্যাম্পলগুলো সিদ্ধান্ত সীমানার (Hyperplane) সবচেয়ে কাছাকাছি অবস্থান করে এবং মার্জিনাল লাইন লক করতে সরাসরি অংশ নেয়, সেগুলোকে **Support Vectors** বলে। ট্রেনিং শেষ হওয়ার পর, SVM বাকি সব সাধারণ ডেটা পয়েন্ট মেমোরি থেকে সম্পূর্ণ ডিলিট করে দেয় এবং শুধুমাত্র এই সাপোর্ট ভেক্টরগুলোকে ধরে রাখে। নতুন কোনো অজানা টেস্ট ডেটা আসলে, মডেলটি কেবল এই সাপোর্ট ভেক্টরগুলোর সাপেক্ষে দূরত্ব মেপে তার ক্লাস নির্ধারণ করে।

২৩. সারাংশ (Summary)

সাপোর্ট ভেক্টর মেশিন (SVM) — সারসংক্ষেপ

১. মূল দর্শন ও জ্যামিতিক উপাদান (Core Philosophy & Geometry)

- হাইপারপ্লেন (Hyperplane):** SVM হলো একটি সুপারভাইজড লার্নিং অ্যালগরিদম, যার মূল লক্ষ্য হলো বহুমাত্রিক উপাত্ত ক্ষেত্রে (n -Dimensional Space) একটি লিনিয়ার সিদ্ধান্ত সীমানা বা হাইপারপ্লেন ($w^T x + b = 0$) তৈরি করা।
- জ্যামিতিক রূপান্তর:** মাত্রাভেদে এই বাউন্ডারিটি ভিন্ন রূপ নেয়; যেমন—দ্বিমাত্রিকে (2D) এটি একটি সরলরেখা, ত্রিমাত্রিকে (3D) একটি সমতল পৃষ্ঠ বা প্লেন, এবং তার অধিক মাত্রায় এটি একটি হাইপারপ্লেন।
- সাপোর্ট ভেক্টরস (Support Vectors):** সিদ্ধান্ত রেখার সবচেয়ে নিকটে এবং মার্জিনাল বাউন্ডারির ($w^T x + b = \pm 1$) ওপর অবস্থিত যে বিশেষ বিন্দুগুলো হাইপারপ্লেনের অবস্থান ও অভিযোজন (Orientation) সরাসরি নির্ধারণ করে, তাদের সাপোর্ট ভেক্টর বলে।

২. মার্জিন সর্বোচ্চকরণ ও গাণিতিক ভিত্তি (Margin Maximization & Mathematics)

- ম্যাক্সিমাম মার্জিন ক্লাসিফায়ার:** SVM কেবল ডেটা আলাদাই করে না, বরং দুই ক্লাসের মধ্যবর্তী সম্ভাব্য সবচেয়ে চওড়া রাস্তা বা সর্বোচ্চ মার্জিন (Maximum Margin) তৈরি করে।
- গাণিতিক সমীকরণ:** জ্যামিতিক প্রতিপাদন অনুযায়ী মোট মার্জিনের দূরত্ব হলো:

$$\text{Margin } (M) = \frac{2}{\|w\|}$$

- অপটিমাইজেশন লজিক:** গাণিতিক নিয়ম অনুযায়ী, ভগ্নাংশের হর $\|w\|$ যত হ্রাস পাবে, মোট মার্জিন (M) তত বৃদ্ধি পাবে। তাই ক্যালকুলাসের সুবিধার্থে SVM-এর মূল অপটিমাইজেশন সমস্যাটিকে একটি সমতুল্য সর্বনিম্নকরণ সমস্যা হিসেবে রূপ দেওয়া হয়:

$$\min \frac{1}{2} \|w\|^2 \quad \text{subject to} \quad y_i(w^T x_i + b) \geq 1$$

৩. লস ফাংশন এবং আউটলায়ার প্রতিরোধ ক্ষমতা (Loss Function & Robustness)

- হিঞ্জ লস (Hinge Loss):** লজিস্টিক রিগ্রেশনের ক্রমাগত পরিবর্তনশীল 'লগ-লস'-এর বিপরীতে SVM ব্যবহার করে হিঞ্জ লস:

$$\text{Hinge Loss} = \max(0, 1 - y_i(w^T x + b))$$

- **পেনাল্টি মেকানিজম:** যদি কোনো ডেটা পয়েন্ট সঠিকভাবে ক্লাসিফাইড এবং মার্জিনের বাইরে থাকে ($y_i(w^T x + b) \geq 1$), তবে তার লস সরাসরি **০ (Zero)** হয়।
- **Robust to Outliers:** সিদ্ধান্ত সীমানাটি শুধুমাত্র অল্প কিছু সাপোর্ট ভেক্টরের ওপর নির্ভর করে। মার্জিনের বাইরে অবস্থিত অন্য হাজারটি বিন্দু পরিবর্তন বা অপসারণ করলেও হাইপারপ্লেন স্থানচ্যুত হয় না, যা একে আউটলায়ারের বিরুদ্ধে অত্যন্ত শক্তিশালী (Robust) করে তোলে।

৪. কার্নেল ট্রিক ও ডেটাসেটের প্রকারভেদ (Kernel Trick & Dataset Types)

- **লিনিয়ারলি সেপারেবল ডেটা:** ডেটা সোজা লাইনে ভাগ করা সম্ভব হলে কোনো ভুল বা ওভারল্যাপিং ছাড়া **"Hard Margin"** ব্যবহার করে নিখুঁত শ্রেণীবিভাগ করা যায়।
- **অরৈখিক উপাত্ত বিন্যাস (Non-Linear Data):** বাস্তব জীবনের জটিল ডেটা সোজা লাইনে আলাদা করা না গেলে SVM তার বিখ্যাত **Kernel Trick** (যেমন: Polynomial, RBF কার্নেল) ব্যবহার করে। এটি মূল ইনপুট স্পেসের ডেটাকে কোনো বাস্তব রূপান্তর ছাড়াই উচ্চ মাত্রার ফিচার স্পেসে (Higher Dimensional Space) গাণিতিক শর্টকাটের মাধ্যমে নিখুঁতভাবে পৃথক করে।

৫. অন্যান্য মডেলের তুলনায় SVM-এর অপরিহার্যতা (Why We Need SVM)

- **লজিস্টিক রিগ্রেশন বনাম SVM:** লজিস্টিক রিগ্রেশনে মার্জিনের ওপর কোনো সুনির্দিষ্ট জ্যামিতিক নিয়ন্ত্রণ থাকে না এবং এটি আউটলায়ার দ্বারা প্রভাবিত হয়। SVM মার্জিন সর্বোচ্চ করে এবং হিঞ্জ লসের মাধ্যমে অপ্রয়োজনীয় বিন্দুর প্রভাব মুক্ত রেখে লজিস্টিক রিগ্রেশনের চেয়ে অনেক বেশি স্থিতিশীল (Robust) সিদ্ধান্ত বাউন্ডারি দেয়।
- **KNN বনাম SVM:** KNN একটি অলস শিক্ষণ পদ্ধতি (Lazy Learner), যা টেস্ট টাইমে প্রতিটি ডেটার জন্য পুরো ডেটাসেটের দূরত্ব মাপে (ধীরগতির ও মেমোরি সাপেক্ষ)। পক্ষান্তরে, SVM ট্রেনিং শেষে শুধু অল্প কিছু সাপোর্ট ভেক্টর মেমোরিতে রাখে, ফলে এর প্রেডিকশন স্পিড অত্যন্ত ফাস্ট।
- **Decision Tree / Random Forest বনাম SVM:** বৃক্ষ-ভিত্তিক মডেলগুলো অক্ষ-বরাবর লম্বালম্বিভাবে উপাত্ত খণ্ডিত করে বৃত্তাকার বা জটিল প্যাটার্নে ওভারফিটিং তৈরি করে। কিন্তু SVM তার আরবিএফ (RBF) কার্নেল দিয়ে অতি সহজেই আঁকাবাঁকা বৃত্তাকার সিদ্ধান্ত সীমানা চিহ্নিত করতে পারে।

১৭-১৮. সুবিধা ও অসুবিধা (Advantages & Disadvantages)

Advantages (সুবিধা)	Disadvantages (অসুবিধা)
High-Dimensional ডেটায় খুব ভালো কাজ করে	বৃহৎ Dataset-এ Training অনেক ধীর (Quadratic Programming-এর কারণে)
Kernel Trick-এর মাধ্যমে জটিল Non-Linear প্যাটার্নও ধরতে পারে	Hyperparameter Tuning (C, gamma) তুলনামূলক জটিল ও সময়সাপেক্ষ
Outlier-এর প্রতি Robust (শুধু Support Vector-এর উপর নির্ভরশীল)	Feature Scaling ছাড়া প্রায় কাজ করে না — বাধ্যতামূলক প্রি-প্রসেসিং প্রয়োজন
Overfitting প্রতিরোধে ভালো (বিশেষত Margin Maximization-এর কারণে)	Probability Estimate সরাসরি পাওয়া যায় না (আলাদা Calibration লাগে)

১৯. প্রয়োগক্ষেত্র (Applications)

- Text ও Document Classification (উচ্চ-মাত্রিক Sparse Feature Space-এ চমৎকার কাজ করে)।
- Image Classification, বিশেষত Face Detection-এর প্রাথমিক যুগে।
- Bioinformatics: Gene Expression ডেটা থেকে রোগ শ্রেণিবিন্যাস।
- Handwriting Recognition (যেমন MNIST Digit Classification)।

২১. সাধারণ ভুলত্রুটি (Common Mistakes)

- Feature Scaling ছাড়া SVM ব্যবহার করা — যেহেতু এটি দূরত্ব-ভিত্তিক, বড় Scale-এর ফিচার সম্পূর্ণ ফলাফল বিকৃত করে দিতে পারে।
- বড় Dataset-এ চিন্তাভাবনা ছাড়া RBF Kernel ব্যবহার করা — Training Time বহুগুণ বেড়ে যেতে পারে; বড় Dataset-এ Linear Kernel বা SGDClassifier বিবেচনা করা উচিত।
- C ও gamma-কে ডিফল্ট রেখে দিয়ে ফলাফল খারাপ পাওয়া — এই দুটি Hyperparameter GridSearchCV দিয়ে যত্ন সহকারে Tune করা দরকার।
- predict_proba()-কে অন্যান্য Probabilistic Model-এর মতো সরাসরি নির্ভরযোগ্য মনে করা, যদিও SVM-এ এটি Platt Scaling নামক একটি আলাদা Approximation কৌশলের মাধ্যমে আসে।

সম্পূর্ণ গাণিতিক বাস্তবায়ন — সম্পূর্ণ ম্যানুয়াল ওয়াকথ্রু (Complete Mathematical Implementation — Full Manual Walkthrough)

Introduction

এই সেকশনে একটি ছোট, Linearly Separable ডেটাসেট নিয়ে সম্পূর্ণ SVM হাতে-কলমে সমাধান করা হবে — Support Vector চিহ্নিতকরণ, Hyperplane সমীকরণ বের করা, Margin গণনা, এবং নতুন স্যাম্পলের Classification পর্যন্ত।

Dataset

দুটি ক্লাস: +1 (Positive) এবং -1 (Negative), প্রতিটি বিন্দু (X1, X2)।

Sample	X1	X2	Class
1	4	3	+1
2	5	4	+1
3	6	3	+1
4	1	1	-1
5	2	1	-1
6	1	2	-1

Hyperplane Equation

$$w \cdot x + b = 0$$

w: Weight Vector (হাইপারপ্লেনের দিক নির্ধারণ করে)। x: ইনপুট ফিচার ভেক্টর। b: Bias (হাইপারপ্লেনটি Origin থেকে কতদূর সরানো)।

Support Vector Identification

প্রথমে দুই ক্লাসের মধ্যে সবচেয়ে কাছের বিন্দুর জোড়া খুঁজে বের করতে হবে, কারণ এরাই Maximum-Margin Hyperplane নির্ধারণ করে (Support Vector প্রার্থী)।

$$\text{distance}((4, 3), (1, 1)) = \sqrt{[(4-1)^2 + (3-1)^2]} = \sqrt{[9+4]} = \sqrt{13} = 3.6056$$

$$\text{distance}((4, 3), (2, 1)) = \sqrt{[(4-2)^2 + (3-1)^2]} = \sqrt{[4+4]} = \sqrt{8} = 2.8284$$

$$\text{distance}((4, 3), (1, 2)) = \sqrt{[(4-1)^2 + (3-2)^2]} = \sqrt{[9+1]} = \sqrt{10} = 3.1623$$

$$\text{distance}((5, 4), (1, 1)) = \sqrt{[(5-1)^2 + (4-1)^2]} = \sqrt{[16+9]} = \sqrt{25} = 5.0000$$

$$\text{distance}((5, 4), (2, 1)) = \sqrt{[(5-2)^2 + (4-1)^2]} = \sqrt{[9+9]} = \sqrt{18} = 4.2426$$

$$\text{distance}((5, 4), (1, 2)) = \sqrt{[(5-1)^2 + (4-2)^2]} = \sqrt{[16+4]} = \sqrt{20} = 4.4721$$

$$\text{distance}((6, 3), (1, 1)) = \sqrt{[(6-1)^2 + (3-1)^2]} = \sqrt{[25+4]} = \sqrt{29} = 5.3852$$

$$\text{distance}((6, 3), (2, 1)) = \sqrt{[(6-2)^2 + (3-1)^2]} = \sqrt{[16+4]} = \sqrt{20} = 4.4721$$

$$\text{distance}((6, 3), (1, 2)) = \sqrt{[(6-1)^2 + (3-2)^2]} = \sqrt{[25+1]} = \sqrt{26} = 5.0990$$

সবচেয়ে কম দূরত্ব পাওয়া গেছে (4, 3) (Class +1) ও (2, 1) (Class -1)-এর মধ্যে, দূরত্ব = 2.8284।
তাই এই দুইটি বিন্দু প্রাথমিক Support Vector প্রার্থী।

Hyperplane নির্ণয় (Two Closest-Point Method)

যেহেতু সর্বোচ্চ-মার্জিন হাইপারপ্লেন সবসময় এই দুই নিকটতম বিন্দুর সংযোগরেখার লম্ব-দ্বিখণ্ডক (perpendicular bisector) বরাবর হয়, তাই w ভেক্টরটি $(x_{\text{pos}} - x_{\text{neg}})$ দিকের সমানুপাতিক ধরা হলো:

$$x_{\text{pos}} - x_{\text{neg}} = (4-2, 3-1) = (2, 2)$$

Support Vector শর্ত অনুযায়ী: $w \cdot x_{\text{pos}} + b = 1$ এবং $w \cdot x_{\text{neg}} + b = -1$ । এই দুইটি সমীকরণ বিয়োগ করলে পাওয়া যায় $w \cdot (x_{\text{pos}} - x_{\text{neg}}) = 2$ ।

$$\text{যেহেতু } w = a \cdot (2, 2), \text{ তাই } a \cdot [(2)^2 + (2)^2] = 2 \rightarrow a \cdot 8 = 2 \rightarrow a = 2/8 = 1/4$$

$$w = (1/4) \cdot (2, 2) = (1/2, 1/2) = (0.5, 0.5)$$

$$b \text{ নির্ণয়: } w \cdot x_{\text{neg}} + b = -1 \rightarrow (1/2) \times 2 + (1/2) \times 1 + b = -1 \rightarrow 3/2 + b = -1 \rightarrow b = -1 - 3/2 = -5/2 = -2.5$$

$$\text{চূড়ান্ত Hyperplane: } (0.5) \cdot X_1 + (0.5) \cdot X_2 + (-2.5) = 0$$

Decision Function ও প্রতিটি Point-এ Margin যাচাই

$$f(x) = w \cdot x + b$$

প্রতিটি ট্রেনিং পয়েন্টে $f(x)$ গণনা করে দেখা হচ্ছে সবাই যথাযথ Margin ($|f(x)| \geq 1$) মেনে চলছে কি না:

$$(4, 3) (+1): f(x) = (0.5) \times 4 + (0.5) \times 3 + (-2.5) = 1.0$$

$$(5, 4) (+1): f(x) = (0.5) \times 5 + (0.5) \times 4 + (-2.5) = 2.0$$

$$(6, 3) (+1): f(x) = (0.5) \times 6 + (0.5) \times 3 + (-2.5) = 2.0$$

$$(1, 1) (-1): f(x) = (0.5) \times 1 + (0.5) \times 1 + (-2.5) = -1.5$$

$$(2, 1) (-1): f(x) = (0.5) \times 2 + (0.5) \times 1 + (-2.5) = -1.0$$

$$(1, 2) (-1): f(x) = (0.5) \times 1 + (0.5) \times 2 + (-2.5) = -1.0$$

Margin Evaluation Table

Point	True Class	$f(x) = w \cdot x + b$	Status
(4, 3)	+1	1.00	Support Vector
(5, 4)	+1	2.00	Non-SV (মার্জিনের বাইরে)
(6, 3)	+1	2.00	Non-SV (মার্জিনের বাইরে)
(1, 1)	-1	-1.50	Non-SV (মার্জিনের বাইরে)
(2, 1)	-1	-1.00	Support Vector
(1, 2)	-1	-1.00	Support Vector

Margin গণনা

$$||w|| = \sqrt{(w_1^2 + w_2^2)}$$

$$||w|| = \sqrt{((0.5)^2 + (0.5)^2)} = \sqrt{0.5} = 0.7071$$

$$\text{Margin (একপাশের প্রস্থ)} = 1 / ||w||$$

$$\text{Margin} = 1/0.7071 = 1.4142, \text{ এবং সম্পূর্ণ Margin Width (দুই পাশ মিলিয়ে)} = 2/||w|| = 2.8284$$

Prediction on a New Sample

নতুন স্যাম্পল: $X = (3, 3)$ ।

$$f(x) = (0.5) \times 3 + (0.5) \times 3 + (-2.5) = 0.5$$

যেহেতু $f(x) = 0.5 \geq 0$, তাই প্রেডিক্টেড ক্লাস = +1।

Distance from Hyperplane

$$\text{distance} = |f(x)| / ||w||$$

$$\text{distance} = |0.5| / 0.7071 = 0.7071$$

Classification (Kernel নোট)

এই ডেটাসেটটি Linearly Separable বলে সরাসরি Linear Kernel (কোনো ট্রান্সফরমেশন ছাড়াই) ব্যবহার করা হয়েছে। যদি ডেটা Linearly Separable না হতো, তাহলে RBF বা Polynomial-এর মতো Kernel Function ব্যবহার করে ডেটাকে উচ্চতর-ডাইমেনশনের একটি Feature Space-এ ম্যাপ করা হতো, যেখানে তারা Linearly Separable হয়ে যায় — তবে এই ম্যানুয়াল উদাহরণে হিসাব শুধুমাত্র

Linear ক্ষেত্রেই সীমাবদ্ধ রাখা হয়েছে, কারণ Kernel Transformation-এর ম্যানুয়াল হিসাব উচ্চ-ডাইমেনশনাল এবং ক্যালকুলেটরে সরাসরি যাচাই করা কঠিন।

Interpretation

লক্ষণীয় যে $(4, 3)$ $(+1)$, $(2, 1)$ (-1) এবং $(1, 2)$ (-1) — এই তিনটি বিন্দুর জন্য $f(x)$ ঠিক ± 1 , অর্থাৎ এরাই প্রকৃত Support Vector — এরাই একমাত্র হাইপারপ্লেন নির্ধারণে ভূমিকা রাখে। বাকি বিন্দুগুলো $(5, 4)$ ও $(6, 3)$ Margin-এর অনেক ভেতরে ($f(x)=2$), তাই তাদের সরিয়ে ফেললেও হাইপারপ্লেনের কোনো পরিবর্তন হতো না — এটাই SVM-এর একটি মূল বৈশিষ্ট্য: শুধুমাত্র Support Vector-গুলোই সিদ্ধান্ত নির্ধারণ করে, বাকি ডেটা পয়েন্ট নয়।